

# Часть II

## Механизмы абстракции

Здесь описываются средства языка C++ для определения и использования новых типов. Представлены методики объектно-ориентированного и обобщенного стилей программирования.

### **Главы**

- 10. Классы**
- 11. Перегрузка операций**
- 12. Наследование классов**
- 13. Шаблоны**
- 14. Обработка исключений**
- 15. Иерархии классов**

*«... нет дела, коего устройство было бы труднее, ведение опаснее, а успех сомнительнее, нежели замена старых порядков новыми. Кто бы ни выступал с подобным начинанием, его ожидает враждебность тех, кому выгодны старые порядки, и холодность тех, кому выгодны новые ...»*

— Николо Макиавелли («Государь» §vi)

# Классы

*Эти типы совсем не «абстрактные»;  
они столь же реальны как **int** или **float**.  
— Дуг МакИлрой*

Концепции и классы — члены класса — управление доступом — конструкторы — *статические* члены класса — копирование по умолчанию — *константные* функции-члены — **this** — *структуры* — определение функции в классе — конкретные классы — функции-члены и функции поддержки — перегруженные операции — использование конкретных классов — деструкторы — конструкторы по умолчанию — локальные переменные — копирование, определяемое пользователем — **new** и **delete** — классовые объекты как члены класса — массивы — статическая память — временные объекты — объединения — советы — упражнения.

## 10.1. Введение

Концепция классов языка C++ нацелена на то, чтобы снабдить программиста средствами создания новых типов данных, пользоваться которыми удобно в той же мере, что и встроенными типами. Кроме того, концепции производных классов (глава 12) и шаблонов (глава 13) позволяют программисту синтаксическим образом извлечь практическую выгоду из определенного рода отношений между классами.

Тип есть конкретное представление концепции. Например, встроенный тип **float** вместе с операциями +, -, \* и т.д. представляет собой конкретную (приближенную) реализацию математической концепции вещественных чисел. *Класс* — это *тип, определяемый пользователем* (программистом). Мы создаем новые типы, чтобы точнее отражать концепции, не представимые встроенными типами. Например, для телефонной программы может пригодиться тип **Trunk line** (магистральная междугородняя телефонная линия), для видеоигры — тип **Explosion** (взрыв), а для текстового процессора — тип **list<Paragraph>** (список параграфов, то есть абзацев). Программы, в которых типы близки концепциям предметной области задачи, и понимать легче, и легче сопровождать (в том числе модифицировать) в течении всего срока их эксплуатации.

Умело подобранный набор пользовательских типов делает программу более компактной и более выразительной. Он также позволяет выполнять тщательный анализ кода. В частности, компилятор может при этом надежно выявлять случаи недопустимого использования объектов указанных типов, которые в противном случае не были бы выявлены иначе, как с помощью процесса массивованной и длительной отладки.

Фундаментальная идея, которой нужно следовать при построении нового типа, заключается в том, чтобы четко отделить случайные детали реализации (например, расположение в памяти отдельных частей объектов этого типа) от таких свойств, которые обеспечивают удобное и безошибочное использование типа в клиентском коде (например, полный набор функций для доступа к данным). Подобное разделение лучше всего реализуется путем предоставления доступа к структурам данных и другим внутренним подробностям исключительно через специально предназначенный для этого интерфейс.

В настоящей главе основное внимание фокусируется на относительно простых конкретных классах, которые с логической точки зрения слабо отличаются от встроенных типов. В идеале, такие типы должны отличаться от встроенных типов не в плане использования клиентским кодом, а только способами их создания.

## 10.2. Классы

*Класс* — этот тип, определяемый пользователем. В данном разделе рассматриваются основные средства определения классов, создания объектов классов и манипулирования объектами классов.

### 10.2.1. Функции-члены

Рассмотрим реализацию концепции даты с помощью *структуры* **Date**, содержащей данные, и набора функций для манипулирования переменными этого типа:

```
struct Date                                // представление
{
    int d, m, y;
};

void init_date (Date& d, int, int, int); // инициализация d
void add_year (Date& d, int n);          // прибавить n лет к d
void add_month (Date& d, int n);         // прибавить n месяцев к d
void add_day (Date& d, int n);           // прибавить n дней к d
```

Явной связи между структурой и набором функций нет. Такую связь можно выразить явно, объявив функции в качестве членов структуры:

```
struct Date
{
    int d, m, y;
    void init (int dd, int mm, int yy); // инициализация
    void add_year (int n);              // прибавить n лет
    void add_month (int n);             // прибавить n месяцев
    void add_day (int n);               // прибавить n дней
};
```



Функции, объявленные в теле определения класса (а *структура* это один из видов класса; §10.2.8), называются *функциями-членами* (*member functions*)<sup>1</sup> и могут вызываться только от имени переменных этого типа, используя стандартный синтаксис доступа к членам структуры. Например:

```
Date my_birthday;

void f()
{
    Date today;
    today.init(16, 10, 1996);
    my_birthday.init(30, 12, 1950);
    Date tomorrow = today;
    tomorrow.add_day(1);
    // ...
}
```

Так как различные структуры могут иметь функции-члены с одинаковыми именами, мы должны явно указывать имя структуры в определении функций-членов:

```
void Date::init(int dd, int mm, int yy)
{
    d = dd;
    m = mm;
    y = yy;
}
```

В теле функций-членов имена членов класса можно использовать без явных ссылок на объект класса. Это означает, что имена ссылаются на поля объекта, для которого функция-член вызвана. К примеру, если функция-член *Date::init()* вызвана для объекта *today*, то *m=mm* означает *today.m=mm*, а если эта функция-член вызвана для объекта *my\_birthday*, то *m=mm* означает *my\_birthday.m=mm*. В процессе своей работы функция-член всегда «знает», для какого объекта она вызвана.

Конструкция

```
class X { ... };
```

называется *определением класса* (*class definition*), так как с ее помощью определяется новый тип данных. По историческим причинам часто определение класса называют *объявлением класса* (*class declaration*). Как и объявления, не являющиеся определениями, определение класса может без нарушения правила одного определения (§9.2.3) присутствовать в разных исходных файлах программы, будучи помещенным в них директивой *#include*.

### 10.2.2. Управление режимом доступа

Объявление *Date* из предыдущего раздела предоставляет набор функций для манипулирования объектами этого типа. Однако оно не фиксирует того факта, что лишь эти функции знают точное устройство объектов типа и только через них осу-

<sup>1</sup> Постепенно, особенно в русскоязычной литературе, вместо термина «функция-член» все чаще применяется термин «метод класса» (англ. термин — *class method*). — *Прим. ред.*

существляется прямой доступ к этим объектам. Чтобы выразить указанное ограничение явным образом, применим ключевое слово **class** вместо **struct**:

```
class Date
{
    int d, m, y;

public:
    void init (int dd, int mm, int yyy) ;    // инициализация
    void add_year (int n) ;                  // прибавить n лет
    void add_month (int n) ;                 // прибавить n месяцев
    void add_day (int n) ;                   // прибавить n дней
};
```

Метка **public** делит тело класса на две части. Имена в первой части (*закрытой* — *private*) могут использоваться лишь функциями-членами. Вторая же часть — *открытая* (*public*) и она формирует открытый интерфейс к объектам класса. Структуры есть по сути те же классы, но с открытым режимом доступа по умолчанию (§10.2.8); функции-члены здесь определяются и используются одинаково. Например:

```
inline void Date::add_year (int n)
{
    y += n;
}
```

Обычные же функции (не функции-члены) теперь лишены возможности обращаться к закрытым полям класса напрямую. Например:

```
void timewarp (Date& d)
{
    d.y -= 200;                      // error: Date::y имеет режим private
}
```

Из ограничения доступа к внутренней структуре класса явно предназначенным для этого интерфейсом вытекает ряд преимуществ. Например, любая ошибка, приводящая к неверным данным в объекте типа **Date** (например, 36 декабря 1985 года), является следствием неверного кода в одной из функций-членов. Это означает, что локализация ошибки (первая стадия отладки) может быть выполнена на ранней стадии разработки программы (чуть ли не до ее запуска). Более общее положение состоит в том, что любое изменение поведения типа **Date** может и должно осуществляться изменением кода функций-членов. Например, любое усовершенствование внутренней структуры данных класса требует лишь изменения функций-членов с тем, чтобы воспользоваться новыми возможностями. Код же клиента, опирающегося на открытый интерфейс класса, остается при этом неизменным (может потребоваться его перекомпиляция). Еще одно достоинство состоит в том, что потенциальному пользователю класса для начала работы с новым типом данных нужно изучить лишь работу с открытым классовым интерфейсом.

Защита закрытых данных класса опирается на защиту имен членов класса. Такую защиту можно обойти с помощью манипуляции адресами и явного преобразования типов (то есть обманным путем). Защита C++ есть *защита от непреднамеренных ошибок*, а не от умышленного нарушения правил. Только аппаратная защита могла бы поставить надежный барьер на пути злонамеренных фокусов с универ-

сальным языком высокого уровня, что, однако, трудновыполнимо в реальных системах.

Из-за закрытия данных в классе *Date* мы вынуждены были ввести функцию *init()*, с помощью которой можно установить конкретное значение объекта класса.

### 10.2.3. Конструкторы

Использование функций типа *init()* для инициализации объектов класса и неэлегантно и чревато ошибками. Действительно, поскольку нигде прямо не указано, что объект класса нужно инициализировать таким образом, программист может забыть об этом или, наоборот, дважды вызвать функцию *init()* (часто с одинаковыми разрушительными последствиями). В итоге, лучше дать возможность программисту объявлять специальные функции-члены, явно предназначенные именно для инициализации объектов. Так как такие функции участвуют в создании (конструировании) объектов, их принято называть *конструкторами (constructors)*. Конструкторы выделяются тем, что *их имена совпадают с именем класса*. Например:

```
class Date
{
    // ...
    Date (int, int, int);           // конструктор
};
```

Если класс имеет конструкторы, то любой объект класса инициализируется вызовом конструктора (§10.4.3), причем если конструктор имеет аргументы, то ему их требуется передать:

```
Date today = Date (23, 6, 1983);
Date xmas (25, 12, 1990);         // сокращенная форма
Date my_birthday;                 // error: отсутствует инициализация
Date release1_0 (10, 12);         // error: отсутствует третий аргумент
```

Часто бывает полезным иметь несколько способов инициализации классовых объектов. Этого можно достичь, определив в классе несколько конструкторов. Например:

```
class Date
{
    int d, m, y;

public:
    // ...
    Date (int, int, int);           // день, месяц, год
    Date (int, int);               // день, месяц, текущий год
    Date (int);                   // день, текущие месяц и год
    Date ();                      // дата по умолчанию: сегодня
    Date (const char*);           // дата в строковом представлении
};
```

Конструкторы подчиняются тем же самым правилам перегрузки, что и обычные функции (§7.4). До тех пор, пока конструкторы существенно отличаются друг от друга набором аргументов, у компилятора есть возможность выбрать правильный вариант:

```

Date today (4) ;
Date july4 ("July 4, 1983") ;
Date guy ("5 Nov") ;
Date now;           // умолчательная инициализация текущей датой

```

Чрезмерное увеличение числа конструкторов, как в показанном примере класса **Date**, является типичным явлением. В процессе проектирования класса у программиста возникает искушение на всякий случай добавить любые средства просто потому, что они могут кому-нибудь понадобиться. Большие умственные усилия требуются, чтобы четко ограничить набор средств действительно необходимым минимумом. Эти дополнительные интеллектуальные усилия приводят в типичном случае к более ясным и более компактным программам. Одним из способов уменьшения количества необходимых функций является применение «аргументов по умолчанию» (§7.5). Для класса **Date** можно предусмотреть умолчательные значения аргументов, трактуемые как «сегодня» (*today*):

```

class Date
{
    int d, m, y;

public:
    Date (int dd = 0, int mm = 0, int yy = 0) ;
    // ...
};

Date::Date (int dd, int mm, int yy)
{
    d = dd?dd:today.d;
    m = mm?mm:today.m;
    y = yy?yy:today.y;
    // проверка допустимости даты
}

```

Для умолчательных значений аргументов нужно предусмотреть, чтобы они не совпадали с возможными «обычными» значениями аргументов. Для *дней* и *месяцев* это достигнуто с очевидностью (не существует нулевых дней и месяцев), но для *года* это также имеет место (хотя и не так очевидно), ибо в европейском календаре нет нулевого года; *первый год* от Рождества Христова следует сразу же за первым годом до Рождества Христова (*минус первый год*).

#### 10.2.4. Статические члены

Удобство умолчательных аргументов в классе **Date** достигнуто за счет следующей скрытой проблемы. Класс **Date** теперь зависит от глобальной переменной *today*. В результате безошибочное использование класса **Date** возможно лишь в контексте определения и корректного использования глобальной переменной *today* всеми частями программы. Такого рода ограничение делает класс бесполезным вне контекста, в котором он был изначально написан. Пользователи получают множество неприятных сюрпризов, когда они пытаются воспользоваться контекстно-зависимыми классами, а сопровождение программ становится слишком сложным. Может, «одна небольшая глобальная переменная» и не повредит, но вообще такой стиль программирования приводит к коду,

бесполезному для всех, кроме написавшего его программиста. Этого следует избегать.

К счастью, имеется возможность и удобства умолчательных значений сохранить, и глобальных переменных избежать. Переменная, являющаяся членом класса, но не частью объекта класса, называется *статическим членом* (*static member*). Она всегда существует в *единственном экземпляре*, в отличие от обычных членов класса, дублирующихся во всех классовых объектах (§С.9). Аналогично, функция, которой нужен доступ к членам класса, но не требуется ее вызов для конкретных объектов класса, называется *статической функцией-членом*.

Вот переработанный вариант класса **Date**, сохраняющий семантику умолчательного конструирования, но не опирающийся при этом на глобальные переменные:

```
class Date
{
    int d, m, y;
    static Date default_date;

public:
    Date (int dd=0, int mm=0, int yy=0) ;
    // ...
    static void set_default (int, int, int) ;
};
```

Теперь мы можем определить конструктор класса **Date** так, чтобы использовать **default\_date**:

```
Date::Date (int dd, int mm, int yy)
{
    d = dd ? dd: default_date.d;
    m = mm ? mm: default_date.m;
    y = yy ? yy: default_date.y;
    // проверка допустимости даты
}
```

Используя **set\_default()**, мы в любое время можем изменить умолчательное значение даты. К статическому члену можно обращаться так же, как и к любому другому. Дополнительно, к нему можно обращаться без указания классового объекта. Вместо этого следует квалифицировать его имя именем класса. Например:

```
void f()
{
    Date::set_default (4, 5, 19445) ;           // обращение к статической функции
                                                // set_default() класса Date
}
```

*Статические члены* — как данные, так и методы — *должны быть определены*. Ключевое слово **static** при этом повторять не надо. Вот пример:

```
Date Date::default_date (16, 12, 1770) ;      //определяем Date::default_date
void Date::set_default (int d, int m, int y)   //определяем Date::set_default()
{
    default_date = Date (d, m, y) ;           //присваиваем новое значение умолчательной дате
}
```

Теперь умолчательная дата совпадает с днем рождения Бетховена (пока кто-то не задаст иное умолчательное значение).

Заметим, что выражение *Date()* служит иным обозначением умолчательного значения *Date: :default\_date*. Например:

```
Date copy_of_default_date = Date() ;
```

Следовательно, нам не нужна отдельная функция для чтения умолчательной даты.

### 10.2.5. Копирование объектов класса

Возможность копирования классовых объектов доступна изначально. В частности, объект класса можно инициализировать копией другого объекта того же класса; это можно делать и в случае, когда определены конструкторы. Например:

```
Date d = today; // инициализация копированием
```

По умолчанию, копия объекта класса содержит копии всех полей объекта. Если это не то, что нужно, следует явно определить *копирующий конструктор* (*copy constructor*) вида *X: :X(const X&)* (подробно обсуждается далее в §10.4.4.1).

Аналогично, классовые объекты можно копировать с помощью *операции присваивания* (*assignment operator*). Например:

```
void f(Date& d)
{
    d = today;
}
```

И снова умолчательной семантикой операции является *почленное копирование*. Если это не подходит для некоторого класса *X*, то можно определить специфическую версию операции присваивания именно для этого класса (§10.4.4.1).

### 10.2.6. Константные функции-члены

В определенном нами классе *Date* имеются функции-члены, позволяющие задавать и изменять значения объектов этого класса. К сожалению, пока что отсутствует способ узнать эти значения. Проблема легко устраняется введением функций для чтения дня, месяца и года:

```
class Date
{
    int d, m, y;

public:
    int day() const {return d; }
    int month() const {return m; }
    int year() const;
    // ...
};
```

Обратите внимание на ключевое слово *const* после круглых скобок в объявлениях (определениях) функций. Оно означает, что эти *функции не изменяют состояния объектов класса Date (доступ по чтению)*. Естественно, что компиляторы обнаруживают случайные попытки нарушить это обещание. Например:

```
inline int Date::year() const
{
    return y++; // error: попытка изменить состояние объекта в константной функции
}
```

Когда константная функция-член определяется вне тела определения класса, требуется повторять ключевое слово **const**:

```
inline int Date::year() const // так правильно
{
    return y;
}

inline int Date::year() // а так ошибка: отсутствует ключевое слово const
{
    return y;
}
```

Другими словами, суффикс **const** является неотъемлемой частью типа **Date::day()**, **Date::month()** и **Date::year()**.

Константные функции-члены могут вызываться для **const**- и не **const**-объектов классов, в то время как неконстантные функции-члены — лишь для не **const**-объектов. Например:

```
void f(Date& d, const Date& cd)
{
    int i = d.year(); // ok
    d.add_year(1); // ok
    int j = cd.year(); // ok
    cd.add_year(1); // error: нельзя изменять значение константы cd
}
```

### 10.2.7. Ссылки на себя

Функции **add\_year()**, **add\_month()** и **add\_day()**, призванные изменять значения объектов типа **Date**, не имели возврата (указан как **void**). Для подобных логически связанных функций было бы желательно иметь возможность их последовательного использования в виде компактной цепочки вызовов. Например, хотелось бы иметь возможность писать что-то вроде

```
void f(Date& d)
{
    // ...
    d.add_day(1).add_month(1).add_year(1);
    // ...
}
```

чтобы в рамках единственного выражения изменить значения дня, месяца и года. Для этого каждая из функций *должна возвращать ссылку на mun Date*:

```
class Date
{
    // ...
    Date& add_year(int n); // прибавить n лет
}
```

```

Date& add_month (int n) ;    // прибавить n месяцев
Date& add_day (int n) ;      // прибавить n дней
};

```

Каждая нестатическая функция класса знает объект, для которого она вызывает-ся, и может ссылаться на него явным образом:

```

Date& Date::add_year (int n)
{
    if (d==29 && m==2 && !leapyear (y+n) )    // не забудьте о February 29
    {
        d = 1;
        m = 3;
    }

    y += n;
    return *this;
}

```

Выражение *\*this* как раз и означает такой объект. В языке Simula это же обозначается как **THIS**, а в языке Smalltalk — как **self**.

В нестатической функции-члене ключевое слово **this** означает *указатель на объект, для которого функция вызвана*. В неконстантной функции-члене класса *X* тип **this** есть *X\**. Однако ж, **this** не является обычной переменной — невозможно взять ее адрес и нельзя ей ничего присвоить. В константной функции-члене класса *X* тип **this** есть **const X\*** для предотвращения изменения самого объекта (см. также §5.4.1).

По большей части **this** используется неявным образом. В частности, каждое обращение к нестатическому члену внутри класса опирается на неявное применение **this** для доступа к полям классического объекта. Например, можно написать следующее эквивалентное (но утомительное) определение функции **add\_year()**:

```

Date& Date::add_year (int n)
{
    // не забудьте о February 29
    if (this->d==29 && this->m==2 && !leapyear (this->y+n) )
    {
        this->d = 1;
        this->m = 3;
    }

    this->y += n;
    return *this;
}

```

В операциях со связными списками **this** применяется явным образом (§24.3.7.4).

### 10.2.7.1. Физическое и логическое постоянство

Иногда так бывает, что *константной* по логике функции-члену все-таки требуется изменить значение поля данных класса. С точки зрения пользователя функция ничего в состоянии объекта не меняет. А на самом деле она изменяет отдельные детали, пользователю недоступные. Такое явление называется *логическим постоянством* (*logical constness*). Например, класс **Date** мог бы возвращать строковое представление даты, которое пользователь использовал бы для вывода. Конструирование та-



кого представления может оказаться довольно дорогой (затратной) операцией. Поэтому имеет смысл хранить актуальную копию строки с тем, чтобы при последовательных запросах возвращать эту копию в случаях, когда дата не изменялась с момента последнего обращения. *Кэширование* (*caching*) данных чаще применяется для более сложных структур, но мы сейчас рассмотрим как все это работает на примере *Date*:

```
class Date
{
    bool cache_valid;
    string cache;
    void compute_cache_value () ;           // заполнить кэш
    // ...

public:
    // ...
    string string_rep () const;             // строковое представление
};
```

С точки зрения пользователя, вызов *string\_rep()* не меняет состояния *Date*, так что естественно, что это константная функция-член класса *Date*. С другой стороны, любой кэш нужно хотя бы один раз проинициализировать до того, как он будет использоваться на чтение. Достигнуть этого можно, например, следующим образом (применяя грубую силу — *brute force*):

```
string Date::string_rep () const
{
    if (cache_valid == false)
    {
        Date* th = const_cast<Date*> (this) ;    // снимаем const
        th->compute_cach_value () ;
        th->cache_valid = true;
    }
    return cache;
}
```

Мы здесь применяем операцию *const\_cast* (§15.4.2.1) для того, чтобы вместо *this* получить указатель типа *Date\**. Это не только не элегантно, но может и не работать, когда объект класса изначально объявлен константой. Например:

```
Date d1;
const Date d2;
string s1 = d1.string_rep () ;
string s2 = d2.string_rep () ;           // неопределенное поведение
```

Для объекта *d1* все прекрасно работает. Но объект *d2* объявлен константой и конкретная реализация C++ может включить дополнительные защитные механизмы, гарантирующие неизменность объекта. Из-за этого нельзя гарантировать, что *d2.string\_rep()* во всех реализациях будет иметь одинаковое и предсказуемое поведение.

### 10.2.7.2. Ключевое слово **mutable**

Можно избежать операции приведения **const\_cast** и последующей зависимости от деталей реализации C++, если пометить кэшируемые данные класса **Date** ключевым словом **mutable**:

```
class Date
{
    mutable bool cache_valid;
    mutable string cache;
    void compute_cache_value () const;    // заполнить (mutable) кэш
    // ...

public:
    // ...
    string string_rep () const;           // строковое представление
};
```

Ключевое слово **mutable** требует обеспечить такое хранение помеченного им поля данных, чтобы это поле можно было гарантированным образом модифицировать, даже для объектов, объявленных *константами*. Иными словами, **mutable** означает «никогда не может быть **const**». В результате упрощается определение функции **string\_rep()**:

```
string Date::string_rep () const
{
    if (!cache_valid)
    {
        compute_cache_value ();
        cache_valid = true;
    }
    return cache;
}
```

и становятся действительными все случаи ее разумного применения. Например:

```
Date d3;
const Date d4;
string s3 = d3.string_rep ();
string s4 = d4.string_rep ();    // ok!
```

Объявление части полей данных с модификатором **mutable** приемлемо в случаях, когда это малая часть общего числа полей данных класса. Если же у логически *константного* объекта большая часть полей подвержена изменению, будет лучше все изменяющиеся поля переместить в отдельный объект с косвенным доступом. В этой технике наш пример с кэшируемой строкой принимает следующий вид:

```
struct cache
{
    bool valid;
    string rep;
```

```

class Date
{
    cache* c;                                // инициализируется конструктором (§10.4.6)
    void compute_cache_value() const;        // заполнить кэш
    // ...

public:
    // ...
    string string_rep() const;                // строковое представление
};

string Date::string_rep() const
{
    if(!c->valid)
    {
        compute_cache_value();
        c->valid = true;
    }
    return c->rep;
}

```

Различные обобщения техник кэширования приводят к тем или иным формам так называемых *ленивых вычислений* (*lazy evaluation*).

### 10.2.8. Структуры и классы

По определению *структура* есть класс с открытыми по умолчанию членами, так что

```
struct s { . . .
```

есть просто сокращенная форма записи для

```
class s {public: . . .
```

Спецификатор доступа **private** говорит, что все последующие за ним члены имеют закрытый режим доступа, точно так же, как **public** сообщает об открытом режиме доступа. За исключением различий в именах следующие объявления эквивалентны:

```

class Date1
{
    int d, m, y;

public:
    Date1(int dd, int mm, int yy);
    void add_year(int n);           // прибавить n лет
};

struct Date2
{
private:
    int d, m, y;

public:
    Date2(int dd, int mm, int yy);
    void add_year(int n);           // прибавить n лет
}

```

Какой стиль объявления использовать, зависит от вкуса и обстоятельств. Я обычно предпочитаю использовать ***struct*** для классов данных с открытыми членами. О таких типах я думаю как о «не совсем классах, просто наборах данных». Конструкторы и функции доступа полезны и в этих случаях, просто для удобства, а не как фундаментальные элементы типов (§24.3.7.1).

Вовсе не обязательно объявлять данные первыми. Часто полезно первыми поместить функции, чтобы подчеркнуть особую важность открытого интерфейса пользователя. Например:

```
class Date3
{
public:
    Date3 (int dd, int mm, int yy) ;
    void add_year (int n) ;           // прибавить n лет

private:
    int d, m, y;
};
```

В реальном коде, где и данные, и открытый интерфейс классов весьма велики по сравнению с учебным кодом, я предпочитаю стиль, использованный только что для объявления ***Date3***.

Спецификаторы доступа могут встречаться в объявлениях классов много раз. Например:

```
class Date4
{
public:
    Date4 (int dd, int mm, int yy) ;

private:
    int d, m, y;

public:
    void add_year (int n) ;           // прибавить n лет
};
```

Множественные открытые и множественные закрытые секции класса (как в ***Date4***) могут запутать программиста, но они нужны, например, для автоматической генерации кода.

### 10.2.9. Определение функций в теле определения класса

Функция-член, *определенная*, а не только объявленная в теле определения класса, по умолчанию *предполагается встраиваемой*. Ясно, что это полезно для небольших и часто применяемых функций. Как и определение класса, частью которого такие функции являются, они могут директивами ***#include*** включаться во многие единицы трансляции. Как и для класса в целом, их определение должно быть всюду одинаковым (§9.2.3).

Стиль, в котором поля данных класса помещаются в конец тела определения класса, вызывает небольшие проблемы в связи со встраиваемыми функциями-членами, ссылающимися на поля данных. Рассмотрим пример:

```

class Date
{
public:
    int day() const {return d;}           // возвращаем Date::d
    // ...
private:
    int d, m, y;
};

```

Это абсолютно правильный код на C++, потому что функция-член класса может обращаться к любому члену класса так, как будто весь класс полностью определен до определения тела функции. Это, однако, может смутить человека, читающего программу.

Поэтому я либо помещаю поля данных в начало определения класса, либо полагаю определения встраиваемых функций-членов после определения класса:

```

class Date
{
public:
    int day() const;
    // ...
private:
    int d, m, y;
};

inline int Date::day() const {return d;}

```

## 10.3. Эффективные пользовательские типы

В предыдущем разделе отдельные части класса *Date* рассматривались в связи с изучением основных средств C++, предназначенных для определения классов. В настоящем разделе я переворачиваю акценты и рассматриваю в первую очередь простой и эффективный класс *Date* и показываю, как средства C++ поддерживают дизайн таких типов данных.

Небольшие, интенсивно используемые абстракции весьма типичны для большинства приложений: латинские буквы, китайские иероглифы, целые числа и числа с плавающей запятой, точки, указатели, координаты, преобразования, пары (указатель, смещение), даты, время, диапазоны, связи, ассоциации, узлы, пары (значение, единица измерения), местоположения на диске, расположение исходного кода, валюты, строки, прямоугольники, масштабируемые числа с фиксированной запятой, обыкновенные дроби, символьные строки, вектора и массивы. Каждое приложение использует хоть что-то из перечисленного набора. Часто некоторые из этих типов используются весьма интенсивно. Типичное приложение использует часть типов непосредственно, а часть — косвенно, через библиотеки.

Язык C++, как и другие языки программирования, непосредственно поддерживает лишь часть из перечисленных абстракций. Большинство же из подобного рода абстракций непосредственно не поддерживается ввиду их чрезвычайной многочисленности. Более того, разработчик универсального языка высокого уровня и не может в деталях представить себе все конкретные нужды каждого приложения. Вместо

этого, пользователю предоставляются стандартные механизмы для определения таких небольших, конкретных типов данных. Эти типы и принято называть конкретными типами или конкретными классами, чтобы противопоставить их абстрактным классам (§12.3) и классам из иерархий наследования (§12.2.4, §12.4).

При разработке языка C++ учитывалась необходимость предоставить средства определения и эффективного использования пользовательских типов данных. Это фундамент эффективного программирования. Ведь, как следует из статистики, простые и приземленные средства намного важнее сложных и изощренных.

В свете изложенного, давайте определим более совершенный вариант класса **Date**:

```
class Date
{
public:
    // открытый интерфейс:
    enum Month {jan=1, feb, mar, apr, may, jun, ju, aug, sep, oct, nov, dec};
    class Bad_date { }; // класс исключений
    Date (int dd = 0, Month mm = Month (0), int yy = 0); // 0 - "взять умолчат. значение"
    //Функции доступа к дате:
    int day () const;
    Month month () const;
    int year () const;
    string string_rep () const; // string представление
    void char_rep (char s[]) const; // представление C-строкой
    static void set_default (int, Month, int);
    //Функции для изменения даты:
    Date& add_year (int n); // прибавить n лет
    Date& add_month (int n); // прибавить n лет
    Date& add_day (int n); // прибавить n лет
private:
    int d, m, y; // представление
    static Date default_date;
};
```

Набор операций класса **Date** весьма типичен для любого пользовательского типа:

1. Конструктор, определяющий, как инициализируются объекты (переменные) типа.
2. Набор функций доступа. Эти функции снабжены модификатором **const**, который указывает, что функции не изменяют состояния объектов, для которых они вызваны.
3. Набор функций, позволяющий легко манипулировать датами без необходимости разбираться в деталях их устройства или осмысливать запутанную семантику типа.
4. Набор неявно определенных операций для свободного копирования объектов.
5. Класс **Bad\_date**, используемый в исключениях с сообщениями об ошибках.

Я определил отдельный тип *Month*, чтобы он имел дело с месяцами и помогал не путаться при записи, например, 7 июня, по американскому стилю в виде *Date* (6, 7), или по европейскому стилю в виде *Date* (7, 6).

Я также рассматривал возможность определения отдельных типов *Day* и *Year*, чтобы избежать путаницы между вариантами *Date* (1995, jul, 27) и *Date* (27, jul, 1995). Однако эти типы не столь полезны, как *Month*. Почти все ошибки такого рода выявляются в процессе выполнения — 26 июля 27 года не часто встречается в моей повседневной практике. Как обрабатывать даты до 1800 года или около того, вопрос довольно тонкий, и его лучше оставить историкам. Кроме всего, число месяца нельзя проверить в отрыве от самого месяца и даже года. В §11.7.1 приводится определение удобного в использовании типа *Year*.

Где-то нужно задать корректную дату по умолчанию. Например:

```
Date Date : : default_date (22, jan, 1901) ;
```

Я здесь изъясняю технику кэширования, рассмотренную в §10.2.7.1, ибо в столь простом типе в ней нет необходимости. В то же время, ее всегда можно внедрить обратно как деталь реализации, не влияющую на открытый интерфейс пользователя.

Вот небольшой пример того, как тип *Date* может использоваться:

```
void f(Date& d)
{
    Date lwb_day = Date (16, Date : : dec, d.year ()) ;
    if(d.day () == 29 && d.month () == Date : : feb)
    {
        // ...
    }

    if(midnight ()) d.add_day (1) ;
    cout<< "day after: " << d+1 << '\n' ;
}
```

Здесь предположено, что для типа *Date* определены операции << и +. Об этом рассказано в §10.3.3.

Обратите внимание на запись *Date* : : *feb*. Дело в том, что функция *f*() не является членом *Date*, и поэтому в ней нужно явно указывать, что речь идет о *feb* из *Date*, а не о какой-либо иной сущности.

Зачем нужно определять отдельный тип данных для такого простого понятия, как дата? Можно было бы ограничиться следующей структурой

```
struct Date
{
    int day, month, year;
};
```

и позволить программистам решать, что с ней делать. Однако если бы мы поступили таким образом, каждый пользователь должен был бы манипулировать полями *Date* непосредственно, или определить специальные функции для выполнения этих действий. В результате, понятие даты было бы «размазано» по всей системе; его было бы сложно понимать, документировать и модифицировать. Реализация концепции в виде простой структуры с неизбежностью приводит к дополнительной работе со стороны каждого пользователя структуры.

Несмотря на то, что класс *Date* и выглядит столь простым, все же нужно потрудиться, чтобы все работало как надо. Например, инкрементирование дат должно учитывать високосность года, что количество дней в месяцах разное и т.д. (§10.6[1]). Также, для многих приложений представление дат в виде «день-месяц-год» может оказаться недостаточным. Если мы решим изменить представление, то нам потребуется изменить лишь ограниченный набор функций. Например, чтобы представить *Date* в виде количества дней до или после 1 января 1970 года, мы должны будем изменить лишь функции-члены класса *Date* (§10.6[2]).

### 10.3.1. Функции-члены

Естественно, требуется выполнить и реализацию функций-членов класса *Date*. Вот пример реализации конструктора класса *Date*:

```
Date : : Date (int dd, Month mm, int yy)
{
    if (yy == 0) yy = default_date.year ();
    if (mm == 0) mm = default_date.month ();
    if (dd == 0) dd = default_date.day ();

    int max;

    switch (mm)
    {
        case feb:
            max = 28 + leapyear (yy);
            break;
        case apr: case jun: case sep: case nov:
            max = 30;
            break;
        case jan: case mar: case may: case jul: case aug: case oct: case dec:
            max = 31;
            break;
        default:
            throw Bad_date ();           // кто-то шутит или напутал
    }

    if (dd < 1 || max < dd) throw Bad_date ();

    y = yy;
    m = mm;
    d = dd;
}
```

Конструктор проверяет, является ли полученная им дата допустимой. Если нет, как например в случае *Date* (30, *Date* : : feb, 1994), он генерирует исключение (§8.3, глава 14), свидетельствующее о том, что что-то пошло не так и это лучше не игнорировать. Если же представленные на вход конструктору данные корректны, то производится очевидная инициализация. В целом, инициализация получилась относительно сложной как раз из-за проверки допустимости даты. Это достаточно типичный случай. С другой стороны, после того, как дата создана, ее можно использовать и копировать без дополнительных проверок. Другими словами, конструктор устанавливает инвариант класса (в данном случае это корректность даты). Остальные функ-



ции-члены могут полагаться на этот инвариант и должны поддерживать его. Такая техника проектирования упрощает код в заметной степени (см. §24.3.7.1).

Для выбора месяца по умолчанию я использую выражение *Month(0)*, не представляющее реального месяца. Я мог бы специально для этой цели предусмотреть элемент в перечислении *Month*. Но я решил, что лучше уж использовать явно аномальное значение, чем представить дело так, что в году 13 месяцев. Обратите внимание, что нулем можно пользоваться потому, что он гарантированно находится внутри диапазона значений перечисления *Month* (§4.8).

Я думал над тем, чтобы вынести проверку корректности дат в отдельную функцию *is\_date()*. Но затем я понял, что при этом пользовательский код станет более сложным и менее надежным, чем в случае опоры на исключения, как в следующем примере, предполагающем, что операция *>>* определена для типа *Date*:

```
void fill(vector<Date>& a)
{
    while(cin)
    {
        Date d;
        try
        {
            cin >> d;
        }
        catch(Date : : Bad_date)    // обработка ошибки
        {
            continue;
        }

        aa.push_back(d);           // см. §3.7.3
    }
}
```

Что вообще типично для простых конкретных типов, определения функций-членов характеризуются диапазоном оценок от «тривиально» до «не слишком сложно». Например:

```
inline int Date : : day() const
{
    return d;
}

Date& Date : : add_month(int n)
{
    if(n==0) return *this;
    if(n>0)
    {
        int delta_y = n / 12;
        int mm = m+n%12;
        if(12 < mm)    // обратите внимание: int(dec)==12
        {
            delta_y++;
            mm -= 12;
        }
    }
}
```

```

    // работа со случаями, когда день d не существует для Month(mm):
    y += delta_y;
    m = Month(mm) ;
    return *this;
}

// отрицательное n:
return *this;
}

```

### 10.3.2. Функции поддержки (helper functions)

В типичном случае с классом логически связан набор вспомогательных *функций поддержки (helper functions)*, определять которые внутри класса нет необходимости, так как они не нуждаются в прямом доступе к внутреннему представлению класса. Например:

```

int diff(Date a, Date b);      // количество дней между a и b
bool leapyear(int y);
Date next_weekday(Date d);
Date next_saturday(Date d);

```

Определение таких функций в качестве функций-членов лишь усложнило бы интерфейс класса и потребовало бы большей работы по просмотру всех функций, подлежащих модификации при изменении внутреннего представления класса.

Но как такие функции следует «связывать» с классом? Традиционно, их объявления просто помещают в тот же самый файл, где объявляется сам класс, так что пользователям они становятся доступными с помощью той же самой директивы **#include**, с помощью которой они включают определение интерфейса класса (§9.2.1). Например:

```
#include "Date.h"
```

Вместо использования заголовочного файла **Date.h** — или в качестве альтернативы — мы могли бы установить связь явно, поместив объявления функций поддержки в одно пространство имен с объявлением класса (§8.2):

```

namespace Chrono                // средства для работы со временем
{
    class Date { /* ... */ };

    int diff(Date a, Date b);
    bool leapyear(int y);
    Date next_weekday(Date d);
    Date next_saturday(Date d);
    // ...
}

```

В типичных случаях пространство имен **Chrono** содержало бы еще и другие логически связанные классы, например, **Time** и **Stopwatch**, а также их собственные функции поддержки. Применение пространства имен лишь для одного класса является избыточным и ведет к напрасному переусложнению.

### 10.3.3. Перегруженные операции

Полезно добавить к классу функции, обеспечивающие возможность пользовательскому коду применять привычные формы записи. Например, функция **operator==**( ) определяет, как *операция ==* (операция сравнения на равенство) работает для объектов класса **Date**:

```
inline bool operator==(Date a, Date b)    // проверка на равенство
{
    return a.day() == b.day() && a.month() == b.month() && a.year() == b.year();
}
```

Другими очевидными кандидатами являются:

```
bool operator!=(Date, Date);           // не равно
bool operator<(Date, Date);           // меньше чем
bool operator>(Date, Date);           // больше чем
// ...
Date& operator++(Date& d);            // увеличить Date на один день
Date& operator--(Date& d);            // уменьшить Date на один день

Date& operator+=(Date& d, int n);      // прибавить n дней
Date& operator-=(Date& d, int n);      // вычесть n дней

Date operator+(Date d, int n);         // прибавить n дней
Date operator-(Date d, int n);         // вычесть n дней

ostream& operator<<(ostream&, Date d); // вывести d
istream& operator>>(istream&, Date& d); // считать d
```

Для класса **Date** перегрузка этих операций совершается исключительно ради дополнительного удобства. Но для таких типов, как комплексные числа (§11.3), вектора (§3.7.1) и классы функторов (функциональных объектов или объектов-функций) (§18.4) — стандартные операции настолько проникли в сознание, что их перегрузка является просто обязательной. Перегрузка операций подробно рассматривается в главе 11.

### 10.3.4. Роль конкретных классов

Я называю простые типы, определяемые пользователем, такие как **Date**, *конкретными типами* (*concrete types*), чтобы противопоставить их *абстрактным классам* (*abstract classes*) (§2.5.4) и классовым иерархиям (§12.3), а также подчеркнуть их сходство со встроенными типами, такими как **int** или **char**. Конкретные типы называют также *типами-значениями* (*value types*), а их применение в программе — программированием, ориентированным на значения (*value-oriented programming*). Модель их применения и философия, стоящая за проектированием таких типов, сильно отличаются от того, что принято называть объектно-ориентированным программированием (§2.6.2).

Конкретные типы предназначены для того, чтобы хорошо и эффективно выполнять отдельную небольшую работу. Как правило, пользователю не предоставляют возможностей для модификации поведения конкретных типов. В частности, конкретные типы не предназначены для демонстрации полиморфного поведения (см. §2.5.5, §12.2.6).

Если вам не подходят отдельные черты конкретного типа, вы строите новый тип с желаемым поведением. При этом вы можете повторно применить старый конкретный тип (*reuse a concrete type*) в качестве строительного кирпичика нового типа точно так же, как и встроенный тип, такой как *int*. Например:

```
class Date_and_time
{
private:
    Date d;
    Time t;

public:
    Date_and_time (Date d, Time t) ;
    Date_and_time (int d, Date::Month m, int y, Time t) ;
    // ...
};
```

Механизм наследования классов, обсуждаемый в главе 12, может быть применен с целью формулирования различий (и сходства) нового типа и заданного конкретного типа. Определение типа *Vec* из типа *vector* (§3.7.2) служит соответствующим примером.

При наличии качественного компилятора конкретные типы вроде *Date* не вносят дополнительных накладных расходов ни в плане скорости выполнения, ни в плане объема занимаемой памяти. Размер конкретного типа известен во время компиляции, так что объекты этих типов можно создавать в стеке (то есть без операций размещения в свободной памяти). Раскладка объектов этих типов известна во время компиляции, так что не составляет труда реализовывать встраиваемые операции. Аналогично, совместимость с языками типа C или Fortran достигается без дополнительных усилий.

Хороший набор конкретных типов может составить надежный фундамент приложения. Отсутствие маленьких эффективных типов ведет к перерасходу компьютерной памяти и вычислительных мощностей из-за применения слишком общих и громоздких классов. Кроме того, отсутствие конкретных типов приводит к снижению эффективности труда программиста, поскольку каждый программист вынужден в очередной раз писать свой собственный код для манипулирования «простыми и широко применяемыми» структурами данных.

## 10.4. Объекты

Объекты можно создавать разными способами. Некоторые объекты являются локальными переменными, некоторые — глобальными переменными, другие входят в состав более крупных объектов иных классов (являются членами классов) и т.д. В настоящем разделе обсуждаются перечисленные альтернативы, правила, которыми при этом руководствуются, как конструкторы используются для инициализации объектов, и как деструкторы очищают выделенные под объекты ресурсы перед тем, как объекты станут недоступными.

### 10.4.1. Деструкторы

Конструкторы предназначены для инициализации объектов. Можно сказать, они создают среду, в которой затем выполняются функции-члены. Иногда создание такой среды включает выделение таких ресурсов, как файлы, блокировки или память, и которые должны быть освобождены после использования объектов класса (§14.4.7). Таким образом, эти классы нуждаются в функции, которая гарантированно вызывается при уничтожении объектов аналогично тому, как конструктор гарантированно вызывается при их создании. Такие функции принято называть *деструкторами* (*destructors*). Их типичное поведение заключается в очистке и/или освобождении ресурсов. Деструкторы автоматически вызываются, когда локальные переменные выходят из области видимости, или когда явным образом уничтожаются объекты, динамически созданные в свободной памяти. Только в крайне необычных ситуациях от пользователя требуется вызывать деструкторы явно (§10.4.11).

Чаще всего деструкторы освобождают память, динамически выделенную конструкторами. Рассмотрим простую таблицу элементов некоторого типа *Name*. Конструктор класса *Table* должен выделить память под элементы таблицы. Когда таблицу тем или иным способом уничтожают, мы должны быть уверены в том, что ранее выделенная в конструкторе память возвращена системе с целью ее дальнейшего повторного использования. Это можно сделать, предоставив специальную, комплиментарную к конструктору функцию:

```
class Name
{
    const char* s;
    // ...
};

class Table
{
    Name* p;
    size_t sz;

public:
    Table(size_t s = 15) {p = new Name[sz = s];} // конструктор
    ~Table() {delete[] p;} // деструктор

    Name* lookup(const char*);
    bool insert(Name*);
};
```

Тильда в имени деструктора *~Table()* является знаком операции дополнения, что намекает на характер этой функции, дополняющей классовой конструктор *Table()*.

Комплиментарные пары конструктор/деструктор служат обычным для языка C++ механизмом реализации объектов переменного размера. Контейнеры стандартной библиотеки, такие как *map*, применяют некоторые вариации этой техники для управления выделением памяти под элементы, так что последующее обсуждение иллюстрирует технику, которой вы фактически пользуетесь всегда, когда используете стандартные контейнеры (и стандартный тип *string* в том числе). Это обсуждение в равной степени распространяется и на типы без явно запрограммиро-

ванных деструкторов, ибо в этом случае можно предполагать, что деструктор просто ничего не делает.

### 10.4.2. Конструкторы по умолчанию

Аналогично, можно считать, что большинство типов имеют конструктор по умолчанию. Конструктор по умолчанию можно вызывать, не предоставляя никаких аргументов. В вышеприведенном примере из-за наличия умолчательного значения 15 для аргумента конструктора **Table**: **Table (size\_t)**, его можно считать конструктором по умолчанию. Если пользователь явным образом определяет в классе конструктор по умолчанию, именно этот вариант и задействуется; в противном случае компилятор пытается сгенерировать свой вариант умолчательного конструктора, но только в случаях, когда отсутствуют явно определенные в классе конструкторы иных типов. Сгенерированный компилятором конструктор по умолчанию пытается неявно вызывать умолчательные конструкторы для членов класса (также имеющими тип классов) и конструкторы базовых классов (§12.2.2). Например:

```
struct Tables
{
    int i;
    int vi[10];
    Table t1;
    Table vt[10];
};

Tables tt;
```

Здесь **tt** будет инициализироваться сгенерированным конструктором по умолчанию, который вызовет **Table (15)** для **tt.t1** и каждого элемента **tt.vt**. С другой стороны, **tt.i** и элементы **tt.vi** не относятся к классовым типам и, соответственно, не инициализируются. Причины такого расхождения заключаются в необходимости обратной совместимости с языком C и опасении излишних накладных расходов для встроенных типов данных.

Из-за того, что *константы* и ссылки обязаны инициализироваться (§5.5, §5.4), класс, содержащий члены, являющиеся константами или ссылками, не может конструироваться по умолчанию, если только программист не предоставит явным образом соответствующий конструктор (§10.4.6.1). Например:

```
struct X
{
    const int a;
    const int& r;
};

X x;           // error: нет умолчательного конструктора для X
```

Конструкторы по умолчанию можно вызвать явно (§10.4.10). Встроенные типы также обладают конструкторами по умолчанию (§6.2.8).

### 10.4.3. Конструирование и уничтожение объектов

Рассмотрим различные способы создания и последующего уничтожения объектов классов. Объект может быть создан в качестве:

- §10.4.4. Именованного автоматического объекта, создаваемого каждый раз при проходе потока управления работающей программы через его определение, и уничтожаемого по выходу из блока, содержащего указанное определение.
- §10.4.5. Объекта в свободной памяти, создаваемого операцией *new* и уничтожаемого операцией *delete*.
- §10.4.6. Нестатического члена классowego типа, создаваемого в виде подобъекта в момент создания объемлющего объекта иного типа, и уничтожаемого в момент уничтожения указанного объемлющего объекта.
- §10.4.7. Элемента массива, создаваемого и уничтожаемого в моменты создания и уничтожения самого массива.
- §10.4.8. Локального статического объекта, создаваемого в момент первого прохождения потока управления работающей программы через его определение, и уничтожаемого в момент окончания работы программы.
- §10.4.9. Глобального объекта, объекта из пространства имен или статического объекта класса, создаваемого при старте программы и уничтожаемого при завершении работы программы.
- §10.4.10. Временного объекта, создаваемого в процессе вычисления выражения, и уничтожаемого после окончания вычисления полного выражения.
- §10.4.11. Объекта, помещаемого в область памяти, предоставляемую функцией пользователя, с учетом параметров, передаваемых конструктору операцией размещения (*placement new*).
- §10.4.12. Члена объединения (*union*), который не может иметь ни конструктора, ни деструктора.

Этот список приблизительно отсортирован в порядке важности. В следующих подразделах подробно рассматриваются все перечисленные способы создания объектов и их использование.

### 10.4.4. Локальные объекты

Конструктор локального объекта выполняется каждый раз, когда поток управления работающей программы проходит через его определение. Деструктор же вызывается при каждом выходе из блока, содержащего указанное объявление. Деструкторы локальных объектов выполняются в порядке, обратном выполнению их конструкторов. Например:

```
void f(int i)
{
    Table aa;
    Table bb;
    if (i > 0)
    {
        Table cc;
```

```

    // ...
}

Table dd;
// ...
}

```

Здесь *aa*, *bb* и *dd* конструируются (в указанном порядке) каждый раз при вызове функции *f()*, и *dd*, *bb* и *aa* уничтожаются (в указанном порядке) при каждом выходе из *f()*. Если же в конкретном вызове *i>0*, *ss* конструируется после *bb*, и уничтожается перед конструированием *dd*.

#### 10.4.4.1. Копирование объектов

Если *t1* и *t2* являются объектами класса *Table*, то присваивание *t2=t1* по умолчанию означает почленное (побитовое) копирование *t1* в *t2* (§10.2.5). Такое поведение операции присваивания может вызывать проблемы для объектов классов, содержащих указатели в качестве членов. Почленное копирование не соответствует семантике копирования объектов классов, управляющих ресурсами с помощью пары конструктор/деструктор. Например:

```

void h ()
{
    Table t1;
    Table t2 = t1;                                // копирующая инициализация: проблема

    Table t3;
    t3 = t2;                                       // копирующее присваивание: проблема
}

```

Умолчательный конструктор класса *Table* вызывается дважды — при конструировании объектов *t1* и *t3*; для объекта *t2* он не вызывается, так как тут работает копирование (из *t1* в *t2*). В то же время деструктор класса *Table* вызывается трижды — при уничтожении объектов *t1*, *t2* и *t3*! Согласно умолчательной трактовке операции присваивания перед выходом из функции *h()* каждый из объектов *t1*, *t2* и *t3* будет содержать указатель на массив имен, память под который была динамически выделена в свободной памяти в момент создания *t1*. Указатель же на массив имен, выделенный в свободной памяти при создании *t3*, не сохранился из-за того, что была выполнена операция *t3=t2* с ее почленным копированием. Ввиду отсутствия автоматической сборки мусора (*garbage collection*; §10.4.5), эта память потеряна для программы навсегда. С другой стороны, массив, выделенный для *t1*, теперь адресуется также из *t2*, *t3*, и в итоге он будет удаляться трижды. Результирующий эффект стандартом не определен, но почти наверняка будет катастрофическим.

Подобного рода аномалий можно избежать, если явно определить, что же нужно понимать под присваиванием объектов класса *Table*:

```

class Table
{
    // ...как в §10.4/1...
    Table (const Table& ) ;                        // копирующий конструктор
    Table& operator= (const Table& ) ;           // присваивание
}

```



Программист волен определить любое приемлемое поведение для операции присваивания, однако традиционным является копирование содержимого контейнера (или создание иллюзии у пользователя о таком копировании; см. §11.12). На пример:

```

Table : Table (const Table & t)           // копирующий конструктор
{
    p = new Name [sz=t.sz] ;
    for (int i=0; i<sz; i++) p [i] = t.p [i] ;
}

Table & Table : operator= (const Table & t)  // присваивание
{
    if (this != &t)    // не забудьте о возможности самоприсваивания: t = t
    {
        delete [] p;
        p = new Name [sz=t.sz] ;
        for (int i=0; i<sz; i++) p [i] = t.p [i] ;
    }
    return *this ;
}

```

Копирующий конструктор (*copy constructor*) и операция присваивания (*assignment*) различаются в следующем принципиальном моменте: копирующий конструктор инициализирует «сырую» (*raw* — ранее не инициализированную) память, в то время как операция присваивания работает над участком памяти, содержащем корректно сконструированный классовый объект.

В ряде случаев допускаются разные схемы оптимизации, но основная стратегия работы операции присваивания проста: защита от присваивания самому себе, удаление старых элементов, инициализация и копирование новых элементов. Как правило, нужно скопировать все нестатические члены (§10.4.6.3). Для сообщения об ошибках копирования можно использовать исключения (§14.4.6.2). По поводу техники написания операций копирования, оставляющих левый операнд в корректном состоянии при возникновении исключений (*exception-safe copy operations*), см. §E.3.3.

### 10.4.5. Динамическое создание объектов в свободной памяти

Конструктор для объекта, создаваемого в свободной памяти, вызывается по операции **new**. Объект в свободной памяти существует до тех пор, пока не будет вызвана операция **delete** с операндом, являющимся указателем на этот объект. Рассмотрим следующий пример:

```

int main ()
{
    Table* p = new Table;
    Table* q = new Table;

    delete p;
    delete p;    // вероятно, приведет к ошибке во время выполнения
}

```

Конструктор `Table::Table()` вызывается дважды, как и деструктор `Table::~~Table()`. К сожалению, операции `new` и `delete` не точно соответствуют друг другу в данном примере: объект, адресуемый указателем `p`, удаляется дважды, а объект, адресуемый указателем `q` — ни разу. То, что объект не удаляется операцией `delete`, не является ошибкой с формальной точки зрения языка C++; это лишь напрасный перерасход памяти (образно говорят об *утечке памяти* — *memory leak*). Но для программы, предлагаемой к длительному непрерывному использованию, это сильно вредит производительности и может трактоваться как откровенный брак в ее реализации. Существуют средства для помощи в нахождении ошибок, приводящих к утечкам памяти. Двойное освобождение памяти является серьезной операционной ошибкой с непредсказуемыми результатами, чаще всего катастрофическими.

Некоторые реализации C++ могут предоставлять услуги автоматической сборки мусора, сканируя объекты в свободной памяти с целью нахождения неиспользуемых объектов. Поведение сборщиков мусора не стандартизовано. При наличии автоматически работающих сборщиков мусора операция `delete` может спровоцировать ошибку двойного удаления объектов. Чаще всего это можно рассматривать как мелкое неудобство, ибо если известно, что работает сборщик мусора, деструкторы, освобождающие память, могут быть просто опущены. Конечно, все это ведет к переносимости программ, к некоторой потере производительности и, возможно, к неточно предсказуемому их поведению (§С.9.1).

После того как выполнена операция `delete`, к объекту нельзя обращаться никоим образом. К сожалению, реализации не в состоянии надежно выявлять такого рода логические ошибки на стадии компиляции.

Пользователь может явным образом определять поведение операций `new` и `delete` (см. §6.2.6.2 и §15.6). Также возможно определять варианты взаимодействия операций выделения памяти, конструирования (инициализации) и генерации/обработки исключений (см. §14.4.5 и §19.4.5). Создание массивов в свободной памяти обсуждается в §10.4.7.

### 10.4.6. Классовые объекты как члены классов

Рассмотрим класс, который содержит информацию о небольшой организации:

```
class Club
{
    string name;
    Table members;
    Table officers;
    Date founded;
    // ...
    Club(const string& n, Date fd);
};
```

Конструктор класса `Club` принимает в качестве аргументов имя клуба и дату его основания. Аргументы конструкторам членов класса `Club` передаются с помощью *списка инициализации членов* (*member initializer list*) в определении конструктора обобщающего их класса:

```

Club : : Club (const string& n, Date fd)
      : name (n) , members () , officers () , founded (fd)
{
    // ...
}

```

Инициализаторы членов класса *предваряются двоеточием* и отделяются друг от друга запятыми.

Конструкторы членов класса вызываются до момента выполнения тела конструктора содержащего их класса. Порядок выполнения конструкторов членов соответствует порядку их объявления в определении объемлющего класса, а не порядку появления инициализаторов в списке инициализации. Во избежание путаницы, лучше упорядочить список инициализации в точном соответствии с порядком объявления членов. Порядок вызова деструкторов членов обратный по отношению к вызову их конструкторов, и выполняются они после выполнения тела деструктора объемлющего класса.

Если конструктор члена класса не требует аргументов, то в списке инициализации соответствующий ему инициализатор можно просто опустить:

```

Club : : Club (const string& n, Date fd) : name (n) , founded (fd)
{
    // ...
}

```

Данная версия конструктора полностью эквивалентна его предыдущей версии. В обеих версиях член *officers* инициализируется умолчательным конструктором класса **Table** (для этого конструктора предусмотрено умолчательное значение аргумента, равное 15).

Когда уничтожается объект объемлющего класса (например, класса **Club**), сначала выполняется тело деструктора этого класса (если таковой определен), а затем выполняются деструкторы членов в порядке, обратном порядку вызова их конструкторов (то есть в порядке, обратном объявлению членов в объемлющем классе). Можно сказать, что конструктор собирает среду исполнения функций-членов в порядке снизу-вверх (сначала конструкторы членов, а затем конструктор объемлющего класса), в то время как деструктор демонтирует эту среду сверху вниз.

#### 10.4.6.1. Обязательная инициализация членов

Инициализаторы членов необходимы (обязательны) в тех случаях, когда инициализация отличается от присваивания — для членов класса с типами, не имеющими умолчательных конструкторов, для *константных* членов и для ссылок. Например:

```

class X
{
    const int i;
    Club cl;
    Club& rc;
    // ...
    X(int ii, const string& n, Date d, Club& c) : i(ii) , cl(n, d) , rc(c) {}
}

```

Не существует никаких других способов инициализации таких членов, а отсутствие их инициализации является ошибкой. Однако для большинства иных типов существует выбор между применением списка инициализации и присваиваниями. В таких случаях я предпочитаю использовать список инициализации, делая тем самым факт инициализации особо наглядным. Часто это приводит и к более эффективному коду. Например:

```
class Person
{
    string name;
    string address;
    // ...
    Person (const Person&);
    Person (const string& n, const string& a);
};

Person::Person (const string& n, const string& a)
    : name (n)
{
    address = a;
}
```

Здесь **name** инициализируется копией **n**. А член **address** сначала инициализируется пустой строкой, а затем ему присваивается значение **a**.

#### 10.4.6.2. Члены-константы

*Статические члены-константы интегральных типов* можно инициализировать с помощью константных выражений *прямо в их объявлениях*. Например:

```
class Curious
{
public:
    static const int c1 = 7;           // ok, но нужно помнить определение
    static int c2 = 11;                // error: не константа
    const int c3 = 13;                // error: нет модификатора static
    static const int c4=f(17);         // error: не константный инициализатор
    static const float c5 = 7.0;       // error: не интегральный тип
    // ...
};
```

Если (и только если) требуется инициализированный член хранить и использовать как объект в памяти, то его где-либо нужно уникально определить. Инициализатор при этом повторять нельзя:

```
const int Curious::c1;                // обязательно (но без повтора инициализатора)
const int* p = &Curious::c1;        // ok: Curious::c1 был определен
```

В качестве альтернативы, в классе можно определить символические константы с помощью перечислений (§4.8, §14.4.6, §15.3). Например:

```
class X
{
    enum {c1 = 7, c2 = 11, c3 = 13, c4 = 17};
    ..
```

Тогда у вас не будет возникать искушение инициализировать в классе переменные, числа с плавающей запятой и т.д.

#### 10.4.6.3. Копирование членов

Умолчательный копирующий конструктор и умолчательный вариант операции присваивания (§10.4.4.1) просто копируют все элементы класса. Когда такое копирование не проходит, при попытке копирования объектов класса возникает ошибка. Например:

```
class Unique_handle
{
private:
    // копирующие операции здесь private для предотвращения явного
    // копирования (§11.2.2)

    Unique_handle(const Unique_handle&);
    Unique_handle& operator=(const Unique_handle&);

public:
    // ...
};

struct Y
{
    // ...
    Unique_handle a; // требуется явная инициализация
};

Y y1;
Y y2 = y1;          // error: нельзя скопировать Y::a
```

Кроме того, умолчательный вариант присваивания не проходит в тех случаях, когда нестатические члены являются ссылками, константами или имеют пользовательский тип, не определяющий умолчательного варианта операции присваивания.

Обратите внимание, что в результате работы умолчательного копирующего конструктора члены-ссылки в обоих объектах (оригинальном и копии) ссылаются на один и тот же объект в памяти, что порождает проблемы при уничтожении этого объекта.

При написании копирующего конструктора нужно проследить за тем, чтобы были скопированы все элементы, для которых действительно требуется копирование. Иначе они проинициализируются умолчательными значениями, а это часто не то, что требуется. Например:

```
Person : Person(const Person& a) : name(a.name) {} // будьте внимательны!
```

Здесь я забыл скопировать **address**, и по умолчанию **address** проинициализируется пустой строкой. При добавлении нового члена к классу всегда тщательно проверяйте, не нужно ли при этом модифицировать ранее определенные в классе конструкторы для инициализации и копирования нового члена.

### 10.4.7. Массивы

Если объект класса можно создать без указания явного инициализатора, то можно определить массив элементов этого класса. Например:

```
Table tbl[10];
```

Создается массив из *10 элементов* типа *Table*, и каждый элемент инициализируется вызовом *Table::Table()* с умолчательным аргументом 15.

За исключением применения списка инициализации (§5.2.1, §18.6.7) не существует иных способов явного задания аргументов конструктора в объявлении массива. Если в обязательном порядке требуется инициализировать элементы массива разными значениями, нужно написать соответствующий умолчательный конструктор, который так или иначе вырабатывает необходимые значения. Например:

```
class Ibuffer
{
    string buf;

public:
    Ibuffer() { cin>>buf; }
    // ...
};

void f()
{
    Ibuffer words[100];    // каждое слово инициализируется из cin
    // ...
}
```

Таких хитростей, все же, лучше избегать.

Когда массив уничтожается, деструктор вызывается для каждого элемента массива. Это происходит неявным образом для массивов, память под которые не выделялась операцией *new*. Как и язык С, язык С++ не отличает указатель на массив от указателя на первый элемент массива (§5.3). Как следствие, программист должен явно указать, что удаляется — массив или отдельный элемент. Например:

```
void f(int sz)
{
    Table* t1 = new Table;
    Table* t2 = new Table[sz];
    Table* t3 = new Table;
    Table* t4 = new Table[sz];

    delete t1;                // правильно
    delete [] t2;              // правильно
    delete [] t3;              // неправильно
    delete t4;                // неправильно
}
```

Точные детали выделения памяти под массивы или под отдельные объекты зависят от конкретной реализации. Поэтому и реакция на ошибочное применение операций *delete* или *delete[]* будет также различаться. В простейших и неинтересных случаях типа приведенного выше примера компилятор в состоянии обнару-

жить потенциальные проблемы, но в общем случае нечто ужасное произойдет лишь во время выполнения программы.

С логической точки зрения нет необходимости в особой форме операции **delete** [], предназначенной специально для массивов. С другой стороны, представим, что мы потребовали от системы управления свободной памятью маркировать выделяемые блоки памяти таким образом, чтобы сразу было ясно, происходило это выделение для массивов или нет. С программиста при этом будет снята дополнительная ноша, однако она тяжелым грузом ляжет на приложения и снизит эффективность их выполнения.

Если же вы находите массивы в С-стиле громоздкими и неуклюжими, используйте вместо них некоторый подходящий класс, например **vector** (§3.7.1, §16.3):

```
void g ()
{
    vector<Table> v (10);           // нет необходимости в удалении
    vector<Table>* p = new vector<Table> (10); // используйте delete, а не delete[]

    delete p;
}
```

Применять контейнеры типа **vector** намного проще, чем то и дело манипулировать операциями **new/delete**. Кроме того, класс **vector** обеспечивает безопасное поведение в плане исключений (приложение E).

#### 10.4.8. Локальные статические объекты

Конструктор для локального статического объекта (§7.1.2) вызывается только тогда, когда поток управления в работающей программе первый раз проходит через определение объекта. Рассмотрим пример:

```
void f(int i)
{
    static Table tbl;
    // ...
    if(i)
    {
        static Table tbl2;
        // ...
    }
}

int main ()
{
    f(0);
    f(1);
    f(2);
    // ...
}
```

Здесь конструктор для **tbl** вызывается только при первом вызове функции **f()**. Так как переменная **tbl** объявлена статической, она не уничтожается при выходе из **f()**, и она повторно не конструируется при повторных вызовах **f()**. Поскольку блок кода, содержащий определение **tbl2**, не выполняется при вызове **f(0)**, то **tbl2** не соз-

дается до вызова  $f(I)$ . При повторных входах в этот блок, имеющих место при последующих вызовах  $f()$ , *tbl2* повторно не конструируется.

При завершении работы программы деструкторы локальных статических объектов вызываются в порядке, обратном порядку их конструирования (§9.4.1.1). Однако точный момент их вызова не специфицируется.

#### 10.4.9. Нелокальные объекты

Переменная, определяемая вне функций (в глобальном пространстве, в пространстве имен или классовой *статическая* переменная; §С.9), инициализируется (конструируется) до вызова функции *main*(), а их деструкторы вызываются после завершения работы функции *main*(). Динамическая компоновка программных модулей слегка усложняет картину, откладывая инициализацию до того момента, когда код будет динамически скомпонован с исполняемой программой.

Конструкторы нелокальных объектов в рамках единицы трансляции исполняются в порядке, в каком появляются их определения. Рассмотрим пример:

```
class X
{
    // ...
    static Table memtbl;
};

Table tbl;
Table X: : memtbl;

namespace Z
{
    Table tbl2;
}
```

Здесь порядок создания объектов следующий: сначала *tbl*, затем *X: : memtbl*, а затем *Z: : tbl2*. Заметьте, что объявления (в противоположность определениям), например объявление *memtbl* в *X*, не влияют на порядок создания объектов. Деструкторы объектов вызываются в обратном порядке: сначала для объекта *Z: : tbl2*, затем для *X: : memtbl*, и потом для *tbl*.

Нет никаких правил для порядка конструирования нелокальных объектов, определенных в разных единицах трансляции. Например:

```
// файл file1.c:
Table tbl1;

// файл file2.c:
Table tbl2;
```

Что создается раньше, *tbl1* или *tbl2*, зависит от конкретной реализации. Даже в рамках одной и той же реализации нельзя точно специфицировать порядок их создания. Динамическая компоновка, или даже небольшие вариации в процессе компиляции способны изменить этот порядок. Аналогично, порядок уничтожения объектов также зависит от реализации.

При создании библиотеки часто бывает необходимо или просто удобно ввести тип, единственной целью которого является выполнение инициализации и завершающей очистки (cleanup). Такой тип предполагается использовать один раз при



создании статического объекта, чтобы обеспечить вызовы конструктора и деструктора. Вот пример на эту тему:

```
class Zlib_init
{
    Zlib_init () ;           // готовит Zlib к использованию
    ~Zlib_init () ;          // очищает ресурсы после использования Zlib
};

class Zlib
{
    static Zlib_init x;
    // ...
};
```

К сожалению, нет гарантии, что этот статический объект будет проинициализирован до его первого использования и уничтожен после последнего, если программа состоит из нескольких отдельно компилируемых единиц. Конкретная реализация может предоставить такую гарантию, но в общем случае такой гарантии нет. Программист можем сам обеспечить такую гарантию с помощью стратегии, обычно применяемой к локальным статическим объектам: стратегии флага первого использования. Например:

```
class Zlib
{
    static bool initialized;
    static void initialize () { initialized = true; }

public:
    // конструктор отсутствует
    void f ()
    {
        if (initialized == false) initialize () ;
        // ...
    }
    // ...
};
```

Если многие функции потребуют проверки флага первого использования, то стратегия станет утомительной, но все же по-прежнему работоспособной. Она опирается на тот факт, что статические объекты без конструкторов инициализируются нулевыми значениями. Большие сложности начинаются тогда, когда первый вызов критичен по времени выполнения, и дополнительные накладные расходы на проверку и возможную инициализацию становятся обременительными. В таких случаях помогут дополнительные трюки (§21.5.2).

Альтернативным подходом в случае простых объектов является их представление в виде функций (§9.4.1):

```
int& obj () { static int x = 0; return x; } // инициализация по первому вызову
```

Флаги первого использования не решают абсолютно всех проблем. Например, речь может пойти об объектах, которые ссылаются друг на друга во время конструирования. Конечно, таких ситуаций лучше избегать, но если все же они необходимы,

то конструирование нужно выполнять аккуратно и поэтапно. Еще стоит отметить, что нет аналогичной простой конструкции с флагом последнего использования (см. §9.4.1.1 и §21.5.2).

### 10.4.10. Временные объекты

*Временные объекты (temporary objects)* чаще всего создаются в процессе вычисления арифметических выражений. Например, при вычислении выражения  $x*u+z$  надо где-то сохранить промежуточный результат умножения  $x*u$ . За исключением случаев, когда эффективность выполнения кода особо критична (§11.6), временные объекты не беспокоят программиста. Однако ж, иногда на них приходится обращать внимание (§11.6, §22.4.7).

Если временный объект не связан ссылкой и не используется для инициализации именованного объекта, он обычно уничтожается по достижении конца полного выражения, в рамках оценки которого он был создан. *Полное выражение (full expression)* — это выражение, которое не является подвыражением (частью) другого выражения.

Стандартный класс *string* имеет функцию-член *c\_str()*, которая возвращает массив символов в С-стиле, то есть с терминальным нулем (§3.5.1, §20.4.1). В нем также определена операция  $+$ , означающая конкатенацию строк. Все это важные и полезные черты класса *string*. Однако их совместное использование порождает малоизвестную проблему. Вот пример:

```
void f(string& s1, string& s2, string& s3)
{
    const char* cs = (s1+s2).c_str();
    cout<< cs;

    if( strlen(cs=(s2+s3).c_str()) < 8 && cs[0] == 'a' )
    {
        // cs используется здесь
    }
}
```

Вероятно вашей первой реакцией будет «не надо так писать», и я абсолютно согласен с вашей реакцией. Но все же стоит рассмотреть этот пример подробнее.

Создается временный объект класса *string* для хранения результата конкатенации  $s1+s2$ . Потом указатель на С-строку извлекается из этого объекта. Затем — в конце выражения — временный объект уничтожается. Теперь зададимся вопросом, где же хранилась возвращаемая с помощью *c\_str()* строка С-стиля? Она, вероятно, хранилась как часть временного объекта, содержащего результат конкатенации  $s1+s2$ . Но этот объект уничтожен, так что *cs* указывает на освобожденную память с непредсказуемым содержимым. Возможно, операция *cout<<cs* и сработает как надо, но это будет в любом случае чистой удачей. Компилятор может обнаруживать и предупреждать о большинстве подобного рода проблем.

Пример с оператором *if* тоньше. Само условие отработает как надо, ибо полным выражением, содержащим временный объект, хранящий результат конкатенации  $s2+s3$ , является само условие. Однако после оценки условия эта временная переменная будет уничтожена, так что дальнейшее применение *cs* под вопросом.

Причина, по которой в рассмотренном примере (и во многих других подобных примерах) возникла проблема с временной переменной, заключается в том, что высокоуровневый тип данных используется в несвойственном ему низкоуровневом стиле. Более адекватный стиль программирования не только позволяет избежать рассмотренных проблем с временными переменными, но и порождает существенно более понятный код. Например:

```
void f(string& s1, string& s2, string& s3)
{
    cout<< s1+s2;
    string s = s2+s3;

    if(s.length() < 8 && s[0] == 'a')
    {
        // s используется здесь
    }
}
```

Временные объекты можно использовать в качестве инициализаторов для *константных* ссылок или именованных объектов. Например:

```
void g(const string&, const string&);

void h(string& s1, string& s2)
{
    const string& s = s1+s2;
    string ss = s1+s2;
    g(s, ss);           // здесь можно использовать s и ss
}
```

Здесь все корректно. Временный объект уничтожается только тогда, когда «его» ссылка или именованный объект выходят из области видимости. Помните, что возврат ссылки на локальную переменную является ошибкой (§7.3), и что временный объект не может связываться с *неконстантной* ссылкой (§5.5).

Временный объект можно создавать прямым вызовом конструктора. Например:

```
void f(Shape& s, int x, int y)
{
    s.move(Point(x, y));    // создается Point для передачи Shape::move()
    // ...
}
```

Такие временные объекты уничтожаются точно так же, как и временные объекты, создаваемые неявно.

### 10.4.11. Размещение объектов в заданных блоках памяти

С помощью операции **new** объекты по умолчанию создаются в свободной памяти. А что, если мы захотим разместить объект где-либо в другом месте? Рассмотрим простой класс:

```
class X
{
public:
```

```

X(int) ;
// ...
};

```

Мы можем разместить динамически создаваемый объект где-угодно, если определим функцию выделения памяти **operator new()** с дополнительным (вторым) формальным параметром, фактическое значение которого будет задаваться при помощи дополнительного параметра в операции **new**:

```

void* operator new (size_t, void* p) {return p; } // см. (§19.4.5)
void* buf = reinterpret_cast<void*> (0xF00F) ; // некоторый значимый адрес
X* p2=new (buf) X; // вызывается функция operator new (sizeof(X), buf)
// и конструируется объект X в буфере buf

```

Из-за такого применения вариант операции **new** в виде **new (buf) X**, предназначенный для передачи дополнительного аргумента функции **operator new()**, называется операцией «размещающее **new**» (*placement new*). Обратите внимание на то, что первым аргументом функции **operator new()** всегда является размер выделяемого блока памяти, и что размер размещаемого объекта передается неявно (§15.6). Выбор конкретного варианта функции **operator new()** для соответствия применяемому варианту операции **new** выполняется, как всегда, по правилу соответствия аргументов (§7.4); при этом первый аргумент каждого варианта функции **operator new()** должен иметь тип **size\_t**.

Приведенный пример функции **operator new()** является простейшим примером распределителя памяти для операции «размещающее **new**». Он определен в стандартном заголовочном файле **<new>**.

В рассмотренном примере мы использовали операцию приведения **reinterpret\_cast**, наиболее «безобразную на вид» и опасную из всех операций приведения (§6.2.7). В большинстве случаев эта операция оставляет последовательность бит операнда нетронутой, изменяя лишь трактовку его типа. Ее применяют, например, в потенциально опасных, но иногда необходимых низкоуровневых системнозависимых преобразованиях целочисленных значений в указатели и наоборот.

Операцию «размещающее **new**» можно использовать для помещения объектов в конкретные участки памяти:

```

class Arena
{
public:
    virtual void* alloc (size_t)=0;
    virtual void free (void*)=0;
    // ...
};

void* operator new (size_t sz, Arena* a)
{
    return a->alloc (sz) ;
}

```

Теперь объекты произвольных типов можно при необходимости размещать в тех или иных специфических участках памяти («аренах»). Например:

```
extern Arena* Persistent;
extern Arena* Shared;

void g (int i)
{
    X* p = new (Persistent) X(i);    // X в арене Persistent
    X* q = new (Shared) X(i);        // X в арене Shared
    // ..
}
```

Помещение объекта в область памяти, не находящуюся под управлением стандартного механизма выделения/освобождения свободной памяти, подразумевает специальные действия при уничтожении объекта. Базовым механизмом является в этом случае *явный вызов деструктора*:

```
void destroy (X* p, Arena* a)
{
    p->~X();                        // вызов деструктора
    a->free (p);                     // освобождение памяти
}
```

Отметим, что явный вызов деструкторов и применение специальных вариантов *глобальных* аллокаторов памяти в обычных случаях нецелесообразны. Все же, в определенных случаях они нужны. Например, трудно было бы реализовать обобщенный контейнер в духе стандартного библиотечного класса **vector** (§3.7.1, §16.3.8) без применения явного вызова деструктора. Новичкам лучше трижды подумать перед тем, как явно вызвать деструктор, а еще лучше проконсультироваться с опытным специалистом.

В §14.4.4 рассмотрено, как исключения взаимодействуют с операциями «размещающее **new**».

Не существует отдельного синтаксиса для размещения массивов. В этом нет нужды, так как операция «размещающее **new**» и так работает с произвольными типами данных. Тем не менее, для массивов можно определить специальный вариант функции **operator delete()** (§19.4.5).

### 10.4.12. Объединения

Именованное объединение определяется как *структура*, в которой всем полям отводится одна и та же область памяти (§C.8.2). Объединение может иметь функции-члены, но не статические члены<sup>1</sup>.

В общем случае, компилятор не может знать, какой член объединения используется; то есть тип объекта, хранящегося в объединении, неизвестен. Следовательно, объединение не может содержать члены, классы которых имеют явно определенные конструктор или деструктор. Иначе не будет возможности предотвратить хранящийся в объединении объект от непреднамеренной порчи, и невозможно будет вызвать правильный деструктор при выходе объединения из области видимости.

<sup>1</sup> Подробнее этот вопрос рассматривается в книге Н.Н. Мартынов, «Программирование для Windows на C/C++», том 2, М., Бином, 2006, стр. 67–69. — *Прим. ред.*

Лучше всего ограничить использование объединений низкоуровневым кодом, или в виде части класса, содержащего информацию о том, что на самом деле хранится в объединении (см. §10.6[20]).

## 10.5. Советы

1. Представляйте концепции в виде классов; §10.1.
2. Используйте открытые данные (*структуры*) только тогда, когда это на самом деле лишь данные, и для них не существует разумных инвариантов; §10.2.8.
3. Конкретные типы являются простейшими видами классов. Где только возможно, предпочитайте конкретные типы более сложным видам классов или открытым структурам данных; §10.3.
4. Определяйте функцию в качестве функции-члена класса только в тех случаях, когда ей нужен непосредственный доступ к его внутренней структуре; §10.3.2.
5. Используйте пространства имен для явного выражения связи класса и его функций поддержки; §10.3.2.
6. Объявляйте функции-члены *константными*, если они не модифицируют состояния (значения) объектов; §10.2.6.
7. Объявляйте функции-члены *статическими* в тех случаях, когда им нужен доступ к представлению класса, но не нужен вызов для конкретных объектов класса; §10.2.4.
8. Используйте конструктор для установления инварианта класса; §10.3.1.
9. Если конструктор класса выделяет ресурсы, то для освобождения ресурсов классу требуется деструктор; §10.4.1.
10. Если в классе содержится член-указатель, то для класса следует определить копирующие операции (копирующий конструктор и операцию присваивания); §10.4.4.1.
11. Если класс содержит член-ссылку, то ему, возможно, следует определить копирующие операции (копирующий конструктор и операцию присваивания); §10.4.6.3.
12. Если классу требуются копирующие операции или деструктор, то ему вероятно потребуются конструктор, деструктор, операция присваивания и копирующий конструктор; §10.4.4.1.
13. В операции присваивания проверяйте возможность самоприсваиваний; §10.4.4.1.
14. При написании копирующего конструктора следите за тем, чтобы были скопированы все элементы (остерегайтесь умолчательной инициализации); §10.4.4.1.
15. Добавляя новый член к существующему классу, проверяйте необходимость модификации конструкторов с целью корректной инициализации новых членов; §10.4.6.3.

16. Пользуйтесь перечислениями для объявлений целых констант в классе; §10.4.6.2.
17. Избегайте зависимости от порядка конструирования глобальных объектов и объектов в пространствах имен; §10.4.9.
18. Для минимизации зависимости от порядка конструирования объектов применяйте флаги первого использования; §10.4.9.
19. Помните, что временные объекты уничтожаются в конце вычисления полных выражений, в которых они созданы; §10.4.10.

## 10.6. Упражнения

1. (\*1) Найдите ошибку в *Date::add\_year()* в §10.2.2. Затем найдите еще две ошибки в версии из §10.2.7.
2. (\*2.5) Скомпилируйте и проверьте работу *Date*. Переработайте этот класс для представления даты в виде «количество дней после 1/1/1970».
3. (\*2) Найдите какой-либо коммерческий вариант типа *Date*. Покритикуйте предоставляемые им возможности. Обсудите это с другими пользователями.
4. (\*1) Как вы обратитесь к *set\_default* из класса *Date*, находящегося в пространстве имен *Chrono* (§10.3.2). Предложите по крайней мере три разных способа.
5. (\*2) Определите класс *Histogram*, который хранит числа из интервалов, указанных аргументами конструктора. Напишите функции для вывода гистограмм. Обрабатывайте ошибки выхода из диапазона значений.
6. (\*2) Определите несколько классов для представления случайных чисел с разными законами распределения (например, равномерным или экспоненциальным). В каждом классе должен быть конструктор, задающий параметры распределения, и функция *draw()*, возвращающая следующее случайное число.
7. (\*2.5) Дополните класс *Table* возможностью хранения пар (имя, значение). Затем модифицируйте программу-калькулятор из §6.1 таким образом, чтобы она использовала класс *Table* вместо *map*. Сравните две версии.
8. (\*2) Перепишите *Tnode* из §7.10[7] в виде класса с конструкторами, деструктором и т.д. Определите дерево с узлами *Tnode* в виде класса с конструкторами, деструктором и т.д.
9. (\*3) Определите, реализуйте и протестируйте класс *Intset* (множество целых). Предоставьте операции объединения множеств, пересечения и симметричной разности.
10. (\*1.5) Модифицируйте класс *Intset* во множество узлов *Node*, где *Node* — определяемая вами структура.
11. (\*3) Определите класс для анализа, хранения, вычисления и печати простых арифметических выражений, состоящих из целых констант и операций +, −, \* и /. Открытый интерфейс должен выглядеть так:

```

class Expr
{
// ...
public:
    Expr(const char*);
    int eval();
    void print();
};

```

Строковый аргумент конструктора *Expr::Expr()* является выражением. Функция *Expr::eval()* возвращает значение выражения, а *Expr::print()* выводит представление выражения в *cout*. Программа может выглядеть примерно так:

```

Expr x("123/4+123*4-3");
cout<<"x= "<<x.eval()<<"\n";
x.print();

```

Определите класс *Expr* дважды: один раз для его представления выберите список связанных узлов, а другой раз — строку. Поэкспериментируйте с разными способами печати выражения: с полной расстановкой скобок, в postfixной нотации и т.д.

12. (\*2) Определите класс *Char\_queue* так, чтобы открытый интерфейс не зависел от представления. Реализуйте *Char\_queue* в виде (а) связанного списка и (б) вектора. Не беспокойтесь о многопоточности.
13. (\*3) Разработайте класс таблицы символов и класс элементов этой таблицы для какого-либо языка. Посмотрите на какой-нибудь компилятор для этого языка, чтобы узнать, как в действительности выглядит таблица символов.
14. (\*2) Модифицируйте класс выражений из §10.6[11] таким образом, чтобы он мог обрабатывать переменные и операцию присваивания. Воспользуйтесь классом таблицы символов из §10.6[13].
15. (\*1) Дана программа:

```

#include <iostream>

int main()
{
    std::cout<<"Hello, world!\n";
}

```

Модифицируйте ее так, чтобы она выводила

```

Initialize
Hello, world!
Clean up

```

Не вносите при этом никаких изменений в функцию *main()*.

16. (\*2) Определите класс *Calculator* так, чтобы большая часть его реализации состояла из функций §6.1. Создавайте калькуляторы и активизируйте их для ввода из *cin*, из командной строки, для строк программы. Реализуйте вывод в разные приемники.



17. (\*2) Определите два класса, каждый со **статическим** членом, так, чтобы конструирование **статического** члена использовало ссылку на другой **статический** член. Где такое может встретиться в реальном коде? Как нужно модифицировать эти классы, чтобы устранить в конструкторах зависимость от порядка?
18. (\*2.5) Сравните класс **Date** (§10.3) с вашими решениями упражнений §5.9[13] и §7.10[19]. Обсудите найденные ошибки и возможные различия в сопровождении каждого из решений.
19. (\*3) Напишите функцию, которая получает в качестве аргументов **istream** и **vector<string>**, а порождает **map<string, vector<int>>**, содержащий каждую строку и частоту ее вхождения. Прогоните программу на текстовом файле с количеством строк, не менее 1000, разыскивая при этом не менее 10 слов.
20. (\*2) Возьмите класс **Entry** из §C.8.2 и модифицируйте его таким образом, чтобы каждый член объединения всегда использовался в соответствии с его типом.



# Перегрузка операций

*Когда я использую слово,  
оно означает то, что я хочу им выразить,  
не больше и не меньше.  
— Шалтай-Болтай*

Нотация операций — функции-операции — бинарные и унарные операции — предопределенный смысл операций — операции и пользовательские типы — операции и пространства имен — комплексный тип — операции в виде функций-членов и глобальных функций — смешанная арифметика — инициализация — копирование — преобразования — литералы — функции поддержки — операции приведения типов — разрешение неоднозначности — друзья — члены и друзья — объекты больших размеров — присваивание и инициализация — индексирование — операция вызова функции — разыменование — инкремент и декремент — класс строк — советы — упражнения.

## 11.1. Введение

В любой технической области — и в большинстве не технических — вырабатываются свои стандартные обозначения, помогающие наглядно представлять и обсуждать часто используемые концепции. Например, из-за длительного знакомства с формой записи

$x+y*z$

она для нас намного яснее и понятнее, чем фраза

*умножить y на z и прибавить результат к x*

Невозможно переоценить важность кратких нотаций для общепринятых операций.

Как и большинство других языков программирования, C++ поддерживает набор операций для встроенных типов. Однако большинство концепций, для которых традиционно используются операции, не относятся ко встроенным типам языка C++, и их нужно представлять пользовательскими типами. Например, если вам

нужны комплексная арифметика, матричная алгебра или символьные строки, в языке C++ вы используете классы для представления этих концепций. А если для этих типов данных определить смысл действия стандартных операций, то работа пользователя с объектами этих классов становится более простой и наглядной по сравнению с использованием лишь традиционной функциональной нотации. Например,

```
class complex                // очень упрощенный класс complex
{
    double re, im;

public:
    complex(double r, double i) : re(r), im(i) {}
    complex operator+ (complex);
    complex operator* (complex);
};
```

реализует концепцию комплексных чисел. *Комплексное* число представлено здесь в виде набора двух вещественных чисел и дополнено арифметическими операциями + и \* над этим новым типом данных. Программист в классе complex определяет функции-члены *operator+* () и *operator\** (), которые и задают смысл и порядок работы операций + (сложение) и \* (умножение) над объектами типа *complex*. Если *b* и *c* имеют тип *complex*, то *b+c* означает функциональный вызов *b.operator+* (*c*). Теперь для типа *complex* мы можем применять нотацию, принятую в математической литературе для комплексных чисел:

```
void f()
{
    complex a = complex(1, 3.1);
    complex b = complex(1.2, 2);
    complex c = b;

    a = b+c;
    b = b+c*a;
    c = a*b+complex(1, 2);
}
```

При этом сохраняются обычные правила приоритета операций, так что второе выражение в представленном примере означает *b=b+(c\*a)*, а не *b=(b+c)\*a*.

Наиболее очевидные случаи перегрузки операций (то есть переопределения операций для нового типа) относятся к конкретным типам данных (§10.3). Но, разумеется, полезность определяемых пользователем операций выходит за эти рамки. Например, при проектировании общих и специализированных интерфейсов программных систем часто приходится переопределять поведение даже таких операций, как *->*, *[]* и *()*.

## 11.2. Функции-операции

Можно перегружать следующие операции:

|    |    |     |            |               |               |                  |
|----|----|-----|------------|---------------|---------------|------------------|
| +  | -  | *   | /          | %             | ^             | &                |
|    | ~  | !   | =          | <             | >             | +=               |
| -- | *= | /=  | %=         | ^=            | &=            | =                |
| << | >> | >>= | <<=        | ==            | !=            | <=               |
| >= | && |     | ++         | --            | ->*           | ,                |
| -> | [] | ()  | <i>new</i> | <i>new []</i> | <i>delete</i> | <i>delete []</i> |

Нельзя перегружать операции:

- :: (разрешение области видимости; §4.9.4, §10.2.4),
- . (выбор члена класса; §5.7) и
- \* (выбор члена класса через указатель на классовые члены; §15.5).

Правым операндом у этих операций является имя, а не значение, и обеспечивают они базовое средство доступа к членам. Возможность перегрузки этих операций привела бы к очень тонким ошибкам [Stroustrup, 1994]. Тернарная условная операция `?:` (§6.3.2) также запрещена к перегрузке. Также нельзя перегружать именованные операции *sizeof* (§4.6) и *typeid* (§15.4.4).

Невозможно использовать не существующие в языке C++ знаки операций, но можно воспользоваться функциями в случаях, когда существующие знаки операций не подходят. Например, используйте *pow()* вместо `**`<sup>1</sup>. Такие ограничения могут показаться драконовскими, но более гибкая система обозначений легко приводит к неоднозначностям. Например, определение новой операции `**` для обозначения возведения в степень кажется на первый взгляд очевидной и несложной в реализации, но это только на первый взгляд. Действительно, должна ли операция `**` быть левоассоциативной (как в языке Fortran), или правоассоциативной (как в языке Algol)? Как должно интерпретироваться выражение *a\*\*p* — как *a\*(\*p)*, или как *(a)\*\*(p)*?

Имя *функции-операции* (*operator function*) начинается с ключевого слова *operator*, за которым следует знак перегружаемой операции, например *operator<<*. Функция-операция объявляется и может быть вызвана так же, как и любая другая функция. Традиционная форма использования операции является сокращением явной функциональной формы вызова. Например:

```
void f(complex a, complex b)
{
    complex c = a + b;           // сокращенная форма
    complex d = a.operator+(b);  // явный вызов
}
```

Оба инициализирующих выражения в данном примере эквивалентны друг другу.

<sup>1</sup> Речь идет об операции возведения в степень. — Прим. ред.

### 11.2.1. Бинарные и унарные операции

Бинарную (двухоперандную) операцию можно перегрузить либо с помощью нестатической функции-члена с одним аргументом, либо с помощью глобальной функции с двумя аргументами. При этом для любой бинарной операции @ выражение *aa@bb* интерпретируется либо как *aa.operator@(bb)*, либо как *operator@(aa, bb)*. Если определены оба варианта, то для выбора одного из них используются критерии разрешения перегрузки (§7.4). Например:

```
class X
{
public:
    void operator+ (int) ;
    X(int) ;
};

void operator+ (X, X) ;
void operator+ (X, double) ;

void f(X a)
{
    a+1;           // a.operator+(1)
    1+a;           // ::operator+(X(1),a)
    a+1.0;         // ::operator+(a,1.0)
}
```

Унарную (однооперандную; префиксную или постфиксную) операцию можно определить либо с помощью нестатической функции-члена без аргументов, либо с помощью глобальной функции с одним аргументом. Для любой префиксной унарной операции @ выражение @*aa* интерпретируется либо как *aa.operator@()*, либо как *operator@(aa)*. Если определены оба варианта, то для выбора одного из них используются критерии разрешения перегрузки (§7.4). Для любой постфиксной унарной операции @ выражение *aa@* интерпретируется либо как *aa.operator@(int)*, либо как *operator@(aa, int)*. Более подробно это объясняется в §11.11. Если определены оба варианта, то для выбора одного из них используются критерии разрешения перегрузки (§7.4). Любая операция при перегрузке должна объявляться в соответствии с ее стандартным синтаксисом (§A.5). Например, пользователь не может определить унарную операцию % или тернарную операцию +. Рассмотрим пример:

```
class X
{
public:
    // members (имеющие неявный указатель this):
    X* operator& () ;           // префиксная унарная & (взятие адреса)
    X operator& (X) ;           // бинарная & ("I")
    X operator++ (int) ;         // постфиксный инкремент (см. §11.11)
    X operator& (X, X) ;         // error: три операнда (тернарная операция)
    X operator/ () ;            // error: унарная операция /
};

// nonmember functions:
X operator- (X) ;              // префиксный унарный минус
```

```

X operator- (X, X) ;      // бинарный минус
X operator-- (X&, int) ;  // постфиксный декремент
X operator- () ;          // error: нет операнда
X operator- (X, X, X) ;    // error: тернарная операция
X operator% (X) ;        // error: унарная операция %

```

Перегрузка операции `[]` рассматривается в §11.8, операции `()` — в §11.9, операции `->` — в §11.10, операций `--` и `++` — в §11.11; перегрузка операций динамического выделения памяти и ее освобождения рассматривается в §6.2.6.2, §10.4.11 и §15.6.

### 11.2.2. Предопределенный смысл операций

Относительно перегружаемых пользователем операций существует лишь ряд дополнительных правил. В частности, ***operator=()***, ***operator[]()***, ***operator()()*** и ***operator->()*** *обязаны быть нестатическими функциями-членами* (и только ими); это гарантирует, что их первый операнд всегда `lvalue` (§4.9.6).

Для некоторых встроенных операций постулируется их эквивалентность определенной комбинации других операций. Например, если `a` имеет тип `int`, то `++a` означает то же, что и `a += 1`, что также эквивалентно `a = a + 1`. Такие соотношения между операциями не поддерживаются для перегружаемых пользователем операций автоматическим образом, а только если пользователь позаботился об этом сам. Например, компилятор не генерирует автоматически определение для `Z : operator+=()` из одного того факта, что пользователь определил `Z : operator+()` и `Z : operator=()`.

Исторически так сложилось, что операции `=` (операция присваивания), `&` (операция взятия адреса) и `,` (операция следования; §6.2.2) имеют предопределенный смысл, когда применяются к классовым объектам. Их можно сделать недоступными для пользователей класса, если объявить операции закрытыми:

```

class X
{
private:
    void operator= (const X&) ;
    void operator& () ;
    void operator, (const X&) ;
    // ...
};

void f(X a, X b)
{
    a = b;           // error: operator= private
    &a;              // error: operator& private
    a, b;            // error: operator, private
}

```

А можно придать им новый смысл, переопределив их в классе соответствующим образом (естественно, в открытом режиме).

### 11.2.3. Операции и пользовательские типы

Любая функция-операция должна быть либо членом класса, либо иметь по крайней мере один аргумент классowego типа (за исключением функций ***operator new()*** и ***operator delete()***, участвующих в перегрузке операций ***new/delete***). Это правило га-

рантирует, что пользователь не может изменить смысл выражения, если оно не содержит объектов пользовательских типов. В частности, нельзя изменить смысл операций, выполняющихся над указателями. Можно сказать, что язык C++ является расширяемым, но он не подвержен мутациям (за исключением операций `=`, `&` и `,` для классовых объектов).

Функция-операция, принимающая первый аргумент встроенного типа (§4.1.1), не может быть функцией-членом класса. Например, рассмотрим сложение комплексной переменной `aa` и целого числа `2`: выражение `aa+2` может при наличии соответствующего определения функции-члена рассматриваться как `aa.operator+(2)`, но выражение `2+aa` не может никак интерпретироваться, ибо нет класса `int`, для которого была бы определена операция `+` с возможностью трактовки выражения как `2.operator+(aa)`. Но даже, если бы такой класс и был, то все равно выражения `aa+2` и `2+aa` требовали бы для интерпретации двух разных функций-членов. Из-за того, что компилятор не понимает смысла определяемых пользователем операций, он не может понять, что операция `+` коммутативна, и интерпретировать `2+aa` как `aa+2`. Рассмотренная проблема легко решается с помощью определения глобальных функций-операций (§11.3.2, §11.5).

Так как *перечисления являются пользовательскими типами*, то для них можно *определять операции*. Например:

```
enum Day {sun, mon, tue, wed, thu, fri, sat};

Day& operator++(Day& d)
{
    return d = (sat==d) ? sun : Day(d+1);
}
```

Любое выражение проверяется на отсутствие неоднозначностей. Когда имеются предоставляемые пользователем операции, неоднозначности в выражениях устраняются по правилам из §7.4.

#### 11.2.4. Операции и пространства имен

Операции определяются либо в классах, либо в пространствах имен (в частности, в глобальном пространстве). Рассмотрим упрощенную версию ввода/вывода строк из стандартной библиотеки:

```
namespace std                // упрощенное std
{
    class ostream
    {
        // ...
        ostream& operator<<(const char*);
    };

    extern ostream cout;

    class string
```



```
ostream& operator<< (ostream&, const string&) ;
}

int main ()
{
    char* p = "Hello";
    std::string s = "world";
    std::cout<< p << ", " << s << "!\n";
}
```

Естественно, представленный код выводит строку *Hello, world!* Но почему? Заметьте, что я не стал делать доступными все средства из пространства имен *std* при помощи директивы

```
using namespace std;
```

Вместо этого я воспользовался префиксом *std::* для *string* и *cout*. Таким образом я не стал засорять глобальное пространство имен и порождать ненужные зависимости.

Для вывода С-строк (то есть *char\**) в классе *std::ostream* имеется соответствующая функция-операция, так что по определению

```
std::cout<< p
```

означает

```
std::cout.operator<< (p)
```

Но для вывода строк типа *std::string* в классе *std::ostream* соответствующей функции-члена нет, так что

```
std::cout<< s
```

означает

```
operator<< (std::cout, s)
```

Операции, определенные в пространствах имен, различаются по типам операндов точно так же, как функции различаются по типам их аргументов (§8.2.6). В частности, так как *cout* определен в пространстве имен *std*, то это пространство имен включается в процесс поиска подходящего определения для операции *<<*. В результате, компилятор находит и использует

```
std::operator<< (std::ostream&, const std::string&)
```

Рассмотрим бинарную операцию *@*. Если *x* имеет тип *X*, а *y* имеет тип *Y*, то для выражения *x@y* выполняется следующий поиск определений:

- Если *X* класс, искать определение функции-члена *operator@* в классе *X* или в его базовых классах.
- Искать объявление *operator@* в контексте, окружающем *x@y*.
- Если *X* определен в пространстве имен *N*, искать объявление *operator@* в *N*.
- Если *Y* определен в пространстве имен *M*, искать объявление *operator@* в *M*.

Если находятся несколько подходящих операций, то для выбора наилучшего соответствия (если оно существует) применяются критерии разрешения перегрузки

(§7.4). Рассмотренный механизм поиска выполняется только в том случае, когда один из операндов операции имеет пользовательский тип; при этом будут учтены операции приведения типа, определенные пользователем (§11.3.2, §11.4). Обратите внимание на то, что имя, введенное оператором *typedef*, является лишь синонимом, а не пользовательским типом (§4.9.7).

Поиск определений унарных операций и разрешение неоднозначностей выполняются аналогично.

Заметьте, что при поиске определений для операций никакого преимущества у функций-членов перед глобальными функциями нет; в этом состоит отличие от поиска именованных функций (§8.2.6). *Меньшее сокрытие операций* приводит к тому, что встроенные операции всегда доступны и что пользователи могут придавать операциям новый смысл без модификации существующих определений классов. Например, в стандартной библиотеке операция вывода << определена по отношению ко встроенным типам. Пользователь же всегда может доопределить эту операцию для вывода своих собственных классовых типов без какой бы то ни было модификации класса *ostream* (§21.2.1).

## 11.3. Тип комплексных чисел

Реализация концепции комплексных чисел, представленная во введении, слишком ограничена, чтобы подойти для реальной работы. Например, заглянув в книгу по математике, мы ожидаем, что следующее будет работать:

```
void f()
{
    complex a = complex(1, 2);
    complex b = 3;
    complex c = a + 2 * 3;
    complex d = 2 + b;
    complex e = -b - c;
    b = c * 2 * c;
}
```

Кроме того, мы ожидаем, что реализованы такие операции, как операция == для сравнения комплексных чисел и операция << для вывода, а также набор подходящих математических функций, таких как *sin()* и *sqrt()*.

Класс *complex* является конкретным типом, так что будем следовать рекомендациям из §10.3. Кроме того, поскольку стандартная комплексная арифметика опирается на применение операций, то нам придется в рамках типа *complex* использовать все, что уже известно про перегрузку операций.

### 11.3.1. Перегрузка операций функциями-членами и глобальными функциями

Я предпочитаю минимизировать количество функций, непосредственно манипулирующих внутренним представлением объекта класса. Для этого в теле класса следует определять лишь те операции, которые по своей внутренней природе должны изменять состояние первого операнда, например +=. Любую иную операцию,

целью которой является порождение нового значения, операцию  $+$  к примеру, следует реализовывать вне класса (при этом она может опираться на фундаментальные операции, имеющие доступ к представлению класса):

```
class complex
{
    double re, im;

public:
    complex& operator+= (complex a) ; // имеет доступ к внутреннему представлению
    // ...
};

complex operator+ (complex a, complex b)
{
    complex r = a;
    return r+= b; // получает доступ к представлению через +=
}
```

Отталкиваясь от данных определений, мы можем писать:

```
void f(complex x, complex y, complex z)
{
    complex r1 = x+y+z; // r1 = operator+(x,operator+(y,z))
    complex r2 = x; // r2 = x
    r2 += y; // r2.operator+=(y)
    r2 += z; // r2.operator+=(z)
}
```

Составные операции присваивания, такие как  $+=$  и  $*=$ , программировать легче, чем их более «простые составные части» — операции  $+$  и  $*$ . Поначалу это многих удивляет, но это вытекает из того факта, что три объекта вовлекаются в работу операции  $+$  (два операнда и временный объект-результат), в то время как у операции  $+=$  — всего два объекта. В последнем случае достигается более высокая эффективность за счет устранения необходимости во временных объектах. Например, следующий код

```
inline complex& complex::operator+= (complex a)
{
    re += a.re;
    im += a.im;
    return *this;
}
```

не требует временных объектов для хранения результатов сложения и упрощает компилятору задачу по реализации встраиваемого кода.

Оптимизирующий компилятор сумеет сгенерировать код, близкий к оптимальному, и для операции  $+$ . К сожалению, не каждый компилятор может похвастаться блестящими оптимизирующими способностями, и не каждый пользовательский тип столь же прост как тип `complex`; поэтому в §11.5 обсуждаются синтаксические средства, позволяющие операциям иметь прямой доступ к представлению класса.

### 11.3.2. Смешанная арифметика.

Чтобы справиться с выражением

```
complex d = 2 + b ;
```

нам нужно определить операцию `+`, работающую с операндами разных типов. В терминологии языка Fortran, нам требуется *смешанная арифметика* (*mixed-mode arithmetic*). Этого можно добиться путем добавления всех нужных версий операции сложения:

```
class complex
{
    double re, im ;

public :
    complex& operator+= (complex a)
    {
        re += a.re ;
        im += a.im ;
        return *this ;
    }

    complex& operator+= (double a)
    {
        re += a ;
        return *this ;
    }
    // ...
};

complex operator+ (complex a, complex b)
{
    complex r = a ;
    return r += b ;                               // вызывает complex::operator+=(complex)
}

complex operator+ (complex a, double b)
{
    complex r = a ;
    return r += b ;                               // вызывает complex::operator+=(double)
}

complex operator+ (double a, complex b)
{
    complex r = b ;
    return r += a ;                               // вызывает complex::operator+=(double)
}
```

Операция сложения с числом типа ***double*** проще, чем сложение двух *комплексных* чисел, что и отражается в коде данного примера. Операции с ***double*** не затрагивают мнимых частей *комплексных* чисел, и поэтому более эффективны.

Отталкиваясь от имеющихся определений, мы можем писать, например, такой клиентский код:

```
void f(complex x, complex y)
{
    complex r1 = x+y;           // вызывает operator+(complex,complex)
    complex r2 = x+2;           // вызывает operator+(complex,double)
    complex r3 = 2+x;           // вызывает operator+(double,complex)
}
```

### 11.3.3. Инициализация

Чтобы иметь возможность копирования и инициализации *комплексных* чисел вещественными скалярами, нам требуется преобразование из такого скаляра в комплексное число. Например:

```
complex b = 3;                 // должно означать b.re=3, b.im=0
```

Конструктор, принимающий *единственный аргумент*, осуществляет *преобразование* (приведение) *типа аргумента к классовому типу*. Например:

```
class complex
{
    double re, im;

public:
    complex(double r) : re(r), im(0) {}
    // ...
};
```

Конструктор предписывает, как сформировать значение классового типа. Конструктор используется в случаях, когда ожидается значение такого типа и когда оно может быть создано конструктором из значений, предоставляемых инициализатором, или из присваиваемых значений. Поэтому *конструктор с единственным аргументом вызывать явно не нужно*. Например,

```
complex b = 3;
```

означает

```
complex b = complex(3);
```

Неявное применение определяемого пользователем приведения типов возможно лишь в случае, если оно уникально (§7.4). В §11.7.1 рассматривается способ определения конструктора, для которого неявный вызов запрещен.

Естественно, нам обязательно нужен конструктор с двумя аргументами типа *double*, а также умолчательный конструктор для построения комплексного числа  $(0, 0)$ :

```
class complex
{
    double re, im;

public:
    complex() : re(0), im(0) {}
    comlex(double r) : re(r), im(0) {}
    comlex(double r, double i) : re(r), im(i) {}
```

Применяя аргументы по умолчанию, мы можем добиться некоторого сокращения кода:

```
class complex
{
    double re, im;

public:
    complex (double r=0, double i=0) : re(r), im(i) {}
    // ...
};
```

При наличии в классе явно определенных конструкторов использование списков инициализации (§5.7, §4.9.5) запрещается. Например:

```
complex z1 = {3};                // error: у типа complex есть конструктор
complex z2 = {3, 4};            // error: у типа complex есть конструктор
```

### 11.3.4. Копирование

В дополнение к определенным в классе *конструкторам*, класс имеет умолчательный вариант (неявно генерируемый компилятором) копирующего конструктора (§10.2.5), который просто копирует все поля данных. То же самое можно явным образом определить и самим:

```
class complex
{
    double re, im;

public:
    complex (const complex& c) : re(c.re), im(c.im) {}
    // ...
};
```

В тех случаях, когда умолчательный вариант копирующего конструктора подходит, я предпочитаю ничего самому не писать. Это лаконичнее и люди заранее понимают его работу. Кроме того, компилятору удобно оптимизировать свое собственное творение. И, наконец, ручное кодирование простейшего копирования полей данных утомительно и чревато внесением нелепых ошибок, особенно для классов со многими полями данных (§10.4.6.3).

Для *копирующего конструктора* я использую *аргумент, передаваемый по ссылке*, и я просто *обязан* это делать, поскольку копирующий конструктор определяет, как проходит процесс копирования, в том числе аргументов этого типа. Таким образом, *передача аргумента копирующему конструктору по значению*

```
complex::complex (complex c) : re(c.re), im(c.im) {}    // error
```

*приводит к бесконечной рекурсии.*

Для остальных же функций я использую передачу аргументов типа *complex* по значению, а не по ссылке. Здесь у разработчика есть выбор. С точки зрения пользователя разница между вариантами аргументов типа *complex* или *const complex&* невелика. Этот вопрос обсуждается далее в §11.6.

В принципе, копирующие конструкторы используются в простых случаях инициализации вроде

```
complex x = 2;           // создается complex(2) и им инициализируется x
complex y = complex (2, 0); // создается complex(2,0) и им инициализируется y
```

В то же время, вызовы копирующих конструкторов могут быть легко заменены на эквивалентные варианты

```
complex x (2);           // x инициализируется значением 2
complex y (2, 0);        // x инициализируется парой чисел (2,0)
```

Для арифметических типов я предпочитаю вариант с использованием `=`. Можно ограничить набор допустимых для стиля инициализации со знаком `=` значений по сравнению с вариантом инициализации, использующим `()`, сделав копирующий конструктор закрытым (§11.2.2) или объявив его с модификатором *explicit* (§11.7.1).

Аналогично инициализации, присваивание по умолчанию одного объекта класса другому эквивалентно почленному присваиванию (§10.2.5). Мы могли бы и явным образом задать такое действие с помощью **complex::operator=** (). Однако для простых типов, таких как **complex**, в этом нет никакой нужды. Достаточно умолчательного варианта.

Копирующий конструктор — и умолчательный, и пользовательский — *используется не только при инициализации переменных, но и при передаче аргументов, возврате значений, а также обработке исключений* (§11.7). Семантика этих операций эквивалентна семантике инициализации (§7.1, §7.3, §14.2.1).

### 11.3.5. Конструкторы и преобразования типов

Мы определили по три версии для каждой из четырех арифметических операций:

```
complex operator+ (complex, complex) ;
complex operator+ (complex, double) ;
complex operator+ (double, complex) ;
// . .
```

Такое дублирование довольно утомительно, а что утомительно, то чревато ошибками. А что, если было бы по три альтернативы для каждого аргумента каждой функции. Тогда потребовались бы три версии для каждой одноаргументной функции, девять версий каждой двухаргументной функции, двадцать семь версий для каждой трехаргументной функции и т.д. Часто, все эти варианты похожи друг на друга — они сначала приводят аргументы к общему типу, а потом выполняют один и тот же стандартный алгоритм.

Альтернатива такому множественному определению функций для каждой из возможных комбинаций типов аргументов может базироваться на автоматических преобразованиях аргументов. Например, наш класс **complex** имеет конструктор, который преобразовывает **double** в **complex**. В результате мы можем определить единственную версию операции сравнения:

```

bool operator== (complex, complex) ;

void f (complex x, complex y)
{
    x==y;                // означает operator==(x,y)
    x==3;                // означает operator==(x,complex(3))
    3==y;                // означает operator==(complex(3),y)
}

```

Конечно, могут существовать причины для предпочтения варианта с несколькими отдельными функциями. Например, в каких-то случаях преобразования типов аргументов могут оказаться обременительными с точки зрения производительности, а в других случаях — может применяться более простой и эффективный алгоритм для специфических аргументов. Во всех остальных случаях целесообразно предоставить один наиболее общий вариант функции, базирующийся на преобразованиях типов аргументов, плюс, возможно, несколько критических вариантов, и это предотвращает *комбинаторный взрыв вариантов*, типичный для *смешанной арифметики*.

При наличии нескольких вариантов функций (операций) компилятор должен выбрать наиболее подходящий вариант, основываясь на типах аргументов (операндов) и доступных преобразованиях (стандартных или определяемых пользователем). Если невозможно выбрать единственный наилучший вариант, то выражение трактуется как ошибочное (неоднозначное) (см. §7.4).

Объект в выражении, созданный явно или неявно конструктором, является автоматическим и будет уничтожен при первой же возможности (см. §10.4.10).

Пользовательские преобразования не применяются неявным образом к левым операндам операций `.` или `->`. Это справедливо и для случаев, когда операция `.` подразумевается (хотя явно и не пишется):

```

void g (complex z)
{
    3+z;                // ok: complex(3)+z
    3.operator+=(z) ;   // error: 3 не является объектом класса
    3+=z;               // error: 3 не является объектом класса
}

```

Таким образом, если операция в качестве левого операнда требует lvalue, то ее нужно перегрузить в классе с помощью функции-члена.

### 11.3.6. Литералы

Невозможно определить литералы пользовательских типов в том же самом смысле, в каком `1.2` или `12e3` являются литералами типа **double**. Однако вместо них можно использовать литералы встроенных типов (§4.1.1), если в классе имеются функции-члены для их интерпретации. Конструктор с единственным аргументом служит основным механизмом реализации этого подхода. Когда такие конструкторы просты и поддаются встраиванию, естественно рассматривать вызов конструкторов с литеральными аргументами в качестве литерала. Например, я рассматриваю `complex(3)` как литерал типа **complex**, хотя технически это не так.



### 11.3.7. Дополнительные функции-члены

До сих пор мы снабдили класс **complex** лишь конструкторами и арифметическими операциями. Для практической работы этого не достаточно. В частности, требуются функции, позволяющие читать вещественную и мнимую части комплексного числа:

```
class complex
{
    double re, im;

public:
    double real() const {return re;}
    double imag() const {return im;}
    // ...
};
```

Поскольку **real()** и **imag()** не модифицируют значения комплексного числа, то естественно объявить их константными.

Остальные полезные операции можно реализовать, отталкиваясь от **real()** и **imag()**, то есть не предоставляя им непосредственного доступа к внутреннему представлению класса **complex**. Например:

```
inline bool operator==(complex a, complex b)
{
    return a.real() == b.real() && a.imag() == b.imag();
}
```

Часто требуется именно читать вещественную и мнимую части комплексных чисел; перезапись этих значений требуется значительно реже. Если же частичная модификация все же нужна, то можно поступить следующим образом:

```
void f(complex& z, double d)
{
    // ...
    z = complex(z.real(), d);           // присвоить z.im значение d
}
```

Оптимизирующий компилятор может здесь сгенерировать единственную операцию присваивания.

### 11.3.8. Функции поддержки (helper functions)

Если собрать все фрагменты вместе, то наш класс **complex** примет вид:

```
class complex
{
    double re, im;

public:
    complex(double r=0, double i=0) : re(r), im(i) {}

    double real() const {return re;}
    double imag() const {return im;}
```

```

complex& operator+=(complex) ;
complex& operator+=(double) ;
// - =, *=, and /=
};

```

Дополнительно представляем набор глобально определяемых (не в качестве членов класса **complex**) функций поддержки (*helper functions*):

```

complex operator+(complex, complex) ;
complex operator+(complex, double) ;
complex operator+(double, complex) ;
// -, *, and /

complex operator-(complex) ;           // унарный минус
complex operator+(complex) ;         // унарный плюс

bool operator==( complex, complex) ;
bool operator!=( complex, complex) ;

istream& operator>>(istream&, complex&) ; // ввод
ostream& operator<<(ostream&, complex) ; // вывод

```

Отметим, что функции-члены **real**() и **imag**() важны для реализации операций сравнения. Они также важны и для иных функций поддержки.

Например, полезно определить функции для работы в полярных координатах:

```

complex polar(double rho, double theta) ;
complex conj(complex) ;

double abs(complex) ;
double arg(complex) ;
double norm(complex) ;

double real(complex) ;           // для удобства записи
double imag(complex) ;         // для удобства записи

```

И, наконец, нужно реализовать подходящий набор стандартных математических функций:

```

complex acos(complex) ;
complex asin(complex) ;
complex atan(complex) ;
// ...

```

С точки зрения пользователя представленный здесь комплексный тип почти идентичен представленному в файле **<complex>** типу **complex<double>** стандартной библиотеки (§22.5).

## 11.4. Операции приведения типов

Применение конструкторов для приведения типов удобно, но не является панацеей. Конструктор не может задавать:

1. Неявное приведение пользовательского типа во встроенный тип (так как встроенный тип не является классом).

2. Преобразование из нового класса в ранее определенный класс (без модификации старого класса).

Перечисленные проблемы решаются при помощи явно программируемых в классе *операций приведения типов* (*conversion operators*). Функция-член *X::operator T()*, где *T* есть имя типа, определяет приведение типа *X* в тип *T*. Например, пусть определен тип «шестибитовое неотрицательное целое» — *Tiny*, который можно смешивать с обычными целыми в арифметических операциях:

```
class Tiny
{
    char v;
    void assign (int i) { if(i < 0) throw Bad_range(); v=i; }

public:
    class Bad_range{};

    Tiny (int i) { assign (i); }
    Tiny& operator= (int i) { assign (i); return *this; }

    operator int () const { return v; }           // операция приведения к типу int
};
```

Диапазон допустимых значений для типа *Tiny* проверяется как при его инициализации целыми, так и в присваиваниях ему целых значений. При копировании *Tiny* проверка диапазона не нужна, и поэтому мы удовлетворяемся умолчательными вариантами копирующего конструктора и операции присваивания.

Чтобы обеспечить возможность обычных вычислений с переменными типа *Tiny*, мы определяем операцию приведения от *Tiny* к *int*: *Tiny::operator int()*. Обратите внимание на то, что тип, к которому выполняется преобразование (приведение), присутствует в имени функции-операции и *не может указываться в качестве возвращаемого значения*:

```
Tiny::operator int () const { return v; }           // правильно
int Tiny::operator int () const { return v; }       // ошибка
```

В этом отношении операции приведения типов имеют сходство с конструкторами.

Теперь, если переменные типа *Tiny* присутствуют там, где нужны переменные типа *int*, необходимое преобразование от *Tiny* к *int* выполняется *неявным образом*:

```
int main ()
{
    Tiny c1 = 2;
    Tiny c2 = 62;
    Tiny c3 = c2 - c1;    // c3 = 60
    Tiny c4 = c3;         // проверка диапазона не выполняется (нет необходимости)
    int i = c1 + c2;      // i = 64

    c1 = c1 + c2;         // выход за пределы диапазона: c1 не может равняться 64
    i = c3 - 64;          // i = -4
    c2 = c3 - 64;         // выход за пределы диапазона: c2 не может равняться -4
    c3 = c4;              // проверка диапазона не выполняется (нет необходимости)
}
```

Операции приведения особо полезны там, где чтение (реализуемое операцией приведения) тривиально, а инициализация и присваивание существенно менее тривиальны.

Классы *istream* и *ostream* полагаются на операции приведения, чтобы сделать осмысленными операторы вроде следующего:

```
while (cin>>x) cout<<x;
```

Операция ввода *cin>>x* возвращает *istream&*. Последнее неявно преобразуется к значению, отражающему состояние потока *cin*. Полученное таким образом значение может использоваться в цикле *while* (§21.3.3). Однако в общем случае, идея реализации операций приведения типа, в которых происходит потеря информации, не слишком хороша.

Как правило, лучше не переусердствовать с операциями приведения типов. При избыточном определении возможны неоднозначности. Подобные неоднозначности выявляются компилятором, но от них бывает трудно избавиться. Вероятно, лучше сначала попробовать ограничиться именованными функциями, например *X: make\_int()*. Когда такая функция становится слишком популярной, в результате чего код теряет элегантность, тогда и стоит подумать о введении операции приведения *X: operator int()*.

Когда же одновременно определяются и пользовательские операции, например *+*, и пользовательские приведения типа, то могут возникать неоднозначности следующего вида:

```
int operator+ (Tiny, Tiny) ;
void f (Tiny t, int i)
{
    t+i; // error, неоднозначность: operator+(t,Tiny(i)) или int(t)+i ?
}
```

Тогда лучше ограничиться чем-либо одним — либо пользовательской операцией сложения, либо пользовательским приведением от *Tiny* к *int*.

### 11.4.1. Неоднозначности

Присваивание значения типа *V* объекту класса *X* допускается в тех случаях, когда определена операция присваивания *X: operator= (Z)*, где *V* есть *Z*, либо имеется уникальное преобразование от *V* к *Z*. Трактовка инициализации абсолютно аналогична.

В ряде случаев значение требуемого типа может быть создано путем многократного применения конструкторов и операций приведения. Это нужно осуществлять при помощи явных преобразований; *неявное* применение пользовательских преобразований типов осуществляется лишь *однократно*. Иногда значение желаемого типа может быть создано более, чем одним способом — такое допустимо. Например:

```
class X { /* ... */ X(int) ; X(char*) ; } ;
class Y { /* ... */ Y(int) ; } ;
class Z { /* ... */ Z(X) ; } ;
```

```

X f(X) ;
Y f(Y) ;
Z g(Z) ;

void k1 ()
{
    f(I) ;           // error: неоднозначность: f(X(I)) или f(Y(I))?
    f(X(I)) ;       // ok
    f(Y(I)) ;       // ok
    g ("Mack") ;      // error: нужны два пользовательских преобразования;
                      // g(Z(X("Mack"))) not tried
    g (X ("Doc")) ;  // ok: g(Z(X("Doc")))
    g (Z ("Suzy")) ; // ok: g(Z(X("Suzy")))
}

```

Пользовательские преобразования типов рассматриваются лишь тогда, когда они необходимы для разрешения неоднозначности вызова. Например:

```

class XX { /* ... */ XX(int) ; } ;

void h (double) ;
void h (XX) ;

void k2 ()
{
    h (I) ;           // h(double(I)) или h(XX(I))? h(double(I))!
}

```

Вызов **h**(**I**) трактуется как **h**(**double**(**I**)), ибо здесь используется стандартное, а не пользовательское преобразование типа (§7.4).

Правила для преобразований и не слишком просты, и весьма сложны в документировании, и не столь общие, как можно было бы подумать. Однако их применение безопасно, а результаты не слишком непредсказуемы. Лучше применить явное разрешение неоднозначности, чем искать ошибку, вызванную неожиданным (неявным) преобразованием.

Требование строгого анализа снизу-вверх предполагает, что тип возвращаемого значения не участвует в разрешении перегрузки. Например:

```

class Quad
{
public:
    Quad (double) ;
    // ...
};

Quad operator+ (Quad, Quad) ;

void f(double a1, double a2)
{
    Quad r1 = a1+a2;           // суммирование с двойной точностью
    Quad r2 = Quad (a1) + a2; // форсируем Quad-арифметику
}

```

Причинами такого подхода являются соображения о том, что анализ снизу вверх более понятен, и что не дело компилятора решать за программиста, какая нужна точность для выполнения сложения.

Когда типы операндов инициализации или присваивания определены, они оба используются для разрешения неоднозначностей. Например:

```
class Real
{
public:
    operator double () ;
    operator int () ;
    // ...
};

void g (Real a)
{
    double d = a;           // d = a.double();
    int i = a;              // i = a.int();
    d = a;                  // d = a.double();
    i = a;                  // i = a.int();
}
```

И здесь анализ типов идет снизу-вверх: в каждом отдельном случае лишь тип операции и типы операндов используются для разрешения неоднозначности.

## 11.5. Друзья класса

Обычное объявление функции-члена имеет три логически разных последствия:

1. Функция имеет доступ к закрытой части класса.
2. Функция находится в области видимости класса.
3. Функция вызывается для объекта класса (получает указатель *this*).

Приписывая функции-члену модификатор **static** (§10.2.4), мы обеспечиваем для нее первые два свойства. А если мы припишем функции модификатор **friend**, то наделим ее одним лишь первым свойством.

Например, можно было бы определить операцию умножения объектов типа **Matrix** и **Vector**. Каждый из этих типов скрывает внутреннее представление и обеспечивает полный набор операций для манипулирования своими объектами. Наша операция умножения не может быть функцией-членом обоих классов, да и вообще, вряд ли мы захотим предоставить любым пользователям низкоуровневые возможности читать/переписывать внутреннее устройство объектов типа **Matrix** и **Vector**. Во избежание этого мы объявим операцию умножения другом обоих классов:

```
class Matrix;

class Vector
{
    float v[4];
    // ...
    friend Vector operator* (const Matrix&, const Vector&);
};
```

```

class Matrix
{
    Vector v[4];
    // ...
    friend Vector operator* (const Matrix&, const Vector&);
};

Vector operator* (const Matrix& m, const Vector& v)
{
    Vector r;
    for (int i = 0; i<4; i++)    // r[i] = m[i] * v;
    {
        r.v[i] = 0;
        for (int j = 0; j<4; j++) r.v[i] += m.v[i].v[j] * v.v[j];
    }
    return r;
}

```

Объявление функции другом можно помещать как в открытые, так и в закрытые секции класса — это *не имеет значения*. Как и функции-члены, функции-друзья явным образом объявляются в классе, друзьями которого они являются. Так что они являются частью интерфейса класса в той же мере, что и функции-члены.

Другом класса может быть объявлена и функция-член другого класса. Например:

```

class List_iterator
{
    // ...
    int* next();
};

class List
{
    friend int* List_iterator::next();
    // ...
};

```

Нередко все функции одного класса являются друзьями другого класса. Это можно объявлять короче:

```

class List
{
    friend class List_iterator;
    // ...
};

```

Такое объявление делает все функции класса **List\_iterator** друзьями класса **List**.

Ясно, что целые классы друзьями объявляют лишь в случаях, когда имеется теснейшая концептуальная связь между ними. Однако для таких случаев существует и иная альтернатива в виде определения класса *внутри* определения другого класса (*вложенный класс* — *nested class*) (см. §24.4).

### 11.5.1. Поиск друзей

Как и объявление функции-члена, объявление друга не вносит его имя в охватывающую область видимости. Например:

```
class Matrix
{
    friend class Xform;
    friend Matrix invert (const Matrix&);
    // ...
};

Xform x; // error: Xform нет в текущей области видимости
Matrix (*p) (const Matrix&) = &invert; // error: invert() нет в текущей области видимости
```

Для больших программ и больших классов это как раз хорошо, что классы не вносят «по-тихому» имена в окружающие области видимости. Это особенно важно для шаблонного класса, который может конкретизироваться в различных контекстах (глава 13).

Дружественный класс должен быть предварительно объявлен в непосредственно охватывающей области видимости, или же определен в той же самой области видимости, где определяется и класс, объявляющий его другом. Более отдаленные (внешние) охватывающие области видимости в расчет не принимаются. Например:

```
class AE{ /* ... */ }; // не друг класса Y

namespace N
{
    class X{ /* ... */ }; // друг класса Y

    class Y
    {
        friend class X;
        friend class Z;
        friend class AE;
    };

    class Z{ /* ... */ }; // друг класса Y
}
```

Дружественная функция может явно объявляться таким же образом, что и дружественный класс; или же ее можно искать по типу аргументов (§8.2.6) даже в случае, когда она объявлена вовсе не в самой близкой из охватывающих областей видимости. Например:

```
void f(Matrix& m)
{
    invert(m); // invert() - друг класса Matrix
}
```

Итак, дружественная классу функция или объявляется в непосредственно охватывающей области видимости, или имеет аргумент типа класса (или производного от него класса) (§13.6), или вообще не может быть найдена. Например:



```
// f() нет в этой области видимости
class X
{
    friend void f();           // бесполезно
    friend void h(const X&);   // можно найти по типу аргумента
};

void g(const X& x);
{
    f();                     // f() нету в области видимости
    h(x);                   // h() - друг класса X
}
```

### 11.5.2. Функции-члены или друзья?

Когда для реализации некоторой операции нужно использовать функции-члены, а когда дружественные функции? Во-первых, следует свести к минимуму набор функций, имеющих прямой доступ ко внутреннему представлению класса, и определить этот набор самым тщательным образом. Поэтому первым является не вопрос «функция-член или статическая функция или дружественная функция?», а совсем другой вопрос — «требуется ли на самом деле прямой доступ к представлению?». В типичных случаях набор функций-членов с прямым доступом значительно меньше, чем это кажется на первый взгляд.

Конечно, некоторые операции (в смысле, действия) обязаны быть членами — конструкторы, деструктор и виртуальные функции (§12.2.6), но во всех остальных случаях выбор возможен. Так как имена членов класса локальны по отношению к классу, функция, нуждающаяся в прямом доступе к представлению, обязана быть функцией-членом.

Рассмотрим класс *X*, реализующий альтернативные способы определения операций:

```
class X
{
    // ...
    X(int);

    int m1();
    int m2() const;

    friend int f1(X&);
    friend int f2(const X&);
    friend int f3(X);
};
```

Функции-члены можно вызывать только для объектов класса; никаких пользовательских приведений типа не применяется к левым операндам операций `.` и `->` (§11.10). Например:

```
void g()
{
    99.m1();           // error: X(99).m1() не рассматривается
    99.m2();           // error: X(99).m2() не рассматривается
}
```

Глобальная функция **f1()** имеет сходные свойства, так как неявные приведения типа не применяются к неконстантным аргументам, передаваемым по ссылке (§5.5, §11.3.5). Однако к аргументам функций **f2()** и **f3()** преобразования типов применяются:

```
void h ()
{
    f1(99);           // error: f1(X(99)) не рассматривается
    f2(99);           // ok: f2(X(99));
    f3(99);           // ok: f3(X(99));
}
```

Любую функцию, модифицирующую состояние классowego объекта, следует определять как функцию-член или глобальную функцию, принимающую объект класса в качестве *неконстантного* аргумента по ссылке (или *неконстантного* указателя). Операции (=, \*=, ++ и т.д.), требующие в качестве левого операнда lvalue, для пользовательских типов обычно перегружаются как члены.

И наоборот, если для аргументов (операндов) желательно неявное приведение типа, реализующая операцию функция не должна быть членом класса и должна иметь либо аргументы, передаваемые по значению, либо константные аргументы, передаваемые по ссылке. Это типично для функций, реализующих операции, не требующие lvalue в качестве левых операндов (это операции +, -, || и т.д.). В то же время, такие операции часто нуждаются в прямом доступе ко внутреннему представлению класса. Поэтому подобного рода бинарные операции чаще всего являются функциями-друзьями.

Если не определяются пользовательские операции приведения типа, то нет никаких причин отдавать предпочтение функциям-членам перед дружественными функциями со ссылочными аргументами, и наоборот. Часто программисты имеют свои собственные предпочтения. Возникает впечатление, что большинство людей предпочитает вызов **inv(m)** для инвертирования матрицы вместо вызова **m.inv()**. Однако ж, если **inv()** на самом деле инвертирует объект **m**, а не возвращает новую матрицу, равную инверсии от **m**, то тут предпочтение имеет функция-член.

При прочих равных условиях предпочитайте определять функцию-член. Невозможно предвидеть, не определит ли кто-нибудь когда-нибудь операцию приведения типа. Невозможно предсказать, не потребуют ли будущие модификации изменения состояний используемых объектов. Синтаксис вызова функций-членов делает очевидным для пользователя, что объект может быть изменен; ссылочный аргумент значительно менее очевиден. Кроме того, выражения в теле функции-члена чаще всего короче и проще, чем эквивалентные выражения в глобальных функциях — последним требуются явно задаваемые аргументы, в то время как функции-члены неявно применяют **this**. Наконец, поскольку имена членов локальны в классе, то они могут быть значительно короче, чем имена глобальных функций.

## 11.6. Объекты больших размеров

Для класса **complex** мы определили типы аргументов как **complex** (эти аргументы передаются по значению, то есть копируются). Накладные расходы на копирование двух **double** имеют определенное значение, но в любом случае это менее накладно, чем применение двух указателей (доступ по указателем относительно дорог). В то

же время, не все классы столь компактны. Для того чтобы избежать накладных расходов на копирование в таких случаях можно задать ссылочные аргументы. Например:

```
class Matrix
{
    double m [ 4 ] [ 4 ];

public :
    Matrix ( ) ;
    friend Matrix operator+ ( const Matrix&, const Matrix& ) ;
    friend Matrix operator* ( const Matrix&, const Matrix& ) ;
};
```

В этом случае использование выражений с перегруженными операциями для объектов большого размера не требует избыточного копирования. Определить операцию сложения можно следующим образом:

```
Matrix operator+ ( const Matrix& arg1, const Matrix& arg2 )
{
    Matrix sum;

    for ( int i=0; i<4; i++ )
        for ( int j=0; j<4; j++ )
            sum.m [ i ] [ j ] = arg1.m [ i ] [ j ] + arg2.m [ i ] [ j ];

    return sum;
}
```

Эта перегруженная операция принимает аргументы (операнды) по ссылке, но возвращает она значение объекта. Возврат по ссылке может показаться более эффективным:

```
class Matrix
{
    // ...
    friend Matrix& operator+ ( const Matrix&, const Matrix& ) ;
    friend Matrix& operator* ( const Matrix&, const Matrix& ) ;
};
```

Это в принципе допустимо, но вызывает проблемы с выделением памяти. Так как возврат выполняется по ссылке, то возвращаемое значение не может быть автоматическим объектом (§7.3). Так как операция чаще всего используется в выражении не один раз, то ее результат не может быть *статической* локальной переменной. Результат, скорее всего, будет размещаться в свободной памяти. В итоге, копирование возврата обойдется дешевле (с точки зрения времени выполнения, объема памяти и кода), чем выделение памяти под результат и ее последующее освобождение. И программировать это легче.

Существуют приемы, позволяющие избежать копирования результата. Самый простой — использование буфера статических объектов. Например:

```

const max_matrix_temp = 7;

Matrix& get_matrix_temp ()
{
    static int nbuf=0;
    static Matrix buf[max_matrix_temp] ;
    if (nbuf==max_matrix_temp) nbuf=0;
    return buf[nbuf++] ;
}

Matrix& operator+ (const Matrix& arg1, const Matrix& arg2)
{
    Matrix& res = get_matrix_temp () ;
    // ...
    return res ;
}

```

Теперь объект типа *Matrix* копируется только тогда, когда результат вычисления выражения присваивается чему-либо. Однако лишь небеса могут помочь вам в том случае, если потребуются более чем *max\_matrix\_temp* временных объектов.

Менее чреватая ошибками техника определяет матрицу как *descriptor* (*handle*; §25.7) к *классу-представлению* (*representation type*), содержащему действительные данные. В этом случае, матричные дескрипторы могут так управлять объектами представления, что затраты на выделение памяти и копирование будут минимальными (§11.12 и §11.14[18]). Эта техника базируется на возврате объектов, а не ссылок или указателей. Еще одна техника опирается на определение тернарных операций и их автоматический вызов для выражений вида  $a=b+c$  или  $a+b*i$  (§21.4.6.3, §22.4.7).

## 11.7. Важные операции

В общем случае, для типа *X* копирующий конструктор *X*(*const X&*) выполняет инициализацию посредством объекта того же самого класса *X*. Важно еще раз напомнить, что инициализация и присваивание — это разные операции (§10.4.4.1). Особенно это важно для классов с деструктором. Когда класс имеет нетривиальный деструктор, освобождающий, например, ранее выделенную конструктором память, скорее всего этот класс нуждается в полном наборе средств, управляющих конструированием объектов, их уничтожением и копированием:

```

class X
{
    // ...
    X(Sometype) ;           // конструктор
    X(const X&) ;           // копирующий конструктор
    X& operator= (const X&) ; // операция присваивания: очистка и копирование
    ~X() ;                 // деструктор: очистка
};

```

Объект копируется еще в трех случаях: при передаче аргумента функции, при возврате из функции, и при генерации исключений. При передаче аргумента ини-

циализируется формальный параметр. Семантика здесь полностью аналогична таковой для любой инициализации. То же самое имеет место и для возвращаемых из функций значений, и для исключений, хотя и в менее очевидной форме. Во всех этих случаях применяется копирующий конструктор. Например:

```
string g (string arg) // передача по значению (используется копирующий конструктор)
{
    return arg;          // возвращается строка (используется копирующий конструктор)
}

int main ()
{
    string s = "Newton";
    s = g (s) ;
}
```

Очевидно, что строка *s* будет содержать "*Newton*" после вызова функции *g()*. Передача копии *s* в аргумент *arg* не составляет труда — вызов конструктора класса *string* поможет это выполнить. Получение еще одной копии при выходе из *g()* требует еще одного вызова *string* (*const string&*) ; на этот раз инициализируется временная переменная (§10.4.10), которая затем присваивается переменной *s*. Часто одна из этих операций (но не обе сразу) может устраняться в процессе оптимизации.

Для класса, в котором копирующий конструктор и операция присваивания не определяются явным образом, предоставляются соответствующие умолчательные варианты от компилятора (§10.2.5). Это подразумевает, что копирующие операции не наследуются (§12.2.3).

### 11.7.1. Конструктор с модификатором *explicit*

Как мы уже знаем, конструктор с одним аргументом определяет неявное приведение типа. Для некоторых типов это является идеальным решением. Например, тип *complex* может инициализироваться типом *int*:

```
complex z = 2;           // инициализация z значением complex(2)
```

В других случаях неявное приведение типа нежелательно и может приводить к ошибкам. Например, если бы мы могли инициализировать *string* размером (имеет тип *int*), кто-нибудь написал бы:

```
string s = 'a' ;          // создается строка s с int('a') элементами
```

Это прошло бы, но вряд ли соответствовало бы настоящему намерению программиста.

Неявные приведения типа можно подавить, объявив конструктор с ключевым словом *explicit*. Такой конструктор можно вызывать только явно. В частности, там, где принципиально требуется копирующий конструктор (§11.3.4), конструктор с модификатором *explicit* вызываться неявно уже не будет. Например:

```
class String
{
    // ...
    explicit String (int n) ;      // выделить n байт
    String (const char* p) ;      // инициализировать C-строкой
}
```

```

String s1 = 'a' ;           // error: неявного преобразования char~>String nem
String s2 (10) ;           // ok: String с местом для 10 символов
String s3 = String (10) ;   // ok: String с местом для 10 символов
String s4 = "Brian" ;       // ok: s4 = String("Brian")
String s5 ( "Faulty" ) ;

void f (String) ;

String g ( )
{
    f (10) ;                 // error: неявного преобразования int->String nem
    f (String (10) ) ;
    f ("Arthur" ) ;          // ok: f(String("Arthur"))
    f (s1) ;

    String* p1 = new String ( "Eric" ) ;
    String* p2 = new String (10) ;

    return 10 ;              // error: неявного преобразования int->String nem
}

```

Может быть разница между

```

String s1 = 'a' ;           // error: неявного преобразования char~>String nem

```

и

```

String s2 (10) ;           // ok: String с местом для 10 символов

```

покажется слишком тонкой, но она более заметна в реальном коде, чем в надуманных примерах.

В классе **Date** мы использовали тип **int** для представления года (§10.3). Будь наше отношение к разработке **Date** более серьезным, мы могли бы создать класс **Year** для обеспечения более строгой проверки во время компиляции. Например:

```

class Year
{
    int y;

public:
    explicit Year (int i) : y (i) {} // создание Year из int
    operator int () const {return y;} // преобразование: Year в int
};

class Date
{
public:
    Date {int d, Month m, Year y} ;
    // ...
};

Date d3 (1978, feb, 21) ;      // error: 21 это не Year
Date d4 (21, feb, Year {1978}) ; // ok

```

Класс **Year** является простой «оберткой» («wrapper») вокруг типа **int**. Благодаря наличию определения для **operator int()** тип **Year** неявно преобразуется в **int** каждый раз, когда это необходимо. Из-за того, что мы снабдили конструктор модифи-

котором *explicit*, мы можем быть уверены, что преобразования из *int* в *Year* происходят только тогда, когда мы об этом просим *явно*, а случайные оказии вылавливаются компилятором. Функции-члены класса *Year* легко превращаются во встраиваемые функции, так что нет никаких дополнительных затрат по памяти и быстрдействию.

Аналогичную технику можно применить и к диапазонам (§25.6.1).

## 11.8. Индексирование

Функцию-операцию *operator* [] определяют с целью придать смысл операциям индексации над объектами классов. Вторым аргументом этой операции (индекс) может иметь любой тип. Это и позволяет определять такие типы стандартной библиотеки, как вектора, ассоциативные массивы и т.д.

Для иллюстрации перепишем пример из §5.5, в котором ассоциативный массив использовался для подсчета количества вхождений слова в содержимое файла. Там для этого использовалась функция. Сейчас же мы определим свой *собственный ассоциативный массив*:

```
class Assoc
{
    struct Pair
    {
        string name;
        double val;
        Pair (string n = " ", double v = 0) : name (n) , val (v) {}
    };

    vector<Pair> vec;

    Assoc (const Assoc&); // private - для предотвращения копирования
    Assoc& operator= (const Assoc&); // private - для предотвращения копирования

public:
    Assoc () {}
    const double& operator [] (const string&);
    double& operator [] (string&);
    void print_all () const;
};
```

Класс *Assoc* хранит вектор элементов типа *Pair*. Реализация использует тот же самый тривиальный и неэффективный метод поиска, что и в §5.5:

```
// поиск s; возврат ее значения в случае нахождения; в противном случае
// создаем новый Pair и возвращаем умолчательное значение 0
//
double& Assoc::operator [] (string& s)
{
    for (vector<Pair>::iterator p = vec.begin (); p != vec.end (); ++p)
        if (s == p->name) return p->val;

    vec.push_back (Pair (s, 0)); // начальное значение: 0
    return vec.back ().val; // возврат последнего элемента (§16.3.3)
```

Так как класс *Assoc* скрывает свое внутреннее представление, то требуется открытый метод для вывода его содержимого:

```
void Assoc::print_all() const
{
    for (vector<Pair>::const_iterator p = vec.begin(); p != vec.end(); ++p)
        cout << p->name << " : " << p->val << '\n';
}
```

Вот пример простого клиентского кода, использующего класс *Assoc*:

```
int main() // подсчитать кол-во вхождений каждого слова во входном потоке
{
    string buf;
    Assoc vec;

    while (cin >> buf) vec[buf]++;

    vec.print_all();
}
```

Дальнейшее развитие идеи ассоциативного массива представлено в §17.4.1.

**Операцию** [] можно перегружать только в виде функции-члена.

## 11.9. Функциональный вызов

Вызов функции, то есть выражение вида *expression(expression-list)*, интерпретируется как бинарная операция с левым операндом *expression* (имя функции или имеющее выражение) и правым операндом *expression-list* (список аргументов). Эту операцию можно перегружать, как и большинство других операций C++. При этом список аргументов для функции-операции *operator()* () вычисляется и проверяется по обычным правилам передачи аргументов. Чаще всего перегрузку операции вызова функции осуществляют для классов, имеющих единственную или доминирующую над другими операцией.

Самым очевидным и, вероятно, наиболее полезным применением перегруженной операции () является предоставление обычного синтаксиса функциональных вызовов классовым объектам, которые в некотором отношении ведут себя как функции. Объекты, которые ведут себя как функции, принято называть функциональными объектами или *объектами-функциями* (*function objects*) (§18.4)<sup>1</sup>. Объекты-функции важны, так как позволяют писать код, принимающий нетривиальные операции в качестве параметров. Например, в стандартной библиотеке реализовано множество алгоритмов, которые вызывают функцию для каждого элемента контейнера. Рассмотрим пример:

```
void negate(complex& c) { c = -c; }

void f(vector<complex>& aa, list<complex>& ll)
{
    for_each(aa.begin(), aa.end(), negate); // поменять знак у всех эл-ов вектора
    for_each(ll.begin(), ll.end(), negate); // поменять знак у всех эл-ов списка
}
```

<sup>1</sup> Также их часто называют *функторами* (*functors*). — Прим. ред.



Этот код предназначен для изменения знака (на противоположный) у каждого элемента вектора и списка.

А что, если нам потребуется добавить *complex(2, 3)* к каждому элементу? Это легко можно сделать следующим образом:

```
void add23 (complex& c)
{
    c += complex (2, 3) ;
}

void g (vector<complex>& aa, list<complex>& ll)
{
    for_each (aa.begin () , aa.end () , add23) ;
    for_each (ll.begin () , ll.end () , add23) ;
}
```

А как написать функцию, прибавляющую к каждому элементу произвольное комплексное значение? В этом случае нам потребуется нечто, чему мы можем передать произвольное значение, и что затем будет использовать это значение при каждом вызове. Это невозможно проделать с функциями, оставаясь лишь в контексте функций. Требуется передавать наше значение в некоторый контекст, окружающий функцию. Все это довольно запутанно. В то же время, нетрудно написать класс, демонстрирующий указанное поведение:

```
class Add
{
    complex val;

public:
    Add (complex c) {val=c; } // сохраняем значение
    Add (double r, double i) {val=complex (r, i) ; }

    void operator () (complex& c) const {c+=val; } // прибавляем значение к аргументу
};
```

Объект класса *Add* инициализируются произвольно задаваемым комплексным числом, а когда затем к этому объекту обращаются с помощью операции *()*, он прибавляет комплексное число к аргументу. Например:

```
void h (vector<complex>& aa, list<complex>& ll, complex z)
{
    for_each (aa.begin () , aa.end () , Add (2, 3) ) ;
    for_each (ll.begin () , ll.end () , Add (z) ) ;
}
```

Здесь комплексное значение *complex(2, 3)* будет добавлено к каждому элементу вектора, и комплексное число *z* будет добавлено к каждому элементу списка. Заметьте, что с помощью выражения *Add(z)* конструируется объект класса *Add*, который затем многократно используется алгоритмом *for\_each()*; *Add(z)* не является функцией, вызываемой однажды или повторно. Повторно вызывается функция-член *operator ()* класса *Add*.

Все это работает потому еще, что *for\_each* является функциональным шаблоном, применяющим операцию *()* к своему третьему аргументу, даже не заботясь о том, чем он является в действительности:

```
template<class Iter, class Fct> Fct for_each (Iter b, Iter e, Fct f)
{
    while (b != e) f(*b++);
    return f;
}
```

На первый взгляд может показаться, что такая техника является чем-то эзотерическим, но на самом деле все это не так уж и сложно, весьма эффективно и чрезвычайно полезно (см. §3.8.5, §18.4).

Другими популярными задачами для применения `operator()` являются задача выделения подстроки и задача индексирования многомерных массивов (§22.4.5).

Функция `operator()` может быть *только функцией-членом*.

## 11.10. Разыменование

Операция разыменования `->` *перегружается в классах как унарная и постфиксная*:

```
class Ptr
{
    // ...
    X* operator->();
};
```

Объектами класса `Ptr` можно пользоваться для доступа к членам класса `X` точно так же, как обычными указателями. Например:

```
void f (Ptr p)
{
    p->m = 7;    // (p.operator->())->m = 7
}
```

Переход от объекта `p` к возвращаемому вызовом `p.operator->()` указателю не зависит от указываемого члена `m`. В этом смысле операция разыменования `->`, перегружаемая с помощью функции `operator->()`, является *унарной* (и *постфиксной*). При этом никакого нового синтаксиса не вводится, так что по-прежнему после знака операции нужно указать имя адресуемого операцией члена. Например:

```
void g (Ptr p)
{
    X* q1 = p->;           // синтаксическая ошибка
    X* q2 = p.operator->(); // ok
}
```

Перегрузка операции `->` используется преимущественно для построения так называемых *интеллектуальных указателей* (*smart pointers*), то есть классовых объектов, которые ведут себя как указатели и дополнительно выполняют некоторые действия, когда через них осуществляется доступ к целевому объекту. Например, можно определить класс `Rec_ptr` для доступа к объектам класса `Rec`, хранящимся на диске. Конструктор класса `Rec_ptr` принимает имя, которое используется для доступа к объекту на диске, `Rec_ptr::operator->()` извлекает объект с диска

в оперативную память, а деструктор записывает измененное значение объекта на диск:

```
class Rec_ptr
{
    const char* identifier;
    Rec* in_core_address;
    // ...

public:
    Rec_ptr(const char* p) : identifier(p), in_core_address(0) {}
    ~Rec_ptr() { write_to_disk(in_core_address, identifier); }

    Rec* operator->() ;
};

Rec* Rec_ptr::operator->()
{
    if(in_core_address==0) in_core_address=read_from_disk(identifier);
    return in_core_address;
}
```

Rec\_ptr можно использовать следующим образом:

```
struct Rec          // Rec, на который указывает Rec_ptr
{
    string name;
    // ...
};

void update(const char* s)
{
    Rec_ptr p(s);          // Rec_ptr для s
    p->name="Roscoe";      // обновить s (при необходимости считать с диска)
    // ...
}
```

В реальной жизни **Rec\_ptr** был бы шаблоном, а тип **Rec** был бы его параметром. Кроме того, реальная программа обязана обрабатывать ошибочные ситуации, а ее взаимодействие с диском было бы не столь наивным, как в нашем учебном примере.

Для обычных указателей языка C++ применение операции **->** синонимично комбинации операций **.** и **\***. Пусть для типа **Y** операции **->**, **.**, **\*** и **[]** имеют стандартный смысл, и указатель **p** имеет тип **Y\***. Тогда справедливы следующие равенства:

```
p->m== (*p) . m          // истина
(*p) . m==p[0] . m      // истина
p->m==p[0] . m          // истина
```

Как обычно, для пользовательских операций таких автоматических гарантий нет. Необходимую эквивалентность операций нужно программировать самому:

```
class Ptr_to_Y
{
    Y* p;
```

*public:*

```
Y* operator->() {return p;}
Y& operator*() {return *p;}
Y& operator[](int i) {return p[i];}
};
```

Действительно, если вы предоставляете для пользовательского типа более одной подобной операции, будет разумно обеспечить ожидаемую пользователями эквивалентность точно так же, как бывает полезно обеспечить эквивалентность  $++x$ ,  $x+=1$  и  $x=x+1$  для объекта  $x$  класса, перегружающего операции  $++$ ,  $+=$ ,  $=$  и  $+$ .

Перегрузка операции  $\rightarrow$  представляет интерес для целого круга задач. Дело в том, что *косвенные обращения (indirection)* составляют важную теоретическую концепцию, а в программировании на C++ операция  $\rightarrow$  реализует эту концепцию самым прямым и эффективным образом. Итераторы (глава 19) служат еще одним примером реализации концепции косвенных обращений. Еще один взгляд на перегрузку операции  $\rightarrow$  состоит в том, что с ее помощью реализуется одна из форм концепции *делегирования (delegation)*; §24.3.6).

Операция  $\rightarrow$  может перегружаться *только в виде функции-члена*. Типом *возвращаемого* ею значения может быть либо *указатель*, либо *объект класса*, *перегружающего операцию*  $\rightarrow$ .

### 11.11. Инкремент и декремент

Если определены «интеллектуальные указатели», то стоит дополнить их перегрузкой сопутствующих традиционным указателям операций  $++$  (инкремент) и  $--$  (декремент). Особенно это важно в случаях, когда интеллектуальные указатели предназначены для замены указателей встроенных. К примеру, рассмотрим следующую традиционную программу со свойственными ей недостатками:

```
void f1(T a)
{
    T v[200];
    T* p = &v[0];

    p--;
    *p=a;           // Oops: p вне корректного диапазона (и это не перехватывается)

    ++p;
    *p=;           // ok
}
```

Для преодоления проблем с традиционными указателями мы могли бы заменить указатель  $p$  объектом класса *Ptr\_to\_T*, разыменовывать который можно лишь в тех случаях, когда он на самом деле указывает на целевой объект. Также было бы неплохо, если операции инкремента и декремента можно было применять к нему только тогда, когда указуемый объект находится в составе массива объектов. То есть нам требуется что-то вроде следующего:

```
class Ptr_to_T
{
```

```

void f2 (T a)
{
    T v[200];
    Ptr_to_T p (&v[0], v, 200);

    p--;
    *p c = a;      // ошибка времени выполнения (run-time error): p вне диапазона

    ++p;
    *p = a;        // ok
}

```

Операции инкремента и декремента уникальны в том отношении, что они могут применяться префиксно и постфиксно. Отсюда следует, что мы должны перегрузить оба варианта для класса **Ptr\_to\_T**. Например:

```

class Ptr_to_T
{
    T* p;
    T* array;
    int size;

public:
    Ptr_to_T (T* p, T* v, int s); // привязка к массиву v размера s; нач. значение p
    Ptr_to_T (T* p);             // привязка к объекту; начальное значение p

    Ptr_to_T& operator++ ();      // префиксный
    Ptr_to_T operator++ (int);    // постфиксный

    Ptr_to_T& operator-- ();      // префиксный
    Ptr_to_T operator-- (int);    // постфиксный

    T& operator* ();             // префиксная операция
};

```

Аргумент типа **int** используется для указания на то, что функция должна вызываться для постфиксной формы операции инкремента/декремента. Причем *величина* аргумента не играет никакой роли; этот аргумент служит лишь тупым указанием на форму операции. Легко запомнить, для какой именно формы операций инкремента/декремента не нужен фиктивный аргумент — он не нужен для обычной префиксной формы, как он не нужен и для остальных обычных унарных арифметических и логических операций, а нужен он лишь для «странных» постфиксных ++ и --.

Приведем пример использования последнего варианта класса **Ptr\_to\_T**.

```

void f3 (T a)
{
    T v[200];
    Ptr_to_T p (&v[0], v, 200);

    P operator-- (0);
    p.operator* () = a; // ошибка времени выполнения (run-time error): p вне диапазона

    p.operator++ ();
    p.operator* () = a; // ok
}

```

Завершение класса *Ptr\_to\_T* оставлено в качестве самостоятельного упражнения (§11.14[19]). Его дальнейшее превращение в шаблон с генерацией исключений для сообщений об ошибках составляет задачу еще одного упражнения (§14.12[2]). Работа операций ++ и -- над итераторами рассматривается в §19.3. «Шаблонные указатели», корректно ведущие себя по отношению к наследованию, рассматриваются в §13.6.3.

## 11.12. Класс строк

В данном разделе представлена достаточно реалистичная версия строкового класса *String*. Это минимальная версия, которая меня устраивает. Она обеспечивает семантику значений, операции посимвольного чтения и записи, контролируемый и неконтролируемый доступ, потоковый ввод/вывод, операции сравнения и конкатенации. Класс хранит строку в виде массива символов с терминальным нулем и использует *счетчик ссылок (reference count)* для минимизации копирования. Дальнейшее усовершенствование этого класса является полезным упражнением (§11.14[7–12]). А затем мы можем выбросить все наши упражнения и использовать класс строк из стандартной библиотеки (глава 20).

Мой почти настоящий класс *String* опирается на три вспомогательных класса: класс *Srep*, который позволяет разделять истинное представление между несколькими объектами типа *String*, имеющими одинаковое значение; класс *Range*, объекты которого генерируются в виде исключений с сообщениями о выходе из допустимого диапазона; класс *Cref* — помогающий в реализации операции индексирования, различающей чтение и запись:

```
class String
{
    struct Srep;           // представление (representation)
    Srep* rep;
    class Cref;           // ссылка на char

public:
    class Range { };      // для исключений
    // ...
};
```

Как и остальные члены класса, *вложенный класс (nested class)* может быть внутри обьемлющего класса лишь объявлен, а определен позже:

```
struct String::Srep
{
    char* s;              // указатель на элементы
    int sz;               // кол-во символов
    int n;               // счетчик ссылок

    Srep(int nsz, const char* p)
    {
        n = 1;
        sz = nsz;
        s = new char[sz+1]; // плюс терминальный нуль
    }
};
```

```

    strcpy (s, p) ;
}

~Srep () { delete [] s; }

Srep* get_own_copy ()           // клонировать при необходимости
{
    if (n==1) return this;
    n--;
    return new Srep (sz, s) ;
}

void assign (int nsz, const char* p)
{
    if (sz!=nsz)
    {
        delete [] s;
        sz = nsz;
        s = new char [sz+1] ;
    }
    strcpy (s, p) ;
}

private:                       // предотвращаем копирование:
Srep (const Srep&) ;
Srep& operator= (const Srep&) ;
};

```

Класс **String** обеспечивает обычный набор конструкторов, деструктор и операции присваивания:

```

class String
{
    // ...
    String () ;                // x = ""
    String (const char*) ;     // x = "abc"
    String (const String&) ;    // x = other_string
    String& operator= (const char*) ;
    String& operator= (const String&) ;
    ~String () ;
    // ...
};

```

Класс **String** реализует семантику значений. То есть после присваивания  $s1=s2$  строки  $s1$  и  $s2$  абсолютно различимы и последующие изменения одной из них никак не затрагивают другую строку. Альтернативой является семантика указателей, при которой после  $s1=s2$  изменение  $s1$  влекло бы за собой и изменение  $s2$ . Для типов с обычными арифметическими операциями, таких как комплексные числа, вектора, матрицы и строки, я предпочитаю семантику значений. Однако чтобы обеспечить семантику значений, класс **String** реализован в виде дескриптора (*handle*) к классу представления (*representation class*), которое копируется лишь при необходимости:

```
String : String ()
```

```

    rep = new Srep (0, "");      // умолчательное значение - пустая строка
}

String::String (const String& x)
{
    x.rep->n++;
    rep = x.rep;                // разделяемое представление
}

String::~String ()
{
    if (--rep->n==0) delete rep;
}

String& String::operator= (const String& x)
{
    x.rep->n++;                  // защита от "st=st"
    if (--rep->n==0) delete rep;
    rep = x.rep;                // разделяемое представление
    return *this;
}

```

Операции псевдокопирования с аргументами типа **const char\*** позволяют работать со строковыми литералами:

```

String::String (const char* s)
{
    rep = new Srep (strlen (s) , s);
}

String& String::operator= (const char* s)
{
    if (rep->n==1)
        rep->assign (strlen (s) , s);    // используем старый Srep
    else
    {
        rep->n--;
        rep = new Srep (strlen (s) , s); //используем новый Srep
    }
    return *this;
}

```

Проектирование операций доступа к строкам является непростой материей, поскольку в идеале хотелось бы доступа в привычной форме (операцией `[]`), максимально эффективного и с проверкой диапазона индексов. К сожалению, невозможно обеспечить все эти свойства одновременно. Мой выбор таков: эффективные операции без проверки диапазона со слегка неудобной формой записи плюс менее эффективные операции с проверкой диапазона и со стандартной формой записи:

```

class String
{
    // ...

    void check (int i) const {if (i<0 || rep->sz<=i) throw Rangee ();}
}

```



```

char read(int i) const {return rep->s[i];}
void write(int i, char c) {rep=rep->get_own_copy(); rep->s[i]=c;}

Cref operator[] (int i) {check(i); return Cref(*this,i);}
char operator[] (int i) const {check(i); return rep->s[i];}

int size() const {return rep->sz;}
// .
};

```

Идея состоит в том, чтобы использовать операцию `[]` с проверкой диапазона в общем случае, но предоставить пользователю оптимизированную возможность однократной проверки диапазона для серии обращений. Например:

```

int hash(const String& s)
{
    int h = s.read(0);
    const int max = s.size();

    for(int i = 1; i < max; i++) h ^= s.read(i) >> 1; // обращение к s без проверки
    return h;
}

```

Определять работу операции `[]` для чтения и записи сложно в тех случаях, когда не допустимо просто вернуть ссылку и позволить делать пользователю, что ему вздумается. В нашем случае такое решение не подходит, поскольку я определил класс **String** таким образом, что между несколькими экземплярами класса **String**, которые присваивались, передавались в качестве аргументов и т.д., представление разделяется до тех пор, пока не потребуется фактическая запись. Тогда, и только тогда, представление копируется. Этот прием называют *копированием по фактической записи (copy-on-write)*. Само копирование выполняется функцией **String::Srep::get\_own\_copy()**.

Чтобы можно было функции доступа сделать встраиваемыми, нужно помещать их определения в области видимости **Srep**, что в свою очередь можно обеспечить либо помещением определения **Srep** внутри определения **String**, либо определением функций доступа с модификатором **inline** вне **String** и после **String::Srep** (§11.14[2]).

Чтобы различать чтение и запись, **String::operator[]()** возвращает **Cref**, если он вызван для неконстантного объекта. Тип **Cref** ведет себя как **char&**, за исключением того, что он вызывает **String::write()** для операции присваивания:

```

class String::Cref // ссылка на s[i]
{
    friend class String;

    String& s;
    int i;

    Cref(String& ss, int ii) : s(ss), i(ii) {}
    Cref(const Cref& r) : s(r.s), i(r.i) {}
    Cref(); // не определяется (не используется)

public:
    operator char() const { s.check(i); return s.read(i); } // выдает значение

```

```
void operator= (char c) { s.write(i, c); }           // изменяет значение
};
```

Например:

```
void f(String s, const String& r)
{
    char c1=s[1];    // c1=s.operator[](1).operator char()
    s[1] = 'c';      // s.operator[](1).operator=('c')

    char c2 = r[1];  // c2 = r.operator[](1)
    r[1] = 'd';      // error: присваивание для non-lvalue char, r.operator[](1) = 'd'
}
```

Обратите внимание на то, что для неконстантного объекта *s.operator[] (1)* есть *Cref(s, 1)*.

Для полноты я снабдил класс *String* набором полезных функций:

```
class String
{
    // ...
    String& operator+= (const String&);
    String& operator+= (const char*);

    friend ostream& operator<< (ostream&, const String&);
    friend istream& operator>> (istream&, String&);

    friend bool operator== (const String& x, const char* s)
    { return strcmp(x.rep->s, s) == 0; }

    friend bool operator== (const String& x, const String& y)
    { return strcmp(x.rep->s, y.rep->s) == 0; }

    friend bool operator!= (const String& x, const char* s)
    { return strcmp(x.rep->s, s) != 0; }

    friend bool operator!= (const String& x, const String& y)
    { return strcmp(x.rep->s, y.rep->s) != 0; }
};

String operator+ (const String&, const String&);
String operator+ (const String&, const char*);
```

Операции ввода/вывода я оставляю в качестве упражнения.

Вот пример простого клиентского кода, использующего операции класса *String*:

```
String f(String a, String b)
{
    a[2] = 'x';
    char c = b[3];
    cout<< "in f: "<< a << ' ' << b << ' ' << c << '\n';
    return b;
}

int main ()
{
    String x, y;
```

```

cout<<"Please enter two strings\n";
cin >> x >> y;
cout << "input: " << x << ' ' << y << '\n';

String z = x;
y = f(x,y) ;
if(x!= z) cout<< "x corrupted!\n";

x[0] = '!';
if(x == z) cout<< "write failed!\n";
cout<<< "exit: " << x << ' ' << y << ' ' << z << '\n';
}

```

В классе *String* вы можете обнаружить отсутствие многих возможностей, которые считаете важными или даже необходимыми. Например, вам может не хватать возможности представления содержимого в виде C-строки (§11.14[10], глава 20).

## 11.13. Советы

1. Перегружайте операции для имитации традиционных форм записи; §11.1.
2. Операнды больших размеров передавайте как *константные* ссылки; §11.6.
3. Для возвратов большого размера рассмотрите возможность оптимизации; §11.6.
4. Предпочитайте умолчательное копирование от компилятора, если оно подходит для вашего класса; §11.3.4.
5. Переопределите или запретите умолчательное копирование от компилятора, если оно не подходит для вашего класса; §11.2.2.
6. Предпочитайте функции-члены, если требуется прямой доступ к представлению класса; §11.5.2.
7. Предпочитайте функциям-членам глобальные функции, если не требуется прямой доступ к представлению класса; §11.5.2.
8. Используйте пространства имен для связывания класса с функциями поддержки; §11.2.4.
9. Используйте глобальные функции для реализации симметричных операций; §11.3.2.
10. Для индексации многомерных массивов используйте операцию (); §11.9.
11. Снабжайте конструктор с одним аргументом для задания размера модификатором *explicit*; §11.7.1.
12. В обычных случаях предпочитайте стандартный класс *string* (глава 20) своим собственным разработкам; §11.12.
13. Будьте осторожны, определяя неявные приведения типа; §11.4.
14. Для реализации операций, требующих lvalue в качестве левого операнда, используйте функции-члены; §11.3.5.

## 11.14. Упражнения

1. (\*2) В следующей программе, какие преобразования выполняются в каждом выражении?

```

struct X
{
    int i;
    X(int) ;
    X operator+ (int) ;
};

struct Y
{
    int i;
    Y(X) ;
    Y operator+ (X) ;
    operator int () ;
};

extern X operator* (X, Y) ;
extern int f(X) ;

X x = 1;
Y y = x;
int i = 2;

int main ()
{
    i+10;      y+10;      y+10*y;
    x+y+i;     x*x+i;     f(7) ;
    f(y) ;     y+y;       106+y;
}

```

Модифицируйте программу так, чтобы при запуске она выводила значения допустимых выражений.

2. (\*2) Завершите и протестируйте класс **String** из §11.12.
3. (\*2) Определите класс **INT**, который ведет себя в точности как **int**. Подсказка: определите **INT: :operator int()**.
4. (\*1) Определите класс **RINT**, который ведет себя как **int**, за исключением того, что допустимыми операциями являются лишь +, - (унарные и бинарные), \*, / и %. Подсказка: не определяйте **RINT: :operator int()**.
5. (\*3) Определите класс **LINT**, который ведет себя как **RINT**, но для представления чисел использует по крайней мере 64 бита.
6. (\*4) Определите класс с арифметикой произвольной точности. Протестируйте этот класс, вычислив факториал от **1000**. Подсказка: вам потребуется управление памятью вроде того, что было сделано для класса **String**.
7. (\*2) Для класса **String** определите внешний итератор:

```

class String_iter           // ссылается на строку и элемент строки
{
public:

```

```

String_iter (String& s) ;    // итератор для s
char& next () ;             // ссылка на следующий элемент
// иные операции по вашему выбору
};

```

Сравните это по удобству, стилю программирования и эффективности с внутренним итератором для **String** (в смысле понятия текущего элемента **String** и операций над этим элементом).

8. (\*1.5) Для строкового класса определите операцию выделения подстроки с помощью (). Какие еще операции вам нужны для работы со строками?
9. (\*3) Разработайте класс **String** таким образом, чтобы операцию взятия подстроки можно было использовать в левой части операции присваивания. Сначала ограничьтесь версией, где длины подстроки и присваиваемой строки одинаковые. Затем напишите версию с разными длинами.
10. (\*2) Напишите операцию для класса **String**, позволяющую получить содержимое в виде C-строки. Обсудите все за и против способа реализации этой операции в виде операции приведения типа. Обсудите возможные альтернативы выделения памяти под C-строку.
11. (\*2.5) Реализуйте простой механизм регулярных выражений для поиска в строках типа **String**.
12. (\*1.5) Модифицируйте механизм поиска из §11.14[11] для работы со стандартным классом **string**. Заметьте, что вы не можете модифицировать класс **string**.
13. (\*2) Напишите программу, которая совершенно нечитабельна из-за применения макросов и определения пользовательских операций. Идея: определите операцию + так, чтобы она означала – (и наоборот) для **INT**, а затем макросом определите **INT** как **int**. Переопределите популярные функции, чтобы они принимали ссылочные аргументы. Ряд запутывающих комментариев окончательно «украсят» ваше произведение.
14. (\*3) Передайте результат §11.14[13] вашему другу. Посмотрите за тем, как ваш друг будет разбираться со смыслом программы. В конце концов, вы поймете, чего вам следует избегать.
15. (\*2) Определите тип **Vec4** как вектор из четырех элементов типа **float**. Определите операцию [] для **Vec4**. Определите операции +, –, \*, /, =, +=, -=, \*= и /= для комбинации операндов типа **Vec4** и типа **float**.
16. (\*3) Определите класс **Mat4** как вектор четырех элементов типа **Vec4**. Определите операцию [] для типа **Mat4** так, чтобы она возвращала **Vec4**. Определите обычные матричные операции для **Mat4**. Определите функцию, реализующую метод исключений Гаусса.
17. (\*2) Определите класс **Vector**, аналогичный **Vec4**, но с размером, задаваемым конструктором **Vector::Vector(int)**.
18. (\*3) Определите класс **Matrix**, аналогичный **Mat4**, но с размерами, задаваемыми конструктором **Matrix::Matrix(int, int)**.

19. (\*2) Завершите класс *Ptr\_to\_T* из §11.11 и протестируйте его. По крайней мере определите для этого класса операции  $*$ ,  $->$ ,  $=$ ,  $++$  и  $--$ . Сделайте так, чтобы ошибка времени выполнения возникала только при попытке разыменования недействительных ссылок.

20. (\*1) При наличии определений

```
struct S {int x, y;};  
struct T {char* p; char* q;};
```

дайте определение класса *C*, позволяющего использовать  $x$  и  $p$  из *S* и *T* примерно так же, как если бы они были членами *C*.

21. (\*1.5) Определите класс *Index* для хранения индекса функции возведения в степень *mypow(double, Index)*. Найдите возможность для  $2^{**I}$  отрабатывать в виде вызова *mypow(2, I)*.
22. (\*2) Определите класс *Imaginary* для представления мнимых чисел. Определите класс *Complex* на базе класса *Imaginary*. Реализуйте основные арифметические операции.

# Наследование классов

*Не множьте объекты без необходимости.*

— В. Оккам

Концепции и классы — производные классы — функции-члены — конструирование и уничтожение — иерархии классов — поля типа — виртуальные функции — абстрактные классы — традиционные иерархии классов — абстрактные классы как интерфейсы — локализация создания объекта — абстрактные классы и иерархии классов — советы — упражнения.

## 12.1. Введение

Из языка Simula язык C++ позаимствовал концепцию класса как пользовательского типа и концепцию классовых иерархий. Дополнительно он заимствовал фундаментальную идею о том, что классы моделируют концепции из мира программ и из реального мира. Язык C++ содержит конструкции, которые непосредственно поддерживают концепции проектирования. Применение таких языковых конструкций с целью прямого воплощения дизайнерских идей свидетельствует об эффективном использовании C++. В то же время, если эти конструкции используются в качестве необязательных подпорок для традиционных стилей программирования, то это не самый эффективный способ программирования на C++.

Никакие концепции не существуют в абсолютной изоляции. Они сосуществуют и взаимодействуют друг с другом, и в этом проявляется вся их мощь. Попробуем выяснить, что такое автомобиль. Скоро у нас появятся понятия колес, двигателей, водителей, пешеходов, грузовиков, скорой помощи, дорог, масла, мотелей и т.д. Поскольку мы хотим использовать классы для представления концепций, то мы неизбежно сталкиваемся с вопросом о том, как можно программным способом отразить связи между концепциями? Ясно, что мы не можем непосредственно в языке отразить какие угодно абстрактные связи. Если бы даже могли, то вряд ли бы захотели это делать. Наши классы должны быть уже реальными концепциями и должны быть более точно определены. Понятие производного класса и реализующий его

языковой механизм наследования классов призваны выразить понятие общности типов. Например, понятия круга и треугольника имеют общее в том отношении, что оба они суть фигуры; это значит, что понятие фигуры является общим для них понятием. Таким образом, нам нужно явным образом указать, что то общее, что есть у классов *Circle* (круг) и *Triangle* (треугольник), сосредоточено в классе *Shape* (фигура). Если же мы будем манипулировать лишь кругами и треугольниками без привлечения более общего понятия формы, то мы упустим что-то весьма существенное. В данной главе подробно изучаются смысл и значение этой простой идеи, лежащей в основе так называемого объектно-ориентированного программирования.

Соответствующие средства языка и приемы программирования изучаются постепенно в направлении от простого и конкретного к более изощренному и абстрактному. Для многих программистов это будет движение от хорошо знакомого к существенно менее известному. Это нельзя назвать движением от «старых плохих приемов программирования» к «единственно правильному стилю». Когда я указываю на ограничения одного метода в качестве мотивации для перехода к другому, я всегда это делаю в контексте некоторой задачи; для другой задачи и в ином контексте первый метод может снова стать наилучшим. Множество полезных программ было написано с применением всех рассматриваемых здесь приемов и методов. Главная цель состоит в том, чтобы помочь вам понять самую суть этих методов, дабы вы могли осознанно и со знанием дела осуществлять их выбор в контексте реальных задач.

Сначала я рассматриваю основные средства языка, поддерживающие объектно-ориентированное программирование. Затем эти средства будут использованы для хорошо структурированной разработки довольно большого практического примера. Иные средства поддержки объектно-ориентированного программирования, такие как множественное наследование и динамическая идентификация типов на этапе выполнения (run-time type identification), рассматриваются в главе 15.

## 12.2. Производные классы

Рассмотрим программу для обработки информации о сотрудниках (employees) некоторой фирмы. Такая программа может содержать следующую структуру данных по сотрудникам фирмы:

```
struct Employee
{
    string first_name, family_name;
    char middle_initial;
    Date hiring_date;
    short department;
    // ...
};
```

Теперь попытаемся определить структуру данных для менеджеров компании:

```
struct Manager
{
    Employee emp;           // общие сведения о менеджере (как о сотруднике вообще)
```



```
list<Employee*> group; // подчиненные
short level;
// ...
};
```

Менеджеры также являются сотрудниками; соответствующие данные хранятся в поле **emp** класса **Manager**. Для читателя программы это может быть вполне ясно, но компилятору и иным программным средствам ничто не говорит о том, что **Manager** является в то же самое время и **Employee**. Указатель **Manager\*** имеет тип, отличный от указателя **Employee\***, так что нельзя просто так использовать один из них там, где требуется другой. В частности, нельзя поместить объект типа **Manager** в список объектов типа **Employee**, не написав для этого специальный код. Можно, например, применить явное преобразование типа к **Manager\***, либо поместить в указанный список адрес члена **emp**. Оба этих решения неэлегантны и могут запутать суть дела. Правильный подход — явно высказать утверждение, что **Manager** есть в то же время и **Employee**:

```
struct Manager : public Employee
{
    set<Employee*> group;
    short level;
    // ...
};
```

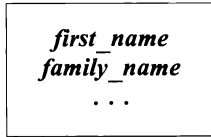
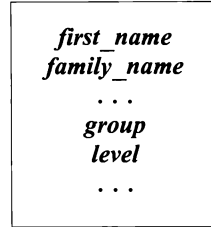
Здесь класс **Manager** указан как *производный* (*derived*) от класса **Employee**, который, в свою очередь, является, тем самым, *базовым классом* (*base class*) для класса **Manager**. В результате этого определения класс **Manager** имеет все члены класс **Employee** (**first\_name**, **department** и т.д.) в дополнение к своим собственным членам (**group**, **level** и т.д.).

Рассмотренный только что механизм *наследования классов* (*classes inheritance*) графически отображается с помощью *стрелки, направленной от производного класса к базовому*, и показывающей, что именно *производный класс ссылается на базовый* (а не наоборот):



Часто говорят, что производный класс наследует свойства базового, и именно поэтому такая связь классов называется наследованием. Иногда базовый класс называют *суперклассом* (*superclass*), а производный класс — *подклассом* (*subclass*). Эта терминология часто смущает людей, которые понимают, что данные в объектах производного класса являются надмножеством данных объектов базового класса. Производный класс шире своего базового класса в том смысле, что он содержит больше данных и функций.

Популярная трактовка механизма наследования представляет объект производного класса как объект базового класса с дополнительной информацией, специфичной лишь для производного класса, добавленной в конец. Например:

*Employee:**Manager:*

Отсюда вытекает, что объекты типа **Manager** представляют собой подтип **Employee**, ибо их можно использовать всюду, где допустимы **Employee**. Например, теперь мы можем составить список сотрудников, некоторые из которых являются менеджерами:

```

void f(Manager m1, Employee e1)
{
    list<Employee*> elist;

    elist.push_front(&m1);
    elist.push_front(&e1);
    // ...
}
  
```

Так как объект типа **Manager** является в то же время и **Employee**, то можно использовать **Manager\*** и как **Employee\***. Обратное же необязательно верно — не каждый сотрудник является менеджером, так что **Employee\*** нельзя использовать вместо **Manager\***. В общем, если класс **Derived** является открытым наследником базового класса (§15.3) **Base**, то переменную типа **Derived\*** можно присвоить переменной типа **Base\*** без необходимости в явном приведении типа. Обратное же присваивание требует явного преобразования типов. Например:

```

void g(Manager mm, Employee ee)
{
    Employee* pe = &mm;           // ok: каждый Manager есть Employee
    Manager* pm = &ee;             // error: не каждый Employee есть Manager

    pm->level = 2;                  // катастрофа: у объекта ee нету поля level

    pm=static_cast<Manager*>(pe);   // грубая сила работает, т.к. pe указывает на mm (Manager)
    pm->level = 2;                  // ok: pm указывает на mm, у которого есть level
}
  
```

Другими словами, с объектом производного класса можно обращаться как с объектом базового класса, если манипулировать им через указатели или ссылки. Обратное же неверно. Применение операций приведения **static\_cast** и **dynamic\_cast** обсуждается в §15.4.2.

Использование класса в качестве базового эквивалентно объявлению (неименованного) объекта этого класса. Следовательно, чтобы класс можно было использовать в качестве базового, он должен быть определен (§5.7):

```

class Employee;                // лишь объявление (не определение)
class Manager : public Employee // error: Employee не определен
{
    // ...
};

```

### 12.2.1. Функции-члены

Столь простые структуры данных, как представленные выше *Employee* и *Manager*, не слишком интересны и не слишком полезны. Нам нужно представить информацию с помощью надлежащего типа, предоставляющего все необходимые операции, и при этом не связываться с деталями конкретных представлений. Например:

```

class Employee
{
    string first_name, family_name;
    char middle_initial;
    // ...
public:
    void print () const;
    string full_name () const
    { return first_name+' '+middle_initial+' '+family_name; }
    // ...
};

class Manager : public Employee
{
    // ...
public:
    void print () const;
    // ...
};

```

Любой член производного класса может обращаться к любому открытому (а также защищенному — см. §15.3) члену базового класса, как будто бы последние были непосредственно объявлены в производном классе. Например:

```

void Manager::print () const
{
    cout<< "name is" << full_name () << '\n';
    // ...
}

```

Закрытые же члены базового класса напрямую не доступны в производном классе:

```

void Manager::print () const
{
    cout<< "name is" << family_name << '\n'; // error!
    // ...
}

```

Эта вторая версия *Manager::print()* компилироваться не будет. Члены производного класса не имеют специального разрешения на доступ к закрытым членам

базового класса, поэтому к *family\_name* нельзя обращаться в теле функции *Manager::print()*.

Для некоторых это является сюрпризом, но рассмотрим альтернативу: функция-член производного класса может иметь прямой доступ к закрытым членам базового класса. Концепция закрытых членов при этом стала бы бессмысленной, поскольку для доступа к закрытой части класса программист должен был бы всего лишь написать производный от него класс. Кроме того, чтобы отыскать все случаи использования закрытой части класса, недостаточно будет в таком случае просмотреть лишь функции-члены класса и его дружественные функции. Теперь нужно будет просматривать каждый исходный файл проекта на предмет обнаружения производных от исходного классов, тщательного изучения всех их функций-членов, а затем нужно будет выявить классы, производные от производных и так далее, и тому подобное. Это в самом лучшем случае утомительно, а часто и просто нереально. В базовых классах, где это возможно, нужно использовать ключевое слово *protected* вместо *private*. Защищенные члены (*protected*) в отношении функций производных классов ведут себя как открытые, а в остальных случаях — как закрытые (§15.3).

Как правило, самым простым решением является использование в производном классе лишь открытых членов базового класса. Например:

```
void Manager::print() const
{
    Employee::print();    // вывести общую информацию
                        // (характерную для любого сотрудника)
    cout<< level;         // вывести информацию, специфичную для типа Manager
}
```

Обратите внимание на то, что нужно использовать операцию `::`, поскольку функция *print()* переопределяется (*redefined*) в классе *Manager*. Очень опрометчиво написать следующее:

```
void Manager::print() const
{
    // печатаем специфичную для типа Manager информацию:
    print();           // oops!
}
```

ибо при этом порождается неожиданная рекурсия.

### 12.2.2. Конструкторы и деструкторы

Некоторые производные классы нуждаются в конструкторах. Если в базовом классе определены конструкторы, то их тоже нужно вызывать. Конструкторы по умолчанию могут вызываться неявно. Остальные же типы конструкторов базового класса должны вызываться явно. Например:

```
class Employee
{
    string first_name, family_name;
    short department;
```

```

public:
    Employee (const string& n, int d) ;
    // ...
};

class Manager : public Employee
{
    list<Employee*> group;    // подчиненные
    short level;
    // ...

public:
    Manager (const string& n, int d, int lvl) ;
    // ...
};

```

Аргументы для конструктора базового класса указываются в аргументах конструктора производного класса. В этом отношении, базовый класс функционирует в точности как член производного класса (§10.4.6). Например:

```

Employee::Employee (const string& n, int d)
    : family_name (n) , department (d)    // инициализация членов класса Employee
{
    // ...
}

Manager::Manager (const string& n, int d, int lvl)
    : Employee (n, d) ,                    // инициализация членов базового класса
      level (lvl)                          // инициализация членов класса Manager
{
    // ...
}

```

Конструктор производного класса может задавать инициализаторы только для своих членов и непосредственных базовых классов (он не может явным образом инициализировать члены базового класса). Например:

```

Manager::Manager (const string& n, int d, int lvl)
    : family_name (n) ,                    // error: family_name не объявлен в Manager
      department (d) ,                    // error: department не объявлен в Manager
      level (lvl)
{
    // ...
}

```

Здесь содержатся три ошибки: не удастся вызвать конструктор базового класса *Employee*, а также дважды осуществляется ошибочная попытка прямой инициализации членов базового класса.

Объекты классов конструируются снизу-вверх: сначала базовый класс, затем инициализируются члены, и только потом остальная инициализация производного класса. Уничтожение объектов выполняется в обратном порядке. Конструирование членов (указанных в списке инициализации конструктора) выполняется строго в порядке появления их объявлений в определении класса, уничтожение же производится точно в обратном порядке. См. также §10.4.6, §15.2.4.1 и §15.4.3.

### 12.2.3. Копирование

Копирование классовых объектов определяется копирующим конструктором и операцией присваивания (§10.4.4.1). Рассмотрим пример:

```
class Employee
{
    // ...
    Employee& operator= (const Employee&) ;
    Employee (const Employee&) ;
};

void f(const Manager& m)
{
    Employee e = m;           // конструируем e из Employee-части m
    e = m;                     // присваиваем Employee-часть m объекту e
}
```

Поскольку копирующие функции класса **Employee** ничего не знают о классе **Manager**, то копируется лишь часть объекта типа **Manager** — та, что достается ему от базового класса **Employee**. Этот эффект часто называют *срезкой* (*slicing*), и он может оказаться причиной недоразумений и ошибок. Одной из причин передачи указателей на объекты классовых иерархий наследования является желание избежать срезы. Другими причинами служат желание обеспечить полиморфное поведение (§2.5.4, §12.2.6) и достичь эффективности.

Помните, что если вы не программируете явно операцию присваивания, то компилятор предоставляет собственный вариант этой операции (§11.7). Это подразумевает, что операции присваивания не наследуются. Конструкторы тоже никогда не наследуются.

### 12.2.4. Иерархии классов

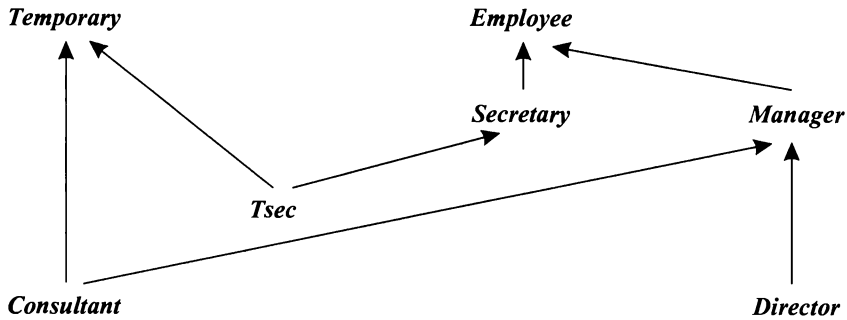
Производный класс, в свою очередь, тоже может быть базовым классом. Например:

```
class Employee { /*...*/ };
class Manager : public Employee{ /*...*/ };
class Director : public Manager { /*...*/ };
```

Такой набор связанных классов традиционно называют классовой иерархией наследования или просто *классовой иерархией* (*class hierarchy*). Ее чаще всего изображают в виде дерева, но она может иметь и более общую структуру графа. Например, для классов

```
class Temporary{ /*...*/ };
class Secretary : public Employee{ /*...*/ };
class Tsec : public Temporary, public Secretary{ /*...*/ };
class Consultant : public Temporary, public Manager{ /*...*/ };
```

в графической форме иерархия имеет вид:



Таким образом, как детально объясняется в §15.2, на С++ можно сформировать направленный ациклический граф классов.

### 12.2.5. Поля типа

Чтобы использовать производные классы в качестве чего-то большего, чем просто удобный способ сокращения исходного кода, мы должны решить следующую проблему: как определить истинный (производный) тип объекта, адресуемого указателем типа *Base\** (то есть указателем на базовый класс)? Существуют четыре фундаментальных способа решения этой проблемы:

1. Гарантировать, что адресуются лишь объекты единственного типа (§2.7, глава 13).
2. Поместить специальное поле типа в базовый класс, чтобы функции могли его проверять.
3. Использовать *dynamic\_cast* (§15.4.2, §15.4.5).
4. Использовать виртуальные функции (§2.5.5, §12.2.6).

Указатели на базовые классы широко используются при проектировании *контейнерных классов* (*container classes*), таких как множества (*set*), вектора (*vector*) и списки (*list*). В таких случаях решение [1] приведет к однородному списку, то есть списку объектов одного и того же типа. Решения [2], [3] и [4] позволяют строить списки разнородных объектов, то есть списки указателей, настроенных на объекты разных типов. Решение [3] является вариантом решения [2], которое поддерживается языком С++ непосредственно. А решение [4] является специальным вариантом решения [2], безопасным по отношению к типам. Комбинация решений [1] и [4] особо интересна и чрезвычайно мощна, и почти всегда способствует более ясному коду, чем код, порожаемый решениями [2] и [3].

Рассмотрим сначала решение, основанное на полях типа с тем, чтобы понять, почему его в большинстве случаев лучше избегать. Наш пример с сотрудниками и менеджерами можно переопределить следующим образом:

```

struct Employee
{
    enum Empl_type {M, E};
    Empl_type type;

    Employee() : type(E) {}

```

```

string first_name, family_name;
char middle_initial;

Date hiring_date;
short department;
// ...
};

struct Manager : public Employee
{
    Manager () { type = M; }

    list<Employee*> group;    // подчиненные
    short level;
    // ...
};

```

Теперь мы можем написать функцию, выводящую информацию о каждом сотруднике:

```

void print_employee (const Employee* e)
{
    switch (e->type)
    {
        case Employee::E:
            cout<< e->family_name<< '\t'<< e->department<< '\n' ;
            // ...
            break;
        case Employee::M:
            {
                cout<< e->family_name<< '\t'<< e->department<< '\n' ;
                // ...
                const Manager* p = static_cast<const Manager*> (e) ;
                cout<< "level" <<p->level<< '\n' ;
                // ...
                break;
            }
    }
}

```

Используем эту функцию для вывода элементов списка:

```

void print_list (const list<Employee*>& elist)
{
    for (list<Employee*>::const_iterator p = elist.begin () ; p!=elist.end () ; ++p)
        print_employee (*p) ;
}

```

Все это прекрасно работает, особенно в маленьких программах, поддерживаемых единственным программистом. В то же время, представленное решение имеет и фундаментальную слабость, так как зависит от манипуляций с типами, которые не контролируются компилятором. Положение становится еще хуже, если функции, вроде `print_employee ()` пытаются извлечь пользу из общности типов:



```

void print_employee (const Employee* e)
{
    cout<< e->family_name<< '\t'<< e->department<< '\n' ;
    // ..
    if( e->type == Employee::M)
    {
        const Manager* p = static_cast<const Manager*>(e) ;
        cout<< "level" << p->level<< '\n' ;
        // ...
    }
}

```

Поиск всех проверок типа, глубоко запрятаных в коде больших функций, да еще и при наличии большого числа производных классов, весьма утомителен. Даже если все проверки найдены, бывает непросто понять, что на самом деле происходит. Более того, если добавляется новый тип сотрудников (производный от *Employee* класс), то приходится вносить изменения во все ключевые функции системы — те, что проверяют поля типа. После внесения изменений программист должен проверить все функции, потенциально зависящие от проверок типа. Это подразумевает, что программист должен иметь доступ к ключевым частям исходного кода программы, а также дополнительные усилия по тестированию программы. В итоге становится понятным, что наличие проверок типа служит отчетливым сигналом о необходимости улучшения кода.

Другими словами, техника применения полей типа способствует ошибкам и усложняет сопровождение. Проблема усиливается чрезвычайно по мере роста размеров программ, ибо поля типа нарушают все принципы модульности и сокрытия данных. Каждая функция, использующая поля типа, должна знать детали представления всех производных классов иерархии (с базовым классом, содержащим поле типа).

Также складывается впечатление, что наличие общих данных, доступных из всех классов иерархии наследования, провоцирует программистов на увеличение числа таких данных. В итоге, общий базовый класс становится чем-то вроде склада всякой «полезной информации». Это, в свою очередь, порождает крайне нежелательные связи между реализациями базовых и производных классов. Ясный дизайн и простота сопровождения требуют, чтобы разные сущности представлялись раздельно, а взаимозависимости между ними минимизировались.

### 12.2.6. Виртуальные функции

Виртуальные функции решают проблему полей типа, предоставляя программисту возможность определить в базовом классе функции, подлежащие замещению в каждом производном классе. Компилятор и загрузчик гарантируют правильное соответствие между объектами и применяемыми к ним функциями. Например:

```

class Employee
{
    string first_name, family_name;
    short department;
    // ...
public:

```

```
Employee (const string& name, int dept) ;
virtual void print () const;
// ...
};
```

Ключевое слово ***virtual*** указывает, что функция ***print***() символизирует общий интерфейс к набору одноименных функций, определенных в базовом и производных от него классах. Если такие функции действительно определены в производных классах, компилятор гарантирует вызов своего варианта функции ***print***() для каждого объекта классовой иерархии.

Чтобы объявление виртуальной функции действительно работало в качестве интерфейса к семейству функций, определенных в базовом и производных классах, *типы аргументов* всех функций *должны быть одинаковыми*, а для возвращаемого значения допускаются лишь незначительные отличия (§15.6.2). Виртуальные функции-члены иногда называют методами<sup>1</sup>.

Виртуальная функция *обязана быть определена* в классе, в котором она впервые объявляется (за исключением случаев чисто виртуальных функций; см. §12.3). Например:

```
void Employee::print () const
{
    cout<< family_name<< ' ' << department<< '\n' ;
    // ...
}
```

Разумеется, виртуальную функцию можно использовать и в случаях, когда у класса отсутствуют производные классы. Производный класс, который не нуждается в собственном варианте функции, *не обязан замещать виртуальную функцию*, определенную в базовом классе. Определяя производный класс, определите и новый вариант виртуальной функции, если он, конечно, нужен. Например:

```
class Manager :public Employee
{
    list<Employee*> group;
    short level;
    // ...
public:
    Manager (const string& name, int dept, int lvl) ;
    void print () const;
    // ...
};

void Manager::print () const
{
    Employee::print () ;

    cout<< " \tlevel" << level << '\n' ;
    // ...
}
```

<sup>1</sup> Чаще всего методами называют любые классовые функции-члены. — Прим. ред.

Функция производного класса, имеющая то же имя и те же параметры, что и виртуальная функция, определенная в базовом классе, *замещает* (*override*) вариант от базового класса. За исключением случаев прямых указаний на конкретный вариант используемой виртуальной функции (например, *Employee::print()*), выбирается замещаемая для класса объекта функция.

Теперь отпадает нужда в глобальной функции *print\_employee()* (§12.2.5), поскольку ее место занято функциями-членами *print()*. Список сотрудников теперь выводится следующим образом:

```
void print_list(const list<Employee*>& s)
{
    for(list<Employee*>::const_iterator p = s.begin(); p!=s.end(); ++p) //см. §2.7.2
        (*p)->print();
}
```

Или еще проще:

```
void print_list(const list<Employee*>& s)
{
    for_each(s.begin(), s.end(), mem_fun(&Employee::print)); // см. §3.8.5
}
```

Информация по каждому сотруднику выводится точно в соответствии с его настоящим статусом. Вот иллюстрация к сказанному, где функция *main()*:

```
int main()
{
    Employee e("Brown", 1234);
    Manager m("Smith", 1234, 2);
    list<Employee*> empl;
    empl.push_front(&e); // см. §2.5.4
    empl.push_front(&m);
    print_list(empl);
}
```

порождает следующий вывод:

```
Smith 1234
    level 2
Brown 1234
```

Заметим, что все это работает даже в случае, когда функция *print\_list()* написана и откомпилирована до того, как класс *Manager* был разработан (и даже задуман). Вот это действительно ключевой аспект работы с классами. При правильном использовании он служит краеугольным камнем объектно-ориентированного проектирования и придает стабильности эволюционирующим проектам.

«Правильное» поведения виртуальных функций *print()* для любых сотрудников компании (то есть для любых объектов типа *Employee*, *Manager* и прочих производных классов) называется *полиморфизмом* (*polymorphism*). Тип с виртуальными функциями называется *полиморфным типом* (*polymorphic type*). Для практической реализации полиморфного поведения в языке C++ тип объектов должен быть полиморфным, а сами объекты должны адресоваться указателями или ссылками. Когда же с объектами работают напрямую (а не косвенно через указатели или

ссылки), то их тип известен компилятору заранее и никакого полиморфизма просто не требуется.

Ясно, что для реализации полиморфизма, компилятор должен хранить с каждым объектом полиморфной классовой иерархии дополнительную информацию о типе, чтобы иметь возможность вызвать правильную версию виртуальной функции. В типичной реализации достаточно дополнительной памяти для хранения указателя (§2.5.5). Эта дополнительная память относится только к объектам классов с виртуальными функциями, а вовсе не ко всем объектам любых классовых иерархий. Отметим, что в рассмотренном выше случае, когда применяется поле типа, величина требуемой избыточной памяти ничуть не меньше.

Вызов функции с применением операции разрешения области видимости `::`, как это было сделано в *Manager::print()*, гарантирует «выключение» механизма виртуальных функций. В противном случае, вызов *Manager::print()* привел бы к бесконечной рекурсии. Использование квалифицированных имен имеет еще один положительный эффект. Если виртуальная функция объявлена встраиваемой (что нередко встречается), то применение операции `::` позволяет действительно встраивать ее вызов. А это помогает программисту уладить деликатную проблему вызова виртуальной функции из другой виртуальной функции для того же самого объекта. Функция *Manager::print()*, как раз служит наглядным примером. Поскольку тип объекта фиксирован уже при вызове *Manager::print()*, нет нужды определять его снова для вызова *Employee::print()*.

Стоит не забывать, что очевидной и традиционной реализацией механизма вызова виртуальных функций является косвенный вызов (§2.5.5), высокая эффективность которого не должна удерживать программиста от применения виртуальных функций там, где приемлем обычный функциональный вызов.

## 12.3. Абстрактные классы

Многие классы схожи с классом *Employee* в том отношении, что они полезны и сами по себе, и как базовые для производных классов. Для подобных классов описанные в предыдущем разделе методики вполне достаточны. Но не все классы вписываются в такую схему работы. Некоторые классы, вроде класса *Shape*, представляют абстрактные концепции, для которых реальных объектов не существует. Класс *Shape* имеет смысл и полезен исключительно как абстрактный базовый класс для порождения конкретных производных классов. Это наглядно видно из того факта, что невозможно разумным образом определить его виртуальные функции:

```
class Shape
{
public:
    virtual void rotate(int) { error("Shape::rotate"); } // не элегантно
    virtual void draw() { error("Shape::draw"); }
    // ...
};
```

Создание таких «бесформенных» объектов хотя и допустимо, но лишено всякого смысла:

**Shape s;** // глупо: "shapeless shape"

Действительно, какой смысл в объекте, любая операция над которым порождает лишь сообщение об ошибке.

Лучше объявить виртуальные функции класса **Shape** как *чисто виртуальные* (*pure virtual*) с помощью инициализатора =0:

```
class Shape                                // абстрактный класс
{
public:
    virtual void rotate (int) = 0;         // чисто виртуальная функция
    virtual void draw () = 0;              // чисто виртуальная функция
    virtual bool is_closed () = 0;         // чисто виртуальная функция
    // ...
};
```

Класс с одной или несколькими чисто виртуальными функциями является *абстрактным классом* (*abstract class*), объекты которого создавать недопустимо:

**Shape s;** // error: переменная абстрактного класса Shape

Абстрактный класс может использоваться только как интерфейсный и базовый для других классов. Например:

```
class Point { /* ... */ };

class Circle : public Shape
{
public:
    void rotate (int) {}                  // заменяем Shape::rotate
    void draw () ;                        // заменяем Shape::draw
    bool is_closed () {return true; }     // заменяем Shape::is_closed

    Circle (Point p, int r) ;

private:
    Point center;
    int radius;
};
```

Чисто виртуальная функция, не определенная и в производном классе, остается чисто виртуальной, и поэтому такой производный класс также является абстрактным. Это открывает нам возможности постепенного осуществления реализации классовых иерархий:

```
class Polygon : public Shape              // абстрактный класс
{
public:
    bool is_closed () {return true; }     // заменяем Shape::is_closed
    // ... draw and rotate не заменяются
};

Polygon b;                               // error: объект абстрактного класса Polygon

class Irregular_polygon : public Polygon
{
    list<Point> lp;
```

```
public:
    void draw() ;                // замещаем Shape::draw
    void rotate(int) ;           // замещаем Shape::rotate
    // ...
};
```

*Irregular\_polygon poly(some\_points) ; // ok. (если есть подходящий конструктор)*

Важным примером применения абстрактных классов является предоставление интерфейса без какой-либо реализации. Например, операционная система может скрывать детали реализации аппаратных драйверов за вот таким абстрактным интерфейсом:

```
class Character_device
{
public:
    virtual int open(int opt)=0;
    virtual int close(int opt)=0;
    virtual int read(char* p, int n)=0;
    virtual int write(const char* p, int n)=0;
    virtual int ioctl(int...)=0;
    virtual ~Character_device(){} // виртуальный деструктор
};
```

Мы можем определить драйверы в форме классов, производных от *Character\_device*, после чего управлять ими через указанный стандартный интерфейс. Важность виртуальных деструкторов разъясняется в §12.4.2.

Теперь, после представления абстрактных классов, мы знакомы со всем инструментарием, позволяющим писать модульные программы, использующие классы в качестве строительных блоков.

## 12.4. Проектирование иерархий классов

Рассмотрим несложную проектную задачу: ввод в программу целого значения через пользовательский интерфейс. Это можно сделать огромным количеством способов. Чтобы позиционировать нашу задачу в рамках этого множества, и чтобы проанализировать различные возможные проектные решения, начнем с определения программной модели ввода. Оставим на потом детали, необходимые для ее конкретной реализации в рамках реальных пользовательских интерфейсов.

Идея состоит в создании класса *Ival\_box*, который знает допустимый диапазон вводимых значений. Программа может запросить *Ival\_box* об этом значении, а также предупредить пользователя. Также программа может попросить *Ival\_box* сообщить, не вводил ли пользователь новых значений после того, как программа получила предыдущее значение ввода.

Поскольку можно по-разному реализовывать эту общую идею, мы предполагаем, что будет множество конкретных разновидностей элементов ввода типа *Ival\_box*, таких как ползунки, диалоговые окна, элементы для голосового ввода и т.д.

В общем, мы хотим реализовать «виртуальную систему пользовательского ввода» для использования в разных приложениях. Она будет демонстрировать часть

функциональности, реализованной в настоящих системах пользовательского интерфейса. Ее желательно реализовать в широком наборе операционных систем, так что нельзя забывать о переносимости кода. Естественно, наш подход не единственно возможный. Я выбрал его потому, что он достаточно общий и позволяет продемонстрировать широкий набор методов и технологий проектирования. Эти методы не только использовались для построения реальных систем пользовательского интерфейса, но они вообще выходят далеко за рамки интерфейсных систем.

### 12.4.1. Традиционные иерархии классов

Наше первое решение сводится к традиционной иерархии классов, типичной для языков Simula, Smalltalk и старых версий C++.

Класс *Ival\_box* определяет базисный интерфейс для всех элементов пользовательского ввода и задает его реализацию, которую более специфические элементы могут переопределять. Кроме того, мы объявляем здесь данные, необходимые для формирования любого элемента ввода:

```
class Ival_box
{
protected:
    int val;
    int low, high;
    bool changed;

public:
    Ival_box (int ll, int hh) { changed = false; val = low = ll; high = hh; }

    virtual int get_value () { changed = false; return val; } // для приложения
    virtual void set_value (int i) { changed=true; val=i; } // для пользователей
    virtual void reset_value (int i) { changed=false; val=i; } // для приложения

    virtual void prompt () {}
    virtual bool was_changed () const { return changed; }
};
```

Представленная здесь реализация функций довольно небрежна и нацелена лишь на демонстрацию основных намерений. В реальном коде осуществлялась бы хоть какая-то проверка введенных значений.

Данный класс можно использовать следующим образом:

```
void interact (Ival_box* pb)
{
    pb->prompt (); // оповестить пользователя
    // ...
    if (pb->was_changed ())
    {
        int i=pb->get_value ();
        // новое значение; что-нибудь делаем
    }
    else
    {
        // старое значение подходит; делаем что-нибудь еще
    }
}
```

```

// ...
}

void some_fct()
{
    Ival_box* p1 = new Ival_slider(0, 5); // Ival_slider наследует от Ival_box
    interact(p1);

    Ival_box* p2 = new Ival_dial(1, 12);
    interact(p2);
}

```

Большая часть кода написана в стиле, использующем доступ к элементу ввода через указатель типа **Ival\_box\*** (см. функцию **interact()**). Поэтому приложению нет нужды знать обо всех потенциально возможных конкретных элементах ввода. Знание же об этих специфических элементах требуется лишь в строго ограниченных частях кода (небольшом количестве функций), имеющих дело с созданием объектов соответствующих типов. Это помогает в значительной степени изолировать пользователя от изменений в реализациях производных классов. Большая часть кода может не обращать внимания на существование множества различных интерфейсных элементов ввода.

Чтобы упростить изложение основных идей проектирования, я сознательно не рассматриваю вопрос о том, как именно программа ожидает ввод. Возможно программа действительно ожидает ввода в функции **get\_value()**, а может она ассоциирует **Ival\_box** с некоторым событием и готовится реагировать на него функцией обратного вызова (callback function); также возможно, что она запускает код **Ival\_box** в отдельном потоке, а потом опрашивает его состояние. Конечно, такие вопросы предельно важны в разработках конкретных систем пользовательского интерфейса. Однако даже минимальное обсуждение этих вопросов отвлекло бы нас от изучения средств языка и общих методов программирования, которые совсем не ограничиваются одними лишь пользовательскими интерфейсами. Они применимы к очень широкому кругу задач.

Конкретные элементы ввода определяются как производные от **Ival\_box** классы. Например:

```

class Ival_slider : public Ival_box
{
    // графические атрибуты, определяющие вид ползунка (slider) и т.д.
public:
    Ival_slider(int, int);
    int get_value();
    void prompt();
};

```

Члены данного класса **Ival\_box** были объявлены защищенными с целью предоставления прямого к ним доступа из производных классов. Таким образом, **Ival\_slider::get\_value()** может держать значение в **Ival\_box::val**. Напоминаем, что защищенные члены доступны самому классу и его производным классам, но не обычному клиентскому коду (§15.3).

В дополнение к **Ival\_slider** мы определим и другие конкретные элементы ввода, реализующие специфические варианты общей концепции **Ival\_box**. Это может быть

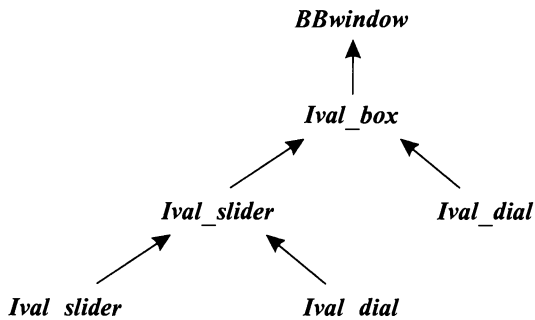


**Ival\_dial**, позволяющий выбрать значение с помощью вращающейся «ручки»; **Flashing\_ival\_slider**, который мерцает при вызове **prompt()**; **Popup\_ival\_slider**, который реагирует на вызов **prompt()** «всплытием» на экране где-нибудь в заметном месте, чтобы пользователь не мог его проигнорировать.

Но откуда при этом возьмется графическая начинка для элементов ввода? Большинство систем графического интерфейса пользователя предоставляют класс, определяющий основные свойства отображения элементов на экране. Тогда, если мы, например, воспользуемся наработками фирмы «Big Bucks Inc.», то все наши классы должны наследовать от стороннего класса **BBwindow**. Для этого достаточно сделать класс **Ival\_box** производным от **BBwindow**. Тогда, например, любой элемент ввода может быть «помещен» на экран и при этом сразу будет подчиняться определенным графическим правилам (может изменять свой визуальный размер, может быть отбуксирован с помощью мыши и т.д.), принятым в **BBwindow**. Наша классовая иерархия примет при этом следующий вид:

```
Class Ival_box : public BBwindow{ /* ... */ }; // переписан для использования
class Ival_slider : public Ival_box{ /* ... */ }; // с BBwindow
class Ival_dial : public Ival_box{ /* ... */ };
class Flashing_ival_slider : public Ival_slider{ /* ... */ };
class Popup_ival_slider : public Ival_slider{ /* ... */ };
```

или в графической форме:



#### 12.4.1.1. Критика

Представленный дизайн во многих случаях работает замечательно, а соответствующая классовая иерархия хорошо подходит для решения многих проблем. Тем не менее, отдельные детали все же заставляют нас рассмотреть и другие альтернативы.

Мы сделали **BBwindow** базовым классом для **Ival\_box**. Это не совсем верно. Использование **BBwindow** не входит в понимание существа **Ival\_box**; это всего лишь деталь реализации. Однако применение этого класса в качестве базового для **Ival\_box** превратило его в важнейший элемент дизайна всей системы. Это в каких-то случаях может быть приемлемым, например, если мы накрепко связываем свои разработки с «Big Bucks Inc.». Но что, если мы захотим визуализировать наш **Ival\_box** в рамках графической системы от «Imperial Bananas», «Liberated Software» или «Compiler Whizzes»? Нам тогда придется поддерживать четыре различные версии программы:

```

class Ival_box : public BBwindow{ /* ... */ }; // BB version
class Ival_box : public CWwindow{ /* ... */ }; // CW version
class Ival_box : public IBwindow{ /* ... */ }; // IB version
class Ival_box : public LSwindow{ /* ... */ }; // LS version

```

Множество версий программы может стать настоящим кошмаром.

Другая проблема состоит в том, что каждый производный класс разделяет данные, объявленные в *Ival\_box*. Эти данные безусловно являются деталью реализации, вкрапшейся в интерфейс *Ival\_box*. С практической же точки зрения, во многих случаях эти данные неадекватны. Например, элемент *Ival\_slider* не нуждается в специальном хранении связанного с ним значения — его можно легко вычислить по положению ползунка этого элемента во время вызова *get\_value()*. С другой стороны, хранить связанные, но различные наборы данных — это значит нарываться на неприятности: рано или поздно они окажутся рассогласованными. Кроме того, многие новички склонны объявлять данные защищенными, что усложняет сопровождение. Данные лучше объявлять закрытыми, чтобы разработчики производных классов не пытались запутывать их в один клубок со своими данными. А еще лучше объявлять данные в производных классах, где их можно определить наиболее точным образом, и они не будут усложнять жизнь разработчикам иных (несвязанных) производных классов. Почти всегда интерфейс должен содержать только функции, типы и константы.

Преимущество наследования от *BBwindow* состоит в том, что его возможности сразу становятся доступными пользователям *Ival\_box*. К сожалению, это же означает, что в случае изменений в *BBwindow* пользователю придется перекомпилироваться (или даже внести изменения в исходный код), чтобы справиться с последствиями этих изменений. В частности, для большинства реализаций C++ изменения в размере базового класса автоматически влекут за собой необходимость перекомпиляции всех производных классов.

И наконец, наша программа может работать в смешанных средах, где сосуществуют различные системы пользовательского интерфейса. Причина может заключаться в том, что две системы каким-то образом совместно используют экран, или что нашей программе потребовалось взаимодействовать с пользователями других систем. Жесткое встраивание какого-либо пользовательского интерфейса в основу классовой иерархии для нашего единственного интерфейса *Ival\_box* не является гибким решением в таких ситуациях.

### 12.4.2. Абстрактные классы

Итак, начнем сначала и построим новую иерархию классов, которая решает проблемы, вытекающие из представленной выше критики традиционной иерархии:

1. Система пользовательского интерфейса должна быть деталью реализации, скрытой от пользователей, не желающих знать о ней.
2. Класс *Ival\_box* не должен содержать данных.
3. Изменения в системе пользовательского интерфейса не должны требовать перекомпиляции кода, использующего классы иерархии *Ival\_box*.
4. Различные варианты *Ival\_box* для разных систем пользовательского интерфейса должны иметь возможность сосуществовать в нашей программе.

Для достижения поставленных целей существует несколько альтернативных подходов. Здесь я представляю один из них, наилучшим образом вписывающийся в возможности C++.

Сначала я определяю *Ival\_box* как чистый интерфейс:

```
class Ival_box
{
public:
    virtual int get_value () = 0;
    virtual void set_value (int i) = 0;
    virtual void reset_value (int i) = 0;
    virtual void prompt () = 0;
    virtual bool was_changed () const = 0;
    virtual ~Ival_box () {}
};
```

Это намного понятнее, чем прежнее объявление *Ival\_box*. Данные исчезли вместе с упрощенными реализациями функций-членов. Ушел также и конструктор, так как отсутствуют данные, подлежащие инициализации. Вместо этого я добавил виртуальный деструктор для того, чтобы гарантировать правильную очистку данных, определяемых в производных классах.

Определение *Ival\_slider* может выглядеть следующим образом:

```
class Ival_slider : public Ival_box, protected BBwindow
{
public:
    Ival_slider (int, int) ;
    ~Ival_slider () ;

    int get_value () ;
    void set_value (int i) ;
    // ...

protected:
    // функции, замещающие виртуальные функции,
    // например, BBwindow::draw(), BBwindow::mouse1hit()

private:
    // данные для slider
};
```

Класс *Ival\_slider* наследуется от абстрактного класса (*Ival\_box*), требующего реализовать его чисто виртуальные функции. Кроме того, *Ival\_slider* наследуется от *BBwindow*, предоставляющего необходимые для реализации средства. Поскольку *Ival\_box* объявляет интерфейс для производных классов, наследование от него выполняется в открытом режиме (с применением ключевого слова *public*). А поскольку *BBwindow* есть просто средство реализации, то наследование от него выполняется в защищенном режиме (с использованием ключевого слова *protected* — см. §15.3.2). Из этого следует, что программист, использующий *Ival\_slider* не может напрямую воспользоваться средствами *BBwindow*. Интерфейс *Ival\_slider* состоит из открыто унаследованного интерфейса *Ival\_box* плюс то, что явно объявляет сам *Ival\_slider*. Я использовал защищенное наследование вместо более ограничительно-го (и обычно более безопасного) закрытого, чтобы сделать *BBwindow* доступным для классов, производных от *Ival\_slider*.

Непосредственное наследование от более, чем одного класса, называется *множественным наследованием* (*multiple inheritance*) (§15.2). Отметим, что *Ival\_slider* должен замещать виртуальные функции из *Ival\_box* и *BBwindow*. Поэтому он прямо или косвенно должен наследовать от обоих этих классов. Как показано в §12.4.1.1, возможно косвенное наследование от *BBwindow*, когда последний служит базовым классом для *Ival\_box*, но это имеет свои нежелательные побочные эффекты. Объявление же поля с типом *BBwindow* членом класса *Ival\_box* не подходит потому, что класс не замещает виртуальные функции своих членов (§24.3.4). Представление графических средств (окна) в виде члена *Ival\_box* с типом *BBwindow*\* приводит к совершенно другому стилю проектирования со своими собственными компромиссными «за» и «против» (§12.7[14], §25.7).

Интересно, что новое объявление *Ival\_slider* позволяет написать код приложения абсолютно так же, как раньше. Все, что мы сделали, это реструктурировали детали реализации более логичным образом.

Многие классы требуют некоторой формы очистки данных перед уничтожением объекта. Поскольку абстрактный класс *Ival\_box* не может знать, требуется ли очистка в производных классах, то ему лучше заранее предположить, что требуется. Мы гарантируем надлежащую очистку, объявляя виртуальный деструктор *Ival\_box*: `~Ival_box()` в базовом классе, и замещая его в производных классах. Например:

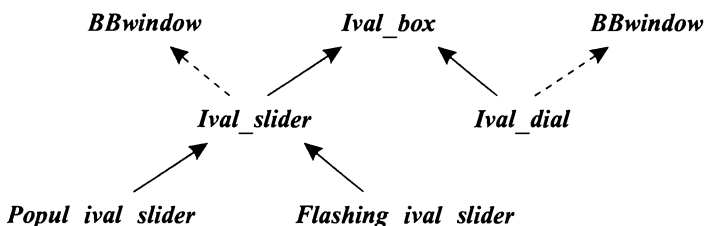
```
void f(Ival_box* p)
{
    // ...
    delete p;
}
```

Операция *delete* явным образом уничтожает объект, адресуемый указателем *p*. Мы не можем знать, какому точно классу принадлежит объект, адресуемый указателем *p*, но благодаря виртуальному деструктору из *Ival\_box* надлежащий деструктор (опционально) будет вызван.

Теперь иерархию *Ival\_box* можно определить следующим образом:

```
class Ival_box { /* ... */ };
class Ival_slider : public Ival_box, protected BBwindow { /* ... */ };
class Ival_dial : public Ival_box, protected BBwindow { /* ... */ };
class Flashing_ival_slider : public Ival_slider { /* ... */ };
class Popup_ival_slider : public Ival_slider { /* ... */ };
```

или в графической форме:



Я использую пунктирную линию для отображения защищенного наследования. Для обычных пользователей это деталь реализации.

### 12.4.3. Альтернативные реализации

Полученный дизайн яснее, чем традиционный, и поддерживать его легче. К тому же, он не менее эффективный. Но пока что он по-прежнему не решает проблему со многими версиями:

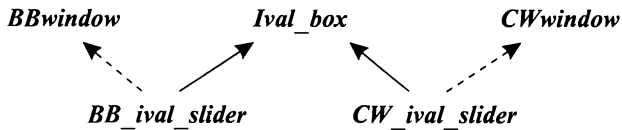
```
class Ival_box { /* ... */ };    // common
class Ival_slider : public Ival_box, protected BBwindow { /* ... */ };    // для BB
class Ival_slider : public Ival_box, protected CWwindow { /* ... */ };    // для CW
// ...
```

И он не позволяет *Ival\_slider* для *BBwindow* сосуществовать с *Ival\_slider* для *CWwindow*, даже если сами эти системы пользовательского интерфейса сосуществуют.

Очевидным решением проблемы является определение нескольких вариантов *Ival\_slider* с разными именами:

```
class Ival_box { /* ... */ };
class BB_ival_slider : public Ival_box, protected BBwindow { /* ... */ };
class CW_ival_slider : public Ival_box, protected CWwindow { /* ... */ };
// ...
```

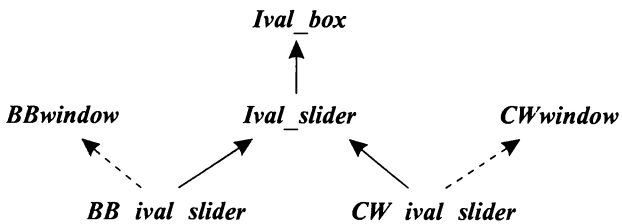
или графически:



Чтобы еще более изолировать наши классы от деталей реализации, мы можем ввести абстрактный класс *Ival\_slider*, который наследует от *Ival\_box*, а системно-зависимые элементы наследовать следующим образом:

```
class Ival_box { /* ... */ };
class Ival_slider : public Ival_box { /* ... */ };
class BB_ival_slider : public Ival_slider, protected BBwindow { /* ... */ };
class CW_ival_slider : public Ival_slider, protected CWwindow { /* ... */ };
// ...
```

или графически:



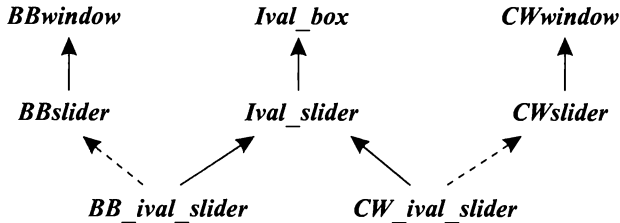
Как правило, ситуация только улучшается, если системно-зависимые классы также выстроены в иерархию. Например, если у фирмы «Big Bucks Inc.» имеется класс ползунков (slider class), то мы производим собственный элемент, наследуя от этого ползунка:

```

class BB_ival_slider : public Ival_slider, protected BBslider { /* ... */ };
class CW_ival_slider : public Ival_slider, protected CWslider { /* ... */ };
// ...

```

или в графической форме:



Последнее улучшение особо велико там, где наши абстракции классов не слишком отличаются от абстракций программной системы, обеспечивающей графическую реализацию. Тогда объем программирования уменьшается и сводится к необходимости установления соответствия между аналогичными концепциями. Наследование же от общих базовых классов, таких как **BBwindow**, производится редко.

Окончательная иерархия будет состоять из нашей оригинальной иерархии интерфейсов, ориентированных на наше приложение:

```

class Ival_box { /* ... */ };
class Ival_slider : public Ival_box { /* ... */ };
class Ival_dial : public Ival_box { /* ... */ };
class Flashing_ival_slider : public Ival_slider { /* ... */ };
class Popup_ival_slider : public Ival_slider { /* ... */ };

```

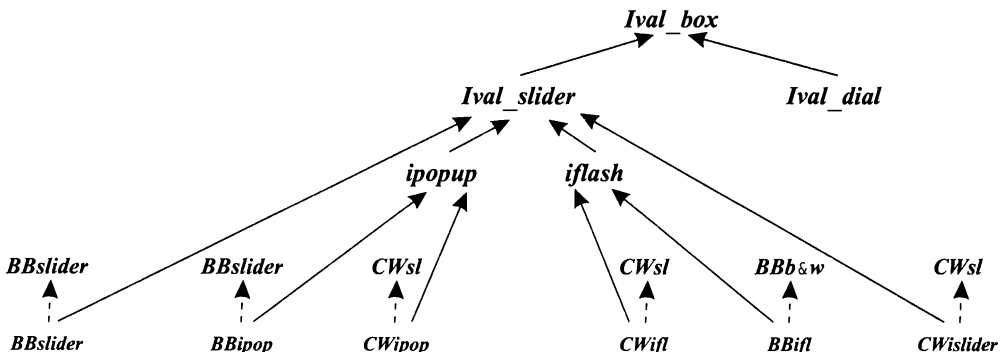
и иерархии реализации для различных сторонних графических систем:

```

class BB_ival_slider : public Ival_slider, protected BBslider { /* ... */ };
class BB_flashing_ival_slider : public Flashing_ival_slider,
    protected BBwindow_with_bells_and_whistles { /* ... */ };
class BB_popup_ival_slider : public Popup_ival_slider, protected BBslider { /* ... */ };
class CW_ival_slider : public Ival_slider, protected CWslider { /* ... */ };
// ...

```

Используя очевидные аббревиатуры, эту иерархию можно представить в следующем графическом виде:



Исходная иерархия классов *Ival\_box* не изменилась — просто она теперь окружена классами графических реализаций.

### 12.4.3.1. Критика

Дизайн на основе абстрактных классов очень гибок, и к тому же он не менее прост, что и традиционный дизайн на основе общего базового класса, определяющего систему пользовательского интерфейса. В последнем случае класс окон находится в корне дерева. А при первом подходе исходная иерархия классов нашего приложения неизменная и выступает в качестве корневой для классов реализации. С точки зрения работы программы, оба подхода эквивалентны в том смысле, что почти весь код выполняется одинаково. В обоих случаях можно большую часть времени оперировать семейством классов *Ival\_box* без оглядки на системы пользовательского интерфейса. Например, нам не нужно переписывать *interact()* из §12.4.1 при смене системы пользовательского интерфейса.

Ясно, что если изменится открытый интерфейс систем пользовательского интерфейса, то придется переписывать реализацию каждого класса из семейства *Ival\_box*. Но в иерархии с абстрактными классами почти весь наш собственный код защищен от изменений в реализации систем пользовательского интерфейса и даже не требует перекомпиляции в таких случаях. Это особенно ценно, когда разработчик систем пользовательского интерфейса выпускает все новые и «почти совместимые» версии. Наконец, пользователи системы с абстрактными классами избавлены от возможности попасть в зависимость от конкретной реализации. Пользователи систем абстрактных классов *Ival\_box* не могут случайно воспользоваться механизмами конкретных реализаций, так как им доступны лишь средства, явно предоставляемые в иерархии *Ival\_box*, и ничто не наследуется неявным образом от базового класса, специфичного для конкретных систем реализации.

### 12.4.4. Локализация создания объектов

Большая часть приложения создается на основе интерфейса *Ival\_box*. Далее, если система интерфейсов эволюционирует, предоставляя новые средства, то большая часть приложения использует *Ival\_box*, *Ival\_slider* и другие интерфейсы. Однако создание объектов требует обращения к конкретным именам, специфичным для систем пользовательского интерфейса — *CW\_ival\_dial*, *BB\_flashing\_ival\_slider* и т.д. Полезно минимизировать число мест в программе, где создаются объекты. Но создание объектов трудно локализовать, если не делать это систематически.

Как всегда решение находится с помощью применения косвенных обращений. Это можно сделать несколькими способами. Самый простой — предоставить абстрактный класс с набором операций создания объектов:

```
class Ival_maker
{
public:
    virtual Ival_dial* dial(int, int) = 0;           // создать dial
    virtual Popup_ival_slider* popup_slider(int, int) = 0; // создать popup slider
    // ...
};
```

Для каждого интерфейса из семейства *Ival\_box*, который знаком пользователю, класс *Ival\_maker* предоставляет функцию создания объекта. Такой класс называют

фабрикой (*factory*), а его функции иногда называют (что может только запутать) виртуальными конструкторами (§15.6.2).

Теперь мы представим каждую систему пользовательского интерфейса классом, производным от *Ival\_maker*:

```
class BB_maker : public Ival_maker    // BB-версия
{
public:
    Ival_dial* dial (int, int) ;
    Popup_ival_slider* popup_slider (int, int) ;
    // ...
};

class LS_maker : public Ival_maker    // LS-версия
{
public:
    Ival_dial* dial (int, int) ;
    Popup_ival_slider* popup_slider (int, int) ;
    // ...
};
```

Каждая функция создает объект с требуемым интерфейсом и нужной реализацией. Например:

```
Ival_dial* BB_maker::dial (int a, int b)
{
    return new BB_ival_dial (a, b) ;
}

Ival_dial* LS_maker::dial (int a, int b)
{
    return new LS_ival_dial (a, b) ;
}
```

Получив указатель на *Ival\_maker*, пользователь может создавать объекты, даже не зная, какая система пользовательского интерфейса задействована. Например:

```
void user (Ival_maker* pim)
{
    Ival_box* pb = pim->dial (0, 99) ;    // создаем подходящий dial
    // ...
}

BB_maker BB_impl;                        // для пользователей BB
LS_maker LS_impl;                        // для пользователей LS

void driver ()
{
    user (&BB_impl) ;                    // используем BB
    user (&LS_impl) ;                    // используем LS
}
```



## 12.5. Классовые иерархии и абстрактные классы

Абстрактный класс является интерфейсом. Классовые иерархии служат средством постепенного и последовательного развертывания классов. Естественно, каждый класс формирует интерфейс для своих пользователей, и некоторые абстрактные классы предоставляют определенную полезную функциональность, но тем не менее, главное предназначение абстрактных классов и иерархий состоит в том, чтобы служить «интерфейсами» и «строительными блоками».

Классическая иерархия — это иерархия, в которой отдельные классы предоставляют пользователям полезную функциональность и служат строительными блоками для реализации более специализированных классов. Такие иерархии идеальны для построения программ путем последовательных улучшений. Они обеспечивают максимум поддержки для реализации новых классов до тех пор, пока новые классы точно вписываются в существующую иерархию.

Классические иерархии имеют тенденцию смешивать вопросы реализации с предоставляемыми пользователям интерфейсами. Здесь могут помочь абстрактные классы. Иерархии абстрактных классов предоставляют ясный и мощный способ выражения абстрактных концепций, не загрязняя их деталями реализации и не требуя дополнительных накладных расходов. В конце концов, вызов виртуальной функции обходится дешево и не зависит от того, к какой абстракции идет при этом обращение. Вызов члена абстрактного класса стоит не дороже вызова любой другой *виртуальной* функции.

Логическим завершением этих размышлений будет система, представленная пользователям как иерархия абстрактных классов и реализованная при помощи классической иерархии.

## 12.6. Советы

1. Избегайте полей типа; §12.2.5.
2. Используйте указатели и ссылки, чтобы избежать срезки; §12.2.3.
3. Используйте абстрактные классы, чтобы сфокусироваться на предоставлении ясных интерфейсов; §12.3.
4. Используйте абстрактные классы для минимизации интерфейсов; §12.4.2.
5. Используйте абстрактные классы для отделения деталей реализации от интерфейсов; §12.4.2.
6. Используйте виртуальные функции, чтобы обеспечить независимость пользовательского кода от добавления новых реализаций; §12.4.1.
7. Используйте абстрактные классы для минимизации необходимых перекомпиляций пользовательского кода; §12.4.2.
8. Используйте абстрактные классы для того, чтобы обеспечить сосуществование альтернативных реализаций; §12.4.3.
9. Класс с виртуальными функциями должен иметь виртуальный деструктор; §12.4.2.

10. Абстрактный класс в типичном случае не нуждается в конструкторах; §12.4.2.
11. Представления различных концепций должны содержаться отдельно; §12.4.1.1.

## 12.7. Упражнения

1. (\*1) Пусть дано определение

```
class base
{
public:
    virtual void iam () {cout<< "base\n"; }
};
```

Напишите два производных от **Base** класса, и для каждого из них определите **iam()**, выводящую имя класса. Создайте объекты этих классов и вызовите **iam()** для них. Присвойте значения указателей на эти объекты указателю типа **Base\*** и вызове с его помощью **iam()**.

2. (\*3.5). Реализуйте простую графическую систему, используя доступные на вашем компьютере графические средства (если их нет — используйте ASCII-представление, где пиксел, это знакоместо): **Window**(*n, m*) создает на экране область размером *nxm*. Координаты декартовы. Окно *w* типа **Window** имеет координаты *w.current()*. Начальные координаты равны **Point**(0, 0). Координаты можно задать с помощью *w.current(p)*, где *p* имеет тип **Point**. Тип **Point** задается парой координат: **Point**(*x, y*). Тип **Line** специфицируется парой точек: **Line**(*w.current()*, *p2*); класс **Shape** является общим интерфейсом для **Dot**, **Line**, **Rectangle**, **Circle** и других фигур. **Point** не является **Shape**. **Dot**(*p*) представляет точку *p* на экране. **Shape** на экране не наблюдается до вызова **draw()**. Например: *w.draw(Circle(w.current(), 10))*. Каждый **Shape** имеет девять контактных точек: *e* (*east*), *w* (*west*), *n* (*north*), *s* (*south*), *ne*, *nw*, *se*, *sw* и *c* (*center*). Например: **Line**(*x.c()*, *y.nw()*) создает линию из центра *x* в левый верхний угол *y*. После вызова **draw()** для **Shape** его текущие координаты равны *se()*. **Rectangle** определяется верхней левой и правой нижней вершинами: **Rectangle**(*w.current()*, **Point**(10, 10)). В качестве простого теста изобразите простой детский рисунок «дом с крышей, два окна и дверь».
3. (\*2) Изображение **Shape** на экране компьютера выполняется в виде множества линейных сегментов. Реализуйте операции, варьирующие внешний вид этих сегментов: *s.thickness(n)* устанавливает толщину линии в 0, 1, 2 или 3 единицы, где 2 соответствует умолчательному значению, а 0 — невидимой линии. Кроме того, сегменты могут быть **solid** (сплошная линия), **dashed** (пунктирная) и **dotted** (из точек), что устанавливается функцией **Shape::outline()**.
4. (\*2.5) Предоставьте функцию **Line::arrowhead()**, добавляющую стрелку к концу линии. У линии два конца и стрелки могут указывать оба направления вдоль линии, так что аргументы функции **arrowhead()** должны уметь задавать по крайней мере четыре альтернативы.

5. (\*3.5) Предоставьте гарантии того, что точки и линии, не попадающие в пределы **Window**, не появятся на экране компьютера. Это часто называют «отсечением» (clipping). В качестве упражнения не пользуйтесь встроенными средствами вашей графической подсистемы.
6. (\*2.5) Добавьте тип **Text** к вашей графической системе. **Text** — это прямоугольный **Shape**, внутри которого выводятся символы. По умолчанию, каждый символ занимает позицию в одну единицу по вертикали и горизонтали.
7. (\*2) Определите функцию, которая рисует соединяющую две фигуры (типа **Shape**) линию, вычисляя для этого две «ближайшие точки» и соединяя их.
8. (\*3) Добавьте к вашей простой графической системе цвет. Цвет может быть у фона, внутренних областей замкнутых фигур и у контуров фигур.
9. (\*2). Рассмотрим:

```
class Char_vec
{
    int sz;
    char element[1];

public:
    static Char_vec* new_char_vec(int s);
    char& operator[] (int i) { return element[i]; }
    // ...
};
```

Определите `new_char_vec()`, выделяющую непрерывную память для **Char\_vec** таким образом, чтобы доступ к элементам мог осуществляться по индексу через `element`. При каких обстоятельствах этот трюк может вызвать серьезные проблемы?

10. (\*2.5) Для классов **Circle**, **Square** и **Triangle**, производных от **Shape**, определите функцию `intersect()` (пересечение), которая принимает два аргумента типа **Shape\*** и вызывает другие функции, необходимые для выявления возможности пересечения фигур. Для решения этой задачи добавьте к классам соответствующие виртуальные функции. Можете не писать реальный код, выявляющий пересечение: просто убедитесь, что вызываются правильные функции. Решение таких задач называют двойной *диспетчеризацией* (*double dispatch*) или *мультиметодом* (*multi-method*).
11. (\*5) Спроектируйте и реализуйте библиотеку для решения задач моделирования, управляемых событиями. Подсказка: `<task.h>`. Это, однако, старая программа, которую вы можете улучшить. Должен быть объявлен класс **task**, объекты которого могут сохранять состояние и восстанавливать его (функции `task::save()` и `task::restore()`), так что они могут работать как индивидуальные задачи. Частные задачи определяются с помощью классов, производных от **task**. Задачи выполняют программы, специфицируемые их виртуальными функциями. Нужно реализовать возможность передачи параметров задаче с помощью аргументов конструкторов. Должен иметься планировщик (scheduler) для реализации концепции виртуального времени. Введите функцию `task::delay(long)`, которая «потребляет» виртуальное время. Сделать планировщик частью класса **task** или нет — одно из ваших собственных важ-

нейших проектных решений. Задачи должны иметь возможность взаимодействовать друг с другом (communicate). Разработайте для этого класс *queue* (очередь сообщений). Придумайте, как задача могла бы ожидать ввод из разных очередей. Обрабатывайте ошибки времени выполнения единым образом. Как можно отлаживать программы, использующие такую библиотеку?

12. (\*2) Определите интерфейсы для *Warrior* (воин), *Monster* (монстр) и *Object* (некий предмет, который можно поднять, бросить и т.д.), применяемых в ролевой игре.
13. (\*1.5) Почему одновременно есть и класс *Point*, и класс *Dot* в §12.7[2]? При каких обстоятельствах будет хорошей идеей снабдить *Shape* конкретными версиями ключевых классов вроде *Line*?
14. (\*3) Определите в общих чертах конкретные стратегии реализации примера *Ival\_box* (§12.4), основываясь на идее о том, что каждый видимый приложению класс является интерфейсом, содержащим единственный указатель на реализацию. Таким образом, каждый интерфейсный класс будет выполнять роль дескриптора для класса реализации, и будут существовать иерархии интерфейсов и иерархии реализации. Напишите фрагменты кода, иллюстрирующие возможные проблемы с приведением типов. Рассмотрите возможности упрощения использования, программирования, повторного применения интерфейсов и реализации при добавлении новых концепций к иерархии, упрощение модификации интерфейсов и реализации, а также необходимость перекомпиляции после внесения изменений в реализацию.

# Шаблоны

*Здесь — Ваша цитата.  
— Б. Страуструн*

Шаблоны — шаблон строк — конкретизация шаблона — параметры шаблонов — проверка типа — шаблоны функций — выводение типов аргументов шаблона — задание аргументов шаблона — перегрузка шаблонов функций — выбор алгоритма через аргументы шаблона — аргументы шаблона по умолчанию — специализация — наследование и шаблоны — шаблонные члены шаблонов — преобразования — организация исходного кода — советы — упражнения.

## 13.1. Введение

Независимые концепции должны представляться независимо и комбинироваться лишь при необходимости. Когда этот принцип нарушается, вы либо связываете воедино разнородные концепции, либо создаете ненужные зависимости. В любом случае вы порождаете негибкие конструкции для построения программных систем. Шаблоны обеспечивают простой способ представления широкого набора общих концепций и простые способы их комбинирования. Получающиеся результирующие классы и функции не уступают ручным специализированным вариантам по эффективности и потребляемой памяти.

Шаблоны обеспечивают непосредственную поддержку *обобщенного программирования (generic programming)* (§2.7), то есть *программирования с использованием типов в качестве параметров*. Механизм шаблонов языка C++ позволяет использовать типы в качестве параметров определений классов или функций. Шаблон зависит лишь от тех свойств параметров-типов, которые он использует явно, и не требует никаких дополнительных зависимостей между ними. В частности, не требуется, чтобы разные параметры-типы шаблона принадлежали к одной иерархии наследования.

В настоящей главе изучение шаблонов фокусируется на тех вопросах, которые заложены в дизайн и реализацию стандартной библиотеки, а также на вопросах ее

практического использования. Стандартная библиотека требует существенно большей степени общности, гибкости и эффективности, чем большинство иных программных систем. Следовательно, использованные в ней технологии действенны и эффективны при разработке программных решений для широкого круга задач и предметных областей. Эти технологии позволяют разработчику скрывать сложные технические детали за простыми интерфейсами и открывать их пользователю лишь тогда, когда в них возникает действительная нужда. Например, через функциональный интерфейс `sort(v)` открывается доступ ко множеству алгоритмов, сортирующих наборы элементов разного типа, содержащихся в самых разных контейнерах. Наиболее подходящая для конкретного `v` функция сортировки будет выбрана автоматически.

Любая существенная абстракция стандартной библиотеки представлена шаблоном (например, ***string***, ***ostream***, ***complex***, ***list*** и ***map***), равно как и фундаментальные операции над ними (сравнение строк, операция вывода `<<`, сложение комплексных чисел, получение следующего элемента списка, сортировка). Специально посвященная стандартной библиотеке часть III настоящей книги является обильным источником по «настоящим» шаблонам и практическим методам работы с ними. А текущая глава опирается на небольшие примеры, которые иллюстрируют следующие технические аспекты шаблонов и фундаментальные способы их использования:

- §13.2: Основные механизмы определения шаблонов классов и их использование.
- §13.3: Шаблоны функций, перегрузка и логический вывод типа.
- §13.4: Параметры шаблонов, управляющие поведением обобщенных алгоритмов.
- §13.5: Множественные определения, обеспечивающие альтернативные реализации шаблонов.
- §13.6: Наследование и шаблоны (полиморфизм на стадии выполнения и полиморфизм на стадии компиляции).
- §13.7: Организация исходного кода.

Начальное знакомство с шаблонами состоялось в §2.7.1 и §3.8. Детальные правила разрешения имен шаблонов, синтаксис шаблонов и т.д. представлены в §С.13.

## 13.2. Простой шаблон строк

Рассмотрим строку символов. Строка является классом, хранящим символы и обеспечивающим доступ по индексу, конкатенацию, сравнение и другие операции, которые мы обычно ассоциируем с понятием «строка». Такое поведение желательно реализовать для разных наборов символов, например, европейского набора символов, набора китайских символов, греческих и т.д. Поэтому целесообразно определить суть понятия «строка» в отрыве от специфических символьных наборов. Это определение базируется в основном на идее о том, что символы можно копировать. Таким образом, общий строковый тип можно создать из строк символов типа `char` (§11.12), сделав тип символов параметром:

```

template<class C> class String
{
    struct Srep;
    Srep* rep;

public:
    String () ;
    String (const C*) ;
    String (const String&) ;

    C read (int i) const;
    // ...
};

```

Префикс *template<class C>* указывает, что объявляется шаблон, и что идентификатор *C* является параметром типа. В теле определения шаблона *C* используется точно так же, как и другие имена типов. Область видимости *C* простирается до конца определения шаблона. Обратите внимание на то, что *template<class C>* означает, что *C* — это любой *тип*, не обязательно *класс*.

В соответствии с определением шаблона его имя с последующим конкретным типом, заключенным в угловые скобки, является именем класса и его можно использовать так же, как имя любого другого класса. Например:

```

String<char> cs;
String<unsigned char> us;
String<wchar_t> ws;

class Jchar
{
    // японские символы
};

String<Jchar> js;

```

За исключением специального синтаксиса имени, *String<char>* ведет себя так же, как и определенный в §11.12 класс *String*. Преобразовав *String* в шаблон, мы получили возможность использовать средства, предназначенные для работы со строками символов типа *char*, в работе со строками символов иных типов. Например, применив строковый шаблон и стандартный библиотечный контейнер *map*, мы превратим наш пример с подсчетом слов из §11.8 в следующий код:

```

int main ()                // подсчет кол-ва вхождений каждого слова
{
    String<char> buf;
    map<String<char>, int> m;

    while (cin>>buf) m[buf]++;
    // вывод результата
}

```

А вариант для японских символов типа **Jchar** будет выглядеть так:

```
int main ()                // подсчет кол-ва вхождений каждого слова
{
    String<char> buf;
    map<String<Jchar>, int> m;

    while (cin>>buf) m[buf]++;
    // вывод результата
}
```

В стандартной библиотеке определяется шаблонный класс **basic\_string**, аналогичный шаблонному варианту **String** (§11.12, §20.3). А **string** определяется в стандартной библиотеке как синоним для **basic\_string<char>**:

```
typedef basic_string<char> string;
```

Это позволяет записать программу подсчета слов в следующем виде:

```
int main ()                // подсчет кол-ва вхождений каждого слова
{
    string buf;
    map<string, int> m;

    while (cin>>buf) m[buf]++;
    // вывод результата
}
```

Вообще, **typedef** часто применяют для сокращения длинных имен классов, сгенерированных из шаблонов. Поскольку мы часто предпочитаем не знать всех деталей определения типа, **typedef** помогает скрыть тот факт, что тип сгенерирован из шаблона.

### 13.2.1. Определение шаблона

Класс, генерируемый из классового шаблона, является абсолютно нормальным классом. Применение шаблонов не требует никаких дополнительных механизмов поддержки времени выполнения по сравнению с применением вручную запрограммированных классов. Не предполагает механизм шаблонов и какого-либо уменьшения объема сгенерированного кода.

Обычно хорошей идеей является тестирование и отладка конкретного класса, например **String**, до его преобразования в шаблон (в нашем примере — в **String<C>**). Таким образом удастся разрешить многие проблемы проектирования и выловить ошибки реализации, оставаясь в рамках конкретного класса. Такой способ отладки знаком всем программистам, а многим из них легче работать с конкретным примером, чем с абстрактной концепцией. Позднее, мы можем сосредоточиться на проблемах, связанных с обобщением, не отвлекаясь при этом на борьбу с обычными ошибками. Аналогично, разбираясь с шаблоном, бывает полезным сначала представить себе его поведение для конкретного типа вместо произвольного типа-параметра, и лишь потом попробовать осознать шаблон во всей его общности.

Члены шаблона класса объявляются и определяются так же, как и в случае обычных классов. Функцию-член шаблона класса не обязательно определять внут-



ри самого шаблона. Ее можно определить где-нибудь еще, как это имеет место и для функций-членов нешаблонных классов (§С.13.7). Члены шаблонных классов сами являются шаблонами, параметризуемыми с помощью параметров шаблона класса. Когда функции-члены определяются вне тела определения шаблона, они должны явным образом указываться как шаблоны. Например:

```
template<class C> struct String<C> : Srep
{
    C* s;                // указатель на элементы
    int sz;              // число элементов
    int n;               // подсчет ссылок
    // ...
};

template<class C> C String<C> : : read (int i) const { return rep->s[i]; }

template<class C> String<C> : : String ()
{
    rep = new Srep (0, C());
}
```

Параметры шаблонов, такие как *C*, являются именно параметрами, а не именами типов, определенными внешним образом по отношению к шаблону. Это, однако, никоим образом не влияет на способ, каким мы его используем при написании кода шаблона. В области видимости *String<C>* квалификация с *<C>* избыточна для имени шаблона, так что *String<C> : : String* есть имя конструктора. Но если уж мы хотим предельно явных формулировок, то можно писать и так:

```
template<class C> String<C> : : String<C>
{
    Rep = new Srep (0, C());
}
```

Аналогично тому, как для обычного класса может существовать лишь одно определение для его функции-члена, также может существовать лишь один шаблон функции в качестве определения функции-члена классowego шаблона. Перегрузка возможно только для шаблонов-функций (§13.3.2), в то время как специализация (§13.5) позволяет обеспечить альтернативные реализации шаблонов.

Перегрузка имени шаблона класса невозможна, поэтому в области видимости шаблона никакая другая сущность с тем же именем объявлена быть не может (см. также §13.5). Например:

```
template<class T> class String { /* ... */ };
class String { /* ... */ }; // error: двойное определение
```

Тип, используемый в качестве аргумента шаблона, должен обеспечивать интерфейс, ожидаемый шаблоном. Например, тип, используемый в шаблоне *String*, должен предоставлять обычные операции копирования (§10.4.4.1, §20.2.1). Еще раз подчеркнем, что разные типы, используемые в качестве аргументов шаблона, не обязаны находиться между собой в отношениях наследования.

### 13.2.2. Конкретизация шаблона (template instantiation)

Процесс генерации объявления класса по классовому шаблону и его параметру типа называют *конкретизацией шаблона* (*template instantiation*) (§С.13.7)<sup>1</sup>. Аналогично, конкретная функция генерируется из шаблона функции и заданного значения для параметра типа. Явно определенные программистом версии шаблона для специфических значений параметров типа называются *специализациями* (*specializations*).

Компилятор автоматически генерирует все конкретные версии функций (так что это не является заботой программиста) для любого набора конкретных значений параметров типа (§С.13.7). Например:

```
String<char> cs;

void f()
{
    String<Jchar> js;

    Cs="Какой код сгенерировать, решает компилятор";
}
```

Здесь компилятор генерирует типы *String<char>*, *String<Jchar>* и соответствующие им *Srep* типы, код для их деструкторов, умолчательных конструкторов и для операции присваивания *String<char>: :operator= (char\*)*. Так как остальные функции-члены здесь не используются, то их код и не генерируется. Сгенерированные классы являются совершенно обычными классами, подчиняющимися всем правилам, имеющим отношение к классам вообще. Аналогично, сгенерированные функции подчиняются всем правилам, относящимся к функциям вообще.

Очевидно, что шаблоны обеспечивают мощный способ генерации кода из относительно небольших определений. Поэтому требуется некоторая осторожность во избежание перегрузки памяти множеством почти идентичных определений функций (§13.5).

### 13.2.3. Параметры шаблонов

Шаблоны могут иметь параметры типа, параметры обычных фиксированных типов (например, *int*) и шаблонные параметры (§С.13.3). Естественно, что один шаблон может иметь несколько параметров. Например:

```
template<class T, T def_val> class Cont { /* ... */ };
```

Из приведенного примера видно, что параметр типа может использоваться для объявления последующих параметров шаблона.

Аргументы целого типа очень удобны для задания размеров и границ. Например:

```
template<class T, int max> class Buffer
{
    T v[max];
};
```

<sup>1</sup> Встречающийся в литературе термин *инстанцирование* труден для произношения. — Прим. ред.

```
public:
    Buffer() {}
    // ...
};

Buffer<char, 128> cbuf;
Buffer<int, 5000> ibuf;
Buffer<Record, 8> rbuf;
```

Простые, ограниченные контейнеры, вроде **Buffer**, полезны в задачах, требующих высочайшей эффективности и компактности кода (так что там не подходят универсальные контейнеры типа **string** и **vector**). Передача размера в виде аргумента шаблона **Buffer** позволяет избежать затратных операций динамического выделения свободной памяти. Другим примером может послужить тип **Range** из §25.6.1.

Аргумент шаблона может быть константным выражением (§C.5), адресом объекта или функции с внешней компоновкой (§9.2); или указателем на член класса (§15.5). Указатель, используемый в качестве аргумента шаблона, должен иметь форму **&of**, где **of** — это имя объекта или функции; или же иметь форму **f**, где **f** — это имя функции. Указатель на член класса должен быть в форме **&X: of**, где **of** — имя члена. Строковые литералы в качестве аргументов шаблонов не допускаются.

Целый аргумент шаблона должен быть константой:

```
void f(int i)
{
    Buffer<int, i> bx;    // error: требуется константное выражение
}
```

Параметр шаблона, не являющийся типом, в теле шаблона является константой, так что попытка изменить его трактуется как ошибка.

### 13.2.4. Эквивалентность типов

Располагая шаблоном, мы можем генерировать конкретные типы, предоставляя конкретные значения для аргументов шаблона. Например:

```
String<char> s1;
String<unsigned char> s2;
String<int> s3;

typedef unsigned char Uchar;
String<Uchar> s4;
String<char> s5;

Buffer<String<char>, 10> b1;
Buffer<char, 10> b2;
Buffer<char, 20-10> b3;
```

Предоставляя один и тот же набор аргументов шаблона, мы имеем дело с одним и тем же сгенерированным типом. Но что в данном контексте означает «один и тот же»? Так как **typedef** не создает нового типа, то **String<Uchar>** есть то же самое, что и **String<unsigned char>**. И наоборот, так как **char** и **unsigned char** суть разные типы (§4.3), то и типы **String<char>** и **String<unsigned char>** тоже разные.

Компилятор может вычислять константные выражения, поэтому тип **Buffer<char, 20-10>** эквивалентен типу **Buffer<char, 10>**.

### 13.2.5. Проверка типов

Шаблоны определяются и впоследствии используются в комбинации с набором аргументов шаблона. В определении шаблона можно обнаружить ошибки, не связанные непосредственно с возможными конкретными значениями аргументов шаблона. Например:

```
template<class T> class List
{
    struct Link
    {
        Link* pre;
        Link* suc;
        T val;
        Link(Link* p, Link* s, const T& v) : pre(p), suc(s), val(v) {}
    } // syntax error: отсутствует точка с запятой

    Link* head;

public:
    Link() : head(7) {} // error: указатель инициализируется типом int
    List(const T& t) : head(new Link(0, 0, t)) {} // error: неопределенный идентификатор 'o'
    // ...
    void print_all() const {for (Link* p = head; p; p=p->suc) cout<< p->val<< '\n'; }
};
```

Компилятор в состоянии распознать простые семантические ошибки в точке определения или позднее в точке использования. Пользователи предпочли бы более раннее выявление ошибок, но не все такие «простые» ошибки легко обнаружить. В примере выше я сделал три «ошибки». Совершенно независимо от того, что есть параметр шаблона, указатель типа **Link\*** не может инициализироваться целым значением 7. Аналогично, идентификатор **o** (опечатка — должен был быть нуль) не может быть аргументом конструктора **List<T>::Link**, поскольку в доступной области видимости такого имени нет.

Любое имя, используемое в определении шаблона, должно либо входить в его область видимости, либо очевидным образом зависеть от параметра шаблона (§С.13.8.1). Наиболее очевидными формами зависимости от параметра шаблона **T** является объявление члена типа **T** или функции-члена с аргументом типа **T**. Более тонким примером служит выражение **cout<<p->val** в функции-члене **List<T>::print\_all()**.

Ошибки, имеющие отношение к использованию конкретных значений параметров шаблонов, нельзя обнаружить до момента конкретизации шаблона. Например:

```
class Rec { /* ... */ };

void f(const List<int>& li, const List<Rec>& lr)
{
    li.print_all();
    lr.print_all();
}
```

Здесь выражение **li.print\_all()** вполне корректно, но выражение **lr.print\_all()** порождает сообщение об ошибке использования типа, ибо операция вывода **<<** не определена для типа **Rec**. Самой ранней точкой обнаружения ошибок, связанных

с параметрами шаблонов, является точка первого указания конкретного типа для параметра шаблона. Такую точку программы принято называть первой точкой конкретизации шаблона или просто *точкой конкретизации шаблона* (*point of instantiation*) (см. §C.13.7). Однако в реальных системах разработки (включающих компиляторы) проверки подобного рода могут откладываться до этапа компоновки. Если бы в представленной выше единице трансляции находилось лишь объявление функции `print_all()`, а не ее определение, то проверка соответствия типов могла быть отложена до более поздних этапов (§13.7). Независимо от того, на каком этапе производится проверка, применяемые для ее выполнения правила одни и те же. Естественно, что для пользователей более ранние проверки предпочтительны. Можно наложить ограничения на допустимые аргументы шаблонов посредством функций-членов (§13.9[16]).

### 13.3. Шаблоны функций

Для большинства людей самым первым и очевидным применением шаблонов является определение или использование таких контейнеров, как *basic\_string* (§20.3), *vector* (§16.3), *list* (§17.2.2) и *map* (§17.4.1). Сразу за этим возникает потребность в шаблонных функциях. Сортировка массивов иллюстрирует сказанное:

```
template<class T> void sort (vector<T>&);    // объявление
void f(vector<int>& vi, vector<string>& vs)
{
    sort (vi);    // sort(vector<int>&);
    sort (vs);    // sort(vector<string>&);
}
```

Когда шаблонная функция вызывается, типы фактических аргументов вызова определяют, какая конкретная версия функционального шаблона используется — аргументы шаблона *выводятся* (*deduced*) из типов аргументов функционального вызова (§13.3.1).

Естественно, что шаблон функции должен быть где-то определен (§C.13.7):

```
template<class T> void sort (vector<T>& v)    // определение
                                           // Shell sort (Knuth, Vol. 3, pg. 84).
{
    const size_t n = v.size();
    for (int gap=n/2; 0<gap; gap /= 2)
        for (int i=gap; i<n; i++)
            for (int j=i-gap; 0<=j; j -= gap)
                if (v[j+gap]<v[j])           // swap v[j] u v[j+gap]
                {
                    T temp = v[j];
                    v[j]=v[j+gap];
                    v[j+gap] = temp;
                }
            else break;
```

Сравните это определение с вариантом функции `ssort()` из §7.7. Шаблонная версия яснее и короче, поскольку опирается на типы сортируемых элементов массива. Скорее всего она и более эффективна, поскольку для сравнения элементов не вызывает функцию сравнения по указателю на эту функцию. То есть никаких косвенных вызовов не используется, а встраивание простой операции `<` весьма несложно.

Для дальнейшего упрощения можно воспользоваться стандартной библиотечной функцией `swap()` (§18.6.8), в результате чего код становится короче и принимает естественный вид:

```
if (v[j+gap] < v[j]) swap(v[j], v[j+gap]);
```

Никаких дополнительных накладных расходов при этом не вносится.

В данном примере для сравнения элементов используется операция `<`. Поскольку не каждый тип априорно располагает такой операцией, то это ограничивает возможности данной версии функции `sort()`. Однако указанное ограничение легко можно обойти (см. §13.4).

### 13.3.1. Аргументы функциональных шаблонов

Шаблоны функций чрезвычайно важны для написания обобщенных алгоритмов, применимых к широкому спектру контейнеров (§2.7.2, §3.8, глава 18). Здесь самым существенным моментом является возможность выведения аргументов шаблона по типам аргументов функционального вызова.

Компилятор осуществляет логический вывод аргументов шаблона (параметров типа и иных параметров) при условии, что список аргументов функции однозначно идентифицирует набор аргументов шаблона (§C.13.4). Например:

```
template<class T, int max> T& lookup(Buffer<T, max>& b, const char* p);

class Record
{
    const char v[12];
    // ...
};

Record& f(Buffer<Record, 128>& buf, const char* p)
{
    return lookup(buf, p);    // вызвать lookup(), где T есть Record, а i - 128
}
```

Для этого примера выводится, что *T* есть *Record*, а *max* равно 128.

Заметьте, что параметры классовых шаблонов никогда не выводятся. Причина заключается в том, что гибкость, порождаемая наличием нескольких конструкторов класса, в ряде случаев делает вывод параметров шаблона невозможным, а во многих других случаях — неоднозначным. Специализация классовых шаблонов предоставляет возможность неявного выбора между разными реализациями класса (§13.5). Если требуется создать объект выводимого типа, можно просто вызвать функцию, специально предназначенную для такого создания объектов (см., например, `make_pair()` из §17.4.1.2).

Если параметр шаблона не удастся вывести из аргументов функционального вызова (§C.13.4), его нужно указать явно. Это делается точно так же, как в случае явного задания параметров для шаблонного класса. Например:

```

template<class T> class vector { /* ... */ };
template<class T> T* create() ;           // создать T и вернуть указатель на T

void f()
{
    vector<int> v;                        // класс; аргумент шаблона int
    int* p=create<int>() ;               // функция; аргумент шаблона int
}

```

Типичным примером явной спецификации типа является необходимость задания конкретного типа возвращаемого шаблонной функцией значения:

```

template<class T, class U> T implicit_cast(U u) {return u; }

void g(int i)
{
    implicit_cast(i) ;                   // error: невозможно вывести T
    implicit_cast<double>(i) ;           // T есть double; U есть int
    implicit_cast<char, double>(i) ;     // T есть char; U есть double
    implicit_cast<char*, int>(i) ;       // T есть char*; U есть int;
                                          // error: невозможно привести int к char*
}

```

Как и в случае умолчательных значений аргументов обычных функций (§7.5), только концевые аргументы могут опускаться в списке явно задаваемых аргументов шаблона.

Явная спецификация аргументов шаблона позволяет создавать семейства функций преобразования и семейства функций, предназначенных для создания объектов (§13.3.2, §C.13.1, §C.13.5). Часто используются явные версии неявных преобразований (§C.6), такие как *implicit\_cast*(). Синтаксис операций *dynamic\_cast*, *static\_cast* и т.д. (§6.2.7, §15.4.1) соответствует синтаксису явной квалификации шаблонных функций. Однако некоторые аспекты операций преобразования встроенных типов невыразимы другими средствами языка.

### 13.3.2. Перегрузка функциональных шаблонов

Можно объявить несколько шаблонов функций с одним и тем же именем, или даже комбинацию шаблонных и обычных функций с совпадающими именами. Когда вызывается функция с перегруженным именем, применяется механизм разрешения перегрузки, позволяющий выбрать наиболее подходящий вариант обычной или шаблонной функции. Например:

```

template<class T> T sqrt(T) ;
template<class T> complex<T> sqrt(complex<T>) ;
double sqrt(double) ;

void f(complex<double> z)
{
    sqrt(2) ;                            // sqrt<int>(int)
    sqrt(2.0) ;                          // sqrt(double)
    sqrt(z) ;                            // sqrt<double>(complex<double>)
}

```

Точно так же, как понятие функционального шаблона является обобщением понятия обычной функции, правила разрешения перегрузки в присутствии функциональных шаблонов являются обобщением правил разрешения перегрузки обычных функций. Сначала отбирается та конкретизация функциональных шаблонов, которая наилучшим образом соответствует фактическим параметрам вызова. Затем она конкурирует с обычными функциями:

1. Отберите конкретизации функциональных шаблонов, которые примут далее участие в процессе разрешения перегрузки. С этой целью рассматривайте каждый функциональный шаблон по отдельности (не обращая внимание на наличие иных шаблонов и обычных функций с тем же именем). Для вызова `sqrt(z)` такой отбор в нашем примере порождает двух кандидатов: `sqrt<double> (complex<double>)` и `sqrt<complex<double> > (complex<double>)`.
2. При наличии нескольких кандидатов, порожденных из разных функциональных шаблонов, для дальнейших шагов разрешения перегрузки отбираются наиболее специфические (более узкие, специальные) варианты (§13.5.1). Среди кандидатов, отобранных для нашего примера на шаге [1], предпочтение отдается варианту `sqrt<double> (complex<double>)` как более специфическому (любой вызов, удовлетворяющий шаблону `sqrt<T> (complex<T>)`, удовлетворяет и шаблону `sqrt<T> (T)`).
3. К отобранным на шаге [2] кандидатам добавляются обычные функции с тем же самым именем и далее применяются правила разрешения перегрузки для обычных функций (§7.4). Если аргумент функционального шаблона был определен в процессе логического вывода (§13.3.1), то к этому аргументу неприменимы «продвижения», стандартные преобразования и пользовательские преобразования. В нашем примере для вызова `sqrt(2)` точным соответствием является `sqrt<int> (int)`, так что он имеет преимущество перед `sqrt(double)`.
4. Если одинаково хорошо подходят и некоторая конкретизация функционального шаблона и обычная функция, то предпочтение отдается обычной функции. Следовательно, для `sqrt(2.0)` предпочтение отдается обычной функции `sqrt(double)`, а не конкретизации `sqrt<double> (double)`.
5. Если соответствий не найдено, вызов ошибочен. Если найдены два или более одинаково подходящих варианта, то вызов трактуется как неоднозначный (ambiguous) и тоже ошибочный.

Например:

```
template<class T> T max (T, T) ;

const int s = 7;

void k ()
{
    max (1, 2) ;           // max<int>(1,2)
    max ('a', 'b') ;       // max<char>('a','b')
    max (2.7, 4.9) ;       // max<double>(2.7,4.9)
    max (s, 7) ;           // max<int>(int(s),7) (тривиальное преобразование)

    max ('a', 1) ;         // error: ambiguous (нет стандартного преобразования)
    max (2.7, 4) ;         // error: ambiguous (нет стандартного преобразования)
}
```



Здесь две неоднозначности можно разрешить либо явной квалификацией вызова

```
void f()
{
    max<int>('a', 1);    // max(int(int('a'), 1)
    max<double>(2.7, 4); // max<double>(2.7, double(4))
}
```

либо добавлением подходящих определений «разрешающих функций»:

```
inline int max(int i, int j) {return max<int>(i, j); }
inline double max(int i, double d) {return max<double>(i, d); }
inline double max(double d, int i) {return max<double>(d, i); }
inline double max(double d1, double d2) {return max<double>(d1, d2); }

void g()
{
    max('a', 1);    // max(int('a'), 1)
    max(2.7, 4);    // max(2.7, 4)
}
```

Для обычных функций применяются обычные правила разрешения перегрузки (§7.4), а использование **inline** гарантирует отсутствие дополнительных накладных расходов. Разрешающие функции **max()** столь тривиальны, что можно было бы полностью написать их определения. Однако применение явных конкретизаций функционального шаблона является простым и общим способом определения разрешающих функций.

Сформулированные правила разрешения перегрузки гарантируют корректное взаимодействие функциональных шаблонов и механизма наследования:

```
template<class T> class B { /* ... */ };
template<class T> class D : public B<T> { /* ... */ };
template<class T> void f(B<T>*) ;

void g(B<int>* pb, D<int>* pd)
{
    f(pb); // f<int>(pb)
    f(pd); // f<int>(static_cast<B<int>*>(pd)); // используется стандартное
                                                // преобразование D<int>* в B<int>*
}
```

В этом примере функциональный шаблон **f()** принимает в качестве аргумента **B<T>\*** для любого типа **T**. Когда фактический аргумент вызова имеет тип **D<int>\***, компилятор легко приходит к выводу, что полагая **T** типом **int**, вызов однозначно разрешается в пользу **f(B<int>\*)**.

Аргумент функции, не используемый в процессе вывода параметра функционального шаблона, трактуется в точности как аргумент обычной (нешаблонной) функции. В частности, к нему применяются обычные правила приведения типов. Рассмотрим пример:

```
template<class T, class C> T get_nth(C& p, int n); // получить n-ый элемент
```

Эта функция предположительно возвращает значение *n*-го элемента контейнера типа **C**. Так как **C** подлежит логическому выводу из фактического аргумента вызова

функции `get_nth()`, то не допускается преобразований ее первого аргумента. А второй ее аргумент абсолютно обычный, так что по отношению к этому аргументу рассматривается полный набор всех преобразований. Например:

```
class Index
{
public:
    operator int () ;
    // ...
};

void f(vector<int>& v, short s, Index i)
{
    int i1 = get_nth(v, 2) ;      // точное соответствие
    int i2 = get_nth(v, s) ;      // стандартное преобразование: short в int
    int i3 = get_nth(v, i) ;      // пользовательское преобразование: Index в int
}
```

### 13.4. Применение аргументов шаблона для формирования различных вариантов поведения кода

Рассмотрим сортировку строк. Здесь присутствуют три концепции: строка, тип элементов и критерий, используемый сортирующим алгоритмом для сравнения элементов строки.

Мы не можем встроить критерий сортировки непосредственно в контейнер, ибо контейнер не должен навязывать свои проблемы типам элементов. Мы не можем также встраивать критерии и в типы элементов, ибо существует много разных критериев сортировки. Таким образом, критерий сортировки не встраивается ни в контейнер, ни в тип элементов.

Критерий сортировки нужно указывать лишь в тот момент, когда возникла нужда в выполнении конкретной операции. Например, какими критериями сортировки нужно воспользоваться в случае строк, содержащих имена шведов? В этом случае могут использоваться две разные сортирующие последовательности (два разных способа нумерации символов). Ясно, что ни общий строковый тип, ни общий алгоритм сортировки не должны ничего знать про способы упорядочения шведских имен. Поэтому любое общее решение нуждается в том, чтобы сортирующий алгоритм формулировался в терминах, допускающих настройку не только по типам, но и по вариантам использования этих типов. В качестве примера рассмотрим обобщение стандартной библиотечной функции `strcmp()` для строк элементов типа `T` (§13.2):

```
template<class T, class C>
int compare(const String<T>& str1, const String<T>& str2)
{
    for (int i=0; i<str1.length() && i<str2.length(); i++)
        if (!C::eq(str1[i], str2[i])) return C::lt(str1[i], str2[i]) ? -1 : 1;
```

```
return str1.length() - str2.length();
}
```

Если потребуется, чтобы *compare()* игнорировала регистр букв, или действовала в соответствии со специфическими национальными настройками и т.п., то тогда нужно определить подходящий вариант *C::eq()* и *C::lt()*. Это позволяет выражать любой алгоритм (сравнение, сортировку и т.п.) в терминах «С-операций» и контейнеров. Например:

```
template<class T> class Cmp    // обычное (умолчательное) сравнение
{
public:
    static int eq(T a, T b) {return a==b;}
    static int lt(T a, T b) {return a<b;}
};

class Literate                // сравнение шведских имен по литературным правилам
{
public:
    static int eq(char a, char b) {return a==b;}
    static int lt(char, char);    // поиск в таблице по коду символа (§13.9[14])
};
```

Теперь мы можем выбирать варианты поведения кода (зависящие в данном случае от правил сравнения строк) явным заданием аргументов шаблона:

```
void f(String<char> swede1, String<char> swede2)
{
    compare<char, Cmp<char>> > (swede1, swede2);
    compare<char, Literate> (swede1, swede2);
}
```

Передача операций сравнения в качестве аргумента шаблона имеет два существенных преимущества перед альтернативными решениями, например, перед передачей указателей на функции. Во-первых, можно передать несколько операций в качестве единственного аргумента без дополнительных затрат на выполнение кода. Кроме того, операции сравнения *eq()* и *lt()* легко встраиваются, в то время как встраивание вызова функции по указателю на нее является довольно сложной задачей для компилятора.

Естественно, что операции сравнения можно реализовать и для встроенных типов, и для пользовательских типов. Это важно для построения обобщенных алгоритмов, работающих с типами, обладающими нетривиальными критериями сравнения (см. §18.4).

Каждый генерируемый из шаблона класс получает копию каждого *статического* члена классового шаблона (см. §С.13.1).

### 13.4.1. Параметры шаблонов по умолчанию

Необходимость явного задания критерия сравнения при каждом вызове функции несколько утомительна. По счастью, в общем случае можно опереться на умолчательные варианты критериев, а в редких случаях явно задавать их более специфические варианты. Это можно реализовать с помощью перегрузки:

```
template<class T, class C>
int compare (const String<T>& str1, const String<T>& str2); // сравниваем, используя C

template<class T>
int compare (const String<T>& str1, const String<T>& str2); // сравниваем, используя Cmp<T>
```

По-другому, обычный вариант критерия сравнения можно указать в качестве аргумента шаблона по умолчанию:

```
template<class T, class C = Cmp<T> >
int compare (const String<T>& str1, const String<T>& str2)
{
    for (int i=0; i<str1.length () && i< str2.length (); i++)
        if (! C::eq (str1 [i], str2 [i]) ) return C::lt (str1 [i], str2 [i]) ? -1 : 1;

    return str1.length () - str2.length ();
}
```

Теперь можно писать так:

```
void f (String<char> swede1, String<char> swede2)
{
    compare (swede1; swede2); // используется Cmp<char>
    compare<char, Literate> (swede1, swede2); // используется Literate
}
```

Или для менее экзотического (по сравнению со шведскими фамилиями) сравнения с учетом и без учета регистра букв:

```
class No_case { /* ... */ };

void f (String<char> s1, String<char> s2)
{
    compare (s1, s2); // учитываем регистр
    compare<char, No_case> (s1, s2); // не учитываем регистр
}
```

Технология, позволяющая осуществлять задание вариантов поведения кода через аргументы шаблонов с использованием их умолчательных значений для наиболее общих случаев, широко применяется в стандартной библиотеке (§18.4). Достаточно странно, но она не используется для типа **basic\_string** (§13.2, глава 20). Параметры шаблонов, применяемые для задания вариантов поведения кода, часто называют «свойствами» (traits). Например, строки стандартной библиотеки используют **char\_traits** (§20.2.1), стандартные алгоритмы полагаются на соответствующие свойства итераторов (§19.2.2), а контейнеры стандартной библиотеки — на **аллокаторы** (allocators) (§19.4).

Проверка семантики умолчательного значения параметра шаблона проводится только в случае его использования. Например, пока мы воздерживаемся от использования умолчательного значения **Cmp<T>**, мы можем использовать функцию **compare()** для сравнения строк элементов типа **X**, для которых **Cmp<X>** не компилируется (например, потому что операция **<** не определена для типа **X**). Это очень важно для стандартных контейнеров, применяющих умолчательные значения аргументов шаблона (§16.3.4).

## 13.5. Специализация

По умолчанию, шаблон является единственным определением, которое должно использоваться для всех конкретных аргументов шаблона, задаваемых пользователем. Это не всегда оптимально для разработчика шаблона. Для него часто актуально рассуждать так: «если аргумент шаблона будет указателем, нужно применить вот эту реализацию, а если нет — то другую реализацию» или «в случае, когда аргумент шаблона не является указателем на класс, производный от *My\_base*, выдать сообщение об ошибке». Такие проблемы можно решить, обеспечив альтернативные определения шаблонов и заставив компилятор выбирать нужный вариант на основе аргументов шаблона, указанных при его использовании. Такие альтернативные определения шаблона называются *специализациями, определяемыми пользователем* (*user-defined specializations*), или просто *пользовательскими специализациями* (*user specializations*)<sup>1</sup>.

Рассмотрим типичные варианты применения шаблона *Vector*:

```
template<class T> class Vector           // general vector type
{
    T* v;
    int sz;

public:
    Vector ();
    explicit Vector (int) ;

    T& elem (int i) {return v[i] ;}
    T& operator[] (int i) ;

    void swap (Vector&) ;
//...
};

Vector<int> vi;
Vector<Shape*> vps;
Vector<string> vs;
Vector<char*> vpc;
Vector<Node*> vpn;
```

Большинство векторов строятся на указателях некоторого типа. На то существует ряд причин, но главная заключается в том, что только указатели обеспечивают полиморфное поведение на этапе выполнения (§2.5.4, §12.2.6). В итоге, любой программист, практикующий объектно-ориентированное программирование и использующий безопасные по отношению к типам контейнеры (такие как контейнеры стандартной библиотеки), так или иначе приходит к контейнерам указателей.

В большинстве реализаций C++ код шаблонов функций реплицируется. Это хорошо с точки зрения производительности, но при недостаточной осторожности может привести к разбуханию кода в критических случаях, как в случае шаблона *Vector*.

<sup>1</sup> Еще чаще их называют просто *специализациями*. — Прим. ред.

По счастью, имеется очевидное решение. Все контейнеры указателей могут разделить единственную специальную реализацию, называемую *специализацией* (*specialization*). Определяем специфическую версию (специализацию) шаблона **Vector** для указателей на **void**:

```
template<> class Vector<void*>
{
    void** p;
    // ...
    void* & operator[] (int i) ;
};
```

Эта специализация может затем использоваться как общая реализация для всех векторов указателей.

Префикс **template<>** говорит о том, что определяется специализация, не нуждающаяся в параметрах шаблона. Аргументы шаблона, для которых эта специализация должна использоваться, указываются в угловых скобках после имени. Таким образом, **<void\*>** означает, что данное определение должно использоваться в качестве реализации для всех **Vector**, у которых **T** есть **void\***.

Определение **Vector<void\*>** называется *полной специализацией* (*complete specialization*), поскольку параметры шаблона отсутствуют. Оно используется в клиентском коде следующим образом:

```
Vector<void*> vpv;
```

Для того чтобы определить специализацию, которую можно использовать для любого вектора указателей (и только для векторов указателей), требуется *частичная специализация* (*partial specialization*):

```
template<class T> class Vector<T*> : private Vector<void*>
{
public:
    typedef Vector<void*> Base;

    Vector() {}
    explicit Vector(int i) : Base(i) {}

    T* & elem(int i) {return reinterpret_cast<T*>(&(Base::elem(i))); }
    T* & operator[] (int i) {return reinterpret_cast<T*>(&(Base::operator[] (i))); }
    // ...
};
```

*Паттерн (схема, форма) специализации* (*specialization pattern*) в виде **<T\*>** после имени означает, что специализация используется для любого типа указателей; то есть это определение используется всегда, когда указывается аргумент шаблона в виде **T\***. Например:

```
Vector<Shape*> vps; // <T*> есть <Shape*>, так что T есть Shape
Vector<int**> vppi; // <T*> есть <int**>, так что T есть int*
```

Отметим, что когда используется частичная специализация, параметр шаблона выводится из паттерна специализации, так что в этом случае параметр шаблона не совпадает с фактическим аргументом шаблона. В частности, для **Vector<Shape\*>** параметр **T** есть **Shape**, а не **Shape\***.

Определив рассмотренную частичную специализацию для шаблона *Vector*, мы получили общую реализацию для любых векторов указателей. При этом по сути дела *Vector<T\*>* является интерфейсом к *Vector<void\*>*, реализованным через наследование и оптимизируемым за счет встраивания.

Важно, что все эти усовершенствования реализации *Vector* выполнены без изменения интерфейса пользователя. Специализация как раз и задумана как альтернативная реализация для специфических случаев использования одного и того же пользовательского интерфейса. Естественно, можно назначить разные имена общему шаблону (вектору вообще) и специализации (вектору указателей), но это будет только путать пользователей, и многие из них могут и не воспользоваться специализацией для векторов указателей, что лишь раздует суммарный объем их кода. Гораздо лучше скрывать важные детали реализации за общим интерфейсом.

Рассмотренная техника программирования, препятствующая разбуханию кода, доказала свою эффективность на практике. Люди, не применяющие этой техники программирования (будь-то в языке C++, или в каком-либо ином языке с аналогичными средствами параметризации), быстро обнаруживают, что излишне продублированный код может запросто потянуть на десятки мегабайт даже в программах весьма умеренного размера. Из-за того, что отсутствует необходимость в компиляции этого избыточного кода, общее время компиляции и компоновки может значительно сократиться.

Применим единственную специализацию для любых списков указателей в качестве еще одного примера минимизации общего кода за счет увеличения доли разделяемого (общего для разных типов) кода. Общий шаблон должен быть объявлен до любой его специализации. Например:

```
template<class T> class List<T*> { /* ... */ };
template<class T> class List { /* ... */ }; // error: общий шаблон после специализации
```

Критически важной информацией в определении общего шаблона является набор параметров шаблона, которые пользователь должен предоставлять, используя этот шаблон или его специализации. Поэтому *одного лишь объявления* общего шаблона достаточно для объявления или определения *специализации*:

```
template<class T> class List;
template<class T> class List<T*> { /* ... */ };
```

Если общий шаблон используется в клиентском коде, он, естественно, должен быть где-то определен (§13.7).

Специализация должна находиться в области видимости кода, в котором пользователь использует эту специализацию. Например:

```
template<class T> class List { /* ... */ };
List<int*> li;
template<class T> class List<T*> { /* ... */ }; // error
```

Здесь *List* был специализирован под *int\** позже того, как с помощью объявляющей конструкции *List<int\*>* был уже использован.

Все специализации шаблона должны объявляться в том же самом пространстве имен, что и общий шаблон. Если специализация используется, то она должна быть где-то явным образом определена (§13.7). Другими словами, явная специа-

лизация шаблона означает, что автоматическая генерация ее определения не выполняется.

### 13.5.1. Порядок специализаций

Одна специализация считается более специализированной (более специфической, узкой), чем другая специализация, если список фактических аргументов шаблона, удовлетворяющий ее паттерну специализации, также соответствует и паттерну второй специализации, но не наоборот. Например:

```
template<class T> class Vector;           // общий шаблон
template<class T> class Vector<T*>;      // специализация для любых указателей
template<> class Vector<void*>;          // специализация для void*
```

Любой тип может использоваться как аргумент для самого общего шаблона **Vector**, но только указатели подходят для специализации **Vector<T\*>**, и только тип **void\*** — для специализации **Vector<void\*>**.

Более специальным версиям отдается предпочтение в объявлениях объектов, указателей и т.д. (§13.5), а также при разрешении перегрузки (§13.3.2).

Паттерн специализации может быть специфицирован в терминах типов с применением конструкций, допустимых при выводе параметра шаблона (§13.3.1, §C.13.4).

### 13.5.2. Специализация шаблонов функций

Естественно, что специализация полезна и для шаблонов функций. Рассмотрим сортировку Шелла из §7.7 и §13.3. В ней производится сравнение элементов операцией **<** и в деталях выполняется перестановка элементов. Предпочтительнее дать следующее определение:

```
template<class T> bool less (T a, T b) { return a<b; }

template<class T> void sort (Vector<T>& v)
{
    const size_t n = v.size();
    for (int gap=n/2; 0<gap; gap /= 2)
        for (int i=gap; i<n; i++)
            for (int j=i-gap; 0<=j; j -= gap)
                if (less (v[j+gap], v[j]))
                    swap (v[j], v[j+gap]);
            else
                break;
}
```

Это не улучшает сам алгоритм, а лишь его реализацию.

В том виде, в каком шаблон **sort()** сейчас определен, он не сможет корректно сортировать тип **Vector<char\*>**, поскольку операция **<** будет сравнивать два указателя на **char**, то есть сравнивать адреса первых символов каждой строки. Нам же нужно, чтобы сравнивались сами символы. Простая специализация функционального шаблона **less()** для типа **const char\*** позаботится об этом:



```
template<> bool less<const char*> (const char* a, const char* b)
{
    return strcmp (a, b) < 0;
}
```

Как и для классовых шаблонов (§13.5), для функциональных шаблонов префикс *template<>* означает специализацию, не нуждающуюся в параметрах шаблона. Конструкция *<const char\*>* после имени функции означает, что эта специализация должна использоваться в случае, когда фактические параметры вызова имеют тип *const char\**. Поскольку параметры функциональных шаблонов могут быть выведены из фактических параметров вызова, нет нужды специфицировать их явно. Поэтому мы можем упростить определение специализации:

```
template<> bool less<> (const char* a, const char* b)
{
    return strcmp (a, b) < 0;
}
```

При наличии префикса *template<>* присутствие вторых пустых угловых скобок излишне, поэтому лучше писать так:

```
template<> bool less (const char* a, const char* b)
{
    return strcmp (a, b) < 0;
}
```

Во многих случаях перегрузка (§13.3.2) является альтернативой специализации. Рассмотрим очевидное определение для *swap()*:

```
template<class T> void swap (T& x, T& y)
{
    T t = x; // копируем x во временную переменную
    x = y;   // копируем y в x
    y = t;   // копируем временную переменную в y
}
```

Такое решение неэффективно для типа *Vector*, поскольку при этом будут копироваться все элементы векторов. Элементы *x* будут даже копироваться дважды. Проблема решается предоставлением версии *swap()*, более подходящей для векторов. Объект типа *Vector* будет представлен только данными, достаточными для косвенного доступа к элементам (как *String*; §11.12, §13.2). Таким образом, работу *swap()* можно свести к обмену этими представлениями. Для этого я определяю *swap()* как функцию-член шаблона *Vector* (§13.5):

```
template<class T> void Vector<T>::swap (Vector& a) // меняем представления
{
    swap (v, a.v);
    swap (sz, a.sz);
}
```

Теперь эту функцию-член можно использовать в качестве альтернативы общему шаблону *swap()*:

```
template<class T> void swap (Vector<T>& a, Vector<T>& b)
{
    a.swap (b) ;
}
```

Рассмотренные здесь специализация *less()* и перегруженная версия *swap()* используются в стандартной библиотеке (§16.3.9, §20.3.16). Кроме того, они отлично иллюстрируют широко применяемые приемы программирования. Специализация и перегрузка полезны тогда, когда имеется более эффективная альтернатива общему алгоритму для конкретных аргументов шаблона (здесь это *swap()*). Кроме того, специализация отлично работает тогда, когда «иррегулярность» типа аргумента приводит к неправильной работе общего алгоритма (здесь это *less()*). Часто такими «иррегулярными» типами являются встроенные типы указателей и массивов.

## 13.6. Наследование и шаблоны

Шаблоны и наследование являются механизмами построения новых типов из уже существующих, а также помогают писать код, извлекающий пользу из самых разных форм общности. Как показано в §3.7.1, §3.8.5 и §13.5, комбинация этих двух механизмов является основой для многих полезных технологий программирования.

Наследование классового шаблона от нешаблонного класса позволяет реализовать общую реализацию для множества шаблонов. Вектор из §13.5 служит хорошим примером:

```
template<class T> class Vector<T*> : private Vector<void*> { /* ... */ ;
```

Под другим углом зрения этот пример иллюстрирует тот факт, что шаблон используется для предоставления элегантного и типобезопасного интерфейса к средству, в противном случае являющегося и опасным, и неудобным.

Естественно, часто бывает полезным создавать классовые шаблоны, производные от других классовых шаблонов. Базовый класс, в частности, служит строительным блоком в реализации производных классов. Если члены базового класса должны соответствовать тому же параметру, что и члены производного класса, то имя базового класса следует соответственно параметризовать; хорошим примером служит *Vec* из §3.7.2:

```
template<class T> class vector { /* ... */ ;
template<class T> class Vec : public vector<T> { /* ... */ ;
```

Правила разрешения перегрузки шаблонных функций гарантируют, что функции работают правильно для таких производных типов (§13.3.2).

Случай, когда производный и базовый классы имеют один и тот же параметр шаблона, является наиболее распространенным, но это не обязательное требование. В редких и интересных случаях в качестве параметра базового класса указывается сам производный класс. Например:

```
template<class C> class Basic_ops // базовые операции над контейнерами
{
public:
    bool operator==(const C&) const; // сравнивает все элементы
```

```

bool operator!= (const C&) const;
// ...
// доступ к операциям типа C:
const C& derived() const {return static_cast<const C&> (*this); }
};

template<class T> class Math_container : public Basic_ops<Math_container<T>> >
{
public:
    size_t size() const;
    T& operator[] (size_t);
    const T& operator[] (size_t) const;
    // ...
};

```

Это позволяет один раз определить базовые операции над контейнерами и отделить их от определений самих контейнеров. В то же время, поскольку определение операций вроде == и != должно выражаться в терминах и контейнера, и его элементов, то тип элементов должен передаваться шаблону контейнера.

Полагая, что *Math\_container* аналогичен традиционному вектору, определения членов *Basic\_ops* будут выглядеть примерно так:

```

template<class C> bool Basic_ops<C>::operator== (const C& a) const
{
    if(derived() .size != a .size()) return false;
    for (int i=0; i<derived() .size(); ++i)
        if(derived() [i] != a [i]) return false;
    return true;
}

```

Другой техникой разделения контейнеров и операций является их комбинирование через параметры шаблонов (а не использование наследования):

```

template<class T, class C> class Mcontainer
{
    C elements;
public:
    T& operator[] (size_t i) {return elements [i]; }

    friend bool operator==<>(const Mcontainer&, const Mcontainer&);
    friend bool operator!=<> (const Mcontainer&, const Mcontainer&);
    // ...
};

template<class T> class My_array { /* ... */ };
Mcontainer<double, My_array<double>> mc;

```

Класс, генерируемый из шаблона класса, является обычным классом. Следовательно, он может иметь дружественные функции (§С.13.2). Я в данном примере воспользовался дружественными функциями, чтобы выразить симметрию операций == и != (§11.3.2).

Можно было бы рассмотреть и вариант с передачей шаблона (шаблонный параметр классowego шаблона) вместо контейнера для аргумента C (§С.13.3).

### 13.6.1. Параметризация и наследование

Шаблоны являются механизмом параметризации определения типа или функции другим типом. Исходный код, реализующий шаблон, одинаков для всех типов параметров, как и большая часть кода, использующего шаблон. Когда требуется дополнительная гибкость, определяют специализации. Абстрактный класс определяет интерфейс. Большая часть кода различных реализаций абстрактного класса может совместно использоваться в классовой иерархии, и большая часть кода, использующего абстрактный класс, не зависит от его реализации. С точки зрения проектирования оба подхода настолько близки, что заслуживают общего названия. Так как оба позволяют выразить алгоритм единожды и использовать его со многими типами, люди все это называют *полиморфизмом*. Чтобы все-таки различать эти подходы, для виртуальных функций применяют термин *полиморфизм времени выполнения* (*run-time polymorphism*), а для шаблонов предлагают термин *полиморфизм времени компиляции* (*compile-time polymorphism*) или *параметрический полиморфизм* (*parametric polymorphism*).

Итак, когда же следует выбирать шаблоны, а когда — абстрактные классы? В обоих случаях мы манипулируем объектами, которые разделяют (совместно используют) общий набор операций. Если отсутствуют иерархические взаимозависимости между объектами, лучше выбирать шаблоны. Когда же истинный тип объектов не известен на этапе компиляции, нужно опираться на классовые иерархии наследования от общего абстрактного класса. Когда же особо важна производительность, обеспечиваемая за счет встраивания операций, следует использовать шаблоны. Эти вопросы подробнее обсуждаются в §24.4.1.

### 13.6.2. Шаблонные члены шаблонов

Класс или классовый шаблон могут иметь члены, которые сами являются шаблонами. Например:

```
template<class Scalar> class complex // детали см. в §22.5
{
    Scalar re, im;

public:
    template<class T> complex(const complex<T>& c) : re(c.real()), im(c.imag()) {}
    // ...
};

complex<float> cf(0, 0);
complex<double> cd = cf; // ok: используется приведение float к double

class Quad
{
    // отсутствует приведение к int
};

complex<Quad> cq;
complex<int> ci=cq; // error: нет приведения Quad к int
```

Другими словами, мы можете создавать **Complex<T1>** из **Complex<T2>** тогда и только тогда, когда вы можете инициализировать **T1** с помощью **T2**. Это выглядит разумно.

К сожалению, C++ допускает некоторые неразумные операции преобразования встроенных типов, такие как *double* в *int*. Сопутствующие проблемы потери точности можно обнаружить во время выполнения с помощью проверяемых преобразований в стиле *implicit\_cast* (§13.3.1) и функции *checked()* (§C.6.2.6):

```
template<class Scalar> class complex // детали см. в §22.5
{
    Scalar re, im;

public:
    complex() : re(0), im(0) {}
    complex(const complex<Scalar>& c) : re(c.real()), im(c.imag()) {}

    template<class T2> complex(const complex<T2>& c)
        :
    re(checked_cast<Scalar>(c.real())), im(checked_cast<Scalar>(c.imag())) {}

    // ...
};
```

Для полноты картины я добавил умолчательный конструктор и копирующий конструктор. Любопытно, что шаблонный конструктор (в этом примере — зависящий от параметра шаблона *T2*) никогда не используется для генерации обычного копирующего конструктора, так что в отсутствие явного определения последнего будет сгенерирован умолчательный вариант копирующего конструктора. В нашем случае такой неявно сгенерированный копирующий конструктор был бы идентичен тому, что я определил явно. Аналогично, операция присваивания (§10.4.4.1, §11.7) также не должна определяться как шаблонная функция-член.

Шаблонная функция-член не может быть виртуальной. Например:

```
class Shape
{
    // ...
    template<class T> virtual bool intersect(const T&) const=0; // error: virtual template
};
```

Такое недопустимо. Если бы это было позволено, то для реализации виртуальных функций нельзя было бы применить традиционную технику виртуальных таблиц (§2.5.5). Компоновщику пришлось бы добавлять новую точку входа в виртуальную таблицу для класса *Shape* каждый раз, когда *intersect()* вызывается с новым типом аргумента.

### 13.6.3. Отношения наследования

Шаблон класса полезно представлять как спецификацию того, как должны создаваться конкретные типы. Другими словами, реализация шаблона является механизмом генерации типов, основанная на предоставляемых пользователем сведениях. Как следствие, шаблоны классов часто называют *генераторами типов* (*type generators*).

С точки зрения языка C++ два типа, сгенерированные из одного шаблона, никак не связаны между собой. Например:

```
class Shape { /* ... */ };
class Circle : public Shape { /* ... */ };
```

Исходя из представленных объявлений люди часто пытаются трактовать `set<Circle*>` как `set<Shape*>`. Это серьезная логическая ошибка, основанная на неверном рассуждении: «*Circle* это *Shape*, так что множество объектов типа *Circle* есть в то же время и множество объектов типа *Shape*; следовательно, я могу использовать множество объектов *Circle* как множество объектов *Shape*». В этом рассуждении последняя часть не верна. Причина состоит в том, что множество объектов *Circle* гарантирует, что каждый член множества есть *Circle*, в то время как множество *Shape* такой гарантии не дает. Например:

```
class Triangle : public Shape { /* ... */ };

void f(set<Shape*>& s)
{
    // ...
    s.insert(new Triangle());
    // ...
}

void g(set<Circle*>& s)
{
    f(s);    // error: несоответствие типов: s есть set<Circle*>, а не set<Shape*>
}
```

Этот код не будет компилироваться, так как нет встроенного преобразования от `set<Circle*>&` к `set<Shape*>&`. И его не должно быть. Гарантия того, что члены множества `set<Circle*>` есть объекты типа *Circle*, позволяет нам безопасно и эффективно применять операции, специфичные для окружностей, например, определение радиуса, ко всем членам множества. Если бы мы позволили с `set<Circle*>` обращаться как с `set<Shape*>`, то мы нарушили бы такую гарантию. Например, функция `f()` внедряет новый треугольник (объект типа *Triangle*) во множество `set<Shape*>`, передаваемое ей в качестве аргумента. Тогда, если бы множество `set<Shape*>` оказалось бы при вызове `f()` множеством `set<Circle*>`, была бы нарушена фундаментальная гарантия того, что все элементы множества `set<Circle*>` есть окружности.

### 13.6.3.1. Преобразования шаблонов

В примере из предыдущего раздела демонстрируется, что не может существовать отношений по умолчанию между классами, сгенерированными из одного и того же шаблона. В то же время, для некоторых шаблонов нам хотелось бы такие отношения выразить. Например, когда мы определяем шаблон указателей (объекты сгенерированных из шаблона классов ведут себя как указатели), нам хотелось бы отразить отношения наследования между адресуемыми объектами. Шаблонные члены шаблонов (§13.6.2) позволяют выразить некоторые из таких взаимоотношений. Рассмотрим пример:

```
template<class T> class Ptr                // указатель на T
{
    T* p;
```

```

public:
    Ptr(T*);
    Ptr(const Ptr&); // копирующий конструктор
    template<class T2> operator Ptr<T2>() ; // преобразование Ptr<T> в Ptr<T2>
    // ...
};

```

Мы хотим определить операции преобразования для пользовательского типа **Ptr** так, чтобы операции преобразования между объектами этого типа были такими же, как между встроенными указателями при наследовании. Например:

```

void f(Ptr<Circle> pc)
{
    Ptr<Shape> ps = pc; // должно работать
    Ptr<Circle> pc2 = ps; // даст ошибку
}

```

Мы хотим допускать первую из указанных в данном коде инициализаций в том и только в том случае, когда **Shape** является прямым или косвенным открытым базовым классом для **Circle**. В общем, нам нужно определить операцию преобразования так, чтобы преобразование от **Ptr<T>** к **Ptr<T2>** допускалось тогда и только тогда, когда значение типа **T\*** можно присваивать объекту типа **T2\***. Этого можно достичь следующим образом:

```

template<class T>
template<class T2>
Ptr<T> : : operator Ptr<T2>() { return Ptr<T2>(p); }

```

Здесь оператор `return` будет компилироваться тогда и только тогда, когда указатель **p** (имеет тип **T\***) может быть аргументом конструктора **Ptr<T2>(T2\*)**. Поэтому, если **T\*** может неявно приводиться к **T2\***, то преобразование **Ptr<T>** в **Ptr<T2>** будет работать. Например:

```

void f(Ptr<Circle> pc)
{
    Ptr<Shape> ps = pc; // ok: можно привести Circle* к Shape*
    Ptr<Circle> pc2 = ps; // error: нельзя привести Shape* к Circle*
}

```

Старайтесь определять лишь логически осмысленные операции преобразования.

Обратите внимание на то, что списки параметров самого шаблона и его шаблонных членов объединять нельзя. Например:

```

template<class T, class T2> // error
Ptr<T> : : operator Ptr<T2>() { return Ptr<T2>(p); }

```

## 13.7. Организация исходного кода

Существует два очевидных способа организации кода, использующего шаблоны:

1. Включать определения шаблонов до их использования в той же самой единице трансляции.

2. Включать лишь объявления шаблонов до их использования в единице трансляции, а определения компилировать отдельно.

Кроме того, шаблонные функции могут сначала объявляться, затем использоваться и лишь после этого определяться в одной и той же единице трансляции.

Для иллюстрации различий между двумя подходами рассмотрим простой шаблон:

```
#include <iostream>

template<class T> void out (const T& t) { std::cerr << t; }
```

Мы могли бы поместить этот фрагмент в файл *out.c*, и включать его директивой *#include* в те места, где вызывается *out()*. Напримеp:

```
// user1.c:
#include "out.c"
// используем out()

// user2.c:
#include "out.c"
// используем out()
```

Таким образом, определение функции *out()* и все объявления, от которых она зависит, должны помещаться в каждую единицу трансляции, где *out()* используется. Далее уже от компилятора зависит, как именно он будет бороться с избыточными определениями и устранять эту избыточность (по возможности). Такая стратегия трактует шаблонные функции так же, как встраиваемые функции. Ее недостатком является излишняя нагрузка на компилятор, который должен перепаривать большие объемы информации. Другая проблема состоит в том, что пользователи могут случайно получить ненужные дополнительные зависимости от объявлений, необходимых лишь определению функционального шаблона *out()*. Опасность этой проблемы можно минимизировать, используя пространства имен, избегая макросов и, вообще, минимизируя количество дополнительно включаемой информации.

Другим подходом является стратегия раздельной компиляции, которая вытекает из следующего рассуждения: если определение шаблона не включается в код пользователя, то никакие нужные шаблону объявления не влияют на пользовательский код. Таким образом, мы разбиваем наш файл *out.c* на два файла:

```
// out.h:
template<class T> void out (const T& t) ;

// out.c:
#include<iostream>
#include "out.h"

export template<class T> void out (const T& t) { std::cerr << t; }
```

Теперь файл *out.c* содержит всю информацию, необходимую для определения *out()*, а файл *out.h* содержит лишь информацию, необходимую для вызова *out()*. При этом подходе в пользовательский код включается только объявление шаблона (интерфейс):



```
// user1.c:  
#include "out.h"  
// используем out()  
  
// user2.c:  
#include "out.h"  
// используем out()
```

Эта стратегия трактует шаблонные функции как обычные невстраиваемые функции. Определение (в файле *out.c*) компилируется отдельно, и задачей реализации является найти его при необходимости. Это также нагружает реализацию, но по-другому, чем в случае борьбы с избыточными определениями.

Отметим, что определение (или объявление) шаблона должно быть дано с ключевым словом **export** (§9.2.3)<sup>1</sup>, или оно будет недоступно из клиентского кода. В противном случае оно должно находиться в той области видимости, где оно используется.

Какая стратегия (или их комбинация) является оптимальной, зависит от используемых компилятора и компоновщика, от внутренних особенностей разрабатываемой программной системы, и от внешних ограничений, накладываемых на нее. Обычно, встраиваемые функции и небольшие шаблонные функции, которые главным образом вызывают другие шаблонные функции, являются кандидатами на включение в каждую единицу трансляции, в которой они используются. Для реализаций с умеренной поддержкой компоновщика в вопросе конкретизации шаблонов такой подход ускоряет процесс компиляции и повышает точность сообщений об ошибках.

Включение определений делает их уязвимыми от макросов и объявлений в контексте использования. Следовательно, большие шаблоны и шаблоны со сложными контекстными зависимостями лучше компилировать отдельно. Кроме того, это позволяет пользователю коду избавиться от влияния объявлений, требующихся для определения шаблона.

Я считаю подход с включением лишь объявлений шаблонов в пользовательский код и раздельной компиляцией определений шаблонов идеальным. Однако практическое воплощение идеалов часто сдерживается практическими ограничениями, к тому же раздельная компиляция шаблонов на некоторых реализациях является слишком громоздкой и дорогой операцией.

Независимо от того, какая стратегия используется, невстраиваемые статические члены (§C.13.1) должны иметь уникальные определения в некоторых единицах трансляции. Это подразумевает, что такие члены лучше не использовать в шаблонах, если последние предполагается включать во многие единицы трансляции.

Идеальным является код, который одинаково хорошо работает как в случае, когда он компилируется из одной единицы трансляции, или собирается из нескольких независимо компилируемых единиц. К такому идеалу нужно стремиться, ограничивая зависимость определений шаблонов от контекста, и не пытаясь втискивать все это в единый процесс конкретизации.

<sup>1</sup> Многие компиляторы (в частности фирмы Microsoft) на платформе Windows не поддерживают это ключевое слово. — *Прим. ред.*

## 13.8. Советы

1. Используйте шаблоны для представления алгоритмов, применимых к разным типам аргументов; §13.3.
2. Используйте шаблоны для реализации контейнеров; §13.2.
3. Для минимизации размера кода создавайте специализации для контейнеров указателей; §13.5.
4. Всегда объявляйте общую форму шаблона до его специализаций; §13.5.
5. Объявляйте специализацию до ее использования; §13.5.
6. Минимизируйте зависимость определения шаблона от контекста его конкретизации; §13.2.5, §C.13.8.
7. Определяйте все объявленные специализации; §13.5.
8. Подумайте, нужны ли специализации вашего шаблона для строк и массивов языка C; §13.5.2.
9. Параметризуйте шаблоны аргументами, задающими различные варианты поведения; §13.4.
10. Для предоставления единого интерфейса к разным реализациям шаблона для различных типов используйте специализации и перегрузку; §13.5.
11. Предоставляйте простой интерфейс для простых случаев и применяйте перегрузку и аргументы по умолчанию для более специальных случаев; §13.5, §13.4.
12. Отлаживайте конкретные примеры до их обобщения в шаблоны; §13.2.1.
13. Не забывайте про ключевое слово **export** в случаях, когда определения шаблонов нужно компоновать с другими единицами трансляции; §13.7.
14. Отдельно компилируйте большие шаблоны и шаблоны с нетривиальными зависимостями от контекста; §13.7.
15. Используйте шаблоны, чтобы выразить преобразования, но делайте это осознанно; §13.6.3.1.
16. При необходимости ограничивайте аргументы шаблона с помощью функции-члена **constraint()**; §13.9[16], §C.13.10.
17. Для минимизации времени компиляции используйте явную конкретизацию; §C.13.10.
18. Когда скорость исполнения имеет первостепенную важность, применяйте шаблоны вместо наследования; §13.6.1.
19. Используйте механизм наследования вместо шаблонов, когда важно иметь возможность добавления новых вариантов без перекомпиляции клиентского кода; §13.6.1.
20. Отдавайте предпочтение шаблонам перед наследованием, когда невозможно определить общий базовый класс; §13.6.1.
21. Отдавайте предпочтение шаблонам перед наследованием, когда использование встроенных типов и структур важно из соображений совместимости; §13.6.1.

## 13.9. Упражнения

1. (\*2) Исправьте ошибки в определении *List* из §13.2.5 и напишите C++-код, эквивалентный тому, что сгенерирует компилятор из определения *List* и функции *f()*. Протестируйте работу обеих версий кода. Если вы можете это сделать, то сравните их машинные коды.
2. (\*3) Напишите классовый шаблон для односвязного списка с элементами, производными от класса *Link*, содержащего информацию для связывания элементов. Такой список называется интрузивным. Отталкиваясь от этого списка, напишите код односвязного списка, который может иметь элементы любого типа (неинтрузивный список). Сравните производительность обоих списков и обсудите их достоинства и недостатки.
3. (\*2.5) Напишите интрузивный и неинтрузивный двусвязные списки. Какие дополнительные операции нужны этим спискам по сравнению с односвязными вариантами списков?
4. (\*2) Завершите шаблон *String* из §13.2, основанный на классе *String* из §11.12.
5. (\*2) Определите шаблонную функцию *sort()*, принимающую критерий сравнения в качестве шаблонного аргумента. Определите класс *Record* с двумя полями — *count* и *price*. Выполните сортировку *vector<Record>* по каждому полю.
6. (\*2) Реализуйте шаблон *qsort()*.
7. (\*2) Напишите программу, которая читает пары (*key*, *value*), а затем вычисляет суммы значений *value*, соответствующих уникальным значениям *key*. Сформулируйте требования к типам значений *value* и *key*.
8. (\*2.5) Реализуйте простой ассоциативный массив *Map*, основанный на классе *Assoc* из §11.8. Убедитесь, что *Map* работает корректно как для C-строк, так и для *string* в качестве типа *key*. Убедитесь, что *Map* работает корректно как для типов с умолчательными конструкторами, так и для типов без оных. Обеспечьте возможность итерации по элементам контейнера *Map*.
9. (\*3) Сравните производительность программы подсчета слов из §11.8 по сравнению с программой, не использующей ассоциативного массива. В обоих случаях используйте одинаковый ввод/вывод.
10. (\*3) Перепрограммируйте *Map* из §13.9[8], используя более подходящие структуры данных (например, «красно-черные» деревья).
11. (\*2.5) Используйте *Map* для выполнения топологической сортировки, описанной в [Knuth,1968] т.1 (второе издание), стр. 262.
12. (\*1.5) Обеспечьте корректную работу программы суммирования из §13.9[7] для имен, содержащих пробелы.
13. (\*2) Напишите шаблоны *readline()* для разных форматов строк ввода, например, (item,count,price).

14. (\*2) Используйте технику, кратко описанную для *Literature* из §13.4 для сортировки строк в обратном лексикографическом порядке. Убедитесь, что указанная техника работает как на системах, где *char* является знаковым типом, так и на системах с беззнаковым *char*. Напишите вариант программы для сортировки без учета регистра символов.
15. (\*1.5) Придумайте пример, в котором демонстрируются, по крайней мере, три различия между функциональными шаблонами и макросами (не считая разницы в синтаксисе их определений).
16. (\*2) Разработайте схему, гарантирующую, что компилятор проверяет общие ограничения на аргументы шаблонов, для которых создаются объекты. Недостаточно просто проверить ограничения вида «*T* должно быть типом, производным от *My\_base*».

# Обработка исключений

*Не перебивайте меня,  
когда я вас перебиваю.  
— Уинстон Черчилль*

Обработка ошибок — группировка исключений — перехват исключений — перехват всех исключений — повторная генерация — управление ресурсами — *auto\_ptr* — исключения и *new* — исчерпание ресурсов — исключения в конструкторах — исключения в деструкторах — исключения, не являющиеся ошибками — спецификация исключений — неожиданные исключения — неперехваченные исключения — исключения и эффективность — альтернативные методы обработки ошибок — стандартные исключения — советы — упражнения.

## 14.1. Обработка ошибок

Как отмечалось в §8.3, автор библиотеки в состоянии обнаружить ошибки времени выполнения, но обычно не имеет представления, что с ними нужно делать. Пользователь библиотеки может знать, как бороться с такими ошибками, но он их не обнаруживает — в противном случае, все было бы сосредоточено в пользовательском коде, а о библиотеке и речь бы не шла. Понятие *исключения* (*exception*) как раз и призвано помочь в разрешении этой коллизии. Центральная идея заключается в том, что функция, обнаружившая ошибку, с которой она не может справиться самостоятельно, *генерирует* (*throws*) исключение в надежде, что вызвавшая ее (непосредственно или косвенно) функция сможет обработать возникшую ошибку. Функции, которые имеют намерение решать проблемы такого рода, должны явно указать, что они *перехватывают* (*catch*) данный тип исключения (§2.4.2, §8.3).

Такой стиль обработки ошибок предпочтительней многих более традиционных способов. Рассмотрим возможные альтернативы. Обнаружив проблему, с которой невозможно справиться локально, функция может:

1. Прекратить выполнение.
2. Возвратить значение, указывающее на ошибку.
3. Вернуть некоторое допустимое значение и оставить программу в ошибочном состоянии.
4. Вызвать функцию, специально предназначенную для вызова в случае ошибки.

Вариант [1], «прекратить выполнение» — это то, что происходит по умолчанию, когда исключения не перехватываются. Для большей части ошибок мы можем и должны придумать что-нибудь получше. В частности, библиотека, которая ничего не знает о цели и общей стратегии использующей ее программы, не может просто так вызвать `exit()` или `abort()`. Такую библиотеку, неожиданно завершающую работу всей программы, невозможно использовать в составе надежных приложений. Центральный взгляд на обработку исключений (динамически возникающих ошибочных ситуаций) состоит в том, что нужно управление передавать в вызывающую функцию в тех случаях, когда локально невозможно принять решение об обработке ошибок.

Вариант [2], «возвратить значение, указывающее на ошибку» — не всегда возможен, ибо часто не существует подходящих значений, указывающих на ошибки. Например, для функции с возвратом типа `int`, любое возвращаемое значение является нормальным, приемлемым результатом. Но даже, если можно выделить специальные «ошибочные возвраты», то все равно такой подход неудобен, поскольку требуется каждый вызов функции проверять на совпадение его возврата с «ошибочными значениями». Это легко может удвоить объем программного кода (§14.8). Как следствие, такой подход редко когда применяют систематически для борьбы со всеми возможными ошибками.

Подход [3], «вернуть некоторое допустимое значение и оставить программу в ошибочном состоянии» — плох тем, что вызывающая функция может не заметить, что программа находится в ненормальном состоянии. Например, многие библиотечные функции языка C для индикации ошибки устанавливают значение глобальной переменной `errno` (§20.4.1, §22.3). Однако ж, многие программы не проверяют его систематически, и в итоге они не застрахованы от последующих ошибок, вызванных использованием значений, полученных от ошибочно отработавших библиотечных функций. Более того, использование глобальных переменных для индикации ошибок проблематично при работе нескольких программ в параллельных процессах.

Обработка исключений не тождественна случаям, когда уместен вариант [4] — «вызвать функцию, специально предназначенную для вызова в случае ошибки». В отсутствие обработки исключений, вариант [4] имеет те же самые три альтернативы в плане того, как именно специальная функция для обработки ошибок выполняет эту работу. Дальнейшее обсуждение функций обработки ошибок и исключений см. в §14.4.5.

Механизм обработка исключений предоставляет альтернативу традиционным методам в случаях, когда последние не достаточны, не элегантны или сами подвержены ошибкам. Он формулирует способ явного отделения кода обработки ошибок от регулярного кода программы, что способствует лучшей читаемости программы человеком и большей удобоваримости ее кода для вспомогательных инструментальных средств. Механизм обработки исключений предоставляет более регуляр-

ную технологию обработки ошибок, упрощающую взаимодействие между отдельно написанными программными модулями.

Один аспект, принятый в схеме обработки исключений, и состоящий в том, что умолчательной реакцией на ошибки (особенно на ошибки в библиотеках) является завершение работы программы, необычен для программистов на языках C или Pascal. Для них традиционной реакцией является попытка «тянуть программу далее, надеясь на лучшее». Этот аспект механизма обработки исключений делает использующие его программы более капризными в том смысле, что он требует больших усилий и внимания для того, чтобы довести работу программы до конца. Однако такой подход более предпочтителен, ибо значительно лучше бороться с проблемами на ранних этапах разработки, чем на поздних, и уж тем более нежелательно передавать якобы готовую программу ничего не подозревающему конечному пользователю, чтобы именно он столкнулся с этими проблемами. Когда завершение работы программы неприемлемо, можно осуществлять перехват всех исключений (§14.3.2) или некоторой их точно специфицированной части (§14.6.2). Таким образом, исключения прекращают выполнение программ только тогда, когда программист позволяет им это делать. Это значительно лучше, чем безоговорочное завершение программ в случаях, когда недостаточная обработка ошибок приводит к их катастрофическому состоянию.

Иногда программисты пытаются смягчить последствия подхода «тянуть до конца» путем вывода сообщений об ошибках, отображением диалоговых окон, обращением к пользователю за подмогой и т.д. Такие подходы действительно полезны при отладке программ, когда пользователем является сам программист, знакомый со структурой программы. Для обычного же пользователя библиотека, запрашивающая помощь у (возможно отсутствующего) пользователя/оператора, неприемлема. Кроме того, в некоторых случаях сообщения об ошибке просто негде появиться (скажем, если программа исполняется в системах, где поток сегмента не подключен к средствам визуализации), но и без этого конечному пользователю они не понятны. Далее, сообщения могут быть на естественном языке, непонятном пользователю (скажем, на финском языке — для пользователя-англичанина). Что еще хуже, сообщения об ошибках от библиотек могут касаться ее внутренних реалий, незнакомых пользователю (например, сообщение «недопустимый аргумент для *atan2*» в ответ на неправильный ввод для графической системы). Хорошие библиотеки не должны так «разговаривать с пользователями». Исключения позволяют коду, обнаружившему проблему, но не знающему, что с ней делать, отпассовать ее в другую часть системы, которая способна принять правильное решение. И только эта часть системы, которая видит весь контекст исполнения программы, может также и сформировать осмысленное сообщение об ошибке.

Механизм обработки исключений можно рассматривать как аналог механизма выявления ошибок типов и разрешения неоднозначностей на стадии компиляции, только механизм исключений работает на стадии выполнения программы. Обработка исключений делает более важным этап проектирования и может увеличить объем работ, необходимых для получения начальных, работающих с ошибками версий программы. Однако в результате получается конечный код, у которого намного больше шансов работать как задумано, как часть большой программы, понятной другим программистам, и который доступен для автоматической обработки различными инструментальными средствами. Как и другие средства языка C++, обработ-

ка исключений предоставляет программисту средства для хорошего стиля программирования, который можно лишь неформально и не в полном объеме практиковать в таких языках как C или Pascal.

Надо понимать, что обработка ошибок является сложной задачей и что механизм обработки исключений, даже с учетом его большей формализации по сравнению с альтернативными методами, все же менее структурирован на фоне иных средств языка, имеющих дело лишь с локальным контекстом потока выполнения программы. Механизм обработки исключений предоставляет программисту возможность обрабатывать ошибки там, где их обрабатывать наиболее сподручно для конкретной системы. Исключения позволяют явным образом продемонстрировать всю сложность обработки ошибок. Но не они являются первопричиной этой сложности (не следует обвинять гонца с плохими новостями).

Сейчас самое подходящее время для повторного прочтения §8.3, где изложены базовый синтаксис, семантика и приемы использования исключений.

#### 14.1.1. Альтернативный взгляд на исключения

Исключения — это такой термин, который разные люди понимают по-разному. Механизм исключений языка C++ разработан для обработки ошибок и других исключительных ситуаций (отсюда название механизма). В частности, он поддерживает обработку ошибок в программах, составленных из независимо разработанных программных компонентов.

Механизм исключений предназначен для обработки лишь синхронных исключений, таких как выход за границы допустимых диапазонов в массивах или ошибки ввода/вывода. Асинхронные события, такие как прерывания от клавиатуры и ряд арифметических ошибок, не обязательно являются исключениями и не обрабатываются непосредственно обсуждаемым механизмом. Асинхронные события требуют для своей обработки механизмов, принципиально отличающихся от механизма исключений (как мы его здесь определили). На многих системах предлагаются механизмы работы с асинхронными событиями, например сигналы, но поскольку эти механизмы системно-зависимые, то они здесь не рассматриваются.

Механизм обработки исключений представляет собой нелокальную управляющую структуру, базирующуюся на *раскрутке стека* (*stack unwinding*) (§14.4), которую можно рассматривать как альтернативный возврат из функции. Вполне законно использовать исключения, которые не имеют никакого отношения к обработке ошибок (§14.5). Но, конечно же, первичной целью этого механизма и предметом данной главы является именно обработка ошибок и обеспечение надежности программных систем.

В стандарте C++ нет понятий потока и процесса. Следовательно, обстоятельства функционирования механизма исключений применительно к потокам и процессам здесь не рассматриваются. Средства параллельного выполнения, доступные на вашей системе, описываются в ее документации. Здесь я просто отмечаю, что механизм исключений разработан таким образом, чтобы быть эффективным и в среде параллельного исполнения программ при соблюдении программистом таких правил, как блокирование разделяемых структур данных при доступе к этим данным.



Механизм обработки исключений в C++ предназначен для того, чтобы обрабатывать ошибки и исключительные ситуации, и информировать о них. Задача программиста состоит в том, чтобы решить, какие ситуации являются исключительными для данной конкретной программы. Это не так легко сделать (§14.5). Можно ли считать исключительным событие, которое происходит почти что в каждом сеансе работы программы? Может ли запланированное и обрабатываемое событие считаться ошибкой? Ответ на оба вопроса — да. «Исключительное» не значит редко встречающееся или катастрофическое. Лучше рассматривать исключительные ситуации как «невозможность некоторой части системы выполнить то, что от нее требуется». Обычно в таких случаях следует предпринять что-то иное. Генерация исключений с помощью *throw* должна быть более редкой по сравнению с обычными вызовами функций, иначе структура программы станет неотчетливой. В то же время, мы можем твердо рассчитывать на то, что большинство грамотно спроектированных крупных программ наверняка генерируют и перехватывают некоторое количество исключений по ходу своего нормального и вполне успешного выполнения.

## 14.2. Группировка исключений

Исключение является объектом некоторого класса, представляющим исключительную ситуацию. Обнаруживший ошибку код (часто это библиотека) генерирует исключение оператором *throw* (§8.3). Часть кода, осуществляющая обработку исключений, сосредотачивается в блоке *catch*. Оператор *throw* порождает раскрутку стека до тех пор, пока не будет найден подходящий *catch*-блок (в функции, прямо или косвенно вызвавшей функцию с этим оператором *throw*).

Часто исключения разбиваются на семейства. Это означает, что наследование может отражать такое структурирование исключений и помогать в их обработке. Например, исключения в математической библиотеке могут быть организованы следующим образом:

```
class Matherr{};
class Overflow: public Matherr{};
class Underflow: public Matherr{};
class Zerodivide: public Matherr{};
// ...
```

Это позволяет нам обрабатывать любое исключение как *Matherr*, не заботясь о точном его типе. Например:

```
void f()
{
    try
    {
        // ...
    }
    catch (Overflow)
    {
        // обработка Overflow или производных от Overflow
    }
}
```

```

catch (Matherr)
{
    // обработка любых Matherr кроме Overflow
}
}

```

Здесь исключения типа ***Overflow*** обрабатываются специальным образом. А все остальные исключения из представленной иерархии подходят для обобщенной (одинаковой) обработки.

Организация исключений в виде наследственной иерархии может быть полезна для обеспечения надежности кода. Например, как бы вам пришлось обрабатывать исключения математической библиотеки без представленной выше иерархии? Ясно, что потребовалось бы занудное перечисление всех типов исключений:

```

void g ()
{
    try
    {
        // ...
    }
    catch (Overflow) { /* ... */ }
    catch (Underflow) { /* ... */ }
    catch (Zerodivide) { /* ... */ }
}

```

И это не только утомительно. Программист вообще может при этом забыть какое-либо исключение. Далее, если мы добавим новое исключение в математическую библиотеку, каждый фрагмент клиентского кода, обрабатывающий все математические исключения, потребовал бы модификации. Как правило, такие глобальные модификации недопустимы после выхода первой рабочей версии библиотеки. Часто они вообще невозможны, если не удастся найти все необходимые куски кода. Рассмотренные проблемы сопровождения кода фактически ставят крест на попытках модификации системы исключений в библиотеках после их первоначального опубликования, что неприемлемо для большинства библиотек. Из рассмотренного вытекает, что в библиотеках или изолированных частях программных систем исключения должны организовываться в наследственные иерархии (§14.6.2).

Обратите внимание на то, что ни встроенные математические операции, ни базовые библиотечные математические функции (языков C/C++) не обрабатывают арифметические ошибки как исключения. Одной из причин такого явления служит факт, что арифметические ошибки вроде деления на ноль проявляются асинхронно на многих машинах с конвейерной архитектурой.

Приведенная выше иерархия исключений ***Matherr*** является просто иллюстрацией. Стандартные библиотечные исключения языка C++ описываются в §14.10.

### 14.2.1. Производные исключения

Использование иерархий классов для обработки исключений естественным образом приводит к обработчикам, интересующимся лишь подмножеством информации, которую несут с собой исключения. Другими словами, в типичном случае ис-

исключение перехватывается обработчиком его базового класса, а не обработчиком его собственного класса. Семантика именованного и перехвата исключений аналогична таковой для функций с аргументом. То есть формальный аргумент инициализируется фактическим значением вызова (§7.2). Это означает, что сгенерированное исключение «срезается» (sliced) до перехваченного (§12.2.3). Например:

```
class Matherr
{
    // ...
    virtual void debug_print() const { cerr << "Math error" ; }
};

class Int_overflow: public Matherr
{
    const char* op;
    int a1, a2;

public:
    Int_overflow(const char* p, int a, int b) { op = p; a1 = a; a2 = b; }
    virtual void debug_print() const { cerr << op << ' (' << a1 << ', ' << a2 << ' ) ' ; }
    // ...
};

void / ()
{
    try
    {
        g();
    }
    catch (Matherr m)
    {
        // ...
    }
}
```

Когда вызывается обработчик для *Matherr*, *m* является объектом типа *Matherr* — даже если функция *g()* во время своей работы сгенерировала исключение типа *Int\_overflow*. Это означает, что добавочная информация, сопутствующая типу *Int\_overflow*, недоступна.

Как всегда, можно использовать указатели или ссылки во избежание потери информации. Например:

```
int add(int x, int y)
{
    if( (x>0 && y>0 && x>INT_MAX-y) || (x<0 && y<0 && x<INT_MIN-y) )
        throw Int_overflow("+", x, y);

    return x+y; // вычисление x+y не вызовет исключений
}

void f()
{
    try
```

```

    int i1 = ad (1, 2) ;
    int i2 = add (INT_MAX, -2) ;
    int i3 = add (INT_MAX, 2) ;    // нуехали!
}
catch (Matherr& m)
{
    // ...
    m.debug_print () ;
}
}

```

В результате последнего из указанных вызовов *add()* генерируется исключение, приводящее к вызову метода *Int\_overflow::debug\_print()*. Если бы исключение перехватывалось «по значению», а не «по ссылке», то был бы вызван метод *Matherr::debug\_print()*.

### 14.2.2. Композитные (комбинированные) исключения

Не каждая группа исключений обязана образовывать древовидную структуру. Бывает, что исключение одновременно принадлежит сразу двум группам. Например:

```
class Netfile_err : public Network_err, public File_system_err { /* ... */ ;
```

Исключения *Netfile\_err* могут перехватываться как функциями, настроенными на работу с сетевыми исключениями

```

void f()
{
    try
    {
        // что-либо
    }
    catch (Network_err& e)
    {
        // ...
    }
}

```

так и функциями, настроенными на работу с файловыми исключениями:

```

void g()
{
    try
    {
        //...
    }
    catch (File_system_err& e)
    {
        // ...
    }
}

```

Такие комбинированные (композитные) исключения важны в тех случаях, когда, например, сетевые службы прозрачны для пользователей — автор функции *g()*

в нашем примере мог не догадываться о том, что сетевые службы вовлечены в дело (см. также §14.6).

## 14.3. Перехват исключений

Рассмотрим пример:

```
void f()  
{  
    try  
    {  
        throw E();  
    }  
    catch (H)  
    {  
        // когда мы сюда попадем?  
    }  
}
```

Обработчик исключений будет вызван в следующих случаях:

1. Если *H* того же типа, что и *E*.
2. Если *H* — это однозначный базовый класс для *E*.
3. Если *H* и *E* — указательные типы, и [1] или [2] справедливы для типов, на которые указывают эти указатели.
4. Если *H* — это ссылка, и [1] или [2] справедливы для типа, на который *H* ссылается.

Мы также можем добавить модификатор *const* к типу исключения, указанному в блоке *catch* (точно так же, как это делается в отношении аргументов функций). Это не изменит набор исключений, который мы можем перехватить, а лишь оградит нас от изменения объекта исключения.

В принципе, исключение при генерации копируется, так что обработчики имеют дело с копиями оригинальных исключений. Исключение может быть даже несколько раз скопировано перед тем, как будет перехвачено для обработки. Отсюда следует, что мы не можем сгенерировать не копируемое исключение. Конкретные реализации могут применять разные стратегии для хранения и передачи исключений. Всегда, однако, гарантируется, что хватит памяти для генерации операцией *new* стандартного исключения нехватки памяти *bad\_alloc* (§14.4.5).

### 14.3.1. Повторная генерация исключений

Перехватив исключение, обработчик рассматривает его и может иногда прийти к выводу, что сам он не в состоянии выполнить исчерпывающую обработку этого исключения. Тогда он делает ту часть работы, которая ему по плечу, после чего вновь генерирует это же исключение. В результате, исключение может обрабатываться там, где это наиболее уместно. Это возможно даже в случаях, когда необходимая для обработки исключения информация не сосредоточена в единственном месте программы, и лучше всего обработку рассредоточить по нескольким обработчикам. Например:

```

void h ()
{
    try
    {
        // код, который может генерировать "математические" исключения
    }
    catch (Matherr)
    {
        if (can_handle_it_completely)
        {
            // обработать Matherr
            return;
        }
        else
        {
            // делаем здесь то, что можем
            throw; // повторно генерируем исключение
        }
    }
}

```

Повторная генерация исключения обозначается ключевым словом **throw**, за которым не указывается никакого выражения. Если попытаться выполнить такой оператор в отсутствие исключений, то будет вызвана функция **terminate()** (§14.7). Компиляторы могут (но не всегда) предупреждать о таких ошибках.

Повторно сгенерированное исключение есть точная копия оригинального исключения, а не его часть, что была доступна как **Matherr**. Другими словами, если было сгенерировано исключение типа **Int\_overflow**, то функция, вызвавшая **h()**, может по-прежнему перехватывать исключение типа **Int\_overflow**, несмотря на то, что в самой функции **h()** исключение перехвачено с типом **Matherr**.

### 14.3.2. Перехват любых исключений

Бывает полезна и вырожденная версия описанной выше техники повторной генерации исключений. Как и для функций, где многоточие означает «любой аргумент» (§7.6), **catch(...)** означает «перехватить любое исключение». Например:

```

void m ()
{
    try
    {
        // что-либо
    }
    catch (...) // обработка любых исключений
    {
        // обработка (очистка)
        throw;
    }
}

```

То есть если основная часть функции **m()** сгенерирует любое исключение, в обработчике будут выполнены необходимые при этом завершающие действия (очист-

ка — cleanup). После выполнения локальной очистки исключение генерируется заново, чтобы имелась возможность выполнить в иных местах программы дополнительную обработку исходной ошибки. В §14.6.3.2 рассмотрена техника получения информации об исключении в обработчике с многоточием.

Важным аспектом обработки ошибок вообще и обработки исключений в частности является поддержка инвариантов программы (§24.3.7.1). Например, если предполагается, что *m*() оставляет некоторые указатели в том же состоянии, в котором она их получила, то в обработчике можно написать код, присваивающий им надлежащие значения. Таким образом, обработчик любых типов исключений может использоваться для поддержки произвольных инвариантов. Тем не менее, такое решение не является для многих важных случаев самым элегантным (§14.4).

### 14.3.2.1. Порядок записи обработчиков

Поскольку исключение производного типа может перехватываться несколькими обработчиками, порядок, в котором в коде представлены эти обработчики, важен. Дело в том, что отбор обработчиков выполняется в соответствие с этим порядком. Например:

```
void f()
{
    try
    {
        // ...
    }
    catch (std::ios_base::failure)
    {
        // обработка любых ошибок потокового ввода/вывода (§14.10)
    }
    catch (std::exception& e)
    {
        // обработка любых исключений стандартной библиотеки (§14.10)
    }
    catch (...)
    {
        // обработка любых других исключений (§14.3.2)
    }
}
```

Поскольку компилятор разбирается в иерархии классов, он может выявить многие логические ошибки. Например:

```
void g()
{
    try
    {
        // ...
    }
    catch (...)
    {
        // обработка любых исключений (§14.3.2)
    }
}
```

```

catch (std::exception & e)
{
    // обработка любых исключений стандартной библиотеки (§14.10)
}
catch (std::bad_cast)
{
    // обработка исключения из операции dynamic_cast (§15.4.2)
}
}

```

Здесь обработчик исключений типа **exception** никогда не будет выполняться. Даже если мы уберем обработчик с многоточием, то **bad\_cast** никогда не будет рассматриваться, потому что тип **bad\_cast** является производным от **exception**.

## 14.4. Управление ресурсами

Если функция получает некоторый ресурс (открывает файл, динамически получает блок свободной памяти, устанавливает блокировку доступа и т.д.), то для будущего функционирования системы очень важно, чтобы этот ресурс был впоследствии корректно освобожден. Часто сама функция и освобождает ресурсы перед возвратом. Например:

```

void use_file (const char* fn)
{
    FILE* f=fopen (fn, "r") ;
    // используем f
    fclose (f) ;
}

```

Это выглядит приемлемым решением до того момента, как мы задумаемся о том, что будет в случае возникновения исключения после вызова **fopen()** и до вызова **fclose()**? Ведь в этом случае возможен выход из **use\_file()** без вызова **fclose()**. Эта проблема характерна для языков программирования, не содержащих встроенных средств обработки исключений. Например, стандартная библиотечная функция **longjmp()** языка C запросто может вызвать подобного рода проблемы. Даже простейший оператор **return** может завершить **use\_file()**, оставив файл открытым.

Первая попытка сделать функцию **use\_file()** устойчивой к таким ошибкам выглядит следующим образом:

```

void use_file (const char* fn)
{
    FILE* f=fopen (fn, "r") ;

    try
    {
        // используем f
    }
    catch (...)
    {
        fclose (f) ;
    }
}

```



```

    throw;
}

fclose (f) ;
}

```

Используемый файл код заключен в блок **try**, что обеспечивает здесь перехват любых исключений, закрытие файла и повторную генерацию исключения.

Проблема с этим решением состоит в том, что оно слишком многословное, утомительное и потенциально дорогостоящее. Более того, любое многословное и утомительное решение подвержено ошибкам, ибо программисту оно надоедает. Но к счастью, существует более элегантное решение. Представим общую постановку проблемы в следующем виде:

```

void acquire ()
{
    // выделяем ресурс 1
    // ...
    // выделяем ресурс n
    // используем ресурсы
    // освобождаем ресурс n
    // ...
    // освобождаем ресурс 1
}

```

В типичном случае ресурсы должны освобождаться в порядке, обратном порядку их выделения. Это сильно напоминает поведение локальных объектов, создаваемых конструкторами и уничтожаемых деструкторами. Следовательно, проблему получения и освобождения ресурсов можно решить с помощью объектов классов, располагающих надлежащими конструкторами и деструкторами. Например, мы могли бы определить класс **File\_ptr**, ведущий себя как указатель на файл (то есть как тип **FILE\***):

```

class File_ptr
{
    FILE* p;

public:
    File_ptr (const char* n, const char* a) { p = fopen (n, a) ; }
    File_ptr (FILE* pp) { p = pp ; }
    // подходящие операции копирования
    ~File_ptr () { if (p) fclose (p) ; }
    operator FILE* () { return p ; }
};

```

Объекты типа **File\_ptr** создаются либо на базе **FILE\***, либо на базе информации, необходимой функции **fopen** (). В любом случае, объект будет автоматически уничтожен по выходу из области видимости, а деструктор закроет ассоциированный с ним файл. Наша функция сократилась теперь до минимума:

```

void use_file (const char* fn)
{
    File_ptr f (fn, "r") ;
    // используем f
}

```

Деструктор будет вызван независимо от того, завершается ли функция нормальным образом или по исключению. Таким образом, механизм обработки исключений позволил нам удалить код обработки ошибок из реализации основного алгоритма задачи. Результирующий код проще и менее подвержен внесению случайных ошибок, чем традиционный вариант.

Процесс восходящей обработки стека вызовов с целью обнаружения подходящего обработчика исключения обычно называют *раскруткой стека* (*stack unwinding*). По мере постепенной раскрутки стека вызовов для всех локально сконструированных в них объектов гарантированно вызываются деструкторы.

### 14.4.1. Использование конструкторов и деструкторов

Техника управления ресурсами через локальные объекты называется так: «получение ресурса есть инициализация». Эта весьма общая техника опирается на свойства конструкторов и деструкторов в связи с их взаимодействием с исключениями.

Объект не считается полностью созданным до тех пор, пока не завершится работа конструктора над этим объектом. Только после этого механизм раскрутки стека с вызовом деструктора станет возможным. Объект, состоящий из подобъектов, считается полностью сконструированным лишь тогда, когда сконструированы все его подобъекты. Создание массива включает создание всех его элементов (и лишь полностью сконструированные элементы массива корректно уничтожаются в процессе раскрутки стека).

Конструктор выполняет попытку корректно и в полном объеме сконструировать объект. Когда же не удастся это сделать, хорошо написанный конструктор восстанавливает (насколько возможно) исходное состояние системы. В идеале, конструкторы должны реализовывать одну из этих альтернатив, никогда не оставляя объекты в полуподготовленном состоянии. Этого можно добиться, применяя к членам класса принцип «получение ресурса есть инициализация».

Рассмотрим класс *X*, конструктор которого должен получить два ресурса — файл *x* и блокировку *y*. В процессе получения ресурсов может генерироваться исключение, сигнализирующее о недоступности ресурса. Конструктор класса *X* не должен нормальным образом завершать работу в том случае, когда файл получен, а блокировка оказалась недоступной. Эту проблему можно решить элегантно (без излишней нагрузки на программиста). Для приобретения указанных ресурсов, используем классы *File\_ptr* и *Lock\_ptr*, соответственно. Получение ресурса формулируем в виде инициализации локальных объектов, представляющих эти ресурсы:

```
class X
{
    File_ptr aa;
    Lock_ptr bb;

public:
    X(const char* x, const char* y)
        : aa(x, "rw"), // приобретаем 'x'
        : bb(y)         // приобретаем 'y'
```

Теперь, как и для локальных объектов, реализация C++ берет на себя все заботы о деталях выполнения процесса (никаких дополнительных действий от программиста не требуется). Например, если исключение возникнет после того, как прошла инициализация **aa**, но до того, как создан **bb**, будет автоматически вызван деструктор для **aa** (но не для **bb**).

Итак, где годится рассмотренная простая модель получения ресурсов, от программиста не требуется писать явный код обработки исключений.

Самым распространенным ресурсом является память. Например:

```
class Y
{
    int* p;
    void init () ;

public:
    Y(int s) { p = new int[s] ; init () ; }
    ~Y() { delete [] p ; }
    // ...
};
```

Такая практика весьма распространена и она может приводить к «утечкам памяти» (memory leaks). Если в **init()** будет сгенерировано исключение, то выделенная в конструкторе память не освобождается — деструктор не будет вызван, ибо объект сконструирован к этому моменту не полностью. Вот более безопасный вариант:

```
class Z
{
    vector<int> p;
    void init () ;

public:
    Z(int s) : p(s) { init () ; }
    // ...
};
```

Память, используемая вектором **p**, теперь находится под управлением типа **vector**. Если **init()** сгенерирует исключение, выделенная память будет (неявно) освобождена вызовом деструктора для **p**.

### 14.4.2. Auto\_ptr

Стандартная библиотека предоставляет классовый шаблон **auto\_ptr**, поддерживающий технику «получение ресурса есть инициализация». В своей основе **auto\_ptr** инициализируется указателем и может быть разыменован аналогично указателю. Кроме того, указуемый объект неявно удаляется (уничтожается) тогда, когда объект типа **auto\_ptr** выходит из области видимости. Например:

```
void f(Point p1, Point p2, auto_ptr<Circle> pc, Shape* pb)
{
    auto_ptr<Shape> p(new Rectangle(p1,p2)) ; // p указывает на rectangle
    auto_ptr<Shape> pbox(pb) ;

    p->rotate(45) ;                               // используем auto_ptr<Shape> как Shape*
    //
```

```

    if (in_a_mess) throw Mess ();
    // ...
}

```

Здесь объекты типа *Rectangle*, *Shape* (адресуется через *pb*) и *Circle* (адресуется с помощью *pc*) корректно уничтожаются независимо от того, генерируется исключение или же не генерируется.

Для достижения *семантики владения* (*ownership semantics*), копирование объектов типа *auto\_ptr* радикально отличается от копирования обычных указателей (реализуется *семантика деструктивного копирования* — *destructive copy semantics*): если некоторый объект типа *auto\_ptr* копируется в другой объект того же типа, то исходный объект после этого уже ни на что не указывает. Из-за того, что копирование объектов типа *auto\_ptr* изменяет их самих, нельзя скопировать объекты типа *const auto\_ptr*.

Шаблон *auto\_ptr* определен в файле *<memory>*:

```

template <class Y> struct auto_ptr_ref { /* ... */ };           // вспомогательный класс

template <class X> class std::auto_ptr
{
    X* ptr;

public:
    typedef X element_type;

    // внимание: копирующий конструктор и присваивания имеют не-const аргументы.
    explicit auto_ptr (X* p=0) throw () { ptr=p; } // throw()-"ничего не генерировать"; см.§14.6
    auto_ptr (auto_ptr& a) throw ();                // копируем, затем a.ptr=0
    template <class Y> auto_ptr (auto_ptr<Y>& a) throw (); // копируем, затем a.ptr=0
    ~auto_ptr () throw () { delete ptr; }

    auto_ptr& operator= (auto_ptr& a) throw ();      // копируем и a.ptr=0
    template <class Y> auto_ptr& operator= (auto_ptr<Y>& a) throw (); // копируем и a.ptr=0
    template <class Y> auto_ptr& operator= (auto_ptr_ref<Y>& a) throw (); // копируем и release

    X& operator* () const throw () { return *ptr; }
    X* operator-> () const throw () { return ptr; }

    X* get () const throw () { return ptr; }         // извлечь указатель
    X* release () throw () { X* t=ptr; ptr=0; return t; } // передача владения
    void reset (X* p=0) throw () { if (p!=ptr) { delete ptr; ptr=p; } }

    auto_ptr (auto_ptr_ref<X>) throw ();             // копирование из auto_ptr_ref
    template <class Y> operator auto_ptr_ref<Y> () throw (); // копирование в auto_ptr_ref
    template <class Y> operator auto_ptr<Y> () throw ();   // деструктив. коп-е из auto_ptr
};

```

Цель *auto\_ptr\_ref* — реализовать семантику деструктивного копирования для *auto\_ptr*, делая невозможным копирование *const auto\_ptr*. Если *D\** может быть преобразован в *B\**, тогда конструктор и операция присваивания шаблона могут (явно или неявно) преобразовывать *auto\_ptr<D>* в *auto\_ptr<B>*. Например:

```

void g (Circle* pc)
{
    auto_ptr<Circle> p2 (pc); // теперь p2 отвечает за удаление
    auto_ptr<Circle> p3 (p2); // теперь p3 отвечает за удаление (а p2 нет)
}

```

```

p2->m = 7;           // ошибка программиста: p2.get()==0
Shape* ps = p3.get(); // извлекаем указатель из auto_ptr

auto_ptr<Shape> aps(p3); // передаем владение и преобразуем тип
auto_ptr<Circle> p4(pc); // ошибка программиста: теперь и p4 отвечает за удаление
}

```

Эффект принадлежности объекта двум **auto\_ptr** не определен; скорее всего, это приведет к двукратному уничтожению объекта (с отрицательными последствиями).

Отметим, что из-за семантики деструктивного копирования объекты типа **auto\_ptr** не удовлетворяют требованиям для элементов стандартных контейнеров и стандартных алгоритмов, таких как **sort()**. Например:

```

vector<auto_ptr<Shape>> & v; //опасно: использование auto_ptr в контейнере
// ...
sort(v.begin(), v.end()); //не делайте этого: сортировка может испортить v

```

Ясно, что **auto\_ptr** не является интеллектуальным указателем (smart pointer); он лишь без дополнительных накладных расходов реализует сервис, для которого и был задуман — повышенную надежность (безопасность) кода в связи с возможностью исключений.

### 14.4.3. Предостережение

Не все программы должны обладать повышенной живучестью по отношению к любым видам ошибочных ситуаций, и не все ресурсы столь критичны, чтобы оправдать совокупные усилия для применения принципа «получение ресурса есть инициализация», типа **auto\_ptr** и механизма **catch(...)**. Например, для большинства программ, которые просто читают ввод и далее работают в направлении своего завершения, наиболее естественной реакцией на возникновение серьезной ошибки будет прекращение работы (после выдачи приемлемого диагностического сообщения). Это позволяет системе освободить ранее выделенные ресурсы, а пользователю — запустить программу заново и ввести корректные данные. Рассмотренная же стратегия предназначена для приложений, которые не могут просто так завершить работу. В частности, разработчик библиотеки не может знать требований программы, использующей библиотеку, в отношении стратегии поведения в случае возникновения ошибочных ситуаций. В результате, он вынужден избегать любых форм безусловного прекращения работы и вынужден заботиться об освобождении ранее выделенных ресурсов перед возвратом в головную программу. Для таких вот случаев и подходит стратегия «получение ресурса есть инициализация» в совокупности с исключениями для сигнализации об ошибочном ходе выполнения библиотечных функций.

### 14.4.4. Исключения и операция new

Рассмотрим пример:

```

void f(Arena& a, X* buffer)
{
    X* p1 = new X;
    X* p2 = new X[10];
}

```

```

X* p3 = new (&buffer[10]) X; // поместить X в buffer (освобождение не нужно)
X* p4 = new (&buffer[11]) X[10];

X* p5 = new (a) X;           // выделить из Arena a (освободить из Arena a)
X* p6 = new (a) X[10];
}

```

Что произойдет, если конструктор класса *X* сгенерирует исключение? Освобождается ли память, выделенная функцией *operator new()*? В обычном случае ответ положителен, так что инициализации *p1* и *p2* не вызывают утечки памяти.

Когда же используется синтаксис размещения (§10.4.11), ответ не так прост; некоторые варианты применения этого синтаксиса включают выделение памяти, подлежащей освобождению, а другие не включают. Более того, главной целью синтаксиса размещения является возможность нестандартного выделения памяти, так что и ее освобождение должно быть нестандартным. Следовательно, необходимые действия зависят от используемой схемы выделения памяти: если для выделения памяти вызывалась функция *Z: operator new()*, то требуется вызвать функцию *Z: operator delete()* (если она определена), а в противном случае память освобождать не нужно. Массивы обрабатываются аналогичным образом (§15.6.1). Данная стратегия работает как с операцией «размещающее *new*» из стандартной библиотеки (§10.4.11), так и в случае, когда пользователь сам реализовал функции для выделения/освобождения памяти.

#### 14.4.5. Истощение ресурсов

То и дело возникает вопрос, что делать, когда попытка получить ресурс завершилась неудачно? Например, мы можем с легкой душой открывать файл (используя функцию *fopen()*) или запрашивать блок свободной памяти (с помощью операции *new*), даже не побеспокоившись о том, что будет, если файла нет в указанном месте или если нет достаточного объема свободной памяти. Сталкиваясь с такими проблемами, программист может выбрать одну из двух стратегий реагирования:

- *Возобновление (resumption)*: попросить вызвавшую функцию справиться с проблемой и продолжить работу.
- *Завершение (termination)*: бросить подзадачу и вернуть управление вызвавшей функции.

В рамках первой стратегии вызвавшая функция должна быть готова помочь решить проблему с выделением ресурса (неизвестным участком кода), а во втором случае — она должна быть готова отказаться от затребованного ресурса. Второй вариант в большинстве случаев намного проще и позволяет лучше реализовать разделение уровней абстракции. Обратите внимание, что при этом речь идет не о прекращении работы программы, а лишь о прекращении частной подзадачи. Стратегия завершения означает стратегию возврата из места, где некоторые вычисления окончились неудачей, в обработчик ошибок, связанный с вызвавшей функцией (может, например, повторно запустить неудавшиеся ранее вычисления), а не попытку устранения проблемы и возобновление вычислений с места, где проблема была обнаружена.

В языке C++ стратегия возобновления поддерживается механизмом вызова функций, а стратегия завершения — механизмом обработки исключений. Обе стра-

тегии можно проиллюстрировать простым примером реализации и использования функции **operator new()**:

```
void* operator new (size_t size)
{
    for (;;)
    {
        if (void* p = malloc (size) ) return p;           // пытаемся найти память
        if ( _new_handler == 0 ) throw bad_alloc ();      // нет обработчика: сдаемся
        _new_handler ();                                   // запрашиваем помощь
    }
}
```

Здесь я использую функцию **malloc()** из стандартной библиотеки языка C для выполнения поиска свободной памяти; другие реализации функции **operator new()** могут избрать иные способы для выполнения такой работы. Если память выделена, **operator new()** может возвратить указатель на нее. В противном случае вызывается функция-обработчик **\_new\_handler()**, призванная помочь отыскать дополнительный объем свободной памяти. Если она в этом преуспевает, то все прекрасно. Если нет — то обработчик не может просто вернуть управление, так как это приведет к бесконечному циклу. Он должен сгенерировать исключение, предоставив вызывающей программе разбираться с проблемой:

```
void my_new_handler ()
{
    int no_of_bytes_found = find_some_memory ();
    if (no_of_bytes_found < min_allocation) throw bad_alloc (); // сдаемся
}
```

Где-то должен быть соответствующий **try**-блок:

```
try
{
    // ...
}
catch (bad_alloc)
{
    // как-нибудь реагируем на исчерпание памяти
}
```

В функции **operator new()** **\_new\_handler** является указателем на функцию-обработчик, регистрируемую с помощью стандартной функции **set\_new\_handler()**. Если мне нужно использовать для этой цели функцию **my\_new\_handler()**, я могу написать:

```
set_new_handler (&my_new_handler);
```

Если же еще и нужно перехватывать исключение **bad\_alloc**, то тогда следует написать такой код:

```
void f()
{
    void (*oldnh) () = set_new_handler (&my_new_handler);
```

```

try
{
    // ...
}
catch (bad_alloc)
{
    // ...
}
catch ( . . . )
{
    set_new_handler (oldnh) ;    // вернуть обработчик
    throw ;                      // повторно сгенерировать исключение
}

Set_new_handler (oldnh) ;    // вернуть обработчик
}

```

А еще лучше вместо **catch ( . . . )** использовать стратегию «получение ресурса есть инициализация» (см. §14.4) применительно к **\_new\_handler** (§14.12[1]).

При использовании **\_new\_handler** никакой дополнительной информации не передается из места обнаружения ошибки в функцию-обработчик. В принципе, передать дополнительную информацию не сложно. Но чем больше дополнительной информации при этом передается, тем больше взаимозависимость этих двух фрагментов кода. Это означает, что изменения в одном фрагменте требуют детального знания другого фрагмента, а может быть даже и изменения второго фрагмента. Поэтому для достижения минимума зависимости между фрагментами лучше ограничивать объем пересылаемой между ними информации. Механизм обработки исключений обеспечивает лучшую изоляцию фрагментов, чем механизм вызова функций-обработчиков, предоставляемых вызывающей стороной.

Как правило, лучше организовать выделение ресурсов послойно (в разных уровнях абстракции), минимизируя зависимости одних слоев от помощи со стороны других (вызывающих) слоев. Опыт работы с крупными системами показывает, что они эволюционируют в указанном направлении.

Генерации исключений сопутствует создание объектов. Конкретные реализации в ситуации исчерпания памяти обязаны гарантировать создание объекта типа **bad\_alloc**. Однако генерация иных исключений может приводить к полному исчерпанию памяти.

#### 14.4.6. Исключения в конструкторах

Исключения решают проблему индикации ошибок в работе конструкторов. Ввиду того, что конструкторы не возвращают никаких значений, традиционными альтернативами исключений являются следующие подходы:

1. Оставить создаваемый объект в нештатном состоянии, в надежде на проверку его состояния пользователем.
2. Установить значение нелокальной переменной (например, **errno**) для индикации об ошибке в работе конструктора, в надежде на проверку ее значения пользователем.



3. Не выполнять инициализацию в конструкторах, оставив эту работу для инициализирующих функций-членов, вызываемых пользователем до первого использования объекта.
4. Пометить объект как «непроинициализированный» и при первом вызове функции-члена для этого объекта выполнить инициализацию уже в этой функции (с вытекающими последствиями в случае невозможности успешно-го завершения инициализации).

Механизм обработки исключений позволяет передать вовне информацию о проблемах в работе конструкторов. Например, векторный класс может защитить свои объекты от попыток затребовать слишком много памяти в момент их создания:

```
class Vector
{
public:
    class Size { };

    enum { max=32000 };

    Vector : Vector (int sz)
    {
        if (sz<0 || max<sz) throw Size ();
        // ...
    }
    // ...
};
```

Код, создающий объекты типа *Vector*, может перехватывать ошибки *Vector : Size*, чтобы сделать в этом случае что-нибудь осмысленное:

```
Vector* f(int i)
{
    try
    {
        Vector* p = new Vector (i);
        // ...
        return p;
    }
    catch (Vector : Size)
    {
        // обработка ошибки с размером
    }
}
```

Обработчик ошибок может использовать стандартный набор основных технологий сигнализирования об ошибках и восстановления. При каждой передаче исключения в вызывающую функцию мнение о том, что же пошло не так, может различаться. Если передавать с исключением всю необходимую информацию, то объем исходных знаний об ошибке возрастает. Другими словами, главной задачей всех технологий обработки ошибок является передача информации об ошибке из точки ее возникновения в такую точку программы, где имеется достаточно сведений для выполнения удобным и надежным способом процесса восстановления системы от последствий возникновения ошибочных ситуаций.

Применение стратегии «получение ресурса есть инициализация» является самым безопасным и элегантным способом обработки ошибок в конструкторах, запрашивающих несколько ресурсов (§14.4). По сути, эта стратегия сводит работу с несколькими ресурсами к последовательному применению техники работы с единственным ресурсом.

#### 14.4.6.1. Исключения и инициализация членов классов

Что происходит, когда инициализирующий некоторый член класса код (прямо или косвенно) генерирует исключение? По умолчанию, исключение передается туда, откуда инициирован конструктор класса этого члена. Тем не менее, сам *конструктор может перехватить исключение*, если все его *тело и список инициализации членов заключить в try-блок*. Например:

```
class X
{
    Vector v;
    // ...
public:
    X(int) ;
    // ...
};

X::X(int s)
try
: v(s)                // инициализация v значением s
{
    // ...
}
catch (Vector::Size) // исключения из v перехватываются здесь
{
    // ...
}
```

#### 14.4.6.2. Исключения и копирование

Как все конструкторы, копирующий конструктор может сигнализировать о невозможности штатного исполнения генерацией исключения. Объект в этом случае не создается. Например, копирующему конструктору класса *vector* часто нужно выделять память и копировать в нее элементы (§16.3.4, §E.3.2), и это может вызвать генерацию исключения. Перед генерацией исключения копирующему конструктору нужно освободить все занятые им ресурсы. Подробное обсуждение вопросов, связанных с управлением ресурсами и обработкой исключений для контейнерных классов, представлено в §E.2 и §E.3.

Копирующее присваивание похоже на копирующий конструктор в том отношении, что тоже может занимать ресурсы и завершаться генерацией исключения. Перед генерацией исключения здесь нужно убедиться, что оба операнда операции находятся в корректном состоянии. В противном случае нарушаются требования стандартной библиотеки, и это может привести к непредсказуемому поведению (§E.2, §E.3.3).

### 14.4.7. Исключения в деструкторах

С точки зрения механизма обработки исключений деструктор может быть вызван двумя способами:

1. *Нормальный вызов*: в результате обычного выхода из области видимости (§10.4.3), или операции `delete` (§10.4.5).
2. *Вызов в процессе обработки исключения*: в процессе раскрутки стека (§14.4) механизм обработки исключений приводит к выходу из области видимости, содержащей объект с деструктором.

В последнем случае исключение не может исходить из самого деструктора, так как это считается ошибкой в работе механизма обработки исключений, приводящей к вызову `std::terminate()` (§14.7). В конце концов, ни механизм обработки исключений, ни деструкторы не в состоянии в общем случае определить, каким исключением пренебречь ради обработки другого исключения. Выход из деструктора путем генерации исключения также считается нарушением требований стандартной библиотеки (§E.2).

Деструктор может обрабатывать исключения, которые генерируются вызываемыми им функциями. Например:

```
X: ~X()
try
{
    f();           // может генерировать исключения
}
catch (...)
{
    // что-нибудь делаем
}
```

Функция стандартной библиотеки `uncaught_exception()` из файла `<exception>` возвращает `true`, если исключение было сгенерировано, но еще не было перехвачено. Это позволяет программисту предпринимать разные действия в случаях, когда объект уничтожается нормальным образом, или же в процессе раскрутки стека.

## 14.5. Исключения, не являющиеся ошибками

Если мы предвидим возникновение исключений и перехватываем их для обработки таким образом, что конечное поведение программы при этом не страдает, то почему нужно в таких случаях считать исключения ошибками? Только потому, что программист считает их ошибками, а механизм обработки исключений рассматривает как механизм обработки ошибок. С другой стороны, можно рассматривать механизм обработки исключений как еще одну управляющую структуру. Например:

```
void f(Queue<X> & q)
{
    try
    {
        for(;;)
```

```

{
    X m = q.get(); // генерирует Empty, если queue пустой (empty)
    // ...
}
}
catch (Queue<X> : : Empty)
{
    return;
}
}

```

В этом примере есть свое очарование, ибо это как раз тот случай, когда невозможно определенным образом сказать, что тут ошибка, а что нет.

Обработка исключений представляет собой менее структурированный механизм управления, нежели локальные управляющие структуры типа *if* или *for*, и он показывает меньшую эффективность в случае реальной генерации исключения. Поэтому использовать такой механизм стоит лишь тогда, когда более традиционные структуры управления либо неэлегантны, либо недопустимы. Отметим также, что стандартная библиотека предоставляет очередь *queue* для произвольных элементов без использования исключений (§17.3.2).

Применение исключений в качестве альтернативного варианта выхода из функции является элегантным приемом, например, для выхода из функций поиска — особенно в случае глубоко рекурсивного поиска в древовидных структурах. Например:

```

void fnd{ Tree* p, const string& s)
{
    if(s == p->str) throw p;           // s найдено
    if(p->left) fnd(p->left, s);
    if(p->right) fnd(p->right, s);
}

Tree* find(Tree* p, const string& s)
{
    try
    {
        fnd(p, s);
    }
    catch (Tree* q)                    // q->str==s
    {
        return q;
    }
    return 0;
}

```

Рассмотренным применением исключений не стоит перебарщивать, ибо это ведет к запутанному коду. Всегда лучше, по возможности, придерживаться традиционной трактовки — «обработка исключений есть обработка ошибок». В этом случае код явным образом разделяется на «обычный код» и «код обработки ошибок». Это сильно улучшает читаемость кода. Однако реальный мир не расслаивается на очевидные подчасти столь же простым образом (и организация программ отражает и должна отражать этот факт).

Обработка ошибок является сложной по своей сути. Все, что помогает четко определить понятие ошибки и выявить разумную стратегию ее обработки, имеет большую ценность.

## 14.6. Спецификация исключений

Генерация и перехват исключений воздействует на механизм, которым функция взаимодействует с другими функциями. Поэтому бесполезно использовать *спецификацию исключений* (*exception specification*), для чего нужно в объявлении функции указать набор исключений, который она может породить. Например:

```
void f(int a) throw(x2, x3) ;
```

Это объявление специфицирует, что функция *f()* может быть источником исключений типа *x2*, *x3* и производных от них типов, и никаких других типов исключений. Объявленная таким образом функция предоставляет своим пользователям определенные гарантии. Если во время своего выполнения она попытается произвести действия, нарушающие объявленную гарантию, такая попытка приведет к вызову стандартной функции *std::unexpected()*. По умолчанию, функция *std::unexpected()* вызывает *std::terminate()*, а та, в свою очередь, вызывает *abort()* (§9.4.1.1).

На самом деле, следующий код

```
void f() throw(x2, x3)
{
    // нечто
}
```

эквивалентен

```
void f()
try
{
    // нечто
}
catch(x2) {throw; }           // генерируем повторно
catch(x3) {throw; }           // генерируем повторно
catch(...) {std::unexpected();} // unexpected() не возвращает управление
```

Важным преимуществом рассмотренной спецификации исключений является тот факт, что объявление функции видимо всем ее пользователям (является частью спецификации ее интерфейса), а определение функции — нет. Но даже если доступны все определения библиотечных функций, вряд ли нам так уж захочется в деталях знакомиться с каждым из них. К тому же, явная спецификация типов исключений в составе объявления функции намного короче варианта с подробным ручным кодированием.

Предполагается, что функция, объявленная *без спецификации исключений*, может породить *любое исключение*. Например:

```
int f() ;           // может генерировать любые исключения
```

Функция, не порождающая никаких исключений, может быть объявлена следующим образом:

```
int g () throw () ; // не генерирует исключений
```

Некоторые могут подумать, что умолчательный вариант больше подходит функциям, запрещающим исключения. Такое правило потребовало бы обязательного наличия спецификации исключений практически для всех функций, вынуждало бы частую перекомпиляцию кода и препятствовало бы взаимодействию с кодом, написанным на других языках программирования. Все это заставляло бы программистов ниспровергать механизм обработки исключений и писать паразитный код для их подавления. А это создало бы ложное чувство безопасности у людей, которые не заметили подобной «подрывной деятельности».

### 14.6.1. Проверка спецификации исключений

Невозможно выявить абсолютно все нарушения спецификации интерфейса на этапе компиляции. Тем не менее, на этапе компиляции реально выполняется весьма существенная проверка. О спецификациях исключений можно думать как о предположении, что функция сгенерирует все указанные в ней исключения. Простые нелепости в связи со спецификацией исключений легко выявляются в процессе компиляции.

Если некоторое объявление функции содержит спецификацию исключений, то тогда и все другие ее объявления (включая определение) обязаны иметь точно такую же спецификацию. Например:

```
int f() throw (std::bad_alloc) ;  
int f() //error: отсутствие спецификации исключений  
{  
  // ...  
}
```

Важно, что не требуется проверять спецификации исключений за пределами единицы трансляции. Конкретные реализации могут это делать. Но для больших и долгоживущих программных систем они этого делать не должны, или если уж делают, то тогда им следует детектировать лишь грубейшие ошибки, которые невозможно перехватить на этапе выполнения. Смысл состоит в том, чтобы добавление исключений в каком-либо месте программы не требовало бы тотального обновления спецификаций и перекомпиляции остальных частей программы. Система может функционировать в частично обновленном состоянии, полагаясь на динамическое обнаружение неожиданных исключений на этапе выполнения. Все это очень важно для поддержки и сопровождения больших программных систем, значительные модификации которых весьма дороги и не весь их исходный код доступен.

Виртуальная функция может замещаться функцией с не менее ограничительной спецификацией исключений. Например:

```
class B  
{  
public:  
  virtual void f() ; // может генерировать любые исключения  
  virtual void g () throw (X, Y) ;  
  virtual void h () throw (X) ;  
}
```

```

class D:public B
{
    void f() throw (X) ;           // ok
    void g() throw (X) ;           // ok: D::g() более ограничительно, чем B::g()
    void h() throw (X, Y) ;        // error: D::h() менее ограничительно, чем B::h()
};

```

Это правило соответствует здравому смыслу. Если в функции производного класса допускается исключение, не объявленное в соответствующей версии базового класса, вызывающая функция (клиент) может не ожидать его. С другой стороны, замещающая виртуальная функция со спецификацией, допускающей меньшее число исключений, не противоречит спецификации исключений замещенной функции.

Аналогично, вы можете присвоить значение указателя на функцию (то есть адрес функции) с более ограничительной спецификацией исключений указателю на функцию с более широкой спецификацией исключений, но не наоборот. Например:

```

void f() throw (X) ;
void (*pf1) () throw (X, Y) = &f;    // ok
void (*pf2) () throw () = &f;        // error: f() менее ограничительна, чем pf2

```

В частности, нельзя присваивать значение указателя на функцию с отсутствующей спецификацией исключений указателю на функцию с явно указанной спецификацией исключений:

```

void g() ;                          // может генерировать любые исключения
void (*pf3) () throw (X) = &g;      // error: g() менее ограничительна, чем pf3

```

Спецификация исключений не является частью типа функции, и в операторе **typedef** она не допускается. Например:

```

typedef void (*PF) () throw (X) ;    // error

```

### 14.6.2. Неожиданные исключения

Спецификация исключений может приводить к вызову **unexpected()**. В принципе, такие вызовы нежелательны, кроме как на этапе тестирования. Их можно избежать посредством тщательной организации исключений и спецификаций интерфейсов. Кроме того, вызовы **unexpected()** могут перехватываться таким образом, что они становятся безвредными.

Исключения хорошо сформированной подсистемы **Y** часто наследуют от одного класса **Yerr**. Например, при наличии определения

```

class Some_Yerr: public Yerr { /* ... */ };

```

функция, объявленная как

```

void f() throw (Xerr, Yerr, exception) ;

```

будет передавать любое исключений **Yerr** вызывающей функции. В частности, функция **f()** будет обрабатывать любое исключение типа **Some\_Yerr**, передавая его вызывающей функции. В итоге, никакое исключение семейства **Yerr** не спровоцирует в **f()** вызов **unexpected()**.

Все исключения стандартной библиотеки наследуются от класса **exception** (§14.10).

### 14.6.3. Отображение исключений

Политика завершения работы программы при обнаружении неожиданного исключения иногда является неоправданно жесткой. В таких случаях поведение *unexpected()* должно быть изменено с тем, чтобы добиться более приемлемых результатов.

Проще всего это можно сделать добавлением стандартного исключения *std::bad\_exception* к спецификации исключений. В этом случае *unexpected()* будет просто генерировать *std::bad\_exception*, а не вызывать функции для обработки проблемы. Например:

```
class X{ };
class Y{ };
void f() throw (X, std::bad_exception)
{
    // ...
    throw Y();    // генерация "bad" exception
}
```

Спецификация исключений зафиксировывает неприемлемое исключение *Y* и в ответ сгенерирует исключение типа *std::bad\_exception*.

Нет ничего плохого в исключении *std::bad\_exception*; оно просто запускает менее радикальный механизм реакции, чем вызов *terminate()*. Механизм этот, правда, все-таки еще сыроват: например, информация о том, какое исключение вызвало проблему, теряется.

#### 14.6.3.1. Отображение исключений пользователем

Рассмотрим функцию *g()*, написанную для несетевого окружения. Предположим также, что функция *g()* была объявлена со спецификацией исключений, допускающей лишь исключения, относящиеся к «подсистеме *Y*»:

```
void g() throw (Yerr);
```

А теперь предположим, что нам нужно вызвать *g()* в сетевом окружении.

Естественно, что *g()* ничего не знает о сетевых исключениях и будет вызывать *unexpected()* в ответ на возникновение последних. Для успешного применения функции *g()* в сетевом (распределенном) окружении мы будем вынуждены либо написать код для обработки сетевых исключений, либо переписать *g()*. Полагая переписывание функции *g()* нежелательным и неприемлемым решением, мы предоставим собственную функцию, вызываемую вместо *unexpected()*.

Как мы уже знаем, ситуация с исчерпанием памяти обрабатывается с помощью указателя *new\_handler*, который регистрируется функцией *set\_new\_handler()*. Аналогично, реакция на неожиданное исключение определяется указателем *unexpected\_handler*, регистрируемым функцией *std::set\_unexpected()* из файла *<exception>*:

```
typedef void (*unexpected_handler)();
unexpected_handler set_unexpected(unexpected_handler);
```

Чтобы обрабатывать неожиданные исключения так, как это нас устраивает, сначала определяем класс, позволяющий реализовать стратегию «получение ресурса есть инициализация» в отношении функций типа *unexpected()*:



```
class STC // класс для хранения и восстановления
{
    unexpected_handler old;

public:
    STC(unexpected_handler f) {old=set_unexpected(f); }
    ~STC() {set_unexpected(old); }
};
```

Затем определяем функцию **throwY()**, которую мы хотим использовать вместо **unexpected()** в нашем случае:

```
class Yunexpected : public Yerr {};
void throwY() throw(Yunexpected) {throw Yunexpected(); }
```

Будучи использованной вместо **unexpected()**, функция **throwY()** отображает любое неожиданное исключение в **Yunexpected**.

Наконец, мы предоставляем версию функции **g()**, призванную работать в сетевой среде:

```
void networked_g() throw(Yerr)
{
    STC xx(&throwY); // теперь unexpected() генерирует Yunexpected
    g();
}
```

Поскольку **Yunexpected** является производным исключением от **Yerr**, то спецификация исключений не нарушается. Если бы **throwY()** генерировала исключение, нарушающее спецификацию исключений, вызывалась бы функция **terminate()**.

Сохраняя и затем восстанавливая значение указателя **\_unexpected\_handler**, мы предоставляем разным подсистемам возможность по-разному обрабатывать неожиданные исключения, не входя в противоречие друг с другом. В общем и целом, техника отображения неожиданных исключений в известные исключения является более гибким решением, нежели предлагаемое системой стандартное отображение в **std::bad\_exception**.

### 14.6.3.2. Восстановление типа исключения

Отображение неожиданного исключения в **Yunexpected** позволит пользователю **networked\_g()** узнать только то, что некоторое неожиданное исключение отображено в **Yunexpected**. Пользователь не будет знать, какое именно неожиданное исключение отображено таким образом. Эта информация теряется в функции **throwY()**. Простой метод позволяет сохранить и передать эту информацию. Например, можно следующим образом собрать информацию об исключениях **Network\_exception**:

```
class Yunexpected : public Yerr
{
public:
    Network_exception* pe;
    Yunexpected(Network_exception* p) : pe(p?p->clone():0) {}
    ~Yunexpected() {delete p; }
```

```

void throwY() throw (Yunexpected)
{
    try
    {
        throw;                // повторно сгенерировать и тут же перехватить!
    }
    catch (Network_exception & p)
    {
        throw Yunexpected (&p); // генерация отображенного исключения
    }
    catch (...)
    {
        throw Yunexpected (0);
    }
}

```

Повторная генерация и перехват исключения позволяют нам управлять любым исключением, имя которого нам известно. Функция **throwY()** вызывается из **unexpected()**, вызов которой концептуально порождается из обработчика **catch(...)**, так что некоторое исключение в самом деле существует и его можно генерировать повторно оператором **throw**. При этом функция **unexpected()** проигнорировать исключение и просто вернуть управление не может — попытайся она это сделать, в ней самой же будет сгенерировано исключение **std::bad\_exception** (§14.6.3).

Функция **clone()** используется для размещения копии исключения в свободной памяти. Эта копия будет существовать и после того, как произойдет очистка локальных переменных обработчика исключений.

## 14.7. Неперехваченные исключения

Если исключение генерируется, но не перехватывается, то вызывается функция **std::terminate()**. Эта функция вызывается также в случаях, когда механизм обработки исключений сталкивается с повреждением стека, или когда деструктор, вызванный в процессе раскрутки стека в связи с исключением, пытается завершить работу генерацией исключения.

Как мы знаем, реакция на неожиданное исключение определяется указателем **\_unexpected\_handler**, регистрируемым функцией **std::set\_unexpected()**. Аналогично, реакция на неперехваченное исключение определяется указателем **\_uncaught\_handler**, регистрируемым функцией **std::set\_terminate()** из файла **<exception>**:

```

typedef void (*terminate_handler)();
terminate_handler set_terminate(terminate_handler);

```

Возвращаемым значением является указатель на прежнюю функцию, переданную для регистрации **set\_terminate()**.

Функция **terminate()** нужна там, где нужно сменить механизм обработки исключений на что-нибудь менее тонкое. Например, функция **terminate()** могла бы использоваться для прекращения процесса и, возможно, для реинициализации систе-

мы. Функция ***terminate***() предназначена для принятия решительных мер в тот момент, когда стратегия восстановления системы с помощью механизма обработки исключений потерпела неудачу и пришло время для перехода на другой уровень борьбы с ошибками.

По умолчанию ***terminate***() вызывает функцию ***abort***() (§9.4.1.1), что подходит в большинстве случаев, особенно во время отладки.

Предполагается, что обработчик, адресуемый указателем ***\_uncaught\_handler***, не возвращает управление вызвавшему его коду. Если он попытается это сделать, ***terminate***() вызовет ***abort***() .

Отметим, что вызов ***abort***() означает ненормальный выход из программы. Для выхода из программы можно также использовать функцию ***exit***() , имеющую возврат, который указывает системе, был ли выход нормальным или ненормальным (§9.4.1.1).

Вызываются ли деструкторы при завершении работы программы из-за перехваченных исключений или нет, зависит от конкретных реализаций. Для некоторых систем важно, чтобы деструкторы не вызывались, так как в этом случае возможно возобновление работы программы в среде отладчика. Для других систем почти невозможно не вызывать деструкторы в процессе поиска обработчика.

Если вы хотите гарантировать корректную очистку для перехватываемых исключений, нужно добавить обработчик ***catch***(...) (§14.3.2) к телу функции ***main***() помимо иных обработчиков исключений, которые вы намерены обрабатывать. Например:

```
int main ()
try
{
    // ...
}
catch (std::range_error)
{
    cerr<< "range error: Not again!\n";
}
catch (std::bad_alloc)
{
    cerr<< "new ran out of memory\n";
}
catch (...)
{
    // ...
}
```

Эта конструкция перехватит любое исключение кроме тех, которые генерируются при конструировании или уничтожении глобальных объектов. Невозможно перехватить исключение, генерируемое в процессе инициализации глобального объекта. Единственный способ получить контроль при генерации исключений в этих случаях — использовать ***set\_unexpected***() (§14.6.2). Это еще одна причина по возможности избегать глобальных переменных.

При перехвате исключения истинная точка программы, в которой оно было сгенерировано, неизвестна. Это меньшая информация о программе по сравнению

с тем, что о ней может знать специальная программа-отладчик. Поэтому в некоторых средах разработки для некоторых программ может оказаться предпочтительным *не* перехватывать исключения, восстановление от которых не предусматривается дизайном программы.

## 14.8. Исключения и эффективность

В принципе, обработку исключений можно организовать таким образом, что если исключение не сгенерировано, то нет и дополнительных накладных расходов. Кроме того, обработку исключений можно организовать таким образом, что она окажется не дороже вызова функции. Можно добиться этого без использования заметных дополнительных объемов памяти, а также соблюдая последовательность вызовов языка С и требования отладчиков. Это конечно не легко, но и альтернативы исключениям тоже не бесплатны — часто можно встретить традиционный код, половина объема которого занята обработкой ошибок. Рассмотрим простую функцию  $f()$ , которая на первый взгляд не имеет никакого отношения к обработке исключений:

```
void g (int) ;
void f ()
{
    string s ;
    // ...
    g (1) ;
    g (2) ;
}
```

Однако функция  $g()$  может генерировать исключение, и  $f()$  должна обеспечить корректное уничтожение строки  $s$  в этом случае. Если бы  $g()$  не генерировала исключений, а использовала бы иной механизм для информирования об ошибках, то приведенная выше простая функция  $f()$  превратилась бы в нечто вроде следующего примера:

```
bool g (int) ;
bool f ()
{
    string s ;
    // ...
    if (g (1) )
        if (g (2) )
            return true ;
        else
            return false ;
    else
        return false ;
}
```

Обычно программисты не столь систематически обрабатывают ошибки, да это и не всегда нужно. В то же время, когда систематичность в обработке ошибок нужна, лучше поручить всю эту «хозяйственную работу» компьютеру, то есть механизму обработки исключений.

Спецификации исключений (§14.6) особо полезны для генерации качественного кода. Если бы мы явно указали, что функция `g()` не может генерировать исключений:

```
void g(int) throw();
```

качество машинного кода для функции `f()` значительно возросло бы. Стоит отметить, что ни одна традиционная библиотечная функция языка С не генерирует исключений, так что их можно практически в любой программе объявлять с пустой спецификацией исключений `throw()`. В конкретных реализациях лишь немногочисленные функции, такие как `atexit()` и `qsort()`, могут генерировать исключения. Зная эти факты, конкретные реализации могут порождать более эффективный и качественный машинный код.

Перед тем, как приписать «С функции» пустую спецификацию исключений, подумайте, не может ли она все-таки генерировать исключения (прямо или косвенно). Например, такая функция могла быть модифицирована с применением операции `new` языка С++, которая генерирует исключение `bad_alloc`, или она использует средства стандартной библиотеки языка С++, генерирующие исключения.

## 14.9. Альтернативы обработке ошибок

Механизм обработки исключений призван обеспечить передачу сообщений об «исключительных обстоятельствах» из одной части программы в другую. Предполагается также, что указанные части программы написаны независимо, и что та часть, которая обрабатывает исключение, может сделать что-либо осмысленное с возникшей ошибкой.

Чтобы использовать обработчики эффективно, нужна целостная стратегия. То есть различные части программы должны договориться о том, как используются исключения и где обрабатываются ошибки. Механизм обработки исключений не локализован по своей сути, и поэтому следование общей стратегии очень важно. Это подразумевает, что рассмотрение стратегии обработки ошибок лучше запланировать на самые ранние стадии проектирования. Также важно, чтобы стратегия была простой (по сравнению со сложностью самой программы) и явной. Никто не будет придерживаться сложной стратегии в таких и без того сложных вопросах, как восстановление системы после возникновения в ней динамических ошибок.

Перво-наперво, нужно отказаться от идеи, что все типы ошибок можно обработать с помощью единственной технологии — это привело бы к излишней сложности. Успешные, устойчивые к ошибкам системы обычно многоуровневые. Каждый уровень разбирается с теми ошибками, которые он способен переварить, а остальные ошибки оставляет вышестоящим уровням. Предназначение `terminate()` состоит в том, чтобы работать в случаях, когда сам механизм обработки исключений испортился, или когда он задействован не полностью и оставляет некоторые исключения неперехваченными. Аналогичным образом, `unexpected()` предоставляет выход в тех случаях, когда отказывает защитный механизм спецификации исключений.

Не каждую функцию следует защищать «по полной программе». В большинстве систем невозможно снабдить каждую функцию достаточным числом проверок, гарантирующих либо ее успешное завершение, либо строго определенное поведение

в случае невозможности выполнить штатные вычисления. Причины, по которым полная стратегия защиты неосуществима, варьируются от программы к программе и от программиста к программисту. Тем не менее, для больших программ типично следующее:

1. Объем работы, обеспечивающий предельную защищенность кода, слишком велик, чтобы его можно было выполнять последовательно и единообразно.
2. Накладные расходы на память и быстродействие неприемлемо велики (вероятна тенденция повторять одни и те же проверки, например корректность аргументов, снова и снова).
3. Функции, написанные на других языках, скорее всего выпадут из установленных правил.
4. Локальное достижение сверхнадежности приводит к сложностям, от которых страдает надежность совокупной системы.

И тем не менее, разбиение программ на подсистемы, которые либо завершаются успешно, либо терпят неудачу определенным образом, очень важно, достижимо и экономично. Так, важная библиотека, подсистема или ключевая функция должны разрабатываться подобным образом. А спецификации исключений участвуют в формировании интерфейсов к таким библиотекам и подсистемам.

Обычно нам не приходится писать код большой системы от самого нуля и до конца. Поэтому, для реализации общей стратегии обработки ошибок во всех частях программы нам придется принять во внимание фрагменты, написанные в соответствии с иными стратегиями. При этом мы столкнемся с большим разнообразием в подходах по управлению ресурсами и вынуждены будем обращать внимание на то, в каком состоянии оставляются фрагменты после обработки ошибок. Конечная цель состоит в том, чтобы заставить работать фрагмент так, как будто он внешне следует общей стратегии обработки ошибок (хотя внутренне и подчиняется своей собственной уникальной стратегии).

Иногда возникает необходимость в смене стиля обработки ошибок. Например, после вызова традиционной библиотечной функции языка C мы можем проверять переменную `errno` и, возможно, генерировать исключение, или же действовать противоположным образом: перехватывать исключение и устанавливать значение `errno` перед возвратом в C-функцию из библиотеки C++:

```
void callC() throw(C_blewit)
{
    errno=0;
    c_function();
    if(errno)
    {
        // очистка(cleanup), если возможна и необходима
        throw C_blewit(errno);
    }
}

extern "C" void call_from_C() throw()
{
    try
```

```
{
    c_plus_plus_function ();
}
catch ( . . . )
{
    // очистка(cleanup), если возможна и необходима
    errno=E_CPLPLFCTBLEWIT;
}
}
```

В таких случаях важно быть достаточно последовательным, чтобы гарантировать переход на некоторый единый стиль обработки ошибок.

Обработку ошибок следует, насколько это возможно, выполнять иерархически. Если функция наталкивается на ошибочную ситуацию в процессе своего выполнения, ей не следует за помощью обращаться к вызвавшей функции с тем, чтобы та справилась с проблемой восстановления и повторного выделения ресурсов — это порождает в системе циклические зависимости. К тому же, это делает программы запутанными и допускает возможность входа в бесконечные циклы обработки ошибок и восстановления.

Упрощающие методики, такие как «получение ресурса есть инициализация» и «исключение представляет ошибку» делают код обработки ошибок более регулярным. В §24.3.7.1 изложены идеи приложения инвариантов и утверждений (*assertions*) к механизму исключений.

14.10. Стандартные исключения

Приведем таблицу стандартных исключений, а также функций, операций и общих средств, генерирующих эти исключения:

| Стандартные исключения (генерируются языком) |                                     |                          |             |
|----------------------------------------------|-------------------------------------|--------------------------|-------------|
| Имя                                          | Чем генерируется                    | Ссылки                   | Загол. файл |
| <i>bad_alloc</i>                             | <i>new</i>                          | §6.2.6.2, §19.4.5        | <new>       |
| <i>bad_cast</i>                              | <i>dynamic_cast</i>                 | §15.4.1.1                | <typeinfo>  |
| <i>bad_typeid</i>                            | <i>typeid</i>                       | §15.4.4                  | <typeinfo>  |
| <i>bad_exception</i>                         | <i>спецификация исключений</i>      | §14.6.3                  | <exception> |
| <i>out_of_range</i>                          | <i>at()</i>                         | §3.7.2, §16.3.3, §20.3.3 | <stdexcept> |
|                                              | <i>bitset&lt;&gt;::operator[]()</i> | §17.5.3                  | <stdexcept> |
| <i>invalid_argument</i>                      | <i>конструктор bitset</i>           | §17.5.3.1                | <stdexcept> |
| <i>overflow_error</i>                        | <i>bitset&lt;&gt;::to_ulong()</i>   | §17.5.3.3                | <stdexcept> |
| <i>ios_base::failure</i>                     | <i>ios_base::clear()</i>            | §21.3.6                  | <ios>       |

Библиотечные исключения являются частью иерархии классов с корневым классом *exception*, представленным в файле <exception>:

```

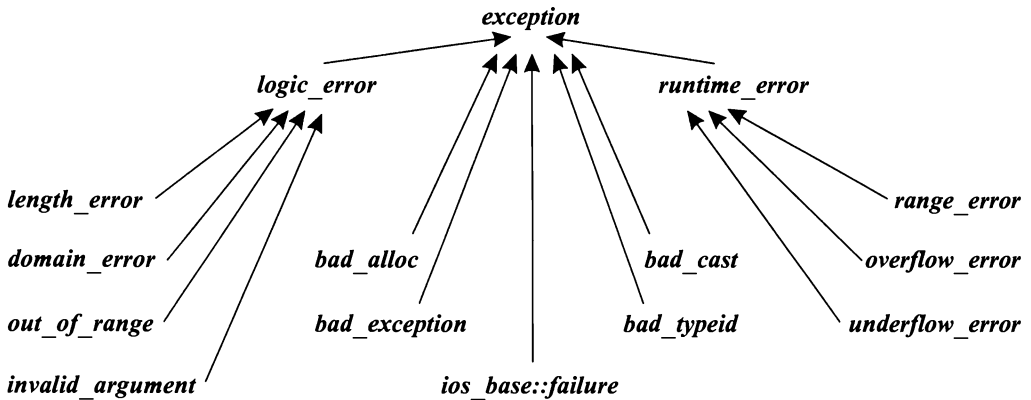
class exception
{
public:
    exception () throw () ;
    exception (const exception&) throw () ;
    exception& operator= (const exception&) throw () ;
    virtual ~exception () throw () ;

    virtual const char* what () const throw () ;

private:
    // ...
};

```

Графически эта иерархия выглядит следующим образом:



Такая организация кажется избыточной для восьми стандартных исключений. Однако она образует каркас для более широкой совокупности, нежели исключения стандартной библиотеки. Логическими называются ошибки (*logic\_error*), которые в принципе можно выявить до начала исполнения программы или путем проверки аргументов функций и конструкторов. Ошибками времени выполнения (*runtime\_error*) являются все остальные ошибки. Многие люди считают представленную схему полезной основой для всех ошибок и исключений. Я так не считаю.

Классы исключений стандартной библиотеки не добавляют функций к набору, предоставляемому корневым классом *exception*; они лишь надлежащим образом определяют необходимые виртуальные функции. Так, мы можем написать:

```

void f()
try
{
    // используем стандартную библиотеку
}
catch (exception& e)
{
    cout<< "standard library exception" << e.what() << '\n' ;
}

```



```

catch ( . . . )
{
    cout << "other exception\n";
    // ...
}

```

Стандартные исключения являются производными от *exception*. Но это неверно для всех исключений вообще, так что будет ошибкой пытаться перехватить все исключения с помощью одного лишь обработчика *exception*. Также ошибкой будет полагать, что любое производное от *exception* исключение является исключением стандартной библиотеки: программисты могут добавлять свои собственные исключения к иерархии *exception*.

Обратите внимание на то, что операции классов иерархии *exception* сами не генерируют исключений. Из этого, в частности, следует, что генерация исключений стандартной библиотеки не порождает исключения *bad\_alloc*. Механизм обработки исключений резервирует некоторое количество памяти для своей работы (исключения хранятся, скорее всего, в стеке), так что в принципе можно написать код, который постепенно исчерпает всю память. Например, ниже приведена функция, которая проверяет, кто первый исчерпает всю память — функциональный вызов или механизм обработки исключений:

```

void perverted ()
{
    try
    {
        throw exception ();    // рекурсивная генерация исключения
    }
    catch (exception& e)
    {
        perverted ();          // рекурсивный вызов функции
        cout << e.what ();
    }
}

```

Цель применения здесь операции вывода состоит единственно в том, чтобы помешать компилятору использовать одну и ту же память под исключения с именем *e*.

## 14.11. Советы

1. Используйте исключения для обработки ошибок; §14.1, §14.5, §14.9.
2. Не используйте исключения там, где достаточно локальных управляющих структур; §14.1.
3. Для управления ресурсами используйте стратегию «получение ресурса есть инициализация»; §14.4.
4. Не каждая программа должна безопасно обрабатывать исключения; §14.4.3.
5. Для поддержки инвариантов используйте обработчики исключений и технику «получение ресурса есть инициализация»; §14.3.2.

6. Минимизируйте использование **try**-блоков. Вместо применения явного кода обработчиков используйте технику «получение ресурса есть инициализация»; §14.4.
7. Не в каждой функции нужно обрабатывать все ошибки; §14.9.
8. Для сигнализации об ошибке в конструкторе генерируйте исключение; §14.4.6.
9. Перед генерацией исключения в операции присваивания убедитесь, что операнды находятся в корректном состоянии; §14.4.6.2.
10. Избегайте генерации исключений из деструкторов; §14.4.7.
11. Заставляйте функцию **main()** перехватывать все исключения и сообщать о них; §14.7.
12. Отделяйте обычный код от кода обработки ошибок; §14.4.5, §14.5.
13. Перед генерацией исключения в конструкторе убедитесь, что все ранее затребованные конструктором ресурсы освобождены; §14.4.
14. Организуйте управление ресурсами иерархически; §14.4.
15. Для важных интерфейсов применяйте спецификацию исключений; §14.9.
16. Не забывайте про возможные утечки памяти, выделенной операцией **new**, и неосвобожденной в случае генерации исключений; §14.4.1, §14.4.2, §14.4.4.
17. Предполагайте, что каждое исключение, которое может быть сгенерировано в функции, будет сгенерировано; §14.6.
18. Не предполагайте, что любое исключение наследует от **exception**; §14.10.
19. Библиотеки не должны безусловно завершать выполнение программ — им следует генерировать исключения и оставлять принятие решения вызывающей функции; §14.1.
20. Библиотека не должна выдавать диагностические сообщения, рассчитанные на конечного пользователя. Им нужно генерировать исключение и оставить остальную работу вызывающей функции; §14.1.
21. Разрабатывайте стратегию обработки ошибок на ранних этапах проектирования; §14.9.

## 14.12. Упражнения

1. (\*2) Обобщите класс **STC** (§14.6.3.1) до шаблона, который использует стратегию «получение ресурса есть инициализация» для хранения и восстановления функций разных типов.
2. (\*3). Завершите класс **Ptr\_to\_T** из §11.11 в виде шаблона, который использует исключения для сигнализации об ошибках времени выполнения.
3. (\*3) Напишите функцию, которая выполняет поиск в бинарном дереве с узлами, содержащими **char\***. Если находится узел, содержащий **hello**, то вызов **find("hello")** возвращает указатель на этот узел. Для индикации того факта, что никакой узел не найден, используйте исключение.

4. (\*3) Определите класс **Int**, который ведет себя как встроенный тип **int**, только он еще генерирует исключения в случаях переполнения (или потери точности).
5. (\*2.5) Возьмите базовые операции открытия, закрытия, чтения и записи из стандартного интерфейса языка C на вашей системе и напишите эквивалентные функции на C++, которые вызывают функции языка C, но в случае возникновения ошибок генерируют исключения.
6. (\*2.5) Напишите полный шаблон **Vector** с исключениями типа **Range** и **Size**.
7. (\*1) Напишите цикл, который суммирует объекты типа **Vector** из предыдущего упражнения без проверки размера векторов. Почему это плохая идея?
8. (\*2.5) Рассмотрите использование класса **Exception** как корневого для всей иерархии исключений. Как это будет выглядеть? Как это нужно использовать? К чему это может привести? Какие недостатки могут вытекать из такого подхода?
9. (\*1) Имеется

```
int main () { /* ... */ }
```

Внесите сюда изменения, направленные на перехват всех исключений, при возникновении которых выдаются сообщения об ошибке и вызывается **abort()**. Подсказка: функция **call\_from\_C()** из §14.9 не полностью обрабатывает все случаи.

10. (\*2) Напишите класс или шаблон, подходящие для реализации обратных вызовов.
11. (\*2.5) Напишите класс **Lock** (блокировка) для некоторой системы с параллельным выполнением.



# Иерархии классов

*Абстракция — это выборочное невежество.*  
— Эндрю Кениг

Множественное наследование — разрешение неоднозначности — наследование и *using-объявление* — повторяющиеся базовые классы — виртуальные базовые классы — использование множественного наследования — управление доступом — защищенные члены — доступ к базовым классам — механизм RTTI — операция *dynamic\_cast* — статическое и динамическое приведения типов — приведение из виртуального базового класса — операция *typeid* — расширенная информация о типе — правильное и неправильное применение RTTI — указатели на члены классов — свободная память — «виртуальные конструкторы» — советы — упражнения.

## 15.1. Введение и обзор

В этой главе обсуждается, как производные классы и виртуальные функции взаимодействуют с другими средствами языка, такими как контроль доступа, поиск имен, управление свободной памятью, конструкторы, указатели и преобразования типов. Глава состоит из пяти разделов:

- §15.2 Множественное наследование.
- §15.3 Контроль доступа.
- §15.4 Механизм RTTI (Run-Time Type Information).
- §15.5 Указатели на члены классов.
- §15.6 Свободная память.

В общем случае, класс создается из некоторой структуры базовых классов. Ввиду того, что исторически такая структура практически всегда была древовидной, ее называют *классовой иерархией* (*class hierarchy*). Мы пытаемся проектировать классы таким образом, чтобы у пользователей не было необходимости интересоваться, каким именно образом классы сформированы. В частности, механизм виртуальных функ-

ций гарантирует, что когда мы вызываем виртуальную функцию  $f()$  для некоторого объекта, вызывается всегда одна и та же функция в независимости от того, какой именно класс в иерархии объявил ее. В данной главе наибольшее внимание уделяется способам композиции классовых иерархий и контролю за доступом к разным частям классов, а также средствам навигации по классовым иерархиям как на этапе компиляции, так и во время выполнения программы.

## 15.2. Множественное наследование

Как показано в §2.5.4 и §12.3, класс может иметь более одного непосредственно базового класса, то есть два и более классов могут указываться после двоеточия в объявлении класса. Рассмотрим задачу моделирования, в которой параллельно выполняющиеся задания представлены классом *Task*, а сбор данных и визуализация представлены классом *Displayed*. Далее мы можем определить класс моделируемых сущностей, например класс *Satellite* (спутник):

```
class Satellite: public Task, public Displayed
{
    // ...
};
```

Использование более одного непосредственного базового класса принято называть *множественным наследованием (multiple inheritance)*. В противоположность этому наличие единственного непосредственного базового класса называют *одиночным наследованием (single inheritance)*.

Кроме операций, определенных специально для *Satellite*, мы можем использовать объединение операций классов *Task* и *Displayed*. Например:

```
void f(Satellite& s)
{
    s.draw();           // Displayed::draw()
    s.delay(10);        // Task::delay()
    s.transmit();       // Satellite::transmit()
}
```

Аналогичным образом, функциям можно передавать *Satellite* вместо ожидаемых ими *Task* или *Displayed*. Например:

```
void highlight(Displayed*);
void suspend(Task*);

void g(Satellite* p)
{
    highlight(p);       // передать указатель на Displayed-часть Satellite
    suspend(p);         // передать указатель на Task-часть Satellite
}
```

Реализация этого механизма предполагает несложные приемы от компилятора, которые обеспечивают видимость разных частей *Satellite* в функциях, ожидающих *Task* или *Displayed*, соответственно.

Виртуальные функции работают как обычно. Например:

```
class Task
{
    // ...
    virtual void pending () = 0;
};

class Displayed
{
    // ...
    virtual void draw () = 0;
};

class Satellite : public Task, public Displayed
{
    // ...
    void pending () ;    // замещаем Task::pending()
    void draw () ;      // замещаем Displayed::draw()
};
```

Это гарантирует, что функции **Satellite::draw()** и **Satellite::pending()** будут вызваны для **Satellite**, проинтерпретированного как **Displayed** и **Task**, соответственно.

В случае одиночного наследования у программиста будет более ограниченный выбор для реализации классов **Displayed**, **Task** и **Satellite**. **Satellite** в таком случае может быть либо **Task**, либо **Displayed**, но не обоими сразу (если только **Task** не был реализован как производный класс от **Displayed**, или наоборот). Выбор любой из этих альтернатив означает потерю гибкости.

А кому вообще может потребоваться такой вот класс **Satellite**? На самом деле, это вполне реальный пример (даже если кому-то он показался абсолютно искусственным). Существовала (а может и сейчас еще существует) программа, построенная по схеме, примененной здесь для иллюстрации множественного наследования. Она использовалась для исследования построения коммуникационных систем, включающих спутники (*satellites*), наземные станции и т.д. В рамках этой модели можно отвечать на вопросы об интенсивности трафика, находить правильную реакцию на ситуацию выведения из строя наземной станции (например, из-за торнадо), отыскивать оптимальное соотношение между нагрузкой на спутниковую связь и связь по Земле и т.д. Такое интенсивное моделирование предполагает множество операций вывода информации и множество отладочных операций. Также, нам нужно хранить состояние объектов **Satellite** и их подкомпонентов для анализа, отладки и восстановления после ошибок.

### 15.2.1. Разрешение неоднозначности

Два базовых класса могут иметь функции-члены с совпадающими именами. Например:

```
class Task
{
    // ...
    virtual debug_info* get_debug () ;
    ,
```

```
class Displayed
{
    // ...
    virtual debug_info* get_debug () ;
};
```

Для объектов типа *Satellite* эти неоднозначности должны устраняться:

```
void f (Satellite* sp)
{
    debug_info* dip=sp->get_debug () ;    // error: неоднозначность
    dip = sp->Task::get_debug () ;        // ok
    dip = sp->Displayed::get_debug () ;    // ok
}
```

Однако явное устранение неоднозначностей довольно неуклюже, так что обычно лучше определить новую функцию в производном классе:

```
class Satellite : public Task, public Displayed
{
    // ...
    debug_info* get_debug ()    // замещаем Task::get_debug() и Displayed::get_debug()
    {
        debug_info* dip1 = Task::get_debug () ;
        debug_info* dip2 = Displayed::get_debug () ;
        return dip1->merge (dip2) ;
    }
}
```

Это локализует информацию о базовых классах для *Satellite*. Так как *Satellite::get\_debug()* замещает функции *get\_debug()* для обоих базовых классов, *Satellite::get\_debug()* вызывается при каждом вызове для объекта *Satellite*.

Квалифицированное имя *Telstar::draw()* может ссылаться либо на функцию *draw()*, объявленную в *Telstar*, либо в одном из базовых классов. Например:

```
class Telstar : public Satellite
{
    // ...
    void draw ()
    {
        draw () ;                // oops!: рекурсивный вызов
        Satellite::draw () ;      // находим Displayed::draw
        Displayed::draw () ;
        Satellite::Displayed::draw () ;    // избыточная двойная квалификация
    }
}
```

Другими словами, если *Satellite::draw()* не разрешается в одноименную функцию класса *Satellite*, то компилятор рекурсивно осуществляет поиск в базовых классах; то есть ищет *Task::draw()* и *Displayed::draw()*. Если находится единственное соответствие, то оно и используется. В противном случае, *Satellite::draw()* либо не найдена, либо неоднозначна.



### 15.2.2. Наследование и using-объявление

Разрешение перегрузки не пересекает границ областей видимости различных классов (§7.4). В частности, неоднозначности между функциями из разных базовых классов не разрешаются на основе типов аргументов.

При комбинировании существенно разных классов, таких как *Task* и *Displayed* в примере с классом *Satellite*, сходство в названиях функций не означает сходства в их назначении. Такого рода конфликты имен часто становятся полным сюрпризом для программистов. Например:

```
class Task
{
    // ...
    void debug (double p) ;
};

class Displayed
{
    // ...
    void debug (int v) ;
};

class Satellite: public Task, public Displayed
{
    // ...
};

void g (Satellite* p)
{
    p->debug (1) ;    // error: неоднозначно - Displayed::debug(int) или Task::debug(double)?

    p->Task::debug (1) ;    // ok
    p->Displayed::debug (1) ;    // ok
}
```

А что, если совпадение имен функций в разных базовых классах было продуманным проектным решением, но пользователь хочет различать их по типам аргументов? В этом случае *объявления using* (§8.2.2) могут ввести обе функции в общую область видимости. Например:

```
class A
{
public:
    int f(int) ;
    char f(char) ;
    // ...
};

class B
{
public:
    double f(double) ;
    ..
```

```

class AB: public A, public B
{
public:
    using A::f;
    using B::f;
    char f(char) ;           // скрывает A::f(char)
    AB f(AB) ;
};

void g (AB& ab)
{
    ab.f(1) ;                // A::f(int)
    ab.f('a') ;              // AB::f(char)
    ab.f(2.0) ;              // B::f(double)
    ab.f(ab) ;               // AB::f(AB)
}

```

Объявления **using** позволяют программисту сформировать набор перегруженных функций из базовых классов и производного класса. Функции, объявленные в производном классе, скрывают в противном случае доступные функции из базовых классов. Виртуальные функции базовых классов могут замещаться как обычно (§15.2.3.1).

Объявление **using** (§8.2.2) в определении класса должно ссылаться на члены базового класса. Вне классов объявление **using** не может ссылаться на члены класса, на его производные классы и их члены. Директивы **using** (§8.2.3) нельзя помещать в определение класса и их нельзя использовать для классов.

Невозможно использовать объявления **using** для получения доступа к дополнительной информации. Это лишь способ упростить доступ к имеющейся информации (§15.3.2.2).

### 15.2.3. Повторяющиеся базовые классы

При наличии нескольких базовых классов становится возможной ситуация, когда какой-то класс дважды окажется базовым для другого класса. Например, если бы каждый из классов **Task** и **Displayed** был бы производным от класса **Link**, то класс **Satellite** получал бы бы класс **Link** в качестве базового дважды:

```

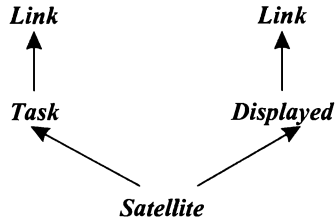
struct Link
{
    Link* next;
};

class Task:public Link
{
    // Link используется для списка задач типа Task (список планировщика)
    // ...
};

class Displayed: public Link
{
    // Link используется для списка отображаемых (Displayed) объектов (список показа)
    //

```

В принципе, это проблем не вызывает. Просто для представления связей используются два объекта типа **Link**, которые не мешают друг другу. В то же время, обращаясь к членам класса **Link**, вы рискуете нарваться на неоднозначности (§15.2.3.1). Объект класса **Satellite** можно изобразить графически следующим образом:



В случаях, когда базовый класс не должен быть представлен двумя отдельными объектами, нужно использовать так называемые виртуальные базовые классы (§15.2.4).

Обычно, реплицируемый базовый класс (такой как **Link** в нашем примере) должен быть деталью реализации и его не следует использовать вне непосредственно производных от него классов. Если к такому базовому классу нужен доступ из места, где видны его множественные копии, нужно явно квалифицировать ссылки во избежание неоднозначностей. Например:

```

void mess_with_links (Satellite* p)
{
    p->next = 0;           // error: неоднозначность (какой Link?)
    p->Link::next = 0;     // error: неоднозначность (какой Link?)

    p->Task::next = 0;     // ok
    p->Displayed::next = 0; // ok
    // ...
}
  
```

Это в точности тот же механизм, что используется для разрешения неоднозначностей при доступе к членам (§15.2.1).

### 15.2.3.1. Замещение

Виртуальная функция реплицируемого базового класса может замещаться (единственной) функцией производного класса. Например, можно следующим образом обеспечить объекту возможность читать свое состояние из файла и записывать его в файл:

```

class Storable
{
public:
    virtual const char* get_file () = 0;
    virtual void read () = 0;
    virtual void write () = 0;
    virtual ~Storable () {}
};
  
```

Естественно, разрабатывая свои собственные специализированные системы, разные программисты могут воспользоваться этим определением для проектирова-

ния классов, которые могут использоваться независимо друг от друга или в некоторой комбинации. Например, в задачах моделирования при остановке работы важно сохранять состояние объектов моделирования с последующим их восстановлением при рестарте. Это можно было бы реализовать следующим образом:

```
class Transmitter : public Storable
{
public:
    void write () ;
    // ...
};

class Receiver : public Storable
{
public:
    void write () ;
    // ..
};

class Radio : public Transmitter, public Receiver
{
public:
    const char* get_file () ;
    void read () ;
    void write () ;
    // ...
};
```

Часто замещающая функция вызывает соответствующую версию базового класса, после чего выполняет специфическую именно для производного класса работу:

```
void Radio::write ()
{
    Transmitter::write () ;
    Receive::write () ;
    // далее пишем информацию, специфичную для Radio
}
```

Преобразование от типа реплицируемого базового класса к производному классу обсуждается в §15.4.2. Техника замещения каждой из функций *write* () отдельными функциями производных классов обсуждается в §25.6.

#### 15.2.4. Виртуальные базовые классы

Пример с классом *Radio* из предыдущего раздела работает потому, что класс *Storable* реплицируется вполне безопасно, удобно и эффективно. Однако это часто не свойственно классам, служащим строительными блоками для других классов. Например, мы могли бы определить класс *Storable* так, чтобы он содержал имя файла, используемого для хранения объектов:

```
class Storable
{
public:
    Storable (const char* s) ;
```

```

virtual void read () = 0;
virtual void write () = 0;
virtual ~Storable () { write () ; }

private:
    const char* store;

    Storable (const Storable& ) ;
    Storable& operator= (const Storable& ) ;
};

```

Несмотря на внешне незначительное изменение класса *Storable*, мы должны изменить дизайн класса *Radio*. Все части объекта должны совместно использовать единственную копию *Storable*; в противном случае становится неоправданно сложно избегать хранения множественных копий объекта. Одним из механизмов, обеспечивающих такое совместное использование, является механизм *виртуального базового класса* (*virtual base class*). Каждый виртуальный базовый класс вносит в объект производного класса единственный (разделяемый) подобъект. Например:

```

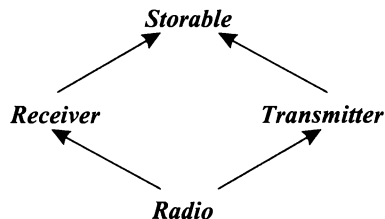
class Transmitter : public virtual Storable
{
public:
    void write () ;
    // ...
};

class Receiver : public virtual Storable
{
public:
    void write () ;
    // ...
};

class Radio : public Transmitter, public Receiver
{
public:
    void write () ;
    // ...
};

```

Или графически:



Сравните эту диаграмму с графическим представлением объекта класса *Satellite* в §15.2.3, чтобы увидеть разницу между обычным и виртуальным наследованием. Любой заданный виртуальный базовый класс в иерархии наследования всегда пред-

ставим единственной копией своего объекта. В противоположность этому, *невиртуальный* базовый класс иерархии представляется своим собственным отдельным подобъектом.

#### 15.2.4.1. Программирование виртуальных базовых классов

Определяя функции класса с виртуальным базовым классом, программист, в общем случае, не может знать, используют ли другие классы тот же самый виртуальный базовый класс. Это приводит к затруднениям, когда в рамках конкретной задачи требуется обеспечить единственный вызов функций базового класса. Например, язык гарантирует, что конструктор виртуального базового класса вызывается (явно или неявно) ровно один раз — из конструктора полного объекта (то есть из конструктора «наиболее производного класса»). Например:

```
class A                                // нет конструктора
{
    // ...
};

class B
{
public:
    B();                                // конструктор по умолчанию
    // ...
};

class C
{
public:
    C(int);                             // нет умолчательного конструктора
};

class D: virtual public A, virtual public B, virtual public C
{
    D() { /* ... */ }                  // error: для C нет умолчательного конструктора
    D(int i) : C(i) { /* ... */ };    // ok
    // ...
};
```

Конструктор виртуального базового класса выполняется до того, как начинают работать конструкторы производных классов.

Там, где это требуется, программист может имитировать такую схему работы, вызывая функции виртуального базового класса лишь из «наиболее производного класса». Например, пусть имеется класс **Window**, знающий, как нужно отрисовывать содержимое окна:

```
class Window
{
    // ...
    virtual void draw();
};
```

Пусть также имеются классы, умеющие изменять внешний вид окна и добавлять к нему вспомогательные интерфейсные средства:

```
class Window_with_border : public virtual Window
{
    // средства для работы с рамкой
    void own_draw() ;           // отобразить рамку
    void draw() ;
};
```

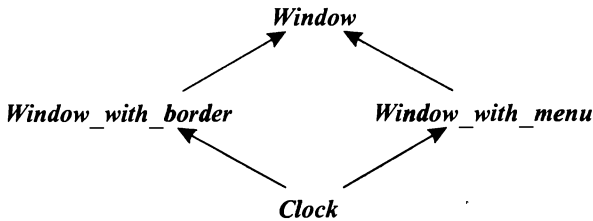
```
class Window_with_menu : public virtual Window
{
    // средства для работы с меню
    void own_draw() ;           // отобразить меню
    void draw() ;
};
```

Функции **own\_draw()** не обязаны быть виртуальными, так как из вызовов предполагается выполнять из виртуальных функций **draw()**, «знающих» тип объекта, для которых они вызваны.

Далее, мы можем создать вполне правдоподобный «хронометрический» класс **Clock**:

```
class Clock : public Window_with_border, public Window_with_menu
{
    // средства для работы с часами
    void own_draw() ;           // отобразить циферблат и стрелки
    void draw() ;
};
```

или в графическом виде:



Функции **draw()** теперь можно написать с использованием функций **own\_draw()** так, что при вызове любых функций отрисовки функция **Window::draw()** будет вызываться ровно один раз. И это достигается независимо от конкретного типа окна, для которого **draw()** вызывается:

```
void Window_with_border::draw()
{
    Window::draw() ;
    own_draw() ;           // отобразить рамку
}
```

```
void Window_with_menu::draw()
{
    Window::draw() ;
    own_draw() ;           // отобразить меню
}
```

```
void Clock::draw()
{
    Window::draw();
    Window_with_border::own_draw();
    Window_with_menu::own_draw();
    own_draw();           // отобразить циферблат и стрелки
}
```

Приведение типа от виртуального базового класса к производному обсуждается в §15.4.2.

### 15.2.5. Применение множественного наследования

Простейшим и наиболее очевидным применением множественного наследования является «склейка» двух несвязанных классов в качестве получения строительного блока для третьего класса. Класс *Satellite*, который мы в §15.2 строили на основе классов *Task* и *Displayed*, иллюстрирует этот случай. Такой вариант применения множественного наследования прост, эффективен и важен, но не очень интересен. В своей основе он позволяет программисту сэкономить на написании большого числа функций, переадресующих вызовы друг другу. Эта техника в целом не сильно влияет на дизайн программной системы, но может вступать в конфликт с необходимостью сокрытия деталей реализации. По счастью, чтобы технике программирования быть полезной, ей не обязательно быть слишком умной.

Применение множественного наследования для реализации абстрактных классов более фундаментально в том смысле, что влияет на проектный дизайн программы. Класс *BB\_ival\_slider* (§12.4.3) служит соответствующим примером:

```
class BB_ival_slider:
    public Ival_slider,           // интерфейс
    protected BBslider          // реализация
{
    // реализация функций Ival_slider и BBslider средствами BBslider
};
```

В этом примере два базовых класса играют различные с логической точки зрения роли. Один базовый класс формирует открытый абстрактный интерфейс, а другой является защищенным (protected) конкретным классом, предоставляющим «детали реализации». Эти роли отражаются и в стилистике классов, и в их режимах наследования (public или protected). Множественное наследование весьма существенно в данном примере, поскольку производный класс вынужден замещать виртуальные функции как «интерфейсного класса», так и «класса реализации».

Множественное наследование позволяет дочерним классам одного уровня иерархии (sibling classes) совместно использовать информацию без необходимости введения зависимости от единственного общего базового класса всей программы. Это как раз тот случай, который называют *ромбовидным наследованием* (diamond-shaped inheritance) (см. класс *Radio* в §15.2.4 и класс *Clock* в §15.2.4.1). Если при этом базовый класс нельзя реплицировать, его нужно использовать в иерархии как виртуальный базовый класс.

Я считаю ромбовидное наследование приемлемым в тех случаях, когда либо используется виртуальный базовый класс, либо когда непосредственно наследуемые

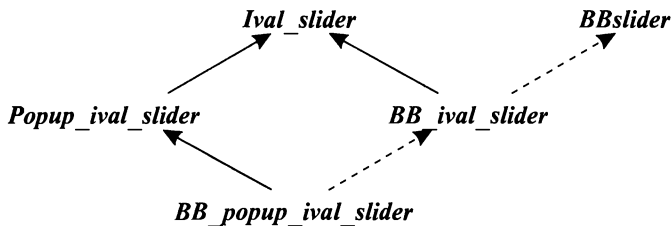


от него классы абстрактные. Рассмотрим снова классы семейства *Ival\_box* из §12.4, где в конце концов я сделал классы этого семейства абстрактными, дабы отразить их чисто интерфейсную роль. Это позволило мне отнести все детали реализации в конкретные классы. Кроме того, совместное использование деталей реализации распространялось лишь на классическую иерархию классов оконной системы, в рамках которой реализация и выполнялась.

Было бы неплохо, если бы класс, реализующий более специализированный интерфейс *Popup\_ival\_slider*, разделял большую часть реализации с классом, реализующим более общий интерфейс *Ival\_slider*. В конце концов, эти реализационные классы могли бы совместно использовать практически все, за исключением деталей взаимодействия с пользователем программы. В этом случае, однако, следует избегать реплицирования подбъектов *Ival\_slider* в объектах специализированных элементов управления (ползунков). Для этого *Ival\_slider* следует использовать как виртуальный базовый класс:

```
class BB_ival_slider : public virtual Ival_slider, protected BBslider { /* ... */ };
class Popup_ival_slider : public virtual Ival_slider { /* ... */ };
class BB_popup_ival_slider :
    public virtual Popup_ival_slider, protected BB_ival_slider { /* ... */ };
```

или графически:

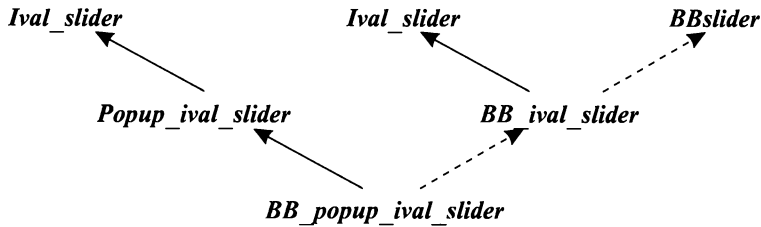


Можно легко себе представить дальнейшие интерфейсы, наследующие от *Popup\_ival\_slider*, и дальнейшие реализационные классы, наследующие от этих интерфейсов и *BB\_popup\_ival\_slider*.

Если довести эту идею до логического конца, то все интерфейсы в системе должны наследоваться от абстрактных классов в виртуальном режиме. Такой подход выглядит наиболее логичным, общим и гибким. Но я не следую этому подходу отчасти по исторической причине, а отчасти из-за того, что наиболее распространенные практические приемы реализации концепции виртуальных базовых классов требуют избыточных затрат памяти и времени выполнения, а это делает их тотальное применение менее привлекательным. Если такие избыточные накладные расходы неприемлемы, то тогда следует обратить внимание на то, что подбъекты класса *Ival\_slider* хранят лишь указатель на виртуальную таблицу и, как отмечалось в §15.2.4, такие абстрактные классы, не имеющие полей данных, могут реплицироваться без отрицательных последствий. Итак, мы отказываемся от виртуального наследования в пользу обычного наследования:

```
class BB_ival_slider : public Ival_slider, protected BBslider { /* ... */ };
class Popup_ival_slider : public Ival_slider { /* ... */ };
class BB_popup_ival_slider :
    public Popup_ival_slider, protected BB_ival_slider { /* ... */ };
```

или графически:



Это можно рассматривать как жизнеспособную оптимизацию более понятной альтернативы, представленной ранее. Некоторой потенциальной проблемой при этом является невозможность неявного приведения от *BB\_popup\_ival\_slider* к *Ival\_slider*.

#### 15.2.5.1. Замещение функций виртуальных базовых классов

В производных классах можно замещать виртуальные функции прямых или косвенных виртуальных базовых классов. В частности, два различных класса могут заместить разные виртуальные функции виртуального базового класса. Таким образом, несколько производных классов могут предоставить реализации интерфейса из виртуального базового класса. Пусть, например, класс *Window* формулирует интерфейс в виде виртуальных функций *set\_color()* и *prompt()*. В этом случае класс *Window with border* может заместить *set\_color()* для управления цветовой схемой, а класс *Window with menu* — заместить *prompt()* для организации интерактивного взаимодействия с пользователем:

```

class Window
{
    // ...
    virtual set_color (Color) = 0;    // задать цвет фона
    virtual void prompt () = 0;
};

class Window_with_border : public virtual Window
{
    // ...
    void set_color (Color);           // управление цветом фона
};

class Window_with_menu : public virtual Window
{
    // ...
    void prompt ();                   // управление взаимодействием с пользователем
};

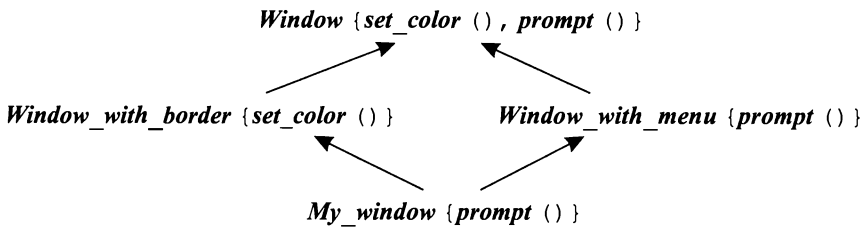
class My_window : public Window_with_menu, public Window_with_border
{
    // ...
};
  
```

А что если разные производные классы заместят одну и ту же функцию? Это допускается только в случаях, когда один из таких классов наследует от всех остальных.

ных классов этой группы. То есть одна функция должна замещать все остальные. Например, класс *My\_window* мог бы заместить *prompt()* для улучшения возможностей, предоставляемых классом *Window\_with\_menu*:

```
class My_window : public Window_with_menu, public Window_with_border
{
    // ...
    void prompt () ;    // не оставляем базов. классу вопросы взаимодействия с пользователем
};
```

или графически:



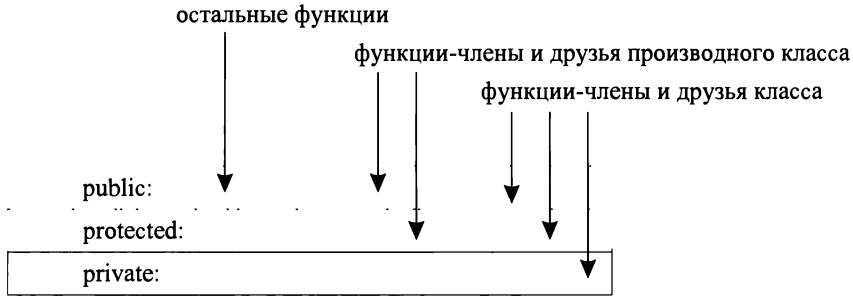
Если в двух классах замещается виртуальная функция базового класса, но при этом ни один из классов не замещает соответствующую функцию из другого, то такая классовая иерархия ошибочна: невозможно создать корректную виртуальную таблицу, ибо вызов виртуальной функции через объект «наиболее производного класса» неоднозначен. Например, если бы класс *Radio* из §15.2.4 не замещал функцию *write()*, то объявления *write()* в *Receiver* и *Transmitter* приводили бы к ошибке в определении класса *Radio*. Такой конфликт разрешается так, как показано в этом примере — путем замещения функции в «наиболее производном классе».

## 15.3. Контроль доступа

Член класса может быть закрытым (*private*), защищенным (*protected*) или открытым (*public*):

- Если он *закрытый*, то его имя могут использовать лишь функции-члены или друзья того же самого класса.
- Если он *защищенный*, то его имя могут использовать функции-члены и друзья того же самого класса, а также функции-члены и друзья непосредственных производных классов (см. §11.5).
- Если он *открытый*, то его имя может использовать любая функция.

Данная классификация отражает точку зрения, что в отношении доступа к классу существует три вида функций: реализующие класс (друзья и члены), реализующие производный класс (друзья и члены) и остальные функции. Это можно изобразить графически следующим образом:



Контроль доступа применяется ко всем именам одинаково. То, к чему относится имя, не влияет на контроль доступа к этому имени. Это означает, что закрытыми могут быть функции-члены, типы, константы и т.д., а также поля данных. Например, классы эффективных неинтрузивных (non-intrusive — ненавязчивых) списков (§16.2.1) часто нуждаются в специальных структурах данных для учета своих элементов. Такую информацию лучше делать закрытой:

```

template<class T> class List
{
private:
    struct Link { T val; Link* next; };

    struct Chunk
    {
        enum { chunk_size = 15 };
        Link v[chunk_size];
        Chunk* next;
    };

    Chunk* allocated;
    Link* free;
    Link* get_free();
    Link* head;

public:
    class Underflow { }; // класс исключения

    void insert(T);
    T get();
    // ...
};

template<class T> void List<T>::insert(T val)
{
    Link* lnk = get_free();
    lnk->val = val;
    lnk->next = head;
    head = lnk;
}
  
```

```

template<class T> List<T>::Link* List<T>::get_free()
{
    if (free == 0)
    {
        // выделить новый блок памяти и включить связи в список free
    }

    Link* p = free;
    free = free->next;
    return p;
}

template<class T> T List<T>::get()
{
    if (head == 0) throw Underflow();

    Link* p = head;
    head = p->next;
    p->next = free;
    free = p;
    return p->val;
}

```

В определениях функций-членов область видимости шаблона *List<T>* вводится с помощью префикса *List<T>::*, но поскольку возврат функции *get\_free()* указывается раньше, то его нужно обозначать с полной квалификацией, то есть *List<T>::Link*, а не просто *Link*.

А глобальные функции (за исключением друзей) такого доступа не имеют:

```

void would_be_meddler(List<T>* p)
{
    List<T>::Link* q = 0;           // error: List<T>::Link - закрыто
    q = p->free;                     // error: List<T>::free - закрыто
    // ...
    if (List<T>::Chunk::chunk_size > 31) // error: List<T>::Chunk::chunk_size - закрыто
    {
        // ...
    }
    // ...
}

```

В определении класса (когда используется ключевое слово *class*) по умолчанию все члены закрытые, а в определении структуры (когда используется ключевое слово *struct*) все члены по умолчанию открытые (§10.2.8).

### 15.3.1. Защищенные члены классов

Рассмотрим пример с иерархией классов *Window* из §15.2.4.1. Функции *own\_draw()* были задуманы как строительные блоки для применения в производных классах, но их использование в остальных случаях небезопасно. А функции *draw()*, наоборот, годятся к использованию в любых контекстах. Это различие можно точно выразить разделением общего интерфейса класса *Window* на защищенную (*protected*) и открытую (*public*) части:

```

class Window_with_border
{
public:
    virtual void draw () ;
    // ...

protected:
    void own_draw () ;
    // ...

private:
    // представление и т.п.
};

```

Прямой доступ к защищенным членам базового класса разрешен в производном классе лишь для объектов этого же класса:

```

class Buffer
{
protected:
    char a [128] ;
    // ...
};

class Linked_buffer : public Buffer { /* ... */ };

class Cyclic_buffer : public Buffer
{
    // ...
    void f (Linked_buffer* p)
    {
        a [0] =0;           // ok: доступ к собственному защищенному члену
        p->a [0] =0;         // error: доступ к защищенному члену другого типа
    }
};

```

Это предохраняет от довольно тонких ошибок, связанных с возможностью одного производного класса портить данные другого производного класса.

#### 15.3.1.1. Применение защищенных членов класса

Простая модель «закрытый/открытый» для сокрытия данных хорошо работает в случае конкретных классов (§10.3). Для иерархий же классов существует два вида пользователей классов: производные классы и «простая публика». От имени этих пользователей и производят свои действия над классовыми объектами функции-члены класса и друзья. Модель «закрытый/открытый» позволяет программисту различать разработчиков классов и обычных пользователей, но она не предоставляет ничего особого в обслуживании производных классов.

Защищенные члены классов более подвержены неправильному использованию, чем закрытые члены. Часто их наличие свидетельствует об ошибках проектирования. Помещение в защищенные секции общих классов заметного объема данных, доступных производным классам, открывает возможность для порчи этих данных. Еще того хуже, защищенные данные, как и открытые, намного труднее реструктурировать ввиду сложности нахождения всех случаев их использования. А это усложняет процесс сопровождения программных систем.

По счастью, использовать защищенные данные необязательно; наиболее предпочтительным вариантом являются закрытые данные, что и имеет место по умолчанию. По моему опыту, всегда имеются альтернативы помещению изрядного количества информации в общий класс для ее непосредственного использования в производных классах.

Обратите внимание, что все это не имеет отношения к защищенным функциям — они прекрасно задают операции для применения их в производных классах. Иллюстрацией служит класс *Ival\_slider* из §12.4.2. Если бы в этом примере функции класса были бы закрытыми, стало бы невозможно создавать дальнейшие производные классы.

Технические примеры доступа к защищенным членам приведены в §C.11.1.

### 15.3.2. Доступ к базовым классам

Аналогично членам класса, можно сам базовый класс объявить в процессе наследования закрытым (*private*), защищенным (*protected*) или открытым (*public*). Например:

```
class X: public B { /* ... */ };
class Y: protected B { /* ... */ };
class Z: private B { /* ... */ };
```

Открытое наследование делает производный класс подтипом базового; это наиболее распространенная форма наследования. Защищенную и закрытую формы наследования применяют для использования деталей реализации. Защищенное наследование полезно в случае классовых иерархий, в которых построение дальнейших производных классов является нормой; класс *Ival\_slider* из §12.4.2 может служить хорошим примером. Закрытое наследование применяется для построения производных классов, закрывающих доступ к интерфейсу базового класса и обеспечивающих большие гарантии, чем базовый класс. Например, шаблон *Vector<T\*>* добавляет проверку типа к своему базовому классу *Vector<void\*>* (§13.5). Также, если бы нам потребовалась гарантия того, что всегда доступ к векторам проверяется (см. §3.7.2), то базовый класс для *Vec* пришлось бы объявить закрытым (чтобы предотвратить преобразование из *Vec* в непроверяемый базовый класс *vector*):

```
template<class T> class Vec: private vector<T> { /* ... */ }; // вектор с проверкой диапазона
```

Если в определении класса опустить спецификатор режима наследования, то для ключевого слова *class* по умолчанию режим наследования будет считаться закрытым (*private*), а для ключевого слова *struct* он будет считаться открытым (*public*). Например:

```
class XX: B { /* ... */ }; // закрытое наследование от B
struct YY: B { /* ... */ }; // открытое наследование от B
```

Чтобы код был более читаемым, лучше явно использовать спецификатор для режима наследования.

Этот спецификатор управляет доступом к членам базового класса и преобразованием указателей и ссылок из типа производного класса к типу базового класса. Рассмотрим класс *D*, производный от базового класса *B*:

- Если ***B*** является *закрытым* базовым классом, его открытые и защищенные члены могут использоваться только функциями-членами и друзьями ***D***. Только функции-члены и друзья класса ***D*** могут преобразовывать ***D\**** в ***B\****.
- Если ***B*** является *защищенным* базовым классом, его открытые и защищенные члены могут использоваться только функциями-членами и друзьями ***D***, а также функциями-членами и друзьями классов, производных от ***D***. Только функции-члены и друзья класса ***D***, а также функции-члены и друзья классов, производных от ***D***, могут преобразовывать ***D\**** в ***B\****.
- Если ***B*** является *открытым* базовым классом, его открытые члены могут использоваться любой функцией. Кроме того, его защищенные члены могут использоваться функциями-членами и друзьями ***D***, а также функциями-членами и друзьями классов, производных от ***D***. Любая функция может преобразовывать ***D\**** в ***B\****.

Это, в основном, является переформулировкой правил для доступа к членам (§15.3). Мы выбираем режим наследования (режим доступа к базовым классам) по тем же соображениям, что и режим доступа к членам. Например, я предпочел сделать ***BBwindow*** защищенным базовым классом для ***Ival\_slider*** (§12.4.2), потому что ***BBwindow*** служит частью реализации ***Ival\_slider***, а не частью его интерфейса. Я, однако, не стал полностью закрывать ***BBwindow*** (то есть наследовать в режиме *private*), так как хотел оставить возможность получать классы, производные от ***Ival\_slider***, которым тоже требуется доступ к реализации.

Технические примеры доступа к базовым классам приведены в §C.11.2.

### 15.3.2.1. Множественное наследование и контроль доступа

Если некоторое имя или базовый класс в принципе доступны по нескольким путям в рамках иерархии множественного наследования, то они достижимы лишь в случае, когда достижимы по любому из этих путей. Например:

```
struct B
{
    int m;
    static int sm;
    // ...
};

class D1 : public virtual B { /* ... */ };
class D2 : public virtual B { /* ... */ };
class DD : public D1, private D2 { /* ... */ };

DD* pd = new DD;
B* pb = pd;           // ok: доступ через D1
int il = pd->m;        // ok: доступ через D1
```

Если некоторая единичная сущность доступна по нескольким путям, мы все равно можем обращаться к ней без возникновения неоднозначностей. Например:

```
class X1 : public B { /* ... */ };
class X2 : public B { /* ... */ };
class XX : public X1, public X2 { /* ... */ };

XX* pxx = new XX;
```



```
int i1 = pxx->m;      // error, неоднозначность: XX::X1::B::m или XX::X2::B::m
int i2 = pxx->sm;     // ok: единственный B::sm в XX
```

### 15.3.2.2. Множественное наследование и контроль доступа

Объявление **using** не может использоваться для получения доступа к дополнительной информации. Оно просто делает доступную информацию более удобной для применения. С другой стороны, если доступ к информации имеется, его можно дать и другим пользователям. Например:

```
class B
{
private:
    int a;

protected:
    int b;

public:
    int c;
};

class D: public B
{
public:
    using B::a;      // error: B::a - закрыто
    using B::b;      // делает B::b общедоступным через D
};
```

Комбинируя объявления **using** с закрытым или защищенным режимами наследования, можно явно открывать доступ к некоторому подмножеству членов класса. Например:

```
class BB : private B    // даем доступ к B::b и B::c, но не к B::a
{
    using B::b;
    using B::c;
};
```

См. также §15.2.2.

## 15.4. Механизм RTTI (Run-Time Type Information)

Вероятным использованием семейства классов **Ival\_box**, определенных в §12.4, будет передача их системе, управляющей экраном, чтобы та вернула объекты этого типа в момент наступления некоторых событий. Так работает большинство систем графических пользовательских интерфейсов. Но эти системы ничего не знают о типах семейства **Ival\_box**, так как задаются в терминах своих собственных классов и объектов, а не классов нашего приложения. В результате возникает неприятный эффект потери типа объектов, которые мы передаем системе, а потом получаем их от нее назад.

Восстановление потерянного типа объекта может заключаться в возможности как-то спросить объект о его типе. Для работы с объектами мы обычно располагаем

подходящего типа указателями или ссылками на них. Поэтому наиболее очевидной и полезной операцией над объектами на этапе выполнения программы будет операция преобразования к ожидаемому типу, которая возвращает корректное значение указателя только если такое преобразование выполнимо (или нулевой указатель в противном случае). Операция *dynamic\_cast* как раз и предназначена для этого. Например, пусть система вызывает функцию *my\_event\_handler()* с указателем на объект (окно) типа *BBwindow*, в котором произошла какая-то активность. Затем я мог бы реагировать на событие вызовом функции *do\_something()* семейства классов *Ival\_box*:

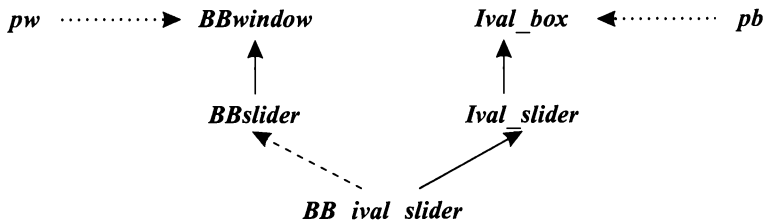
```
void my_event_handler (BBwindow* pw)
{
    if (Ival_box* pb = dynamic_cast<Ival_box*> (pw) ) // указывает pw на Ival_box?
        pb->do_something() ;
    else
    {
        // Oops! неожиданное событие
    }
}
```

Происходящее можно условно объяснить так, что *dynamic\_cast* переводит с языка реализации системы графического интерфейса пользователя на язык нашего приложения. Очень важно отметить, что в примере отсутствует точный тип объекта. Объект лишь должен иметь тип одного из классов семейства *Ival\_box*, скажем *Ival\_slider*, реализованный одним из видов *BBwindow*, скажем *BBslider*. Нет никакой необходимости в знании точного типа объекта в этом взаимодействии между «системой» и приложением. Любой интерфейс существует лишь для открытия важных аспектов взаимодействия. А несущественные детали любой хороший интерфейс скрывает.

Графически, работу операции

```
pb = dynamic_cast<Ival_box*> (pw)
```

можно изобразить следующим образом:



Стрелки от *pw* и *pb* символизируют указатели на передаваемый объект, а остальные стрелки отражают отношения наследования между различными частями передаваемого объекта.

Получение и использование информации о типе объекта на этапе выполнения программы обычно называют механизмом *RTTI* (*run-time type information*).

Приведение типа от базового класса к производному обычно называют *понижающим приведением* (*downcast*) из-за традиции рисовать классовые иерархии свер-

ху вниз. Соответственно, приведение от производного класса к базовому называют *повышающим приведением* (*upcast*). Преобразования между классами одного уровня иерархии называют *перекрестным приведением* (*crosscast*).

### 15.4.1. Операция `dynamic_cast`

Операция **`dynamic_cast`** имеет два операнда: тип, заключенный в угловые скобки, и указатель (или ссылка), заключенный в круглые скобки.

Сначала рассмотрим вариант с указателем:

**`dynamic_cast<T*> (p)`**

Если *p* имеет тип *T\** или *D\**, где *T* является базовым классом для *D*, результат будет таким же, как при простом присваивании *p* указателю типа *T\**. Например:

```
class BB_ival_slider : public Ival_slider, protected BBslider
{
    // ...
};

void f(BB_ival_slider* p)
{
    Ival_slider* pi1 = p;                // ok
    Ival_slider* pi2 = dynamic_cast<Ival_slider*>(p); // ok

    BBslider* pbb1 = p; // error: BBslider - защищенный базовый класс
    BBslider* pbb2 = dynamic_cast<BBslider*>(p); // ok: pbb2 станет 0
}
```

Рассмотренный пример большого интереса не представляет. Тем не менее, все равно приятно, что операция **`dynamic_cast`** не допускает случайных нарушений уровня доступа к закрытым и защищенным базовым классам.

Свое основное предназначение операция **`dynamic_cast`** демонстрирует в случаях, когда компилятор не может выяснить, корректно преобразование типов или нет. Тогда операция

**`dynamic_cast<T*> (p)`**

обращается к объекту, на который указывает *p*. Если это объект типа *T* или типа, у которого имеется уникальный базовый класс *T*, то операция **`dynamic_cast`** возвращает указатель типа *T\** на этот объект; в противном случае возвращается *нуль*. Если значение *p* равно *нулю*, то **`dynamic_cast<T*> (p)`** возвращает *нуль*. Обратите внимание на необходимость преобразования к уникально идентифицируемому объекту. Можно сконструировать примеры, когда преобразование невозможно и возвращается *нуль* из-за того, что объект, на который показывает *p*, содержит несколько подобъектов базового класса *T* (см. §15.4.2).

Для выполнения понижающего или перекрестного приведения нужно, чтобы аргумент операции **`dynamic_cast`** был ссылкой или указателем на полиморфный тип. Например:

```
class My_slider:public Ival_slider // базовый класс - полиморфный (имеет виртуаль. ф-ии)
```

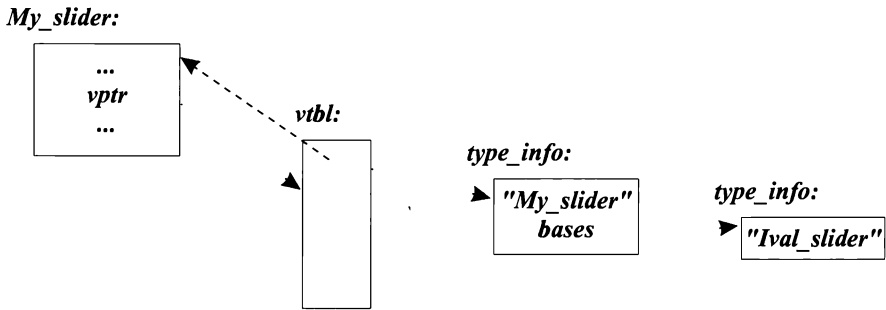
```

class My_date : public Date           // базовый класс - непалиморфный (нет виртуаль. ф-ий)
{
    // ...
};

void g (Ival_box* pb, Date* pd)
{
    My_slider* pd1 = dynamic_cast<My_slider*> (pb) ; // ok
    My_date*   pd2 = dynamic_cast<My_date*> (pd) ;   // error: Date - не полиморфный тип
}

```

Требование полиморфности указателя облегчает компилятору реализацию **dynamic\_cast**, ибо в этом случае легко найти место для хранения необходимой информации о типе объекта. В типичном случае специальный объект с информацией о типе прикрепляется к полиморфному объекту посредством добавления указателя на этот специальный объект к виртуальной таблице класса (§2.5.5). Например:



Пунктирная линия изображает смещение, позволяющее найти начало полного объекта при наличии лишь указателя на полиморфный подобъект. Ясно, что **dynamic\_cast** можно реализовать весьма эффективно. Все, что для этого требуется, это несколько сравнений объектов **type\_info**, представляющих базовые классы (и никаких дорогостоящих табличных поисков или сравнений строк).

Ограничение диапазона действия операции **dynamic\_cast** лишь полиморфными типами оправдано и с логической точки зрения. Действительно, если у объекта отсутствуют виртуальные функции, им невозможно безопасно пользоваться без знания его типа. Поэтому его нужно аккуратно применять лишь в контекстах, в которых его тип известен. А если тип объекта известен, то нет необходимости в операции **dynamic\_cast**.

Результирующий тип операции **dynamic\_cast** не обязан быть полиморфным. Это позволяет нам «завернуть» (wrap) конкретный тип в полиморфный с целью, скажем, его передачи системе ввода/вывода (§25.4.1), а позднее «развернуть» (unwrap) этот конкретный тип. Например:

```

class Io_obj                                     // базовый класс для системы ввода/вывода
{
    virtual Io_obj* clone () = 0;
};

class Io_date: public Date, public Io_obj { };

```

```
void f( Io_obj* pio)
{
    Date* pd = dynamic_cast<Date*>(pio) ;
    // ...
}
```

Применение *dynamic\_cast* для приведения к типу *void\** можно использовать, чтобы вычислить адрес начала объекта полиморфного типа. Например:

```
void g( Ival_box* pb, Date* pd)
{
    void* pd1 = dynamic_cast<void*>(pb) ; // ok
    void* pd2 = dynamic_cast<void*>(pd) ; // error: Date - не полиморфный тип
}
```

Это полезно лишь при взаимодействии с очень низкоуровневыми функциями.

#### 15.4.1.1. Применение *dynamic\_cast* к ссылкам

Для обеспечения полиморфного поведения доступ к объекту должен выполняться через указатель или по ссылке. Когда *dynamic\_cast* используется с указателями, возврат нуля означает невозможность приведения. Но к ссылкам все это абсолютно неприменимо.

Работая с указателями, мы всегда должны учитывать возможность его равенства нулю (это означает, что указатель не адресует никакого объекта). То есть результат работы операции *dynamic\_cast* над указателями должен всегда проверяться явным образом, а саму операцию *dynamic\_cast<T\*>(p)* можно трактовать как вопрос: «Действительно ли адресуемый указателем *p* объект имеет тип *T*?».

С другой стороны, мы можем твердо рассчитывать на то, что ссылка действительно указывает на некоторый объект. Следовательно, *dynamic\_cast<T&>(r)* для ссылки *r* рассматривается уже не как вопрос, а как утверждение: «Объект, на который ссылается *r*, имеет тип *T*». Результат операции *dynamic\_cast* тестируется в этом случае самой системой, и если ссылка имеет неправильный (несовместимый с заявленным) тип, то генерируется исключение *bad\_cast*. Например:

```
void f( Ival_box* p, Ival_box& r)
{
    if( Ival_slider* is = dynamic_cast<Ival_slider*>(p) ) // указывает ли p на Ival_slider?
    {
        // используем is
    }
    else
    {
        // *p это не ползунок
    }

    Ival_slider& is = dynamic_cast<Ival_slider&>(r) ; // r ссылается на Ival_slider!
    // используем is
}
```

Различие в поведении операции *dynamic\_cast* в ее работе над указателями и ссылками отражает фундаментальный факт различия между самими указателями и ссылками,

Если пользователю нужен надежный код в случае, когда *dynamic\_cast* применяется к ссылкам, нужно перехватывать и обрабатывать исключение *bad\_cast*. Например:

```
void g ()
{
    try
    {
        f(new BB_ival_slider, *new BB_ival_slider) ; // аргументы передаются как Ival_box
        f(new BBdial, *new BBdial) ;                 // аргументы передаются как Ival_box
    }
    catch (bad_cast)                                  // §14.10
    {
        // ...
    }
}
```

Первый вызов *f()* завершится нормально, в то время как второй вызовет исключение *bad\_cast*, которое будет перехвачено и обработано.

Явные проверки на ноль в случае работы с указателями могут быть случайно пропущены. Если это вас беспокоит, вы можете написать преобразующую функцию, которая вместо возврата нуля генерирует исключение (§15.8[1]).

### 15.4.2. Навигация по иерархиям классов

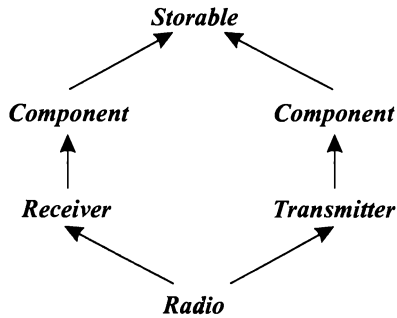
Когда используется лишь одиночное наследование, класс совместно с его базовыми классами образуют дерево с корнем в единственном базовом классе. Это просто, но часто слишком ограничительно. Когда используется множественное наследование, единственного корня не существует. Само по себе это не усложняет суть дела. Однако если класс появляется в иерархии более одного раза, нам следует проявлять осторожность при обращении к объектам такого класса.

Естественно, мы пытаемся конструировать иерархии настолько простыми, насколько нам позволяет делать это само приложение. Но если иерархия получилась нетривиальной, перед нами встает проблема навигации по этой иерархии с целью выявления класса с подходящим интерфейсом. Эта потребность проявляется в двух аспектах. Иногда нам нужно явно поименовать объект базового класса или член базового класса (см. §15.2.3 и §15.2.4.1). В иных случаях нам бывает необходимо получить указатель на объект, представляющий базовый или производный класс объекта, при наличии указателя на полный объект или некоторый подобъект (см. §15.4 и §15.4.1).

Рассмотрим способы навигации по иерархии классов с применением приведенных типов для получения указателя желаемого типа. Для иллюстрации самого механизма и регулирующих его правил рассмотрим классовую иерархию, содержащую как повторяющийся базовый класс, так и виртуальный базовый класс:

```
class Component: public virtual Storable { /*...*/ };
class Receiver: public Component { /*...*/ };
class Transmitter: public Component { /*...*/ };
class Radio : public Receiver, public Transmitter { /*...*/ };
```

или в графическом виде:



В этом примере объект типа **Radio** содержит два подобъекта класса **Component**. Следовательно, динамическое приведение операцией **dynamic cast** типа **Storable** в тип **Component** в рамках объекта **Radio** неоднозначно и вернет нуль. Просто нет способа узнать, какой именно **Component** запрашивает программист:

```

void h1 (Radio& r)
{
    Storable* ps = &r;
    // ...
    Component* pc = dynamic_cast<Component*> (ps);    // pc = 0
}
  
```

Эта неоднозначность в общем случае на этапе компиляции не выявляется:

```

void h2 (Storable* ps)    // ps может указывать на Component, а может и нет
{
    Component* pc = dynamic_cast<Component*> (ps);
    // ...
}
  
```

Такое выявление неоднозначности на этапе выполнения требуется лишь для виртуальных базовых классов. Для обычных базовых классов всегда при понижающих преобразованиях существует единственный подобъект (§15.4), в то время как для повышающих преобразований и здесь возникает аналогичная неоднозначность, обнаруживаемая на этапе выполнения.

#### 15.4.2.1. Операции **static\_cast** и **dynamic\_cast**

Операция **dynamic\_cast** может преобразовывать полиморфный виртуальный базовый класс в производный класс или в класс того же уровня иерархии («братский» класс) (§15.4.1). Операция **static\_cast** (§6.2.7) не анализирует объект, тип которого она приводит, и поэтому она не может этого сделать:

```

void g (Radio& r)
{
    Receiver* prec = &r;                // Receiver обычный базовый класс для Radio
    Radio* pr = static_cast<Radio*> (prec); // ok, без проверки
    pr = dynamic_cast<Radio*> (prec);    // o k, проверка на этапе выполнения
}
  
```

```

Storable* ps = &r;           // Storable - виртуальный базовый класс для Radio
pr = static_cast<Radio*>(ps);  // error: приведение из virtual base невозможно
pr = dynamic_cast<Radio*>(ps); // ok, проверка на этапе выполнения
}

```

Операция **dynamic\_cast** требует полиморфного операнда, потому что в непотоморфном объекте нет информации, которую можно использовать для того, чтобы решить, является ли указанный класс базовым для него. Объекты типов, имеющие ограничения на размещение в памяти, накладываемые языками Fortran или C, могут использоваться в качестве виртуальных базовых классов. Для таких объектов имеется лишь статическая информация о типе. Информация, необходимая для определения типа на этапе выполнения, включает в себя информацию, необходимую для реализации операции **dynamic\_cast**.

А зачем вообще может потребоваться операция **static\_cast** в контексте навигации по классовым иерархиям? Ведь дополнительные накладные расходы по применению операции **dynamic\_cast** невелики (§15.4.1). Однако существуют миллионы строк кода, написанные до появления **dynamic\_cast**. Они опираются на альтернативные способы проверки корректности преобразований, так что проверка операцией **dynamic\_cast** кажется избыточной. В типичном случае этот код использует приведения в стиле языка C (§6.2.7), и поэтому в нем остаются тонкие и трудноуловимые ошибки. В общем, где возможно используйте существенно более безопасные приведения операцией **dynamic\_cast**.

Компилятор не имеет представления о том, что адресуется указателем типа **void\***. Отсюда следует, что операция **dynamic\_cast**, которой требуется «заглянуть в объект» для выявления его типа, не может преобразовывать тип **void\***. Здесь-то и требуется **static\_cast**. Например:

```

Radio* f(void* p)
{
    Storable* ps = static_cast<Storable*>(p); // под ответственность программиста
    return dynamic_cast<Radio*>(ps);
}

```

Обе операции приведения — **dynamic\_cast** и **static\_cast**, учитывают модификатор **const** и уровни доступа. Например:

```

class Users: private set<Person> { /* ... */ };

void f(Users* pu, const Receiver* pcr)
{
    static_cast<set<Person*>>(pu); // error: нарушение доступа
    dynamic_cast<set<Person*>>(pu); // error: нарушение доступа

    Static_cast<Receiver*>(pcr); // error: невозможно "снять" const
    dynamic_cast<Receiver*>(pcr); // error: невозможно "снять" const

    Receiver* pr = const_cast<Receiver*>(pcr); // ok
    // ...
}

```

Невозможно осуществить преобразование к закрытому базовому классу, а для преодоления действия модификатора **const** (или **volatile**) требуется операция **const\_cast** (§6.2.7). И даже в этом случае, результат надежен, лишь если объект не был с самого начала объявлен как **const** (или **volatile**) (§10.2.7.1).



### 15.4.3. Конструирование и уничтожение классовых объектов

Объект класса — это нечто большее, чем просто область памяти (§4.9.6). Объект класса создается поверх «сырой памяти» («raw memory») с помощью конструкторов, и эта память становится снова «сырой» после отработки деструкторов. Создание протекает снизу вверх, а уничтожение идет сверху вниз; при этом объект класса является таковым в той мере, в какой он был создан, и до того, как был уничтожен. Этот факт отражен в правилах для RTTI, обработки исключений (§14.4.7) и виртуальных функций.

Крайне неразумно полагаться на порядок создания и уничтожения объектов, но порядок этот отражается на вызовах виртуальных функций, работе операций *dynamic\_cast* и *typeid* (§15.4.4) в момент, когда объект создан не полностью. Например, если конструктор класса *Component* из иерархии (§15.4.2) вызывает виртуальную функцию, будет вызвана версия классов *Storable* или *Component*, но не версии классов *Receiver*, *Transmitter* или *Radio*. В этой фазе конструирования объект еще не является объектом *Radio*, а является лишь частично созданным объектом. Лучше всего избегать вызова виртуальных функций на этапе создания или уничтожения объектов.

### 15.4.4. Операция typeid и расширенная информация о типе

Операция *dynamic\_cast* удовлетворяет большинство потребностей в информации о типе на этапе выполнения программы. Особо важно, что она гарантирует, что использующий эту операцию код корректно работает с классами, производными от некоторого указанного программистом класса. Таким образом, *dynamic\_cast* демонстрирует гибкость и расширяемость, свойственную виртуальным функциям.

Тем не менее, бывают случаи, когда нужно узнать точный тип объекта. Например, потребовалось узнать имя класса объекта или его расположение в памяти. Для этого имеется операция *typeid*, возвращающая объект, содержащий имя операнда этой операции. Если бы *typeid*() была функцией, ее объявление выглядело бы примерно следующим образом:

```
class type_info;
const type_info& typeid(type_name) throw();           // псевдообъявление
const type_info& typeid(expression) throw(bad_typeid); // псевдообъявление
```

То есть *typeid*() возвращает ссылку на тип из стандартной библиотеки, называемый *type\_info* (определен в файле <typeinfo>). Для операндов *type\_name* (имя типа) или *expression* (выражение) операцией возвращается ссылка на объект *type\_info*, содержащий собственно тип или тип выражения. Чаще всего операция *typeid*() вызывается для объектов, адресуемых указателем или ссылкой:

```
void f(Shape& r, Shape* p)
{
    typeid(r);    // тип объекта, на который ссылается r
    typeid(*p);   // тип объекта, на который указывает p
    typeid(p);    // тип указателя (здесь это Shape*)
}
```

Если значение операнда-указателя полиморфного типа равно *нулю*, операция *typeid*() генерирует исключение *bad\_typeid*. Если операнд операции *typeid*() имеет

неполиморфный тип или он не является lvalue, то результат определяется во время компиляции без вычисления выражения операнда.

Часть класса *type\_info* (не зависящая от конкретной реализации) выглядит следующим образом:

```
class type_info
{
public:
    virtual ~type_info () ;           // полиморфный

    bool operator== (const type_info&) const; // можно сравнивать
    bool operator!= (const type_info&) const;
    bool before (const type_info&) const;     // упорядочение

    const char* name () const;               // имя типа

private:
    type_info (const type_info&) ;           // предотвращает копирование
    type_info& operator= (const type_info&) ; // предотвращает копирование
    // ...
};
```

Функция *before* () помогает сортировать объекты типа *type\_info*. Нет никакой связи между отношениями упорядочения функцией *before* () и отношениями наследования.

Не гарантируется, что есть только один объект *type\_info* для каждого типа в системе. На самом деле, в системах с библиотеками динамической компоновки трудно избежать дублирования объектов *type\_info*. Следовательно, нужно применять операцию == для выявления эквивалентности объектов *type\_info*, а не сравнивать между собой значения указателей на такие объекты.

Иногда нам нужно знать точный тип объекта для выполнения некоторых действий с полным объектом (а не с отдельными его фрагментами базовых типов). В идеале такие действия должны выполняться виртуальными функциями, когда знать точный тип объекта нет необходимости. Но в некоторых случаях невозможно построить общий интерфейс ко всем типам объектов, и экскурс по типам объектов становится неизбежным (§15.4.4.1). Еще одним, более простым случаем является ситуация, когда имена типов нужно выводить для диагностики работы программы:

```
#include <typeinfo>

void g (Component* p)
{
    cout << typeid (*p) .name () ;
}
```

Символьное представление имени класса является системнозависимым. Эта C-строка располагается в системной области памяти, так что программист не должен освобождать ее операцией *delete* [].

#### 15.4.4.1. Расширенная информация о типе

Как правило, выявление точного типа объекта является лишь первым шагом получения более детальной информации о типе и его возможностях.

Рассмотрим вопрос о том, как приложение или инструментальное средство может сделать информацию о типе доступной пользователю на этапе выполнения программы. Допустим, что у меня есть инструментальное средство, генерирующее раскладку объектов каждого используемого класса. Я могу поместить эти описания в контейнер типа *map*, чтобы пользовательский код получил доступ к этой информации:

```
map<string, Layout> layout_table;

void f(B* p)
{
    Layout& x = layout_table[typeid(*p).name()];
    // используем x
}
```

Кто-нибудь еще мог бы предоставить информацию совершенно иного характера:

```
struct TI_eq
{
    bool operator() (const type_info* p, const type_info* q) {return *p==*q; }
};

struct TI_hash
{
    int operator() (const type_info* p);    // вычисляем hash-значение (§17.6.2.2)
};

hash_map<const type_info*, Icon, TI_hash, TI_eq> icon_table;    // §17.6

void g(B* p)
{
    Icon& i = icon_table[&typeid(*p)];
    // используем i
}
```

Такие варианты связывания результатов операции *typeid* с информационными структурами позволяют разным людям или инструментам независимо предоставлять разную информацию о типах:

*layout\_table*:

|     |  |
|-----|--|
| "T" |  |
| ... |  |

➤ раскладка  
объекта

*icon\_table*:

|            |  |
|------------|--|
| ...        |  |
| &typeid(T) |  |
| ...        |  |

➤ представление  
типа в виде  
иконки

Это очень важно, так как вероятность того, что всех устроит одинаковая информация о типах, близка к нулю.

### 15.4.5. Корректное и некорректное применение RTTI

Информацию о типах на этапе выполнения следует использовать только в случае реальной необходимости. Статическая (на этапе компиляции) проверка надежнее, влечет меньшие накладные расходы и приводит (как правило) к более структурированным программам. Например, можно использовать RTTI и для написания тонко замаскированных *switch-операторов*:

```
// неразумное применение RTTI:
void rotate (const Shape& r)
{
    if (typeid (r) == typeid (Circle) )
    {
        // ничего не делаем
    }
    else if (typeid (r) == typeid (Triangle) )
    {
        // вращаем треугольник
    }
    else if (typeid (r) == typeid (Square) )
    {
        // вращаем квадрат
    }
    // ...
}
```

Применение *dynamic\_cast* вместо *typeid* значительно улучшит этот код.

К сожалению, это не надуманный пример — такой код действительно пишется. Для многих людей, использовавших языки C, Pascal, Modula и Ada, очень трудно отказаться от написания кода в виде *switch-операторов*. Этому следует сопротивляться. В большинстве случаев, когда в зависимости от типа требуется осуществить вариативное поведение кода, вместо RTTI лучше использовать виртуальные функции (§2.5.5, §12.2.6).

Примеры надлежащего применения RTTI возникают, когда сервисный код выражен в терминах некоторого класса, а пользователь хочет добавить функциональность посредством наследования. Хорошей иллюстрацией сказанному может служить семейство классов *Ival\_box* из §15.4. Если бы пользователь хотел и имел возможность модифицировать библиотечные классы, скажем *BBwindow*, необходимость в RTTI отпала бы. В противном случае этот механизм нужен. Даже если пользователь хочет модифицировать базовые классы, в такой модификации имеются свои проблемы. Например, возможно пришлось бы писать фиктивные реализации виртуальных функций в классах, для которых они не нужны (или не имеют смысла). Эта проблема обсуждается более подробно в §24.4.3. Применение RTTI для реализации простой системы объектного ввода/вывода рассматривается в §25.4.1.

Люди, имеющие большой опыт работы с языками Smalltalk или Lisp, сильно опираются на динамическую проверку типов и испытывают искушение использовать RTTI совместно с чрезмерно общими типами. Рассмотрим пример:

```

// неразумное применение RTTI:
class Object { /* ... */ }; // полиморфный

class Container: public Object
{
public:
    void put (Object* ) ;
    Object* get ( ) ;
    // ...
};

class Ship: public Object { /* ... */ };

Ship* f(Ship* ps, Container* c)
{
    c->put (ps) ;
    // ...
    Object* p = c->get ( ) ;
    if (Ship* q = dynamic_cast<Ship*> (p) ) // проверка на этапе выполнения
    {
        return q ;
    }
    else
    {
        // что-нибудь еще (как правило, обработка ошибки)
    }
}

```

Здесь класс **Object** — необязательный артефакт реализации. Он слишком общий и не коррелирует с абстракциями какой-либо предметной области; он заставляет прикладного программиста использовать абстракции реализации. Проблемы такого рода часто решаются применением шаблонных контейнеров, хранящих единственный тип указателей:

```

Ship* f(Ship* ps, list<Ship*>& c)
{
    c.push_front (ps) ;
    // ...
    return c.pop_front ( ) ;
}

```

В комбинации с виртуальными функциями этот подход годится для большинства случаев.

## 15.5. Указатели на члены классов

Многие классы объявляют простые и весьма общие интерфейсы, которые предполагается использовать самыми разными способами. Например, для большинства «объектно-ориентированных» графических интерфейсов пользователя определяется некоторый набор запросов, реакцию на которые должен обеспечить каждый представленный на экране объект. Кроме того, программы могут формировать такие запросы прямо или косвенно. Рассмотрим простой вариант этой идеи:

```

class Std_interface
{
public:
    virtual void start () = 0;
    virtual void suspend () = 0;
    virtual void resume () = 0;
    virtual void quit () = 0;
    virtual void full_size () = 0;
    virtual void small () = 0;

    virtual ~Std_interface () {}
};

```

Точный смысл каждой операции определяется объектом, для которого она вызвана. Часто, между порождающей запрос программой (от имени пользователя) и получающим запрос объектом находится некоторый промежуточный слой кода. В идеале, этот промежуточный код ничего не должен знать про индивидуальные операции типа *resume()* или *full\_size()*. В противном случае, промежуточный код пришлось бы модифицировать при каждом изменении набора операций. Следовательно, промежуточный код должен лишь передавать от источника запроса к его получателю некоторые данные, идентифицирующие требуемую операцию.

Одним из простых способов реализации такого подхода является пересылка символической строки, содержащей название операции. Например, для вызова операции *suspend()* пересылается строка "*suspend*". Естественно, что должен быть кто-то, кто формирует такую строку, и кто-то другой, кто ее декодирует, чтобы выявить необходимую операцию (или отсутствие таковой). Часто такой подход может показаться слишком непрямым и слишком утомительным. Вместо этого можно было бы просто пересылать целые числа, представляющие операции. Например, число **2** могло бы означать *suspend()*. Однако пусть компьютерам и удобно работать с числами, человеку они мало что говорят. Кроме того, все равно нужно писать код, выявляющий, что число **2** соответствует операции *suspend()*, после чего вызывать эту операцию.

Теперь рассмотрим такое средство языка C++, как косвенная ссылка на член класса. Значение указателя на член класса идентифицирует этот член класса. Вы можете думать о нем как о позиции члена класса в объекте этого класса, но компилятор, конечно же, учитывает и различия в членах класса: поля данных класса, виртуальные и не виртуальные функции и т.д.

Рассмотрим интерфейс *Std\_interface*. Если я хочу вызвать функцию *suspend()* для некоторого объекта, не указывая эту операцию напрямую, мне потребуется указатель на *Std\_interface::suspend()*. Естественно, что мне также будет нужен указатель или ссылка на объект, для которого и нужно вызвать функцию *suspend()*. Рассмотрим тривиальный пример:

```

typedef void (Std_interface::*Pstd_mem) (); // указатель на функцию-член

void f(Std_interface* p)
{
    Pstd_mem s = &Std_interface::suspend;

    p->suspend (); // прямой вызов
    (p->*s) (); // вызов по указателю на функцию-член
}

```

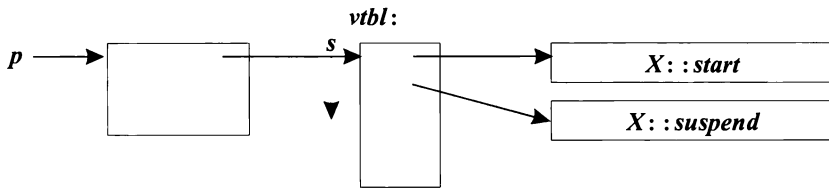
Указатель на член класса (*pointer to member*) можно получить с помощью операции взятия адреса, примененную к полностью квалифицированному имени члена класса, например, `&Sid_interface::suspend`. Переменная типа «указатель на член класса *X*» объявляется с применением декларатора вида *X*: \*.

Часто применяют оператор *typedef* для улучшения читаемости неудобоваримого синтаксиса деклараторов языка C. Обратите внимание на то, что синтаксис декларатора *X*: \* в точности соответствует обычному декларатору \* для указателей.

Указатель *m* на некоторый член класса можно применять как с указателем на объект, так и с самим объектом, для чего нужно использовать операции  $\rightarrow$  \* и . \*, соответственно. Например, выражение *p*  $\rightarrow$  \* *m* связывает *m* с объектом, на который указывает *p*, а выражение *obj*. \* *m* — связывает *m* с объектом *obj*. Результат этих операций используется в соответствии с типом *m*. Невозможно сохранить результат операций  $\rightarrow$  \* и . \* для его дальнейшего использования.

Конечно же, если бы мы заранее знали, какой именно член класса нам нужен, то мы могли бы работать с ним напрямую, а не затевать всю эту катавасию с указателями на члены классов. Как и в случае обычных указателей на функции, мы применяем указатели на функции-члены классов тогда, когда нам нужно сослаться на функцию-член, имя которой неизвестно. В отличие, однако, от обычных указателей на переменные или функции, которые представляют собой готовые целевые адреса в памяти, указатели на члены классов скорее являются смещениями в рамках некоторых структур (или индексами массивов). Когда указатель на член класса комбинируется с указателем на объект класса соответствующего типа, тогда-то и вырабатывается точный адрес конкретного члена для этого конкретного объекта.

Графически это можно изобразить следующим образом:



Так как указатель на виртуальную функцию-член класса (*s* в нашем примере) является в некотором смысле смещением, то он не зависит от точного расположения объекта в памяти. Поэтому указатель на виртуальную функцию-член можно безопасно передавать из одного адресного пространства в другое адресное пространство, при условии одинаковой раскладки объекта в них обоих. В то же время, указатели на неvirtуальные функции-члены (как и обычные указатели на функции) нельзя передавать в другие адресные пространства.

Очевидно, что функции, вызываемые через указатели на функции-члены, могут быть виртуальными. Например, когда мы вызываем *suspend*() через указатель на функцию-член, вызывается правильная версия функции, соответствующая объекту, к которому применялся указатель на функцию-член. Это является важным аспектом поведения указателей на функции-члены классов.

Интерпретирующий код может использовать указатели на функции-члены для вызова функций-членов, представленных в строковом виде:

```
map<string, Std_interface*> variable;
map<string, Pstd_mem> operation;

void call_member (string var, string oper)
{
    (variable [var] -> *operation [oper]) ();    // var.oper()
}
```

Чрезвычайно полезное применение указателей на функции-члены рассматривается в связи со стандартным шаблоном *mem\_fun()* в §3.8.5 и §18.4.

Так как статический член класса не ассоциируется с конкретным объектом класса, то указатель на статический член похож на обычный указатель. Например:

```
class Task
{
    // ...
    static void schedule ();
};

void (*p) () = &Task::schedule;    // ok
void (Task::*pm) () = &Task::schedule; // error: обычный указатель присваивается
                                         // указателю на функцию-член класса
```

Указатели на классовые поля данных рассматриваются в §C.12.

### 15.5.1. Базовые и производные классы

Производный класс в любом случае содержит члены, достаемые ему от базовых классов. Кроме того, часто у него есть и собственные члены. Из этого следует, что мы можем безопасно присваивать значения указателей на члены базовых классов указателям на члены производных классов, но не наоборот. Например:

```
class text: public Std_interface
{
public:
    void start ();
    void suspend ();
    // ...
    virtual void print ();

private:
    vector s;
};

void (Std_interface::*pmi) () = &text::print;    // error
void (text::*pmt) () = &Std_interface::start;    // ok
```

Это правило кажется противоположным известному правилу, что можно присваивать значения указателей на производные классы указателям на базовые классы. Однако оба правила действуют абсолютно согласованно: они гарантируют, что указатели никогда не адресуют объекты, у которых отсутствуют свойства, подразумеваемые типом указателя. В нашем примере указатели типа *Std\_interface::\** могут использоваться с любыми объектами иерархии *Std\_interface*, часть из которых не относится к типу *text*. Следовательно, у них может не быть функции-члена *text::print()*,



которым мы пытались проинициализировать *pmi*. Отказывая в инициализации, компилятор предотвращает ошибки времени выполнения.

## 15.6. Свободная память

Можно управлять выделением памяти для класса, определив в нем функции *operator new()* и *operator delete()* (§6.2.6.2). Однако замена глобальных функций *operator new()* и *operator delete()* — занятие не для слабонервных. В конце концов, другие программисты могут рассчитывать на предопределенное поведение механизма работы с динамической памятью, или представить свои собственные версии.

Более ограниченный и более надежный подход состоит в реализации этих операций в рамках конкретного класса. Причем этот класс может быть и базовым для многих других классов. Например, для класса *Employee* из §12.2.6 можно было бы определить специализированные операции выделения и освобождения памяти:

```
class Employee
{
    // ...
public:
    // ...
    void* operator new (size_t);
    void operator delete (void*, size_t);
};
```

Функции-члены *operator new()* и *operator delete()* неявно статические, так что у них отсутствует возможность работы с указателем *this* и они не изменяют объекты. Они просто предоставляют память, которую конструктор может инициализировать, а деструктор — очищать (деинициализировать):

```
void* Employee::operator new (size_t s)
{
    // выделить s байт памяти и вернуть указатель на эту память
}

void Employee::operator delete (void* p, size_t s)
{
    if (p)
    {
        // удаляем только если p!=0; see §6.2.6, §6.2.6.2
        // полагаем, что p указывает на s байт памяти, выделенной при помощи
        // Employee::operator new() и освобождаем эту память для повторного использования
    }
}
```

Использование загадочного до сих пор аргумента типа *size\_t* теперь становится понятным — это размер фактически уничтожаемого объекта. Если, например, удаляется объект типа *Employee*, то этот аргумент равен *sizeof(Employee)*, а если удаляется объект типа *Manager* — то *sizeof(Manager)*. Это позволяет специфическому для класса аллокатору не хранить объем выделенной памяти для каждого объекта. Но, естественно, он может и хранить эту информацию (аллокаторы общего назначения

обязаны это делать) и игнорировать аргумент типа *size\_t* у функции *operator delete()*. Последний подход затрудняет повышение производительности (в плане скорости и непроизводительных расходов памяти) механизмов работы с памятью по сравнению с таковыми же для общего назначения.

Как компилятор узнает правильное значение второго аргумента (размер удаляемого объекта) при вызове функции *operator delete()*? Пока операция *delete* ассоциируется с истинным типом объекта, это просто. Однако так бывает далеко не всегда:

```
class Manager: public Employee
{
    int level;
    // ...
};

void f()
{
    Employee* p = new Manager;    // беда: потерял истинный тип
    delete p;
}
```

В этом случае компилятор не знает истинного размера. Так же, как и при удалении массива, требуется помощь от программиста. Это делается добавлением виртуального деструктора к базовому классу *Employee*:

```
class Employee
{
public:
    void* operator new (size_t);
    void operator delete (void*, size_t);
    virtual ~Employee();
    // ...
};
```

Подойдет даже пустой деструктор:

```
Employee: ~Employee() {}
```

В случае виртуальных деструкторов необходимый для освобождения размер памяти так или иначе связывается с вызовом правильного деструктора (который знает истинный размер объекта).

Присутствие в классе *Employee* виртуального деструктора гарантирует, что каждый производный класс будет располагать деструктором (обеспечивающим информацию о правильном размере объекта), даже если таковой в этом производном классе явно и не определяется. Например:

```
void f()
{
    Employee* p = new Manager;
    Delete p;    // теперь все правильно (Employee - полиморфный)
}
```

Выделение кода осуществляется при помощи генерируемого компилятором вызова

```
Employee: :operator new (sizeof(Manager))
```

а освобождение памяти — при помощи другого генерируемого компилятором вызова:

```
Employee : : operator delete (p, sizeof(Manager) )
```

Другими словами, если вы хотите предоставить собственную пару аллокатор/деаллокатор, которая корректно работает с производными классами, то вы должны либо определить виртуальный деструктор в базовом классе, либо воздержаться от использования аргумента типа *size\_t* в деаллокаторе. Естественно, можно было бы спроектировать этот аспект языка таким образом, чтобы избавить программиста от сопутствующих сложностей. Но тогда бы и не было возможности осуществлять оптимизацию работы с памятью, присущую лишь менее безопасным системам.

### 15.6.1. Выделение памяти под массивы

Функции *operator new*() и *operator delete*() позволяют программисту брать на себя управление выделением/освобождением памяти для индивидуальных объектов; функции *operator new* [] () и *operator delete* [] () играют ту же роль для массивов. Например:

```
class Employee
{
public:
    void* operator new[] (size_t) ;
    void operator delete[] (void*, size_t) ;
    // ...
};

void f(int s)
{
    Employee* p = new Employee [s] ;
    // ...
    delete[] p;
}
```

В этом случае память выделяется при помощи вызова

```
Employee : : operator new[] (sizeof(Employee) *s+delta)
```

(где *delta* — некоторая вспомогательная память, зависящая от реализации), а освобождается память при помощи вызова

```
Employee : : operator delete[] (p) ;    // освобождаем s*sizeof(Employee)+delta байт
```

Количество элементов *s* и избыточная память *delta* запоминаются системой. Если бы в классе *Employee* была объявлена двухаргументная версия функции *operator delete* [] (), то она бы вызывалась со вторым аргументом, равным *s\*sizeof(Employee) + delta*.

### 15.6.2. «Виртуальные конструкторы»

После знакомства с виртуальными деструкторами возникает очевидный вопрос: «Может ли конструктор быть виртуальным?». Краткий ответ — нет; чуть менее краткий ответ такой: «Нет, но обеспечить искомый эффект возможно».

Для инициализации (создания) объекта конструктор должен знать его точный тип. Отсюда следует, что конструктор не может быть виртуальным. Более того, конструктор вообще не является обычной функцией. В частности, он взаимодействует со средствами управления памятью не так, как это делают обычные функции. Следовательно, вы не можете располагать указателем на конструктор.

Оба указанных ограничения можно обойти, определив функцию, которая вызывает конструктор и возвращает объект. Это полезная возможность, поскольку часто возникает необходимость в создании объекта, точный тип которого неизвестен. Класс *Ival\_box\_maker* (§12.4.4) служит примером класса, специально спроектированного для решения этой задачи. Теперь же я представлю иную вариацию данной идеи, когда объекты класса могут предоставлять пользователю клоны (копии) самих себя или полностью новые объекты своего собственного типа. Рассмотрим следующий код:

```
class Expr
{
public:
    Expr();           // умолчательный конструктор
    Expr(const Expr&); // копирующий конструктор

    virtual Expr* new_expr() const {return new Expr();}
    virtual Expr* clone() const {return new Expr(*this);}
    // ...
};
```

Поскольку функции вроде *new\_expr()* и *clone()* являются виртуальными и они (косвенно) конструируют объекты, их часто называют «виртуальными конструкторами» (шутка английского языка). Каждая из этих функций просто вызывает конструктор для создания необходимого объекта.

Производный класс может заместить функции *new\_expr()* и/или *clone()*, чтобы они возвращали объекты производного класса:

```
class Cond: public Expr
{
public:
    Cond();
    Cond(const Cond&);

    Cond* new_expr() const {return new Cond();}
    Cond* clone() const {return new Cond(*this);}
    // ...
};
```

Это означает, что получив указатель типа *Expr\** на объект данной иерархии, можно создать объект «точно такого же типа». Например:

```
void user(Expr* p)
{
    Expr* p2 = p->new_expr();
    // ...
}
```

Указатель, который здесь присваивается указателю *p2*, имеет правильный, но неизвестный тип.

Возвраты функций **Cond**: *new\_expr()* и *Cond*: *clone()* имеют тип **Cond\***, а не **Expr\***. Это позволяет клонировать **Cond** без потери информации о типе. Например:

```
void user2 (Cond* pc, Expr* pe)
{
    Cond* p2 = pc->clone ();
    Cond* p3 = pe->clone ();    // error
    // ...
}
```

Тип замещающей функции должен быть в точности таким же, как у замещаемой виртуальной функции, за исключением небольшого послабления по отношению к типу возвращаемого значения. Например, если у исходной функции возврат имел тип **B\***, то возврат у замещающей функции может быть **D\***, где **B** является открытым базовым классом для **D**. Аналогично, вместо **B&**, тип возврата может быть ослаблен до **D&**.

Заметьте, что аналогичные правила ослабления для типов аргументов привели бы к нарушению защиты типов (см. §15.8[12]).

## 15.7. Советы

1. Используйте обычное множественное наследование для того, чтобы выразить объединение свойств; §15.2, §15.2.5.
2. Используйте множественное наследования для отделения деталей реализации от интерфейса; §15.2.5.
3. Используйте *виртуальные* базовые классы там, где требуется выразить общность некоторой части (но не всех) классов иерархии; §15.2.5.
4. Избегайте явных приведений типов; §15.4.5.
5. Используйте *dynamic\_cast* там, где навигация по иерархии классов неизбежна; §15.4.1.
6. Предпочитайте *dynamic\_cast* операции *typeid*; §15.4.4.
7. Предпочитайте доступ *private* доступу *protected*; §15.3.1.1.
8. Не объявляйте поля данных защищенными (*protected*); §15.3.1.1.
9. Если класс определяет функцию *operator delete()*, он должен иметь виртуальный деструктор; §15.6.
10. Не вызывайте виртуальные функции в процессе создания или уничтожения объекта; §15.4.3.
11. Умеренно используйте явную квалификацию для разрешения перегрузки имен членов класса и применяйте ее в основном в замещающих функциях; §15.2.1.

## 15.8. Упражнения

1. (\*1) Напишите шаблон *ptr\_cast*, который работает как *dynamic\_cast*, только он вместо возврата нуля генерирует исключение *bad\_cast*.
2. (\*2) Напишите программу, которая иллюстрирует влияние последовательности вызова конструкторов на состояние объекта (с точки зрения RTTI). Аналогичным образом проиллюстрируйте процесс уничтожения объекта.
3. (\*3.5) Реализуйте версию настольной игры Reversi/Othello. Каждый игрок может быть либо человеком, либо компьютером. Сфокусируйтесь на корректной работе программы, а затем доведите качество игры компьютера до такого уровня, чтобы с ним было интересно играть.
4. (\*3) Улучшите пользовательский интерфейс игры из §15.8[3].
5. (\*3) Определите класс графических объектов с достаточным набором операций, чтобы он мог служить в качестве базового класса для графической библиотеки (подсмотрите необходимый набор операций в какой-либо коммерческой графической библиотеке). Определите класс для работы с базами данных, который служил бы базовым классом для библиотеки типов, хранящихся как последовательность полей в базе данных (подсмотрите необходимый набор операций в какой-либо коммерческой системе управления базами данных). Определите графические объекты базы данных, используя подходы со множественным наследованием и без него (сравните преимущества каждого из подходов).
6. (\*2) Напишите версию функции *clone()* из §15.6.2, которая помещала бы клонированные объекты в область памяти типа *Arena* (см. §10.4.11), переданную в качестве аргумента. Реализуйте простой конкретный класс для работы с фиксированными областями памяти, как производный от абстрактного класса *Arena*.
7. (\*2) Не заглядывая в книгу, напишите как можно больше ключевых слов языка C++.
8. (\*2) Напишите удовлетворяющую стандартам программу на C++, содержащую последовательность из по крайней мере десяти разных ключевых слов, не разделенных идентификаторами, знаками операций, пунктуации и т.д.
9. (\*2.5) Изобразите возможное распределение памяти для класса *Radio* из §15.2.3.1. Объясните, как можно реализовать вызов виртуальной функции.
10. (\*2) Изобразите возможное распределение памяти для класса *Radio* из §15.2.4. Объясните, как можно реализовать вызов виртуальной функции.
11. (\*3) Рассмотрите вопрос о том, как можно реализовать операцию *dynamic\_cast*. Определите и реализуйте шаблон *dcast*, который ведет себя как *dynamic\_cast*, но использует лишь данные и функции, определенные вами. Проверьте, что вы можете добавлять в систему новые классы, не изменяя определения *dcast* и других ранее написанных классов.
12. (\*2) Предположим, что правила проверки типов для аргументов ослаблены аналогично правилам для типов возвращаемых значений с тем, чтобы можно было заместить функцию, имеющую аргумент типа *Base\**, на функцию с аргументом *Derived\**. Напишите программу, которая может испортить объект типа *Derived* без использования приведения типов. Опишите безопасное ослабление правил для типов аргументов замещаемых функций.